

```

// ==UserScript==
// @name          HH3D Auto - v2.4.8
// @namespace     hh3d-tool
// @version       v2.4.9
// @description   Auto HH3D - t[i]u t[ư]c độ + thông báo/chuông
// @author        xnxx
// @include       *://hoathinh3d.*/*
// @exclude       *://hoathinh3d.*/*khoang-mach*
// @exclude       *://hoathinh3d.*/*luyen-dan-duong*
// @exclude       *://hoathinh3d.*/*tu-bao-cac-hh3d*
// @exclude       *://hoathinh3d.*/*me-cung*
// @exclude       *://hoathinh3d.*/*dua-top-hh3d*
// @require       https://cdn.jsdelivr.net/npm/sweetalert2@11.26.12/dist/sweetalert2.all.min.js
// @run-at        document-start
// @grant         unsafeWindow
// @connect       raw.githubusercontent.com
// @icon          []

// ==/UserScript==
(async function () {
    'use strict';

    const currentPath = window.location.pathname.replace(/\/$/, '');

    // =====
    // PERFORMANCE PROFILE (v2.4.9)
    // Ch[ế] t[i]u t[ư]c độ thời gian chờ nội bộ c[ủa] script; v[à] n[ữ]n giữ request gap + backoff đ[ể]
    // tránh tăng t[ần] s[ố] t[hi]n request một cách không c[ần] thi[ế]t.
    // Có th[ể] t[hi]n ch[ế] độ nhanh bằng localStorage: hh3d_fast_mode = '0'.
    // =====
    const HH3D_FAST_MODE = localStorage.getItem('hh3d_fast_mode') !== '0';
    const HH3D_BASE_REQUEST_GAP = 1500;
    const HH3D_FAST_HUMAN_DELAY_MIN = 80;
    const HH3D_FAST_HUMAN_DELAY_MAX = 220;
    const HH3D_FAST_STAGGER = 75;
    const HH3D_SAFE_STAGGER = 200;
    const excludes = [
        '/luyen-dan-duong',
        '/tu-bao-cac-hh3d',
        '/me-cung',
        '/dua-top-hh3d'
    ];
    if (excludes.some(p => currentPath === p || currentPath.startsWith(p + '/')) {
        console.log(['HH3D Script'] Excluded URL, script will not run.);
        return;
    }

    // =====
    // RESILIENT CLIENT FETCH WRAPPER (SEQUENTIAL & RATE-LIMIT COMPLIANT)
    // =====
    const originalFetch = window.fetch.bind(window);
    let lastRequestTime = 0;
    let currentMinRequestGap = HH3D_BASE_REQUEST_GAP;

    async function sleep(ms) {
        return new Promise(resolve => setTimeout(resolve, ms));
    }

    // Hàng đợi t[ư]n t[ự] nhưng có kh[ả] năng t[ự] phục h[ồi] sau l[ỗi].
    class RequestSerializer {
        constructor() {
            this.promise = Promise.resolve();
        }
        enqueue(action) {
            const run = this.promise.catch(() => undefined).then(action);
            this.promise = run.catch(err => {
                console.error(['RequestSerializer'] Error in queued task:', err);
                return undefined;
            });
            return run;
        }
    }
    const requestSerializer = new RequestSerializer();

    // Deduplicate GET cùng URL đang bay; ch[ế] áp dụng với request không khai báo no-store.
    const inFlightGetRequests = new Map();

    function getBackoffTime(retryCount, baseDelay = 2000, maxDelay = 15000) {
        const tempDelay = baseDelay * Math.pow(2, retryCount);
        const delay = Math.min(tempDelay, maxDelay);
        const jitter = Math.random() * (delay * 0.5);
        return delay + jitter;
    }

    function getFetchMethod(options = {}) {
        return String(options?.method || 'GET').toUpperCase();
    }

    function getFetchKey(url, options = {}) {
        const method = getFetchMethod(options);
        if (method !== 'GET') return null;
        if (options?.cache === 'no-store') return null;
        if (options?.signal) return null;
        return `${String(url)}|${method}|${options?.credentials || ''}`;
    }

    async function antiBotFetch(url, options = {}, retryCount = 0) {
        const MAX_RETRIES = 3;

        // Giữ request gap an toàn hiện có nhưng loại b[ỏ] sleep ng[ủ]u nhiên 0.8-2s
        // v[à] cộng thêm độ tr[ễ] cho MỌI request c[ủa] tool.
        const now = Date.now();
        const elapsed = now - lastRequestTime;
        if (elapsed < currentMinRequestGap) {
            await sleep(currentMinRequestGap - elapsed);
        }
        lastRequestTime = Date.now();

        try {
            const res = await originalFetch(url, options);

            if ((res.status === 429 || res.status === 503) && retryCount < MAX_RETRIES) {
                const retryAfterHeader = res.headers.get('Retry-After');
                let backoff = 0;

                if (retryAfterHeader) {

```

```

        backoff = isNaN(retryAfterHeader)
        ? Math.max(0, Date.parse(retryAfterHeader) - Date.now())
        : parseInt(retryAfterHeader, 10) * 1000;
    }

    if (backoff <= 0) {
        backoff = getBackoffTime(retryCount);
    }

    currentMinRequestGap = Math.min(currentMinRequestGap + 1000, 5000);
    console.warn(`[Client-Compliance] Nhận mã ph[  
n h[  
i ${res.status}. Tạm dừng và th[  
 lại sau ${Math.round(
backoff)}ms...`);
    await sleep(backoff);
    return antiBotFetch(url, options, retryCount + 1);
}

if (res.ok && currentMinRequestGap > HH3D_BASE_REQUEST_GAP) {
    currentMinRequestGap = Math.max(currentMinRequestGap - 500, HH3D_BASE_REQUEST_GAP);
}

return res;
} catch (err) {
    const message = String(err?.message || err || '');
    if (retryCount < MAX_RETRIES && /Failed to fetch|NetworkError|Load failed/i.test(message)) {
        const backoff = getBackoffTime(retryCount);
        console.warn(`[Client-Compliance] L[  
i mạng. Th[  
 lại sau ${Math.round(backoff)}ms...`);
        await sleep(backoff);
        return antiBotFetch(url, options, retryCount + 1);
    }
    throw err;
}
}

const fetch = function(url, options = {}) {
    const key = getFetchKey(url, options);
    if (!key) {
        return requestSerializer.enqueue(() => antiBotFetch(url, options));
    }

    const existing = inFlightGetRequests.get(key);
    if (existing) {
        return existing.then(response => response.clone());
    }

    const requestPromise = requestSerializer.enqueue(() => antiBotFetch(url, options));
    inFlightGetRequests.set(key, requestPromise);

    return requestPromise
        .then(response => response.clone())
        .finally(() => {
            if (inFlightGetRequests.get(key) === requestPromise) {
                inFlightGetRequests.delete(key);
            }
        });
};

console.log('%c[HH3D Script] T[  
i thành công. Đang kh[  
i tạo UI tùy ch[  
nh.',
'background: #222; color: #bada55; padding: 2px 5px; border-radius: 3px;');

// =====
// HÀM TIỆN ÍCH CHUNG
// =====
const weblink = window.location.origin.replace(/\/+$/, '') + '/';
const baseUrl = "https://raw.githubusercontent.com";
const repoPath = "/shinylee1205/Anonymous";
const branch = "/refs/heads/main";
//const Weblink = baseUrl + repoPath + branch + WeblinkFileName;
//const Weblink = baseUrl + repoPath + branch + WeblinkFileName;
// const Weblink = await fetch(Weblink);
// const weblink = (await Weblink.text()).trim();

// l[  
y chu[  
i và b[  
 kho[  
ng tr[  
ng thừa

const ajaxUrl = weblink + 'wp-content/themes/halimmovies-child/hh3d-ajax.php';
let accountId = '';
let questionDataCache = null;
let isCssInjected = false;
let userBetCount = 0;
let userBetStones = [];
// loggedIn: '1',
// userId: '',
// securityToken: '...',
// tokenType: 'normal',
// pageId: '622123',
// adminAjax: 'https://hoathinh3d.co/wp-admin/admin-ajax.php',
// themeAjax: 'https://hoathinh3d.co/wp-content/themes/halimmovies-child/hh3d-ajax.php',
// restAction: 'https://hoathinh3d.co/wp-json/hh3d/v1/action',
// restBase: 'https://hoathinh3d.co/wp-json',
// restNonce: 'b756294d06',
// act: {
//   tltmOpen: '1354a8a1',
//   tltmTimer: '2a2e1a5a',
//   plOpen: '9e73a16b',
//   plTimer: '5bb9c745',
//   plClaim: 'a278453e',
//   kmList: '64eaa497',
//   kmEnter: '5d6cbc77',
//   kmUsers: '0763c2d5',
//   kmClaim: '7a445af6',
//   kmBuy: '241d4835',
//   kmReward: '787974c8',
//   kmCheck: '7e4560ce',
//   kmNotif: '165d5ce3',
//   kmOwner: 'fafb8844',
//   kmAttack: '660d10f0',
//   kmLeave: '54608a24',
//   kmRefresh: '7ea284d4',
//   bossCheckElem: '72ced4dc',
//   bossTimer: '75b04751',
//   bossHistory: 'e1594bb2',
//   bossBoard: 'aa509745',
//   bossAttack: '6af45008',
//   bossGet: '2ad28e41',
//   bossBalance: '13ecac72',
//   bossBuy: '9cfa093a',
//   bossInventory: 'ce5635a7',

```

```

// bossOpenChest: '61ffdf1a',
// bossActivate: '1c31f726',
// dtExchange: 'a06553ee',
// dtLoad: 'e3e86547',
// dtBet: '9edf2e7e',
// dtClaim: '5e269d3c',
// dtNewbie: 'b8427634',
// vdLoad: 'a265eca2',
// vdSave: 'ba4ef77c',
// lotterySpin: '815b016f',
// lotteryHistory: '7b227116'
// hdnReward: 'b82f4d91'
// }
let cachedSecurityToken = null;
const fetchedUrlsCache = new Set();
fetchedUrlsCache.add(window.location.href);
fetchedUrlsCache.add(window.location.origin + window.location.pathname);
let hData = null; // Biện toàn cục lưu trữ hh3dData đã được parse và decode
/**
 * Parse và decode hh3dData từ HTML
 * @param {string} html - HTML chứa hh3dData mặc định
 * @param {string} k - Key để decode (optional, lấy từ HTML nếu không truy cập)
 * @param {string} d - Encrypted data (optional, lấy từ HTML nếu không truy cập)
 * @returns {object} hh3dData hoàn chỉnh với thuộc tính act đã được decode
 */
function parseHh3dData(html, k = null, d = null) {
    try {
        let hh3dData = {};

        // Bước 1: Lấy hh3dData mặc định từ HTML
        const hh3dMatch = html.match(/var\s+hh3dData\s*=\s*{([^}]+)}/);
        if (hh3dMatch) {
            try {
                hh3dData = JSON.parse(hh3dMatch[1]);
                // console.log('Đã parse hh3dData mặc định:', Object.keys(hh3dData));
            } catch (e) {
                console.log('Không thể parse hh3dData từ HTML:', e.message);
            }
        } else {
            console.log('Không tìm thấy hh3dData trong HTML');
        }

        // Bước 2: Lấy k và d từ HTML nếu không được truy cập vào
        if (!k || !d) {
            // Tìm script chứa k và d
            // Pattern: var k="..."d="...";
            const scriptMatch = html.match(/var\s+k\s*=\s*"([^"]+)"\s*,\s*d\s*=\s*"([^"]+)"\s*;/);
            if (scriptMatch) {
                k = k || scriptMatch[1];
                d = d || scriptMatch[2];
                // console.log('Đã lấy k và d từ HTML');
            } else {
                console.log('Không tìm thấy k và d trong HTML');
                return hh3dData; // Trả về hh3dData mặc định
            }
        }

        // Bước 3: Decode encrypted data
        if (k && d) {
            try {
                // Base64 decode (normalize URL-safe base64 → standard base64, then add padding)
                const b64 = d.replace(/\\\/g, '').replace(/-/g, '+').replace(/_/g, '/').replace(/\/s/g, '');
                const b64Padded = b64.padEnd(b64.length + (4 - b64.length % 4) % 4, '=');
                // Debug: log invalid chars
                const invalidChars = b64Padded.replace(/[A-Za-z0-9+]/g, '');
                // if (invalidChars.length > 0) {
                //     console.log('Đã có ký tự không hợp lệ (charCode):', [...invalidChars].map(c => c.charCodeAt(0)));
                // }
                // console.log('d (first 80):', d.substring(0, 80), '| len:', d.length);
                const decodedBytes = atob(b64Padded).split('').map(c => c.charCodeAt(0));

                // XOR với key
                let result = '';
                for (let i = 0; i < decodedBytes.length; i++) {
                    result += String.fromCharCode(decodedBytes[i] ^ k.charCodeAt(i % k.length));
                }

                // Parse JSON
                const actData = JSON.parse(result);
                hh3dData.act = actData;

                // console.log('Đã decode và gán hh3dData.act thành công');
            } catch (e) {
                console.log('Lỗi khi decode data:', e.message);
            }
        }

        const m = html.match(/var\s+boss_attack_token\s*=\s*"([a-f0-9]+)"\s*;/i);
        hh3dData.attackToken = m ? m[1] : null; // dành cho hoang vuc

        return hh3dData;
    } catch (error) {
        console.log('Lỗi trong parseHh3dData:', error.message);
        return {};
    }
}

// Override khi đang ở trang Khoáng Mạch
if (location.pathname.includes('khoang-mach') || location.href.includes('khoang-mach')) {
    const fastAttack = localStorage.getItem('khoangmach_fast_attack') === 'true';
    if (fastAttack) {
        const NEW_DELAY = 50;
        const originalSetInterval = window.setInterval;
        window.setInterval = function (callback, delay, ...args) {
            let actualDelay = delay;
            if (typeof callback === 'function' && callback.toString().includes('countdown--') &&
                callback.toString().includes('clearInterval(countdownInterval)') &&
                callback.toString().includes('executeAttack')) {
                actualDelay = NEW_DELAY;
                showNotification('Không được đánh dấu khi hết thông báo này', 'error', 5500);
            }
            return originalSetInterval(callback, actualDelay, ...args);
        };
    }
}

```

```

    };
  }
}
// Cấu trúc menu
// CHÚ Ý: Tất cả các task buttons đã được chuyển sang Quest List
// Giữ lại Autorun controls trong menu này
const LINK_GROUPS = []; // Autorun đã được chuyển lên xu-info

function addStyle(css) {
  const style = document.createElement('style');
  style.type = 'text/css';
  style.appendChild(document.createTextNode(css));
  document.head.appendChild(style);
}

// ===== Khoáng Mạch-only UI addons =====
addStyle(`
.km-punch-btn{
  background:#e74c3c;color:#fff;border:none;
  width:28px;height:28px;border-radius:6px;
  cursor:pointer;display:inline-flex;align-items:center;justify-content:center;
  font-weight:700;
}
.km-punch-btn:hover{filter:brightness(1.05);}
`);

let hh3dTTSVoices = null;
let hh3dTTSVoicePromise = null;

function getTTSVoices() {
  const voices = speechSynthesis.getVoices();
  if (voices.length) {
    hh3dTTSVoices = voices;
    return Promise.resolve(voices);
  }
  if (hh3dTTSVoicePromise) return hh3dTTSVoicePromise;

  hh3dTTSVoicePromise = new Promise(resolve => {
    const finish = () => {
      speechSynthesis.removeEventListener('voiceschanged', finish);
      hh3dTTSVoices = speechSynthesis.getVoices();
      resolve(hh3dTTSVoices || []);
    };
    speechSynthesis.addEventListener('voiceschanged', finish, { once: true });
    setTimeout(finish, 800);
  });
  return hh3dTTSVoicePromise;
}

async function speak(textVN, textEN) {
  try {
    const voices = hh3dTTSVoices || await getTTSVoices();
    if (!voices.length) return console.error('[TTS] Không tìm thấy voice kh dụng');

    let voice = voices.find(v => /vi[-_]?VN/i.test(v.lang));
    let lang = 'vi-VN';
    let text = textVN;
    if (!voice) {
      voice = voices.find(v => /en[-_]?US/i.test(v.lang)) || voices[0];
      lang = 'en-US';
      text = textEN;
    }

    const u = new SpeechSynthesisUtterance(text);
    u.voice = voice;
    u.lang = lang;
    u.rate = 0.8;
    u.onerror = e => console.error('[TTS] Lỗi:', e.error);

    speechSynthesis.cancel();
    speechSynthesis.speak(u);
  } catch (e) {
    console.error('[TTS] Lỗi khi tạo:', e);
  }
}

/**
 * Lấy securityToken bằng cách fetch một URL (nếu có)
 * hoặc quét HTML của trang hiện tại (nếu không có URL).
 *
 * @param {string} [url] - (Tùy chọn) URL để fetch.
 * @returns {Promise<string|null>} - Một Promise sẽ resolve với token, hoặc null nếu thất bại.
 */

async function getSecurityToken(url) {
  const logPrefix = "[SecurityTokenFetcher]";
  const targetUrl = url ? url.trim() : null;

  if (cachedSecurityToken && (!targetUrl || fetchedUrlsCache.has(targetUrl))) {
    console.log(`${logPrefix} × S dụng Security Token đã cache: ${cachedSecurityToken.substring(0, 8)}... (URL: ${targetUrl || 'trang hiện tại'})`);
    return cachedSecurityToken;
  }

  console.log(`${logPrefix} ▶ Bắt đầu lấy security token từ ${targetUrl || 'trang hiện tại'}...`);
  let htmlContent = null;

  try {
    // 1. Lấy nội dung HTML (Fetch hoặc quét trang hiện tại)
    if (targetUrl) {
      const response = await fetch(targetUrl);
      if (!response.ok) return null;
      htmlContent = await response.text();
    } else {
      htmlContent = document.documentElement.outerHTML;
    }
  }
  hData = parseHh3dData(htmlContent); // Cập nhật hh3dData từ html mới lấy được

  // 2. Quét Regex lấy Token mới
  const regex = /"securityToken"\s*:\s*"([^"]+)"/;
  const match = htmlContent.match(regex);

  if (match && match[1]) {
    const token = match[1];
    cachedSecurityToken = token;
  }
}

```

```

    if (targetUrl) {
        fetchedUrlsCache.add(targetUrl);
    }

    // LOGIC MỚI: Kiểm tra xem URL yêu cầu có phải là trang hiện tại không
    // Nếu không truy cập URL (!url) -> Mặc định là trang hiện tại
    // Nếu có URL -> Phải trùng khớp tuyệt đối với window.location.href
    const isCurrentPage = !targetUrl || (targetUrl === window.location.href);

    if (isCurrentPage) {
        console.log(`${logPrefix} URL trùng khớp trang hiện tại. Tiến hành cập nhật Global State...`);

        // =====
        // SỬA LỖI: CẬP NHẬT XUYẾN SANDBOX
        // =====

        // Cách 1: Dùng unsafeWindow (Cách chuẩn của Tampermonkey)
        if (typeof unsafeWindow !== 'undefined' && unsafeWindow.hh3dData) {
            unsafeWindow.hh3dData.securityToken = token;
            console.log(`${logPrefix} Đã cập nhật hh3dData thông qua unsafeWindow.`);
        }
        // Cách 2: Fallback nếu không có unsafeWindow
        else if (typeof window.hh3dData !== 'undefined') {
            window.hh3dData.securityToken = token;
            console.log(`${logPrefix} Đã cập nhật hh3dData qua window thường.`);
        }

        // Cách 3: "Tiêm thuốc" trực tiếp
        try {
            const script = document.createElement('script');
            script.textContent = `
                try {
                    if (typeof hh3dData !== 'undefined') {
                        hh3dData.securityToken = "${token}";
                        console.log('[Inject] Token đã được cập nhật từ bên trong trang web.`);
                    }
                } catch(e) {}
            `;
            (document.head || document.body || document.documentElement).appendChild(script);
            script.remove();
        } catch (injectErr) {
            console.warn(`${logPrefix} Lỗi tiêm script:`, injectErr);
        }
        // =====
    } else {
        // - Token không được trả về cho hàm gọi, không ảnh hưởng trang hiện tại
        console.log(`${logPrefix} Token lấy từ URL khác (${targetUrl}).`);
    }

    return token;
}
return null;

} catch (e) {
    console.error(`${logPrefix} Lỗi:`, e);
    return null;
}

}

//Lấy Nonce
async function getNonce() {
    if (typeof restNonce !== 'undefined' && restNonce) {
        return restNonce;
    }

    const scripts = document.querySelectorAll('script');
    for (const script of scripts) {
        let match = script.innerHTML.match(/"restNonce"\s*:\s*"([a-f0-9]+)"/);
        if (match) {
            return match[1];
        } else {
            match = script.innerHTML.match(/"nonce"\s*:\s*"([a-f0-9]+)"/);
            if (match) {
                return match[1];
            }
        }
    }
}

try {
    const nonce = await getSecurityNonce(weburl + '?t', "restNonce");
    if (nonce) {
        return nonce;
    }
} catch (error) {
    console.error("Failed to get security nonce", error);
}

return null;
}

/**
 * Lấy security nonce một cách chung chung từ một URL.
 */
@param {string} url - URL của trang web cần lấy nonce.
@param {RegExp} regex - Biểu thức chính quy (regex) để tìm và trích xuất nonce.
@returns {Promise<string|null>} Trả về security nonce nếu tìm thấy, ngược lại trả về null.
*/

async function getSecurityNonce(url, actionName, maxRetries = 3, retryCount = 0) {
    // Sử dụng một tiến trình log để đơn giản
    const logPrefix = '[HH3D Auto]';

    console.log(`${logPrefix} Đang tải trang từ ${url} để lấy security cho action: ${actionName}...`);
    try {
        const response = await fetch(url);
        if (!response.ok) {
            // Kiểm tra lỗi 503 (Service Unavailable) hoặc 429 (Too Many Requests)
            if ((response.status === 503 || response.status === 429) && retryCount < maxRetries) {
                const waitTime = 2000 + (retryCount * 1000); // 2s, 3s, 4s...
                console.warn(`${logPrefix} Lỗi ${response.status}, đang thử lại sau ${waitTime / 1000}s... (lần ${retryCount + 1}/${maxRetries})`);
                await new Promise(resolve => setTimeout(resolve, waitTime));
                return getSecurityNonce(url, actionName, maxRetries, retryCount + 1);
            }
        }
    }
}

```

```

        throw new Error(`HTTP error! status: ${response.status}`);
    }
    const html = await response.text();

    hhData = parseHh3dData(html); // Cập nhật hh3dData từ html

    // □ CẬP NHẬT: Trích xuất và cập nhật securityToken nếu có trong HTML
    const tokenRegex = /"securityToken"\s*:\s*"([^\"]+)"/;
    const tokenMatch = html.match(tokenRegex);
    if (tokenMatch && tokenMatch[1]) {
        const token = tokenMatch[1];
        console.log(`${logPrefix} □ Phát hiện securityToken mới trong HTML, đang cập nhật...`);
        cachedSecurityToken = token;

        // Kiểm tra URL có phải trang hiện tại không
        const isCurrentPage = window.location.href.includes(url);

        if (isCurrentPage) {
            // Cập nhật xuyên sandbox bằng getSecurityToken
            if (typeof unsafeWindow !== 'undefined' && unsafeWindow.hh3dData) {
                unsafeWindow.hh3dData.securityToken = token;
                console.log(`${logPrefix} □ Đã cập nhật hh3dData.securityToken thông qua unsafeWindow.`);
            } else if (typeof window.hh3dData !== 'undefined') {
                window.hh3dData.securityToken = token;
                console.log(`${logPrefix} □ Đã cập nhật hh3dData.securityToken qua window thường.`);
            } else {
                // Tiêm script trực tiếp
                try {
                    const script = document.createElement('script');
                    script.textContent = `
                        if (typeof hh3dData !== 'undefined') {
                            hh3dData.securityToken = "${token}";
                            console.log('□ [Inject] Token đã được cập nhật từ getSecurityNonce.');

```

```

        this.data = this.loadData();
        this.dothachTimeoutId = null;
    }
    // Tải dữ liệu từ localStorage
    loadData() {
        const storedData = localStorage.getItem(this.storageKey);
        return storedData ? JSON.parse(storedData) : {};
    }
    // Lưu dữ liệu vào localStorage
    saveData() {
        localStorage.setItem(this.storageKey, JSON.stringify(this.data));
    }
    /** Lọc thông tin của một tài khoản cụ thể và tự động cập nhật nó sang ngày mới
     * @param {string} accountId - ID của tài khoản.
     * @return {object} Tr về dữ liệu tài khoản, bao gồm các nhiệm vụ và trạng thái.
     * Nếu tài khoản chưa có dữ liệu, nó sẽ tự động tạo mới và lưu vào localStorage.
     * Nếu ngày hôm nay đã được cập nhật, nó sẽ reset các nhiệm vụ cho ngày mới.
     * Nếu đã đến giờ chuyển sang lượt 2 của Địch, nó sẽ tự động chuyển trạng thái.
     */
    getAccountData(accountId) {
        if (!this.data[accountId]) {
            this.data[accountId] = {};
            this.saveData();
        }

        const accountData = this.data[accountId];
        const today = new Date().toLocaleDateString('vi-VN', { timeZone: 'Asia/Ho_Chi_Minh' });

        // Danh sách tất cả nhiệm vụ mặc định
        const defaultTasks = {
            diemdanh: { done: false },
            thiluyen: { done: false, nextTime: null },
            bicanh: { done: false, nextTime: null },
            phucloi: { done: false, nextTime: null },

            hoangvuc: { done: false, nextTime: null },
            dothach: { betplaced: false, reward_claimed: false, turn: 1 },
            // luanvo: { battle_joined: false, auto_accept: false, done: false },
            khoangmach: { done: false, nextTime: null },
            tienduyen: { last_check: null, done: false },
            hoatdongngay: { done: false },
            luyenDan: { done: false, nextTime: null }
        };

        if (accountData.lastUpdatedDate !== today) {
            console.log('[TaskTracker] Cập nhật dữ liệu ngày mới cho tài khoản: ${accountId}');
            accountData.lastUpdatedDate = today;
            // Reset toàn bộ nhiệm vụ
            Object.assign(accountData, defaultTasks);
            this.saveData();
        } else {
            // Ngày chưa đổi → merge các nhiệm vụ mới
            let updated = false;
            for (const taskName in defaultTasks) {
                if (!accountData[taskName]) {
                    accountData[taskName] = defaultTasks[taskName];
                    updated = true;
                }
            }
            if (updated) this.saveData();
        }
        // X lý Địch Lượt 2
        const now = new Date();
        const hourInVN = parseInt(new Date().toLocaleString('en-US', { timeZone: 'Asia/Ho_Chi_Minh', hour:
'numeric', hour12: false }), 10);
        if (accountData.dothach.turn === 1 && hourInVN >= 16) {
            accountData.dothach = {
                betplaced: false,
                reward_claimed: false,
                turn: 2,
            };
            this.saveData();
        }
        // Lên lịch tự động reset vào 16h hàng ngày nếu chưa có timer
        if (!this.dothachTimeoutId) {
            const now = new Date();

            // Tạo danh sách các reset theo thứ tự
            const resetTimes = [
                new Date(now.getFullYear(), now.getMonth(), now.getDate(), 16, 1, 0, 0), // 16h hôm nay
                new Date(now.getFullYear(), now.getMonth(), now.getDate() + 1, 1, 0, 0, 0) // 01h sáng mai
            ];

            // Tìm các reset gần nhất so với hiện tại
            let nextResetTime = resetTimes.find(t => t > now);
            if (!nextResetTime) {
                // Nếu đã qua tất cả các reset → chọn 16h ngày mai
                nextResetTime = new Date(now.getFullYear(), now.getMonth(), now.getDate() + 1, 16, 0, 0, 0);
            }

            const timeToWait = nextResetTime - now;

            console.log('[TaskTracker] Reset sau ${Math.floor(timeToWait / 1000 / 60)} phút.');
```

```

* @param {string} accountId - ID của tài khoản.
* @param {string} taskName - Tên nhiệm vụ: 'diemdanh', 'thiluyen', 'bicanh', 'phucloi', 'hoangvuc'.
* @return {object|null} Trùng vớ đố tượng nhiệm vụ hoặc null nếu không tởn tại.
* Ví dụ: getTaskStatus('123', 'luanvo').battle_joined => 'true'
*/
getTaskStatus(accountId, taskName) {
  const accountData = this.getAccountData(accountId);
  return accountData[taskName] || null;
}
/**
* Kiểm tra xem một nhiệm vụ đã hoàn thành hay chưa
* @param {string} accountId - ID của tài khoản.
* @param {string} taskName - Tên nhiệm vụ: 'diemdanh', 'thiluyen', 'bicanh', 'phucloi', 'hoangvuc'.
* @return {boolean} Trùng vớ 'true' nếu nhiệm vụ đã hoàn thành, ngược lại là 'false'.
*/
isTaskDone(accountId, taskName) {
  const accountData = this.getAccountData(accountId);
  return accountData[taskName] && accountData[taskName].done;
}
/**
* Đánh dờu một nhiệm vụ là đã hoàn thành
* @param {string} accountId - ID của tài khoản.
* @param {string} taskName - Tên nhiệm vụ: 'diemdanh', 'thiluyen', 'bicanh', 'phucloi', 'hoangvuc'.
* @return {void}
*/
markTaskDone(accountId, taskName) {
  const accountData = this.getAccountData(accountId);
  if (accountData[taskName]) {
    accountData[taskName].done = true;
    this.saveData();

    // Trigger Hoạt Động Ngày khi cở 4 nhiệm vụ yêu cởu đã hoàn thành
    if (!['diemdanh', 'thiluyen', 'phucloi', 'hoangvuc'].includes(taskName)) {
      setTimeout(() => {
        if (typeof window.hh3dAutomatic !== 'undefined' && window.hh3dAutomatic) {
          const auto = window.hh3dAutomatic;
          const isHoangVucDone = this.isTaskDone(accountId, 'hoangvuc');
          const isPhucLoiDone = this.isTaskDone(accountId, 'phucloi');
          const isDiemDanhDone = this.isTaskDone(accountId, 'diemdanh');
          const isThiluyenDone = this.isTaskDone(accountId, 'thiluyen');
          const isHoatDongNgayDone = this.isTaskDone(accountId, 'hoatdongngay');

          if (isHoangVucDone && isPhucLoiDone && isDiemDanhDone && isThiluyenDone &&
!isHoatDongNgayDone) {
            console.log('[Auto Trigger] Phát hiện cở 4 nhiệm vụ (Hoang Vực, Phúc Lợi, Đỉm Danh,
Thí Luyện) đã hoàn thành! Kích hoạt ngay Hoạt Động Ngày...');
            auto.scheduleHoatDongNgay();
          }
        }
      }, 500);
    }
  } else {
    console.error('[TaskTracker] Nhiệm vụ "${taskName}" không tởn tại cho tài khoỏn "${accountId}"');
  }
}
/**
* Bỏ đánh dờu một nhiệm vụ đã hoàn thành
* @param {string} accountId - ID của tài khoản.
* @param {string} taskName - Tên nhiệm vụ.
* @return {void}
*/
unmarkTaskDone(accountId, taskName) {
  const accountData = this.getAccountData(accountId);
  if (accountData[taskName]) {
    accountData[taskName].done = false;
    this.saveData();
  } else {
    console.error('[TaskTracker] Nhiệm vụ "${taskName}" không tởn tại cho tài khoỏn "${accountId}"');
  }
}
/**
* Reset tởt cở trạng thái hoàn thành cởa các nhiệm vụ
* @param {string} accountId - ID của tài khoản.
* @return {void}
*/
resetAllTasks(accountId) {
  const accountData = this.getAccountData(accountId);
  const taskNames = ['diemdanh', 'thiluyen', 'bicanh', 'phucloi', 'hoangvuc', 'khoangmach', 'hoatdongngay',
'tienduyen', 'luyenDan'];
  let resetCount = 0;
  taskNames.forEach(taskName => {
    if (accountData[taskName] && accountData[taskName].done) {
      accountData[taskName].done = false;
      resetCount++;
    }
  });
  // Reset đờu thách vớ trạng thái ban đờu
  if (accountData.dothach) {
    accountData.dothach.betplaced = false;
    accountData.dothach.reward_claimed = false;
  }
  this.saveData();
  console.log('[TaskTracker] Đã reset ${resetCount} nhiệm vụ cho tài khoỏn ${accountId}');
  return resetCount;
}
/**
* Đỉu chỏnh thời gian cởa một nhiệm vụ
* @param {string} accountId - ID của tài khoản.
* @param {string} taskName - Tên nhiệm vụ: 'thiluyen', 'bicanh', 'phucloi', 'hoangvuc'.
* @param {string} newTime - Thời gian mới theo định dạng timestamp.
* @return {void}
*/
adjustTaskTime(accountId, taskName, newTime) {
  //console.log('[TaskTracker] adjustTaskTime called for ${taskName}, newTime=', newTime, "stack=", new Error(
).stack);

  const accountData = this.getAccountData(accountId);
  if (accountData[taskName]) {
    accountData[taskName].nextTime = newTime;
    this.saveData();
  } else {
    console.error('[TaskTracker] Nhiệm vụ "${taskName}" không tởn tại cho tài khoỏn "${accountId}"');
  }
}

getNextTime(accountId, taskName) {
  const accountData = this.getAccountData(accountId);

```

```

const ts = accountData[taskName]?.nextTime;
if (!ts || ts === "null") {
    return null; // chưa có thời gian
}
const date = new Date(Number(ts));
return isNaN(date.getTime()) ? null : date;
}
/** Return dạng Date */
getLastCheckTienDuyen(accountId) {
    const accountData = this.getTaskStatus(accountId, 'tienduyen');
    const timestamp = Number(accountData.last_check); // Chuyển chuỗi miligiây thành số
    return new Date(timestamp); // Tạo đối tượng Date
}
/** Lấy c timestamp dạng string hay Date đều được */
setLastCheckTienDuyen(accountId, timestamp) {
    let finalTimestamp = timestamp; // Khi tạo bài lưu giá trị cuối cùng
    // Kiểm tra nếu timestamp là một đối tượng Date
    if (timestamp instanceof Date) {
        finalTimestamp = timestamp.getTime(); // Lấy giá trị timestamp dạng số
    } else if (typeof timestamp === 'string') {
        finalTimestamp = Number(timestamp); // Chuyển chuỗi thành số
    }
    this.updateTask(accountId, 'tienduyen', 'last_check', finalTimestamp);
}
}
/**
 * Cộng thêm phút và giây vào thời điểm hiện tại và trả về một đối tượng Date mới.
 * @param {string} timeString - Chuỗi thời gian định dạng "mm:ss" (phút:giây).
 * @returns {Date} - String dạng timestamp cho thời gian được cộng thêm
 */

function timePlus(timeString) {
    const now = new Date();
    const [minutes, seconds] = timeString.split(':').map(Number);
    const millisecondsToAdd = (minutes * 60 + seconds) * 1000;
    return now.getTime() + millisecondsToAdd;
}

function isSecurityProtectionActive() {
    try {
        if (typeof window.__TD_SECURITY__ !== 'undefined' && window.__TD_SECURITY__) {
            return true;
        }
        const title = document.title || '';
        const bodyText = document.body ? document.body.textContent || '' : '';
        return title.includes('Thiên Đạo Bảo Vệ') || bodyText.includes('Thiên Đạo Bảo Vệ');
    } catch (error) {
        return false;
    }
}

// =====
// ĐỊNH NGHĨA CÁC NHIỆM VỤ / QUEST CONFIGURATION
// =====
const QUEST_CONFIG = [
    {
        taskId: 'diemdanh',
        taskName: 'Điểm Danh, Tỉ Lệ, Vượt Đáp',
        taskIcon: '<i class="fas fa-calendar-check"></i>',
        autorunEnabled: true,
        autorunKey: 'autoDiemDanh',
        hasButton: true,
        buttonText: 'Thực hiện',
        async action() {
            const nonce = await getNonce();
            if (!nonce) {
                showNotification("Không tìm thấy nonce! Vui lòng tải lại trang.", "error");
                return;
            }
            await doDailyCheckin(nonce);
            await doClanDailyCheckin(nonce);
            await vandap.doVanDap(nonce);
            console.log("[HH3D Script] Điểm danh, tỉ lệ, vượt đáp đã hoàn thành.");
        }
    },
    {
        taskId: 'thiluyen',
        taskName: 'Thí Luyện Tổng Môn',
        taskIcon: '<i class="fas fa-fire-alt"></i>',
        autorunEnabled: true,
        autorunKey: 'autoThiLuyen',
        hasButton: true,
        buttonText: 'Thực hiện',
        async action() {
            await doThiLuyenTongMon();
            console.log("[HH3D Script] Thí Luyện Tổng Môn đã hoàn thành.");
        }
    },
    {
        taskId: 'phucloi',
        taskName: 'Phúc Lợi Đường',
        taskIcon: '<i class="fas fa-gift"></i>',
        autorunEnabled: true,
        autorunKey: 'autoPhucLoi',
        hasButton: true,
        buttonText: 'Nhận',
        hasExtraButton: true,
        extraButtonText: ' ',
        extraButtonTitle: 'Nhận Bonus',
        async action() {
            await doPhucLoiDuong();
            console.log("[HH3D Script] Phúc Lợi đã hoàn thành.");
        },
        async extraAction() {
            await phucloiclaibonus();
            console.log("[HH3D Script] Nhận thưởng Bonus Phúc Lợi đã chạy.");
        }
    },
    {
        taskId: 'hoangvuc',
        taskName: 'Hoàng Vực',
        taskIcon: '<i class="fas fa-dragon"></i>',
        autorunEnabled: true,
        autorunKey: 'autoHoangVuc',
        hasButton: true,
        buttonText: 'Đánh',
    }
]

```

```

        hasExtraButton: true,
        extraButtonText: '🔍',
        extraButtonTitle: 'Mua 5 Rương Linh Bảo',
        hasSettings: true,
        hasDamageToggle: true,
        async action() {
            await hoangvuc.doHoangVuc();
        },
        async extraAction() {
            await hvmuaruong.muaRuongLinhBao(5);
        }
    },
    {
        taskId: 'mecung',
        taskName: 'Mê Cung',
        taskIcon: '<i class="fas fa-dungeon"></i>',
        hasButton: true,
        buttonText: 'Vào',
        async action() {
            window.location.href = '/me-cung';
        }
    },
    {
        taskId: 'khoangmach',
        taskName: 'Khoảng Mạch',
        taskIcon: '<i class="fas fa-gem"></i>',
        autorunEnabled: true,
        autorunKey: 'autoKhoangMach',
        hasButton: true,
        buttonText: 'Đào',
        hasSettings: true,
        hasExtraButton: true,
        extraButtonText: 'Vào',
        extraButtonTitle: 'Vào Khoảng Mạch',
        async extraAction() {
            window.location.href = '/khoang-mach';
        },
        async action() {
            await khoangmach.doKhoangMach();
        }
    },
    {
        taskId: 'dothach',
        taskName: 'Đồ Thạch',
        taskIcon: '<i class="fas fa-dice"></i>',
        autorunEnabled: true,
        autorunKey: 'autoDoThach',
        hasButton: true,
        buttonText: 'Cuộc',
        hasSelect: true,
        selectOptions: [
            { value: 'tai', label: 'Tài' },
            { value: 'xiu', label: 'Xỉu' }
        ],
        async action() {
            const choice = localStorage.getItem('dice-roll-choice') || 'tai';
            await dothach.run(choice);
        }
    },
    {
        taskId: 'bicanh',
        taskName: 'Bí Cầm',
        taskIcon: '<i class="fas fa-skull-crossbones"></i>',
        autorunEnabled: true,
        autorunKey: 'autoBiCanh',
        hasButton: true,
        buttonText: 'Đánh',
        hasInput: true,
        inputType: 'number',
        inputMin: 0,
        inputMax: 5,
        inputTitle: 'Số lượt giữ lại',
        hasToggle: true,
        toggleTitle: 'Theo dõi boss qua socket',
        async action() {
            await bicanh.doBiCanh();
        }
    },
    {
        taskId: 'tienduyen',
        taskName: 'Tiên Duyên',
        taskIcon: '<i class="fas fa-heart"></i>',
        autorunEnabled: true,
        autorunKey: 'autoTienDuyen',
        hasButton: true,
        buttonText: 'Thực hiện',
        hasExtraButton: true,
        extraButtonText: '🔍',
        extraButtonTitle: 'Cậu Nguyễn Đạo Lữ',
        hasExtra2Button: true,
        extra2ButtonText: '🔍',
        extra2ButtonTitle: 'Tặng Hoa',
        hasSelect: true,
        selectOptions: [
            { value: '1', label: 'tặng 1ng' },
            { value: '2', label: 'tặng 2ng' },
            { value: '3', label: 'tặng 3ng' },
            { value: '4', label: 'tặng 4ng' },
            { value: '5', label: 'tặng 5ng' }
        ],
        async action() {
            await tienduyen.doTienDuyen(true);
        },
        async extraAction() {
            const accountId = await getAccountId();
            await docaunguyen(accountId);
        },
        async extra2Action() {
            const quantity = parseInt(localStorage.getItem('tienduyen-choice') || '5', 10);
            if (!tanghoa.initialized) await tanghoa.init();
            await tanghoa.run(quantity);
        }
    }
],
{

```

```

        taskId: 'hoatdongngay',
        taskName: 'Hoạt Động Ngày - Vòng Quay',
        taskIcon: '<i class="fas fa-calendar-day"></i>',
        hasButton: true,
        buttonText: 'M',
        hasExtraButton: true,
        extraButtonText: 'Vào',
        extraButtonTitle: 'Vào Hoạt Động Ngày',
        hasExtra2Button: true,
        extra2ButtonText: '',
        extra2ButtonTitle: 'Nhận lượt Khc Trần Văn',
        async action() {
            await hoatdongngay.doHoatDongNgay();
        },
        async extraAction() {
            window.location.href = '/nhiem-vu-hang-ngay';
        },
        async extra2Action() {
            await hoatdongngay.claimDailyTurns();
        }
    },
    {
        taskId: 'luyenDan',
        taskName: 'Luyện Đan',
        taskIcon: '<i class="fas fa-mortar-pestle"></i>',
        autorunEnabled: true,
        autorunKey: 'autoLuyenDan',
        hasButton: true,
        buttonText: 'Luyện',
        hasSettings: true,
        async action() {
            await luyendan.doLuyenDan(true);
        }
    }
];

// =====
// TẠO SKELETON UI CHO DANH SÁCH NHIỆM VỤ
// =====
function createQuestSkeletonUI() {
    const questsHTML = QUEST_CONFIG.map(quest => {
        const extraControls = [];

        // Thêm select nếu có
        if (quest.hasSelect) {
            const savedValue = localStorage.getItem(`${quest.taskId}-choice`) || quest.selectOptions[quest.
selectOptions.length - 1].value;
            const optionsHTML = quest.selectOptions.map(opt =>
                `<option value="${opt.value}" ${opt.value === savedValue ? 'selected' : ''}>${opt.label}</option>`
            ).join('');
            extraControls.push(`<select class="quest-select" data-task="${quest.taskId}">${optionsHTML}</select>`);
        }

        // Thêm input nếu có
        if (quest.hasInput) {
            const savedValue = localStorage.getItem(`reserve${quest.taskId.charAt(0).toUpperCase() + quest.taskId.
slice(1)}Attacks`) || '0';
            extraControls.push(`<input type="${quest.inputType}" class="quest-input" data-task="${quest.taskId}"
min="${quest.inputMin}" max="${quest.inputMax}" value="${savedValue}"
title="${quest.inputTitle}" />`);
        }

        // Thêm toggle nếu có
        if (quest.hasToggle) {
            const savedState = localStorage.getItem(`${quest.taskId}SocketEnabled`);
            const isEnabled = savedState === '1';
            extraControls.push(`<button class="quest-toggle" data-task="${quest.taskId}"
title="${quest.toggleTitle}">${isEnabled ? '': '':''}</button>`);
            // Apply socket changes immediately
            if (isEnabled) {
                if (typeof bicanhhiente !== 'undefined' && bicanhhiente.startBossSocketListener) {
                    bicanhhiente.startBossSocketListener();
                }
            } else {
                if (typeof bicanhhiente !== 'undefined' && bicanhhiente.stopBossSocketListener) {
                    bicanhhiente.stopBossSocketListener();
                }
            }
        }

        // Thêm settings button nếu có
        if (quest.hasSettings) {
            extraControls.push(`<button class="quest-settings-btn" data-task="${quest.taskId}" title="Cài đặt"></button>`);
        }

        // Thêm toggle tỉ lệ ưu hóa sát thương (Hoang Vực)
        if (quest.hasDamageToggle) {
            const dmgOn = localStorage.getItem('hoangvucMaximizeDamage') === 'true';
            extraControls.push(`<button class="quest-toggle hoangvuc-damage-toggle" title="Tỉ lệ ưu hóa sát thương"
style="background: ${dmgOn ? 'rgba(34,197,94,0.25)' : 'rgba(255,255,255,0.05)'}; color: ${dmgOn ? '#22c55e' : '#9ca3af'};
font-size: 10px; font-weight: 700; min-width: 34px">${dmgOn ? '+15%' : '0%'}</button>`);
        }

        // Tạo nút chính
        let buttonHTML = '';
        if (quest.hasButton) {
            buttonHTML = `<button class="quest-action-btn" data-task="${quest.taskId}">${quest.buttonText}</button>`;
        }

        // Custom controls: Luân Võ inline mode + target ID
        // if (quest.hasCustomControls && quest.taskId === 'luanvo') {
        //     const lvMode = localStorage.getItem('luanVoChallengeMode') || 'auto';
        //     const lvTargetId = localStorage.getItem('luanVoTargetUserId_${accountId}') || '';
        //     buttonHTML = `
        //         <select class="quest-select luanvo-mode-select" title="Chọn độ thách đấu">
        //             <option value="auto" ${lvMode !== 'manual' ? 'selected' : ''}>Tự động</option>
        //             <option value="manual" ${lvMode === 'manual' ? 'selected' : ''}>Nhập ID</option>
        //         </select>
        //         <input type="text" class="quest-input luanvo-target-input" placeholder="ID đối thủ"
        //             value="${lvTargetId}"
        //             style="display: ${lvMode === 'manual' ? 'inline-block' : 'none'}; width: 70px">
        //     ` + buttonHTML;
        // }
        // Tạo nút extra nếu có

```

```

        if (quest.hasExtraButton) {
            buttonHTML += `<button class="quest-extra-btn" data-task="${quest.taskId}" title="${quest.extraButtonTitle}">${quest.extraButtonText}</button>`;
        }

        // Tạo nút extra2 nếu có
        if (quest.hasExtra2Button) {
            buttonHTML += `<button class="quest-extra-btn quest-extra2-btn" data-task="${quest.taskId}" title="${quest.extra2ButtonTitle}">${quest.extra2ButtonText}</button>`;
        }

        // Thêm indicator dot nếu quest có thể autorun
        let indicatorHTML = '';
        if (quest.autorunEnabled) {
            const isEnabled = localStorage.getItem(quest.autorunKey) !== '0';
            indicatorHTML = `<span class="quest-autorun-indicator ${isEnabled ? 'enabled' : 'disabled'}" data-task="${quest.taskId}"></span>`;
        }

        let progressText = '';
        if (quest.taskId === 'luyenDan') {
            const savedProgress = localStorage.getItem(`luyenDanLastProgress_${accountId}`);
            if (savedProgress) {
                progressText = ` (${savedProgress})`;
            }
        }

        return `
            <div class="nv-quest-item" data-task-id="${quest.taskId}">
                ${indicatorHTML}
                <span class="nv-quest-icon" data-task="${quest.taskId}" style="cursor: ${quest.autorunEnabled ? 'pointer' : 'default'};" title="${quest.autorunEnabled ? 'Bấm để bắt/tự chạy tự động' : ''}">${quest.taskIcon}</span>
                <span class="nv-quest-name">${quest.taskName}</span>
                <span class="quest-progress" style="color:#9ca3af;font-size:10px;">${progressText}</span>
                <div class="quest-controls">
                    ${extraControls.join('')}
                    ${buttonHTML}
                </div>
                <div class="quest-next-time" data-task="${quest.taskId}"></div>
            </div>
        `;
    }).join('');

    return `
        <div class="nv-overview">
            <div class="nv-ov-header">
                <h3>Nhiệm Vụ</h3>
                <span class="quest-next-time" data-task="restart" style="font-size:10px;color:#9ca3af;flex-shrink:0;margin-right:5px;"></span>
                <span class="percent">0%</span>
            </div>
            <div class="nv-progress-bar">
                <div class="nv-progress-fill" style="width: 0%"></div>
            </div>
            <p class="nv-ov-summary">0/5 nhiệm vụ</p>
            <div class="nv-chips"></div>
            <button class="progress-toggle-btn" onclick="const details = document.querySelector('.nv-quest-details'); if(details) { details.classList.toggle('show'); this.textContent = details.classList.contains('show') ? '▲ Chi tiết' : '▼ Xem chi tiết'; ">▼ Xem chi tiết</button>
            </div>
            <div class="nv-quest-details ">${questsHTML}</div>
        `;
    }

    /* ===== Update Profile Info ===== */
    async function loadHH3DProfile() {
        // Xu
        try {
            const res = await fetch("/vip-hh3d", { credentials: "include" });
            const html = await res.text();
            //console.log("HTML /vip-hh3d:", html); // log toàn bộ HTML trả về

            const xuMatch = html.match(/id="current-coins">([\d.,]+)/); // Bấm để có xu hoặc ,
            const xukhoaMatch = html.match(/id="current-coins-locked">([\d.,]+)/);

            //console.log("xuMatch:", xuMatch);
            //console.log("xukhoaMatch:", xukhoaMatch);

            const xu = xuMatch ? xuMatch[1] : "?";
            const xukhoa = xukhoaMatch ? xukhoaMatch[1] : "?";

            //console.log("Parsed Xu:", xu);
            //console.log("Parsed Xu Khóa:", xukhoa);

            document.getElementById("xu-info").innerHTML = `
                <div class="xu-display">
                    <span class="xu-left">Xu: <strong>${xu}</strong> <span> <strong>${xukhoa}</strong></span>
                    <span class="xu-right">
                        <span class="autorun-indicator-dot"></span>
                        <span class="autorun-icon" title="Bấm để bắt/tự chạy tự động khi tti trang"><i class="fas fa-robot"></i></span>
                        <button id="autorun-main-btn" class="autorun-main-btn">Bấm để</button>
                        <button id="profile-refresh-btn" title="Làm mới thông tin"></button>
                    </span>
                </div>
                <div id="promo-form" class="promo-form">
                    <input type="text" id="promo-code-input" placeholder="Nhập CODE" value="" />
                    <button id="promo-code-submit">Hộp Thử</button>
                    <button id="settings-btn" class="settings-btn" title="Cài đặt chung">⚙️</button>
                    <button id="guide-btn" class="settings-btn" title="Hướng dẫn sử dụng"></button>
                </div>
            `;

            const refreshBtn = document.getElementById('profile-refresh-btn');
            if (refreshBtn) {
                refreshBtn.addEventListener('click', async () => {
                    refreshBtn.textContent = '⏳';
                    refreshBtn.disabled = true;
                    await loadHH3DProfile();
                    refreshBtn.textContent = '⏳';
                    refreshBtn.disabled = false;
                    showNotification('Đã làm mới thông tin nhiệm vụ hàng ngày.', 'success', 2000);
                });
            }
        }
    }

```

```

}
// Bind autorun button in xu-info
const autorunBtn = document.getElementById('autorun-main-btn');
if (autorunBtn) {
  autorunBtn.addEventListener('click', async () => {
    // let enabled = localStorage.getItem('isRunning') !== '0';
    if (!window.isRunning) {
      autorunBtn.classList.add('running');
      autorunBtn.textContent = 'Dừng Lại';
      if (window.hh3dAutomatic) {
        await window.hh3dAutomatic.start();
      }
      window.isRunning = true;
      showNotification('Bắt đầu chạy tự động', 'info');
    } else {
      autorunBtn.classList.remove('running');
      autorunBtn.textContent = 'Bắt đầu';
      if (window.hh3dAutomatic) {
        await window.hh3dAutomatic.stop();
      }
      window.isRunning = false;
      showNotification('Đã dừng chạy tự động', 'info');
    }
  });
}

// Gọi sự kiện
setTimeout(() => {
  const promoSubmit = document.getElementById('promo-code-submit');
  const promoInput = document.getElementById('promo-code-input');

  // Submit code
  if (promoSubmit && promoInput) {
    promoSubmit.addEventListener('click', async () => {
      const code = promoInput.value.trim();
      if (!code) {
        showNotification('⚠ Vui lòng nhập mã CODE', 'warning');
        return;
      }
      localStorage.setItem('hh3d_promo_code', code);
      await submitPromoCode(code);
    });

    // Nhấn Enter cũng submit
    promoInput.addEventListener('keypress', async (e) => {
      if (e.key === 'Enter') {
        const code = promoInput.value.trim();
        if (code) {
          localStorage.setItem('hh3d_promo_code', code);
          await submitPromoCode(code);
        }
      }
    });
  }

  // Autorun icon toggle
  const autorunIcon = document.querySelector('.autorun-icon');
  const autorunDot = document.querySelector('.autorun-indicator-dot');
  if (autorunIcon && autorunDot) {
    // Initialize state
    let autorunEnabled = localStorage.getItem('autorunEnabled') !== '0';
    autorunDot.classList.toggle('enabled', autorunEnabled);
    autorunDot.classList.toggle('disabled', !autorunEnabled);

    autorunIcon.addEventListener('click', () => {
      autorunEnabled = !autorunEnabled;
      localStorage.setItem('autorunEnabled', autorunEnabled ? '1' : '0');
      autorunDot.classList.toggle('enabled', autorunEnabled);
      autorunDot.classList.toggle('disabled', !autorunEnabled);
      autorunIcon.classList.toggle('enabled', autorunEnabled);
      autorunIcon.classList.toggle('disabled', !autorunEnabled);

      const message = autorunEnabled ? 'Tự động chạy khi tắt trang đã được bật' : 'Tự động chạy khi';
      tti trang đã được tắt';
      showNotification(message, 'info');
    });

    // Robot icon reflects autorunEnabled (auto-start on page load)
    autorunIcon.classList.toggle('enabled', autorunEnabled);
    autorunIcon.classList.toggle('disabled', !autorunEnabled);
    // Button reflects actual running state (window.isRunning)
    if (window.isRunning) {
      autorunBtn.classList.add('running');
      autorunBtn.textContent = 'Dừng Lại';
    } else {
      autorunBtn.classList.remove('running');
      autorunBtn.textContent = 'Bắt đầu';
    }
  }

  const settingsBtn = document.getElementById('settings-btn');
  if (settingsBtn) {
    settingsBtn.addEventListener('click', () => {
      showQuestSettings();
    });
  }

  const guideBtn = document.getElementById('guide-btn');
  if (guideBtn) {
    guideBtn.addEventListener('click', () => {
      showGuideModal();
    });
  }
}, 100);
} catch (e) {
  console.error("Error fetching Xu:", e);
}

// Tải độ - Cập nhật dữ liệu vào skeleton UI
try {
  const html = await (await fetch("/nhiem-vu-hang-ngay?t=" + Date.now(), { credentials: "include" })).text();
  const doc = new DOMParser().parseFromString(html, "text/html");

  // Parse tải độ
  const ringLabel = doc.querySelector(".nv-ring-label");
  const percent = ringLabel ? ringLabel.textContent.trim() : "0%";
  const percentValue = parseInt(percent) || 0;

  // Parse thông tin nhiệm vụ

```

```

const heading = doc.querySelector(".nv-ov-right h3");
const summary = doc.querySelector(".nv-ov-right p");
const headingText = heading ? heading.textContent.trim() : "Nhiệm Vụ";
const summaryText = summary ? summary.textContent.trim() : "0/5 nhiệm vụ";

// Parse danh sách nhiệm vụ từ chips
const chips = [...doc.querySelectorAll(".nv-chip")];
const chipsHTML = chips.map(chip => {
  const isDone = chip.classList.contains("chip-done");
  const text = chip.textContent.trim();
  return `<span class="nv-chip ${isDone ? 'chip-done' : 'chip-pend'}">${text}</span>`;
}).join('');

const isFull = percentValue >= 100;

// Cập nhật UI Overview (không tạo mới)
const progressWrap = document.getElementById("reward-progress-wrap");
if (progressWrap) {
  // Cập nhật các phần tử đã có
  const percentElem = progressWrap.querySelector('.percent');
  const progressFill = progressWrap.querySelector('.nv-progress-fill');
  const summaryElem = progressWrap.querySelector('.nv-ov-summary');
  const chipsContainer = progressWrap.querySelector('.nv-chips');
  const headingElem = progressWrap.querySelector('.nv-ov-header h3');

  if (percentElem) {
    percentElem.textContent = percent;
    percentElem.className = `percent ${isFull ? 'full' : ''}`;
  }
  if (progressFill) {
    progressFill.style.width = percent;
    progressFill.className = `nv-progress-fill ${isFull ? 'full' : ''}`;
  }
  if (summaryElem) summaryElem.textContent = summaryText;
  if (chipsContainer) chipsContainer.innerHTML = chipsHTML;
  if (headingElem) headingElem.textContent = headingText;
}

// Parse danh sách nhiệm vụ chi tiết từ server và cập nhật trạng thái
const quests = [...doc.querySelectorAll(".nv-quest")];

// Mapping tên UI -> tên server (nhiem-vu-hang-ngay) khi khác nhau
const questNameAliases = {
  'Tiên Duyên': ['Hùng Sự Đường'],
};

// Gom progress theo từng quest item trong UI
const questItems = document.querySelectorAll('.nv-quest-item');
questItems.forEach(item => {
  if (item.getAttribute('data-task-id') === 'luyenDan') {
    return;
  }
  const nameEl = item.querySelector('.nv-quest-name');
  const progressContent = item.querySelector('.quest-progress').textContent || '';
  const itemName = (nameEl?.textContent || '').replace(progressContent, '').trim();
  const itemParts = itemName.split(',').map(s => s.trim());
  const aliases = questNameAliases[itemName] || [];
  const progressParts = [];
  let allDone = true;
  let anyMatch = false;

  quests.forEach(quest => {
    const isDone = quest.classList.contains("done");
    const name = quest.querySelector(".nv-qb h4")?.textContent.trim() || "";
    const progress = quest.querySelector(".nv-prog-txt")?.textContent.trim() || "";

    if (itemName.includes(name) || name.includes(itemName.split('<')[0].trim()) || itemParts.some(part
=> name.includes(part)) || aliases.some(alias => name.includes(alias) || alias.includes(name))) {
      anyMatch = true;
      if (!isDone) allDone = false;
      if (progress) progressParts.push(progress);
    }
  });

  if (anyMatch) {
    if (allDone) {
      item.classList.add('done');
    } else {
      item.classList.remove('done');
    }
    const progressSpan = item.querySelector('.quest-progress');
    if (progressSpan && progressParts.length > 0) {
      let progressText = '';
      if (progressParts.length > 1) {
        let curSum = 0, maxSum = 0;
        progressParts.forEach(p => {
          const m = p.match(/(\d+)\s*\s*(\d+)/);
          if (m) { curSum += parseInt(m[1]); maxSum += parseInt(m[2]); }
        });
        progressText = maxSum > 0 ? `${curSum}/${maxSum}` : `${progressParts.join(' | ')}`;
      } else {
        progressText = `${progressParts[0]}`;
      }
      // Nếu là Boss và có window.bicanhBossHp thì append phần trăm máu boss
      if (item.dataset.taskId === 'bicanh' && typeof window.bicanhBossHp === 'number') {
        progressText += ` (${window.bicanhBossHp.toFixed(2)}% HP)`;
      }
      progressSpan.textContent = progressText;
    }
  }
});

// Cập nhật trạng thái nút dựa trên taskTracker
await updateAllQuestButtons();

} catch (e) {
  console.error("Error loading progress:", e);
  const progressWrap = document.getElementById("reward-progress-wrap");
  if (progressWrap && !progressWrap.querySelector('.nv-quest-details')) {
    // Chưa hiển thị lần đầu chưa có skeleton
    progressWrap.innerHTML =
<div style="font-size:12px;color:#999;">
    Không thể tải dữ liệu
  </div>
  `;
  }
}

```

```

    }
}

// =====
// CẬP NHẬT TRẠNG THÁI TẤT CẢ CÁC NÚT NHIỆM VỤ
// =====
async function updateAllQuestButtons() {
    const accountId = await getAccountId();
    if (!accountId) return;

    QUEST_CONFIG.forEach(async (quest) => {
        const button = document.querySelector(`.quest-action-btn[data-task="${quest.taskId}"]`);
        if (!button) return;

        const questItem = document.querySelector(`.nv-quest-item[data-task-id="${quest.taskId}"]`);

        // Xử lý từng loại task
        switch (quest.taskId) {
            case 'diemdanh':
            case 'thiluyen':
            case 'phucloi':
            case 'hoangvuc':
            case 'khoangmach':
            case 'tienduyen':
            case 'hoatdongngay':
                if (taskTracker.isTaskDone(accountId, quest.taskId)) {
                    // button.disabled = true;
                    button.textContent = '✓ Xong';
                    countdownTimer.remove(quest.taskId);
                    if (questItem) questItem.classList.add('done');
                } else {
                    // button.disabled = false;
                    button.textContent = quest.buttonText;
                    if (questItem) questItem.classList.remove('done');
                }
                break;

            case 'luyendan':
                if (taskTracker.isTaskDone(accountId, quest.taskId)) {
                    // button.disabled = true;
                    button.textContent = '✓ Xong';
                    // Chờ remove timer khi KHÔNG đang xử lý để tránh xóa span progress giữa chừng
                    if (!luyendan || !luyendan.isProcessing) {
                        countdownTimer.remove('luyendan');
                        countdownTimer.remove('luyenDanCheck');
                    }
                    if (questItem) questItem.classList.add('done');
                } else {
                    // button.disabled = false;
                    button.textContent = quest.buttonText;
                    if (questItem) questItem.classList.remove('done');
                }
                break;

            case 'dothach':
                const currentHour = parseInt(new Date().toLocaleString('en-US', {
                    timeZone: 'Asia/Ho_Chi_Minh',
                    hour: 'numeric',
                    hour12: false
                }), 10);
                const isBetTime = (currentHour >= 6 && currentHour < 13) || (currentHour >= 16 && currentHour < 21);
                const status = taskTracker.getTaskStatus(accountId, 'dothach');
                if ((status.betplaced && isBetTime) || (status.reward_claimed && !isBetTime)) {
                    // button.disabled = true;
                    button.textContent = '✓ Xong';
                } else {
                    // button.disabled = false;
                    button.textContent = quest.buttonText;
                }
                break;

            case 'bicanh':
                const isDailyLimit = await bicanh.isDailyLimit();
                if (isDailyLimit) {
                    // button.disabled = true;
                    button.textContent = '✓ Xong';
                    countdownTimer.remove('bicanh');
                    if (questItem) questItem.classList.add('done');
                } else {
                    // button.disabled = false;
                    button.textContent = quest.buttonText;
                    if (questItem) questItem.classList.remove('done');
                }
                break;
        }
    });
}

// =====
// GỒN SỰ KIỆN CHO CÁC NÚT VÀ CONTROLS TRONG QUEST
// =====
function attachQuestButtonHandlers() {
    // Gõn sự kiện cho tất cả các nút action
    document.querySelectorAll('.quest-action-btn').forEach(button => {
        const taskId = button.getAttribute('data-task');
        const questConfig = QUEST_CONFIG.find(q => q.taskId === taskId);

        if (!questConfig || !questConfig.action) return;

        button.addEventListener('click', async () => {
            const originalText = button.textContent;
            button.disabled = true;
            button.textContent = 'Đang xử lý...';

            try {
                await questConfig.action();
                // Với luyendan: không gọi loadHH3DProfile() vì nó sẽ kích hoạt
                // updateAllQuestButtons → countdownTimer.remove → xóa span progress
                if (taskId !== 'luyendan') {
                    await loadHH3DProfile();
                } else {
                    await updateAllQuestButtons();
                }
            } catch (error) {
                console.error(`[Quest ${taskId}] Error:`, error);
                showNotification(`Lỗi khi thực hiện ${questConfig.taskName}`, 'error');
            }
        });
    });
}

```

```

        } finally {
            button.textContent = originalText;
            button.disabled = false;
        }
    });
});

// Gõn sự kiện cho các nút extra (như bonus)
document.querySelectorAll('.quest-extra-btn:not(.quest-extra2-btn)').forEach(button => {
    const taskId = button.getAttribute('data-task');
    const questConfig = QUEST_CONFIG.find(q => q.taskId === taskId);

    if (!questConfig || !questConfig.extraAction) return;

    button.addEventListener('click', async () => {
        const originalText = button.textContent;
        button.disabled = true;
        button.textContent = '⏳';

        try {
            await questConfig.extraAction();
            showNotification(`${questConfig.taskName} - Extra hoàn thành`, 'success');
        } catch (error) {
            console.error(`[Quest ${taskId} Extra] Error:`, error);
            showNotification(`Lỗi khi thực hiện extra ${questConfig.taskName}`, 'error');
        } finally {
            button.textContent = originalText;
            button.disabled = false;
        }
    });
});

// Gõn sự kiện cho các nút extra2
document.querySelectorAll('.quest-extra2-btn').forEach(button => {
    const taskId = button.getAttribute('data-task');
    const questConfig = QUEST_CONFIG.find(q => q.taskId === taskId);

    if (!questConfig || !questConfig.extra2Action) return;

    button.addEventListener('click', async () => {
        const originalText = button.textContent;
        button.disabled = true;
        button.textContent = '⏳';

        try {
            await questConfig.extra2Action();
            showNotification(`${questConfig.taskName} - ${questConfig.extra2ButtonText} hoàn thành`,
'success');
        } catch (error) {
            console.error(`[Quest ${taskId} Extra2] Error:`, error);
            showNotification(`Lỗi khi thực hiện ${questConfig.extra2ButtonText}`, 'error');
        } finally {
            button.textContent = originalText;
            button.disabled = false;
        }
    });
});

// Gõn sự kiện cho các select
document.querySelectorAll('.quest-select').forEach(select => {
    const taskId = select.getAttribute('data-task');
    select.addEventListener('change', () => {
        localStorage.setItem(`${taskId}-choice`, select.value);
        console.log(`[Quest ${taskId}] Đã lưu lựa chọn: ${select.value}`);
    });
});

// Gõn sự kiện cho các input
document.querySelectorAll('.quest-input').forEach(input => {
    const taskId = input.getAttribute('data-task');
    input.addEventListener('input', () => {
        let value = parseInt(input.value, 10);
        if (isNaN(value)) value = 0;
        if (value < input.min) value = input.min;
        if (value > input.max) value = input.max;
        input.value = value;
        const storageKey = `reserve${taskId.charAt(0).toUpperCase() + taskId.slice(1)}Attacks`;
        localStorage.setItem(storageKey, value.toString());
        console.log(`[Quest ${taskId}] Đã lưu giá trị reserve: ${value}`);
    });
});

// Gõn sự kiện cho các toggle button
document.querySelectorAll('.quest-toggle').forEach(button => {
    const taskId = button.getAttribute('data-task');
    button.addEventListener('click', () => {
        const isEnabled = button.textContent === '✅';
        const newState = !isEnabled;
        button.textContent = newState ? '✅' : '❌';
        localStorage.setItem(`${taskId}SocketEnabled`, newState ? '1' : '0');
        console.log(`[Quest ${taskId}] Socket tracking: ${newState ? 'Enabled' : 'Disabled'}`);

        // Xử lý logic đặc biệt cho Bí Cảnh
        if (taskId === 'bicanh') {
            if (newState) {
                if (typeof bicanhhiente !== 'undefined' && bicanhhiente.startBossSocketListener) {
                    bicanhhiente.startBossSocketListener();
                }
            } else {
                if (typeof bicanhhiente !== 'undefined' && bicanhhiente.stopBossSocketListener) {
                    bicanhhiente.stopBossSocketListener();
                }
            }
        }
    });
});

// Gõn sự kiện cho các settings button
document.querySelectorAll('.quest-settings-btn').forEach(button => {
    const taskId = button.getAttribute('data-task');
    button.addEventListener('click', () => {
        showQuestSettings(taskId);
    });
});

// Gõn sự kiện toggle tị ưu hóa sát thương Hoang Vực

```

```

const damageToggleBtn = document.querySelector('.hoangvuc-damage-toggle');
if (damageToggleBtn) {
    damageToggleBtn.addEventListener('click', () => {
        const current = localStorage.getItem('hoangvucMaximizeDamage') === 'true';
        const next = !current;
        localStorage.setItem('hoangvucMaximizeDamage', String(next));
        damageToggleBtn.style.background = next ? 'rgba(34,197,94,0.25)' : 'rgba(255,255,255,0.05)';
        damageToggleBtn.style.color = next ? '#22c55e' : '#9ca3af';
        damageToggleBtn.textContent = next ? '+15%' : '0%';
        showNotification(`Tăng ưu hóa sát thương: ${next ? '+15%' : '0%'}`, 'info', 1500);
    });
}

// Gắn sự kiện Luận Võ inline controls
const luanvoModeSel = document.querySelector('.luanvo-mode-select');
const luanvoTargetInput = document.querySelector('.luanvo-target-input');

if (luanvoModeSel) {
    luanvoModeSel.addEventListener('change', () => {
        const isManual = luanvoModeSel.value === 'manual';
        if (luanvoTargetInput) luanvoTargetInput.style.display = isManual ? 'inline-block' : 'none';
        localStorage.setItem('luanVoChallengeMode', luanvoModeSel.value);
    });
}

if (luanvoTargetInput) {
    luanvoTargetInput.addEventListener('input', () => {
        localStorage.setItem('luanVoTargetUserId_${accountId}', luanvoTargetInput.value);
    });
}

// Gắn sự kiện click vào icon để toggle autorun
document.querySelectorAll('.nv-quest-icon[data-task]').forEach(icon => {
    const taskId = icon.getAttribute('data-task');
    const questConfig = QUEST_CONFIG.find(q => q.taskId === taskId);

    if (!questConfig || !questConfig.autorunEnabled) return;

    icon.addEventListener('click', (e) => {
        e.stopPropagation();
        toggleQuestAutorun(taskId, questConfig.autorunKey);
    });
});
}

// =====
// TOGGLE AUTORUN CHO QUEST
// =====
function toggleQuestAutorun(taskId, autorunKey) {
    const indicator = document.querySelector(`.quest-autorun-indicator[data-task="${taskId}"]`);
    if (!indicator) return;

    // Lấy trạng thái hiện tại
    const currentState = localStorage.getItem(autorunKey) !== '0';
    const newState = !currentState;

    // Lưu vào localStorage
    localStorage.setItem(autorunKey, newState ? '1' : '0');

    // Cập nhật UI
    if (newState) {
        indicator.classList.remove('disabled');
        indicator.classList.add('enabled');
    } else {
        indicator.classList.remove('enabled');
        indicator.classList.add('disabled');
    }
}

// Hiển thị thông báo
const questConfig = QUEST_CONFIG.find(q => q.taskId === taskId);
const statusText = newState ? 'bật' : 'tắt';
showNotification(`${questConfig.taskName}: Đã ${statusText} chạy tự động`, newState ? 'success' : 'info', 2000);

console.log(`[Quest Autorun] ${taskId}: ${newState ? 'Enabled' : 'Disabled'}`);

// Dừng hoặc bắt đầu tác vụ ngay lập tức mà không cần tải lại trang
if (!newState) {
    if (typeof automatic !== 'undefined' && automatic) {
        // Dừng timeout của task này (bao gồm cả luyenDan)
        if (automatic.timeoutIds && automatic.timeoutIds[taskId]) {
            clearTimeout(automatic.timeoutIds[taskId]);
            automatic.timeoutIds[taskId] = null;
        }
        if (taskId === 'tienduyen' && automatic.tienduyenTimeout) {
            clearTimeout(automatic.tienduyenTimeout);
            automatic.tienduyenTimeout = null;
        }
        if (taskId === 'dothach' && automatic.dothachTimeout) {
            clearTimeout(automatic.dothachTimeout);
            automatic.dothachTimeout = null;
        }
    }
    // Xóa countdown timer (kể cả luyenDan và luyenDanCheck)
    countdownTimer.remove(taskId);
    if (taskId === 'luyenDan') {
        countdownTimer.remove('luyenDanCheck');
        luyendan.updateProgress(''); // xóa text progress
    }
} else {
    if (typeof automatic !== 'undefined' && automatic && automatic.isRunning) {
        if (taskId === 'luyenDan') {
            automatic.scheduleTask('luyenDan', () => luyendan.doLuyenDan(), automatic.INTERVAL_LUYEN_DAN);
        } else if (taskId === 'hoangvuc') {
            automatic.scheduleTask('hoangvuc', () => hoangvuc.doHoangVuc(), automatic.INTERVAL_HOANG_VUC);
        } else if (taskId === 'thiluyen') {
            automatic.scheduleTask('thiluyen', () => doThiLuyenTongMon(), automatic.INTERVAL_THI_LUYEN);
        } else if (taskId === 'phucloi') {
            automatic.scheduleTask('phucloi', () => doPhucLoiDuong(), automatic.INTERVAL_PHUC_LOI);
        } else if (taskId === 'khoangmach') {
            automatic.scheduleTask('khoangmach', () => khoangmach.doKhoangMach(), automatic.INTERVAL_KHOANG_MACH);
        } else if (taskId === 'bicanh') {
            automatic.scheduleTask('bicanh', () => bicanh.doBicanh(), automatic.INTERVAL_BICANH);
        }
    }
}

```

```

        automatic.scheduleTask('bicanh', async () => {
            await bicanh.doBiCanh();
        }, automatic.INTERVAL_BI_CANH);
    } else if (taskId === 'tienduyen') {
        automatic.scheduleTienDuyenCheck();
    } else if (taskId === 'dothach') {
        automatic.scheduleDoThach();
    }
}
}
}

// =====
// HẸN THI UNIFIED SETTINGS MODAL
// =====
function showQuestSettings(taskId) {
    // Tạo modal nếu chưa tồn tại
    let modal = document.getElementById('unified-settings-modal');
    if (!modal) {
        modal = createUnifiedSettingsModal();
        document.body.appendChild(modal);
    }

    // Chuyển đến tab tương ứng (mặc định tab Chung)
    switchSettingsTab(taskId || 'general');

    // Hiển thị modal
    modal.style.display = 'flex';
}

// =====
// HẸN THI HƯỚNG DẪN SỬ DỤNG
// =====
function showGuideModal() {
    let modal = document.getElementById('guide-modal');
    if (!modal) {
        modal = document.createElement('div');
        modal.id = 'guide-modal';
        modal.style.cssText = 'position:fixed;top:0;left:0;width:100%;height:100%;background:rgba(0,0,0,0.7);display:flex;align-items:center;justify-content:center;z-index:100000;';
        modal.innerHTML = `
            <div style="background:rgba(26, 27, 46, 0.7);backdrop-filter:blur(12px);-webkit-backdrop-filter:blur(12px);border-radius:12px;padding:20px 24px;max-width:520px;width:90%;max-height:80vh;overflow-y:auto;color:#c0caf5;font-size:13px;line-height:1.7;box-shadow:0 8px 32px rgba(0,0,0,0.5);border:1px solid rgba(255,255,255,0.1);">
                <div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:14px;">
                    <h3 style="margin:0;color:#7aa2f7;font-size:16px;">Hướng Dẫn Sử Dụng</h3>
                    <button id="guide-close-btn" style="background:none;border:none;color:#888;font-size:20px;cursor:pointer;padding:0 4px;">&times;</button>
                </div>
                <div style="border-top:1px solid rgba(255,255,255,0.1);padding-top:12px;">
                    <p><strong style="color:#bb9af7;">Bắt Đầu / Dừng Lại:</strong> Chạy tự động tắt công việc.</p>
                    <p><strong style="color:#bb9af7;">Bật/Tắt chế độ tự động chạy khi load trang.</p>
                    <p><strong style="color:#bb9af7;">Nút làm mới:</strong> Tải lại thông tin xu và tiến độ nhiệm vụ.</p>
                    <p><strong style="color:#bb9af7;">Cài Đặt:</strong> Cấu hình chi tiết cho từng nhiệm vụ.</p>
                    <p><strong style="color:#bb9af7;">Hộp Thụ:</strong> Nhập mã CODE để nhận thưởng.</p>
                    <hr style="border-color:rgba(255,255,255,0.1);margin:10px 0;">
                    <p><strong style="color:#9ece6a;">Danh sách nhiệm vụ:</strong></p>
                    <ul style="padding-left:18px;margin:4px 0;">
                        <li>Nhấn nút hành động (Đánh, Nhận, Khúc,...) để chạy thử công.</li>
                        <li>Các nút phụ (□□□□□) thực hiện hành động bổ sung.</li>
                        <li>Bấm vào icon trên mỗi task để bật/tắt tự động chạy mỗi task.</li>
                        <li>Thời gian đếm ngược hiển thị khi task đang chờ lượt tiếp.</li>
                    </ul>
                    <hr style="border-color:rgba(255,255,255,0.1);margin:10px 0;">
                    <p><strong style="color:#f7768e;">Lưu ý:</strong> Đảm bảo đã đăng nhập trước khi sử dụng.</p>
                </div>
            </div>
        `;
        modal.addEventListener('click', (e) => {
            if (e.target === modal || e.target.id === 'guide-close-btn') modal.style.display = 'none';
        });
        document.body.appendChild(modal);
    } else {
        modal.style.display = 'flex';
    }
}

// =====
// TẠO UNIFIED SETTINGS MODAL
// =====
function createUnifiedSettingsModal() {
    const modal = document.createElement('div');
    modal.id = 'unified-settings-modal';
    modal.className = 'settings-modal';

    const modalContent = document.createElement('div');
    modalContent.className = 'settings-modal-content';

    // Header
    const header = document.createElement('div');
    header.className = 'settings-modal-header';
    header.innerHTML = `
        <h2>Cài đặt nhiệm vụ</h2>
        <button class="settings-close-btn" onclick="document.getElementById('unified-settings-modal').style.display='none'">&times;</button>
    `;

    // Tabs container (horizontal scrolling)
    const tabsContainer = document.createElement('div');
    tabsContainer.className = 'settings-tabs-container';

    // Tab Chung (General) đầu tiên
    const generalTab = document.createElement('button');
    generalTab.className = 'settings-tab';
    generalTab.setAttribute('data-task', 'general');
    generalTab.innerHTML = `<i class="fas fa-sliders-h"></i> Chung`;
    generalTab.onclick = () => switchSettingsTab('general');
    tabsContainer.appendChild(generalTab);

    // Lấy các quest có settings
    const questsWithSettings = QUEST_CONFIG.filter(q => q.hasSettings);

    questsWithSettings.forEach(quest => {
        const tab = document.createElement('button');
        tab.className = 'settings-tab';
    });
}

```

```

        tab.setAttribute('data-task', quest.taskId);
        tab.innerHTML = `${quest.taskIcon} ${quest.taskName}`;
        tab.onclick = () => switchSettingsTab(quest.taskId);
        tabsContainer.appendChild(tab);
    });

    // Tab Log (luôn có cũ)
    const logTab = document.createElement('button');
    logTab.className = 'settings-tab';
    logTab.setAttribute('data-task', 'log');
    logTab.innerHTML = `

```

```

        <label>Truy cập nhanh:</label>
        <div style="display:flex;align-items:center;gap:8px;margin-bottom:6px;margin-top:6px">
            <span style="font-size:11px;color:#9ca3af">M</span>
            <label style="display:flex;align-items:center;gap:4px;font-size:11px;color:#e0e0e0;cursor:
pointer">
                <input type="radio" name="link-open-mode" id="link-mode-new" value="new" style="cursor:
pointer"> Tab mới
            </label>
            <label style="display:flex;align-items:center;gap:4px;font-size:11px;color:#e0e0e0;cursor:
pointer">
                <input type="radio" name="link-open-mode" id="link-mode-current" value="current"
style="cursor:pointer"> Tab hiện tại
            </label>
        </div>
        <div style="display:flex;flex-wrap:wrap;gap:5px">
            <button class="general-link-btn" data-path="/cai-dat-tai-khoan">Tài khoản</button>
            <button class="general-link-btn" data-path="/bang-xep-hang-truyen-thua">BXH Truyền Thừa</
button>
            <button class="general-link-btn" data-path="/bang-xep-hang">BXH Tư Vĩ</button>
            <button class="general-link-btn" data-path="/bang-phu-hao-tinh-thach">BXH Phú Hào</button>
            <button class="general-link-btn" data-path="/bang-xep-hang-tong-mon">BXH Tổng Môn</
button>
            <button class="general-link-btn" data-path="/bxh-tien-duyen">BXH Tiên Duyên</button>
            <button class="general-link-btn" data-path="/tu-bao-cac">Tự Bảo Các</button>
            <button class="general-link-btn" data-path="/vip-hh3d">Xu HH3D</button>
            <button class="general-link-btn" data-path="/mini-games-hh3d">Mini Game</button>
            <button class="general-link-btn" data-path="/thien-dao-ban-thuong">Thiên Đạo</button>
            <button class="general-link-btn" data-path="/cap-nhat-he-thong-tu-luyen">Điểm Hệ Thống</
button>
            <button class="general-link-btn" data-path="/danh-sach-thanh-vien-tong-mon">Thành Viên
TM</button>
            <button class="general-link-btn" data-path="/do-kiep-dai">Độ Kiếp Đài</button>
            <button class="general-link-btn" data-path="/thong-bao-tu-chu-phu">Thông Báo</button>
        </div>
        <div class="settings-section" style="border-top: 1px solid rgba(255,255,255,0.08); padding-top:
15px; margin-top: 15px;">
            <h3>Sao lưu / Khôi phục cấu hình</h3>
            <div style="display:flex;gap:8px;margin-top:6px">
                <button id="settings-export-btn" class="settings-save-btn" style="background:rgba(59,130,
246,0.2);color:#3b82f6;border:1px solid rgba(59,130,246,0.3);flex:1;margin:0;padding:6px 12px;font-size:12px;">Xuat
Cấu Hình</button>
                <button id="settings-import-btn" class="settings-save-btn" style="background:rgba(16,185,
129,0.2);color:#10b981;border:1px solid rgba(16,185,129,0.3);flex:1;margin:0;padding:6px 12px;font-size:12px;">Nhập
Cấu Hình</button>
            </div>
        </div>
    `;

    case 'hoangvuc':
        return `
        <div class="settings-section">
            <h3>Cài đặt Hoàng Vực</h3>
            <div class="settings-option">
                <label class="settings-checkbox-label">
                    <input type="checkbox" id="hoangvuc-maximize-damage"
                    ${localStorage.getItem('hoangvucMaximizeDamage') === 'true' ? 'checked' : ''}>
                    <span>Tăng gấp đôi sát thương (buff ngũ hành)</span>
                </label>
                <p class="settings-description">Tự động chọn ngũ hành để tăng 15% sát thương</p>
            </div>
        </div>
    `;

    case 'khoangmach': {
        const _savedRewardMode = localStorage.getItem('khoangmach_reward_mode') || 'any';
        const _savedRewardTime = localStorage.getItem('khoangmach_reward_time') || 'max';
        return `
        <div class="settings-section">
            <h3>Cài đặt Khoáng Mạch</h3>

            <div class="settings-option" style="display:flex;align-items:center;gap:6px;flex-wrap:wrap">
                <label for="khoangmach-mine-select" style="white-space:nowrap">Chọn Khoáng Mạch:</label>
                <select id="khoangmach-mine-select" class="settings-select" style="flex:1;min-width:120px">
                    <option value="">Đang tải danh sách...</option>
                </select>
                <button id="khoangmach-reload-mines-btn" title="Tải lại danh sách" style="padding:3px 8px;
font-size:11px;border-radius:4px;border:1px solid rgba(255,255,255,0.15);background:rgba(255,255,255,0.07);color:
#d0d8f0;cursor:pointer;white-space:nowrap">Load</button>
            </div>

            <div class="settings-option">
                <label for="khoangmach-reward-mode">Chế độ Nhận Thưởng:</label>
                <select id="khoangmach-reward-mode" class="settings-select">
                    <option value="110" ${_savedRewardMode === '110' ? 'selected' : ''}>110%</option>
                    <option value="100" ${_savedRewardMode === '100' ? 'selected' : ''}>100%</option>
                    <option value="20" ${_savedRewardMode === '20' ? 'selected' : ''}>20%</option>
                    <option value="any" ${_savedRewardMode === 'any' ? 'selected' : ''}>Bỏ qua</option>
                </select>
            </div>

            <div class="settings-option">
                <label for="khoangmach-reward-time">Nhận thưởng khi thời gian đạt:</label>
                <select id="khoangmach-reward-time" class="settings-select">
                    <option value="max" ${_savedRewardTime === 'max' ? 'selected' : ''}>Đạt tối đa</option>
                    <option value="20" ${_savedRewardTime === '20' ? 'selected' : ''}>20 phút</option>
                    <option value="10" ${_savedRewardTime === '10' ? 'selected' : ''}>10 phút</option>
                    <option value="4" ${_savedRewardTime === '4' ? 'selected' : ''}>4 phút</option>
                    <option value="2" ${_savedRewardTime === '2' ? 'selected' : ''}>2 phút</option>
                </select>
            </div>

            <div class="settings-option">
                <label class="settings-checkbox-label">
                    <input type="checkbox" id="khoangmach-auto-takeover"
                    ${localStorage.getItem('khoangmach_auto_takeover') === 'true' ? 'checked' : ''}>
                    <span>Tự động đoạt mine khi chưa buff</span>
                </label>
            </div>

            <div class="settings-option">
                <label class="settings-checkbox-label">
                    <input type="checkbox" id="khoangmach-auto-takeover-rotation"
                    ${localStorage.getItem('khoangmach_auto_takeover_rotation') === 'true' ? 'checked' : ''}>
                    <span>Tự động xoay mine khi đoạt mine</span>
                </label>
            </div>
        `;
    }
}
```

```
'')>
    <span>Tự động đoạt m[] khi có th[] (đ[]o key)</span>
  </label>
</div>

<div class="settings-option">
  <label class="settings-checkbox-label">
    <input type="checkbox" id="khoangmach-leave-mine"
      ${localStorage.getItem('khoangmach_leave_mine_to_claim_reward_' + accountId) ===
'true' ? 'checked' : ''}>
    <span>Rời m[] đ[] nhận thu[]ng (cao t[]ng đ[]o key)</span>
  </label>
</div>

<div class="settings-option">
  <label class="settings-checkbox-label">
    <input type="checkbox" id="khoangmach-use-buff"
      ${localStorage.getItem('khoangmach_use_buff') === 'true' ? 'checked' : ''}>
    <span>Tự động mua Linh Quang Phù</span>
  </label>
</div>

<div class="settings-option">
  <label class="settings-checkbox-label">
    <input type="checkbox" id="khoangmach-outer-notification"
      ${localStorage.getItem('khoangmach_outer_notification') === 'true' ? 'checked' : ''}>
    <span>Thông báo ngoại tổng vào khoảng</span>
  </label>
</div>

<div class="settings-option">
  <label class="settings-checkbox-label">
    <input type="checkbox" id="khoangmach-fast-attack"
      ${localStorage.getItem('khoangmach_fast_attack') === 'true' ? 'checked' : ''}>
    <span>B[] qua thời gian chờ khi t[]n công</span>
  </label>
</div>

<div class="settings-option">
  <label for="khoangmach-check-interval" title="Kho[]ng thời gian (phút) đ[] ki[]m tra và thực hiện
các hành động liên quan đ[]n Khoáng Mạch.">Thời gian ki[]m tra khoáng (phút):</label>
  <input type="number" id="khoangmach-check-interval" class="settings-input-number"
    value="${localStorage.getItem('khoangmach_check_interval') || '5'}" min="1" max="60"
    style="width:60px">
  </div>
</div>
`;
}

case 'luyenDan': {
  const minStars = localStorage.getItem('luyenDanMinStars') || '4';
  const targetTier = localStorage.getItem('luyenDanTargetTier') || 'auto';
  const autoDecompose = localStorage.getItem('luyenDanAutoDecompose') === 'true';
  const autoTune = localStorage.getItem('luyenDanAutoTune') === 'true';
  const autoUse = localStorage.getItem('luyenDanAutoUse') === 'true';
  const autoStart = localStorage.getItem('luyenDanAutoStart') === 'true';

  // C[]u hình Đan Đ[]ng
  const autoInvite = localStorage.getItem('luyenDanAutoInvite') === 'true';
  const waitSeconds = localStorage.getItem('luyenDanWaitInviteSeconds') || '60';
  const autoAccept = localStorage.getItem('luyenDanAutoAcceptInvite') === 'true';
  const acceptAll = localStorage.getItem('luyenDanAcceptAllInvites') === 'true';
  const autoLeave = localStorage.getItem('luyenDanAutoLeave') === 'true';

  return `
<div class="settings-section">
  <h3>Cài đặt Luyện Đan</h3>

  <div class="settings-option">
    <label for="luyendan-target-tier">Loại đan sẽ luyện:</label>
    <select id="luyendan-target-tier" class="settings-select" style="width: 100%; margin-top: 6px;">
      <option value="auto" ${targetTier === 'auto' ? 'selected' : ''}>Tự động (Ưu tiên ph[]m cao
nh[]t)</option>
      <option value="cuc" ${targetTier === 'cuc' ? 'selected' : ''}>Cực Ph[]m Đan</option>
      <option value="thuong" ${targetTier === 'thuong' ? 'selected' : ''}>Thượng Ph[]m Đan</option>
      <option value="trung" ${targetTier === 'trung' ? 'selected' : ''}>Trung Ph[]m Đan</option>
      <option value="ha" ${targetTier === 'ha' ? 'selected' : ''}>Hạ Ph[]m Đan</option>
    </select>
    <p class="settings-description">Chọn ph[]m đan cụ th[] mu[]n luyện hoặc đ[] tự động.</p>
  </div>

  <div class="settings-option">
    <label for="luyendan-min-stars">Mức sao t[]i thi[]u đ[] giữ/s[] dụng (Sao):</label>
    <select id="luyendan-min-stars" class="settings-select" style="width: 100%; margin-top: 6px;">
      <option value="1" ${minStars === '1' ? 'selected' : ''}>[] 1 Sao tr[] lên</option>
      <option value="2" ${minStars === '2' ? 'selected' : ''}>[] 2 Sao tr[] lên</option>
      <option value="3" ${minStars === '3' ? 'selected' : ''}>[] 3 Sao tr[] lên</option>
      <option value="4" ${minStars === '4' ? 'selected' : ''}>[] 4 Sao tr[] lên (Mặc định)</
option>
      <option value="5" ${minStars === '5' ? 'selected' : ''}>[] 5 Sao tr[] lên (Phân gi[]i c[]
4*)</option>
    </select>
    <p class="settings-description">Mức sao làm m[]c đ[] phân biệt đan được ph[]m ch[]t t[]t hay kém.</p>
  </div>

  <div class="settings-option">
    <label class="settings-checkbox-label">
      <input type="checkbox" id="luyendan-auto-start" ${autoStart ? 'checked' : ''}>
      <span>Tự động Khai lò khi lò tr[]ng</span>
    </label>
    <p class="settings-description">N[]u t[]t, tool sẽ không tự dùng nguyên liệu đ[] khai lò mới (
thuận tiện cho việc đi làm Đan Đ[]ng).</p>
  </div>

  <div class="settings-option">
    <label class="settings-checkbox-label">
      <input type="checkbox" id="luyendan-auto-use" ${autoUse ? 'checked' : ''}>
      <span>Tự động s[] dụng đan ph[]m ch[]t cao</span>
    </label>
    <p class="settings-description">N[]u bật, đan đạt mức sao [] trên tr[] lên sẽ tự động được s[] dụng.
</p>
  </div>

  <div class="settings-option">
```

```

        <label class="settings-checkbox-label">
            <input type="checkbox" id="luyendan-auto-decompose" ${autoDecompose ? 'checked' : ''}>
            <span>Tự động phân gíđi đán phám chát kém</span>
        </label>
        <p class="settings-description">Nhu bát, đán có số sao thấp hơn mức sao đ trên sẽ tự động bị
phân gíđi đá thu nguyên liệu.</p>
    </div>

    <div class="settings-option">
        <label class="settings-checkbox-label">
            <input type="checkbox" id="luyendan-auto-tune" ${autoTune ? 'checked' : ''}>
            <span>Tự động đđiu hoà (giữ độ đnh)</span>
        </label>
        <p class="settings-description">
            Tự động bđiu Hoà khi độ đnh xuung thấp (<= 68%).<br>
            <i>* Lưu ý: Khi làm Đán Đđng (acc phụ) cũng sẽ tự động đđiu hoà giúp Đán Chá nhu bát tùy
chọn này, ngược lại nhu tđiđ thì cá Đán Đđng cũng tđiđ tự đđiu hoà.</i>
        </p>
    </div>
</div>

<div class="settings-section" style="border-top: 1px solid rgba(255,255,255,0.08); padding-top: 15px;
margin-top: 15px;">
    <h3>Cài đặt Đán Đđng (Hỗ trợ Luyện Đán)</h3>

    <div class="settings-option">
        <label class="settings-checkbox-label">
            <input type="checkbox" id="luyendan-auto-invite" ${autoInvite ? 'checked' : ''}>
            <span>Tự động mời Đán Đđng khi lò trởng</span>
        </label>
        <p class="settings-description">Tự động gửi lời mời tới bạn bè được chọn trước khi tđiđn hành
Khai lò.</p>
    </div>

    <div class="settings-option" id="luyendan-wait-time-option" style="display: ${autoInvite ? 'block' :
'none'};">
        <label for="luyendan-wait-seconds">Thời gian tđiđ đa chờ Đán Đđng vào phòng (giấy):</label>
        <input type="number" id="luyendan-wait-seconds" class="settings-input-number"
            value="${waitSeconds}" min="10" max="300" style="width:70px; margin-top:4px;">
        <p class="settings-description">Quá thời gian này mà Đán Đđng chưa vào đđ, tool sẽ tự động Khai
lò đán.</p>
    </div>

    <div class="settings-option">
        <label class="settings-checkbox-label">
            <input type="checkbox" id="luyendan-auto-accept" ${autoAccept ? 'checked' : ''}>
            <span>Tự động nhận làm Đán Đđng</span>
        </label>
        <p class="settings-description">Tự động chấp nhận khi có Đán Chá gửi lời mời hỗ trợ luyện đán.</p>
    </div>

    <div class="settings-option" id="luyendan-accept-all-option" style="display: ${autoAccept ? 'block'
: 'none'};">
        <label class="settings-checkbox-label">
            <input type="checkbox" id="luyendan-accept-all" ${acceptAll ? 'checked' : ''}>
            <span>Nhận lời mời từ bát kỳ ai</span>
        </label>
        <p class="settings-description">Nhu tđiđ, chấp tự động nhận lời mời từ những người được tích chọn
trong danh sách bạn bè phía dưới.</p>
    </div>

    <div class="settings-option">
        <label class="settings-checkbox-label">
            <input type="checkbox" id="luyendan-auto-leave" ${autoLeave ? 'checked' : ''}>
            <span>Tự động rời Đán Đđng sau 5 phút</span>
        </label>
        <p class="settings-description">Sau khi hỗ trợ Đđiu Hàa xong (hết 5 phút đđu) hoặc khi lò nh,
tool sẽ tự rời vị trí Đán Đđng.</p>
    </div>

    <div class="settings-option" id="luyendan-friends-list-section" style="margin-top:12px; display: ${
autoInvite || autoAccept} ? 'block' : 'none'};">
        <label style="font-weight:bold; color:#d0d8f0;">Chọn bạn bè hỗ trợ (Mời / Nhận lời mời):</label>
        <div style="margin-top:6px; display:flex; gap:6px;">
            <input type="text" id="luyendan-friend-search" class="settings-input" placeholder="Tìm tên
bạn bè..." style="flex:1; padding:4px 8px; font-size:11px; background:rgba(0,0,0,0.2); border:1px solid rgba(255,255,
255,0.1); border-radius:4px; color:#fff;">
        </div>
        <div id="luyendan-friends-container" style="max-height:160px; overflow-y:auto; background:rgba(
0,0,0,0.25); border:1px solid rgba(255,255,255,0.08); border-radius:4px; margin-top:6px; padding:6px; display:flex;
flex-direction:column; gap:4px;">
            <p style="font-size:11px; color:#888; text-align:center; padding:10px 0; margin:0;">Đang
tđiđ danh sách bạn bè...</p>
        </div>
    </div>
</div>
</div>
`;
}

default:
    return `<div class="settings-section"><p>Không có cài đặt cho nhiệm vụ này</p></div>`;

case 'log':
    return
    <div class="settings-section" style="padding:8px">
        <div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:8px">
            <h3 style="margin:0">Lich Số Log</h3>
            <div style="display:flex;gap:6px">
                <select id="hh3d-log-filter" style="font-size:11px;padding:2px 6px;border-radius:4px;border:
1px solid rgba(255,255,255,0.2);background:rgba(255,255,255,0.07);color:#d0d8f0">
                    <option value="all">Tđiđ cá</option>
                    <option value="success">Số Success</option>
                    <option value="warn">⚠ Warn</option>
                    <option value="error">❌ Error</option>
                    <option value="info">ℹ Info</option>
                    <option value="debug">🐛 Debug</option>
                </select>
                <button id="hh3d-log-clear" style="font-size:11px;padding:2px 8px;border-radius:4px;border:
1px solid rgba(239,68,68,0.4);background:rgba(239,68,68,0.1);color:#f87171;cursor:pointer">Xóa</button>
            </div>
        </div>
        <div id="hh3d-log-list" style="height:380px;overflow-y:auto;background:rgba(0,0,0,0.3);border-
radius:4px;border:1px solid rgba(255,255,255,0.08)"></div>
    </div>
`;
```

```

    }
}

// =====
// BIND EVENTS CHO SETTINGS
// =====
function bindSettingsEventsForTask(taskId) {
    switch (taskId) {
        case 'general': {
            const resetBtn = document.getElementById('general-reset-tasks-btn');
            if (resetBtn) {
                resetBtn.addEventListener('click', () => {
                    const accId = localStorage.getItem('hh3d_account_id') || accountId;
                    const count = taskTracker.resetAllTasks(accId);
                    showNotification('Đã reset ${count} nhiệm vụ', 'success', 2000);
                    updateAllQuestButtons();
                });
            }
            // Restore saved open-mode preference
            const _linkModeKey = 'general_link_open_mode';
            const _savedMode = localStorage.getItem(_linkModeKey) || 'new';
            const _modeNewRadio = document.getElementById('link-mode-new');
            const _modeCurrentRadio = document.getElementById('link-mode-current');
            if (_modeNewRadio) _modeNewRadio.checked = _savedMode === 'new';
            if (_modeCurrentRadio) _modeCurrentRadio.checked = _savedMode === 'current';
            document.querySelectorAll('input[name="link-open-mode"]').forEach((radio) => {
                radio.addEventListener('change', () => localStorage.setItem(_linkModeKey, radio.value));
            });

            document.querySelectorAll('.general-link-btn').forEach((btn) => {
                btn.addEventListener('click', () => {
                    const path = btn.getAttribute('data-path');
                    if (!path) return;
                    const url = weblink.replace(/\/$/, '') + path;
                    const mode = localStorage.getItem(_linkModeKey) || 'new';
                    if (mode === 'current') window.location.href = url;
                    else window.open(url, '_blank');
                });
            });

            const exportBtn = document.getElementById('settings-export-btn');
            if (exportBtn) {
                exportBtn.addEventListener('click', exportSettings);
            }
            const importBtn = document.getElementById('settings-import-btn');
            if (importBtn) {
                importBtn.addEventListener('click', importSettings);
            }
            break;
        }

        case 'hoangvuc':
            // No special events needed, just checkbox
            break;

        case 'khoangmach': {
            // Async load danh sách mìn
            const km_sel = document.getElementById('khoangmach-mine-select');
            const km_reloadBtn = document.getElementById('khoangmach-reload-mines-btn');
            const km_accountId = localStorage.getItem('hh3d_account_id') || '';

            const km_populateMines = (force = false) => {
                if (km_sel) km_sel.innerHTML = '<option value="">Đang tải...</option>';
                if (km_reloadBtn) { km_reloadBtn.disabled = true; km_reloadBtn.textContent = ''; }
                khoangmach.getAllMines(force).then(({ optionsHtml, minesData }) => {
                    const savedMineSetting = localStorage.getItem(`khoangmach_selected_mine_${km_accountId}`);
                    let savedMineId = '';
                    try { savedMineId = savedMineSetting ? JSON.parse(savedMineSetting).id : ''; } catch (e) { }
                    km_sel.innerHTML = '<option value="">Chọn mìn --</option>' + optionsHtml;
                    if (savedMineId) {
                        const opt = km_sel.querySelector(`option[value="${savedMineId}"]`);
                        if (opt) opt.selected = true;
                    }
                    minesData.forEach((mine) => {
                        const opt = km_sel.querySelector(`option[value="${mine.id}"]`);
                        if (opt) opt.dataset.mine = JSON.stringify({ id: mine.id, type: mine.type });
                    });
                    if (force) showNotification('Đã tải ${minesData.length} mìn', 'success', 1500);
                }).catch(() => {
                    km_sel.innerHTML = '<option value="">Không tải được danh sách mìn</option>';
                }).finally(() => {
                    if (km_reloadBtn) { km_reloadBtn.disabled = false; km_reloadBtn.textContent = 'Load'; }
                });
            };

            km_populateMines(false);

            if (km_reloadBtn) {
                km_reloadBtn.addEventListener('click', () => km_populateMines(true));
            }
            break;
        }

        case 'luyenDan': {
            const autoInviteCheck = document.getElementById('luyendan-auto-invite');
            const autoAcceptCheck = document.getElementById('luyendan-auto-accept');
            const waitOption = document.getElementById('luyendan-wait-time-option');
            const acceptAllOption = document.getElementById('luyendan-accept-all-option');
            const friendsListSection = document.getElementById('luyendan-friends-list-section');

            const updateFriendsVisibility = () => {
                const inviteVal = autoInviteCheck?.checked || false;
                const acceptVal = autoAcceptCheck?.checked || false;

                if (waitOption) waitOption.style.display = inviteVal ? 'block' : 'none';
                if (acceptAllOption) acceptAllOption.style.display = acceptVal ? 'block' : 'none';
                if (friendsListSection) friendsListSection.style.display = (inviteVal || acceptVal) ? 'block' :
'none';
            };

            if (autoInviteCheck) autoInviteCheck.addEventListener('change', updateFriendsVisibility);
            if (autoAcceptCheck) autoAcceptCheck.addEventListener('change', updateFriendsVisibility);

            // Tải danh sách bạn bè bất đồng bộ

```

```

const friendsContainer = document.getElementById('luyendan-friends-container');
const searchInput = document.getElementById('luyendan-friend-search');

if (friendsContainer) {
    luyendan.sendLdRequest("/friends", "GET").then(friendsRes => {
        const friends = friendsRes?.data?.friends || [];
        if (!friends.length) {
            friendsContainer.innerHTML = '<p style="font-size:11px; color:#888; text-align:center; padding:10px 0; margin:0;">❏ Không tìm thấy đạo hữu nào</p>';
            return;
        }

        const savedIds = (localStorage.getItem('luyenDanSelectedFriendIds') || '').split(',').filter(
Boolean);

        const renderFriends = (filterText = '') => {
            const normalizedFilter = filterText.toLowerCase().trim();
            const filtered = friends.filter(f => !normalizedFilter || (f.name || '').toLowerCase().
includes(normalizedFilter));

            if (!filtered.length) {
                friendsContainer.innerHTML = '<p style="font-size:11px; color:#888; text-align:center; padding:10px 0; margin:0;">❏ Không khớp tên bạn bè</p>';
                return;
            }

            friendsContainer.innerHTML = filtered.map(f => {
                const uid = String(f.userId != null ? f.userId : f.id);
                const isChecked = savedIds.includes(uid);
                const lvlName = f.rank_level_name || (f.rank_level ? `Bậc ${f.rank_level}` : '');
                return `
                <label style="display:flex; align-items:center; gap:6px; font-size:11px; padding:
2px 4px; cursor:pointer;">
                <input type="checkbox" class="luyendan-friend-checkbox" value="${uid}" ${
isChecked ? 'checked' : ''} style="margin:0;">
                <span style="color:#d0d8f0;">${f.name || `Đạo hữu #${uid}`}</span>
                ${lvlName ? `<span style="color:#10b981; font-size:9px;">{${lvlName}</span>` : ''}
                </label>
                `;
            }).join('');
        };

        renderFriends();

        if (searchInput) {
            searchInput.addEventListener('input', (e) => {
                renderFriends(e.target.value);
            });
        }
    }).catch(err => {
        console.error('Error fetching friends list:', err);
        friendsContainer.innerHTML = '<p style="font-size:11px; color:#ef4444; text-align:center; padding:10px 0; margin:0;">⚠️ Lỗi tải danh sách bạn bè</p>';
    });
}

break;

case 'bicanh':
case 'general':
    // No special events
    break;

case 'log': {
    const logList = document.getElementById('hh3d-log-list');
    const logFilter = document.getElementById('hh3d-log-filter');
    const logClear = document.getElementById('hh3d-log-clear');

    const renderLogs = (filter = 'all') => {
        if (!logList) return;
        logList.innerHTML = '';
        const entries = (window.hh3dLogBuffer || []).filter(e => filter === 'all' || e.type === filter);
        // Oldest first → newest at bottom
        for (let i = 0; i < entries.length; i++) {
            _hh3dRenderLogLine(logList, entries[i], false);
        }
        logList.scrollTop = logList.scrollHeight;
    };

    renderLogs('all');

    if (logFilter) logFilter.addEventListener('change', () => renderLogs(logFilter.value));
    if (logClear) logClear.addEventListener('click', () => {
        window.hh3dLogBuffer = [];
        renderLogs('all');
    });
    break;
}

default:
    break;
}
}

// =====
// XUẤT CÀI ĐẶT
// =====
function exportSettings() {
    const exactKeys = [
        'selfSchedule_h',
        'selfSchedule_m',
        'generalVipMode',
        'hoangvucMaximizeDamage',
        'khoangmach_use_buff',
        'khoangmach_fast_attack',
        'khoangmach_auto_takeover',
        'khoangmach_auto_takeover_rotation',
        'khoangmach_outer_notification',
        'khoangmach_reward_mode',
        'khoangmach_reward_time',
        'khoangmach_check_interval',
        'luyenDanMinStars',
        'luyenDanTargetTier',
        'luyenDanAutoStart',
        'luyenDanAutoUse',
    ];
}

```

```

        'luyenDanAutoDecompose',
        'luyenDanAutoTune',
        'luyenDanAutoInvite',
        'luyenDanWaitInviteSeconds',
        'luyenDanAutoAcceptInvite',
        'luyenDanAcceptAllInvites',
        'luyenDanAutoLeave',
        'luyenDanSelectedFriendIds',
        'reservebicanhAttacks',
        'bicanhSocketEnabled',
        'luanVoChallengeMode',
        'dice-roll-choice',
        'tienduyen-choice'
    ];

    const prefixKeys = [
        'khoangmach_selected_mine_',
        'khoangmach_leave_mine_to_claim_reward_',
        'luanVoTargetUserId_',
        'luyenDanLastProgress_',
        'reserve',
        'SocketEnabled',
        '-choice'
    ];

    const exported = {};
    for (let i = 0; i < localStorage.length; i++) {
        const key = localStorage.key(i);
        if (!key) continue;

        const isExact = exactKeys.includes(key);
        const isPrefix = prefixKeys.some(prefix => key.startsWith(prefix) || key.endsWith(prefix));

        if (isExact || isPrefix) {
            exported[key] = localStorage.getItem(key);
        }
    }

    const jsonStr = JSON.stringify(exported, null, 2);

    Swal.fire({
        title: '📄 Xuýt Cúu Hình',
        html: `
            <p style="font-size:13px;color:#d0d8f0;margin-bottom:10px">Bạn có thđ copy mã cúu hình bên dưới hoặc
bđm nút tđi file cúu hình.</p>
            <textarea id="swal-export-json" readonly style="width:100%;height:150px;font-family:monospace;font-size:
11px;background:#1e1e2e;color:#a6adc8;border:1px solid #313244;border-radius:4px;padding:8px;box-sizing:border-box;
resize:none">${jsonStr}</textarea>
        `,
        showCancelButton: true,
        confirmButtonText: '📄 Sao chép',
        cancelButtonText: '📄 Tđi File .json',
        customClass: {
            confirmButton: 'swal2-confirm swal2-styled',
            cancelButton: 'swal2-deny swal2-styled'
        },
        preConfirm: () => {
            const textarea = document.getElementById('swal-export-json');
            if (textarea) {
                textarea.select();
                document.execCommand('copy');
                showNotification('📄 Đđ sao chép cúu hình vào clipboard!', 'success', 2000);
            }
        }
    }).then((result) => {
        if (result.dismiss === Swal.DismissReason.cancel) {
            const blob = new Blob([jsonStr], { type: 'application/json' });
            const url = URL.createObjectURL(blob);
            const a = document.createElement('a');
            a.href = url;
            a.download = `hh3d_settings_${Date.now()}.json`;
            document.body.appendChild(a);
            a.click();
            document.body.removeChild(a);
            URL.revokeObjectURL(url);
            showNotification('📄 Đđ tđi file cúu hình thành công!', 'success', 2000);
        }
    });
}

// =====
// NHẬP CÀI ĐẶT
// =====
function importSettings() {
    Swal.fire({
        title: '📄 Nhập Cúu Hình',
        html: `
            <p style="font-size:13px;color:#d0d8f0;margin-bottom:10px">Đán mã cúu hình JSON vào ô dưới đây HOẶC
chọn file cúu hình (.json) từ máy tính cúa bạn:</p>
            <input type="file" id="swal-import-file" accept=".json" style="width:100%;margin-bottom:10px;color:
#d0d8f0;font-size:12px;">
            <textarea id="swal-import-json" placeholder="Đán chúđi JSON cúu hình vào đây..." style="width:100%;
height:150px;font-family:monospace;font-size:11px;background:#1e1e2e;color:#a6adc8;border:1px solid #313244;border-
radius:4px;padding:8px;box-sizing:border-box;resize:none"></textarea>
        `,
        showCancelButton: true,
        confirmButtonText: '📄 Xác nhận nhập',
        cancelButtonText: 'Hđy',
        didOpen: () => {
            const fileInput = document.getElementById('swal-import-file');
            const textarea = document.getElementById('swal-import-json');
            if (fileInput && textarea) {
                fileInput.addEventListener('change', (e) => {
                    const file = e.target.files[0];
                    if (file) {
                        const reader = new FileReader();
                        reader.onload = (event) => {
                            textarea.value = event.target.result;
                        };
                        reader.readAsText(file);
                    }
                });
            }
        },
        preConfirm: () => {
            const jsonStr = document.getElementById('swal-import-json')?.value?.trim();

```

```

        if (!jsonStr) {
            Swal.showValidationMessage('Vui lòng dán mã c[u] hình hoặc chọn file JSON!');
            return false;
        }
        try {
            const settings = JSON.parse(jsonStr);
            if (typeof settings !== 'object' || settings === null) {
                throw new Error('Dữ liệu không phải là một Object hợp lệ');
            }
            return settings;
        } catch (err) {
            Swal.showValidationMessage('Dữ liệu JSON không hợp lệ: ' + err.message);
            return false;
        }
    }
    }).then((result) => {
        if (result.isConfirmed && result.value) {
            const settings = result.value;
            let count = 0;
            for (const [key, value] of Object.entries(settings)) {
                if (value !== null && value !== undefined) {
                    localStorage.setItem(key, String(value));
                    count++;
                }
            }
            Swal.fire({
                icon: 'success',
                title: 'Thành công',
                text: `Đã nhập thành công ${count} khóa c[u] hình! Đang t[i] lại trang...`,
                timer: 2000,
                showConfirmButton: false
            }).then(() => {
                window.location.reload();
            });
        }
    });
}

// =====
// LƯU T[i]T C[i] CÀI ĐẶT
// =====
function saveAllSettings() {
    // Xác định tab hiện tại
    const activeTab = document.querySelector('.settings-tab.active');
    if (!activeTab) return;

    const taskId = activeTab.getAttribute('data-task');
    const accountId = localStorage.getItem('hh3d_account_id') || '';
    let saved = false;

    try {
        switch (taskId) {
            case 'general': {
                const h = parseInt(document.getElementById('general-restart-hour')?.value ?? '0', 10) || 0;
                const m = parseInt(document.getElementById('general-restart-minute')?.value ?? '30', 10);
                const vipMode = document.getElementById('general-vip-mode')?.checked || false;
                localStorage.setItem('selfSchedule_h', String(h));
                localStorage.setItem('selfSchedule_m', String(m));
                localStorage.setItem('generalVipMode', String(vipMode));

                saved = true;
                break;
            }

            case 'hoangvuc': {
                const maximizeDamage = document.getElementById('hoangvuc-maximize-damage')?.checked || false;
                localStorage.setItem('hoangvucMaximizeDamage', maximizeDamage.toString());
                saved = true;
                break;
            }

            case 'khoangmach': {
                const km_useBuff = document.getElementById('khoangmach-use-buff')?.checked || false;
                const km_fastAttack = document.getElementById('khoangmach-fast-attack')?.checked || false;
                const km_autoTakeover = document.getElementById('khoangmach-auto-takeover')?.checked || false;
                const km_autoTakeoverRotation = document.getElementById('khoangmach-auto-takeover-rotation')?.checked || false;
                const km_outerNotification = document.getElementById('khoangmach-outer-notification')?.checked || false;
                const km_leaveMine = document.getElementById('khoangmach-leave-mine')?.checked || false;
                const km_rewardMode = document.getElementById('khoangmach-reward-mode')?.value || 'any';
                const km_rewardTime = document.getElementById('khoangmach-reward-time')?.value || 'max';
                const km_checkInterval = parseInt(document.getElementById('khoangmach-check-interval')?.value || '5', 10);

                // Lưu m[u] đã chọn (data-mine chứa JSON {id, type})
                const km_mineSelect = document.getElementById('khoangmach-mine-select');
                if (km_mineSelect && km_mineSelect.value) {
                    const mineData = km_mineSelect.options[km_mineSelect.selectedIndex].dataset?.mine;
                    if (mineData) localStorage.setItem('khoangmach_selected_mine_${accountId}', mineData);
                }

                localStorage.setItem('khoangmach_use_buff', String(km_useBuff));
                localStorage.setItem('khoangmach_fast_attack', String(km_fastAttack));
                localStorage.setItem('khoangmach_auto_takeover', String(km_autoTakeover));
                localStorage.setItem('khoangmach_auto_takeover_rotation', String(km_autoTakeoverRotation));
                localStorage.setItem('khoangmach_outer_notification', String(km_outerNotification));
                localStorage.setItem('khoangmach_leave_mine_to_claim_reward_${accountId}', String(km_leaveMine));
                localStorage.setItem('khoangmach_reward_mode', km_rewardMode);
                localStorage.setItem('khoangmach_reward_time', km_rewardTime);
                localStorage.setItem('khoangmach_check_interval', isNaN(km_checkInterval) || km_checkInterval < 1 ? '5' : String(km_checkInterval));
                saved = true;
                break;
            }

            case 'bicanh': {
                const reserveAttacks = parseInt(document.getElementById('bicanh-reserve-attacks')?.value || '0', 10);
                const socketEnabled = document.getElementById('bicanh-socket-enabled')?.checked || false;

                localStorage.setItem('reservebicanhAttacks', Math.max(0, Math.min(5, reserveAttacks)).toString());
                localStorage.setItem('bicanhSocketEnabled', socketEnabled ? '1' : '0');

                // Apply socket changes immediately
            }
        }
    }
}

```

```

        if (socketEnabled) {
            if (typeof bicanhhiente !== 'undefined' && bicanhhiente.startBossSocketListener) {
                bicanhhiente.startBossSocketListener();
            }
        } else {
            if (typeof bicanhhiente !== 'undefined' && bicanhhiente.stopBossSocketListener) {
                bicanhhiente.stopBossSocketListener();
            }
        }
        saved = true;
        break;
    }

    case 'luyenDan': {
        const minStars = document.getElementById('luyendan-min-stars')?.value || '4';
        const targetTier = document.getElementById('luyendan-target-tier')?.value || 'auto';
        const autoUse = document.getElementById('luyendan-auto-use')?.checked ?? false;
        const autoStart = document.getElementById('luyendan-auto-start')?.checked ?? false;
        const autoDecompose = document.getElementById('luyendan-auto-decompose')?.checked ?? false;
        const autoTune = document.getElementById('luyendan-auto-tune')?.checked ?? false;

        const autoInvite = document.getElementById('luyendan-auto-invite')?.checked ?? false;
        const waitSeconds = document.getElementById('luyendan-wait-seconds')?.value || '60';
        const autoAccept = document.getElementById('luyendan-auto-accept')?.checked ?? false;
        const acceptAll = document.getElementById('luyendan-accept-all')?.checked ?? false;
        const autoLeave = document.getElementById('luyendan-auto-leave')?.checked ?? false;

        const checkedCheckboxes = document.querySelectorAll('.luyendan-friend-checkbox:checked');
        const selectedIds = Array.from(checkedCheckboxes).map(cb => cb.value).join(',');

        localStorage.setItem('luyenDanMinStars', minStars);
        localStorage.setItem('luyenDanTargetTier', targetTier);
        localStorage.setItem('luyenDanAutoStart', String(autoStart));
        localStorage.setItem('luyenDanAutoUse', String(autoUse));
        localStorage.setItem('luyenDanAutoDecompose', String(autoDecompose));
        localStorage.setItem('luyenDanAutoTune', String(autoTune));

        localStorage.setItem('luyenDanAutoInvite', String(autoInvite));
        localStorage.setItem('luyenDanWaitInviteSeconds', waitSeconds);
        localStorage.setItem('luyenDanAutoAcceptInvite', String(autoAccept));
        localStorage.setItem('luyenDanAcceptAllInvites', String(acceptAll));
        localStorage.setItem('luyenDanAutoLeave', String(autoLeave));
        localStorage.setItem('luyenDanSelectedFriendIds', selectedIds);

        saved = true;
        break;
    }
}

if (saved) {
    showNotification('Đã lưu cài đặt!', 'success', 2000);
    // Close modal after short delay
    setTimeout(() => {
        document.getElementById('unified-settings-modal').style.display = 'none';
    }, 500);
}
} catch (error) {
    console.error('[Settings] Error saving:', error);
    showNotification('Lỗi khi lưu cài đặt', 'error');
}
}

// =====
// NHẬP MÃ THUẬNNG (PROMO CODE)
// =====

async function submitPromoCode(promoCode) {
    const logPrefix = '[Nhập mã]';

    if (!promoCode || promoCode.trim() === '') {
        showNotification(`${logPrefix} Mã CODE không hợp lệ`, 'warning');
        return;
    }

    try {
        // Lấy nonce từ trang linh thach
        const nonce = await getSecurityNonce(weburl + "linh-thach?t", "redeem_linh_thach");

        if (!nonce) {
            showNotification(`${logPrefix} Không thấy nonce`, 'error');
            return;
        }

        showNotification(`${logPrefix} Đang nhập mã: ${promoCode}...`, 'info');

        const response = await fetch(ajaxUrl, {
            credentials: "include",
            headers: {
                "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:143.0) Gecko/20100101 Firefox/143.0",
                "Accept": "*/*",
                "Accept-Language": "vi,en-US;q=0.5",
                "Content-Type": "application/x-www-form-urlencoded",
                "X-Requested-With": "XMLHttpRequest",
                "Sec-Fetch-Dest": "empty",
                "Sec-Fetch-Mode": "cors",
                "Sec-Fetch-Site": "same-origin",
                "Priority": "u=0"
            },
            body: `action=redeem_linh_thach&code=${encodeURIComponent(promoCode)}&nonce=${nonce}&lt_token=${securityToken}&hold_timestamp=${Math.floor(Date.now() / 1000)}`,
            method: "POST",
            mode: "cors"
        });

        const data = await response.json();

        if (data.success) {
            showNotification(`${logPrefix} ${data.data.message}`, 'success');
            // Lưu mã đã nhập vào localStorage
            localStorage.setItem('promo_code_${accountId}', promoCode);
            // Xóa input
            const input = document.getElementById('promo-code-input');
            if (input) input.value = '';
        } else if (data.data?.message === "Bạn đã đạt hạn mức nhập mã") {
            showNotification(`${logPrefix} ${data.data.message || JSON.stringify(data)}`, 'warn');
        }
    }
}

```

```

        localStorage.setItem('promo_code_${accountId}', promoCode);
    } else {
        showNotification(`${logPrefix} ❌ ${data.data?.message || data.message || "Không xác định"}`, 'error');
    }
} catch (error) {
    console.error(`${logPrefix} Lỗi:`, error);
    showNotification(`${logPrefix} ❌ Lỗi: ${error.message}`, 'error');
}
}

// =====
// VẤN ĐÁP
// =====

class VanDap {
    constructor(nonce) {
        this.nonce = nonce;
        this.ajaxUrl = ajaxUrl;
        this.QUESTION_DATA_URL = "https://raw.githubusercontent.com/Enormit/test/main/VanDap.json";
        this.taskTracker = taskTracker;
        this.questionDataCache = null;
    }

    // ❌ normalizeText riêng
    normalizeText(str) {
        return str
            .normalize("NFC") // chuẩn hóa Unicode
            .toLowerCase()
            .trim()
            .replace(/[.,\/#!%&^*~;:}=\\-_|~()?\s]/g, ' ');
    }

    tokenize(str) {
        if (typeof str !== "string") return [];
        return str
            .toLowerCase()
            .replace(/[.,\/#!%&^*~;:}=\\-_|~()?\s]/g, ' ')
            .trim()
            .split(/\s+/)
            .filter(x => x);
    }

    /**
     * Tải dữ liệu đáp án và lưu vào cache.
     */
    async loadAnswersFromHub() {
        const cacheKey = "hh3d_vandap_cache";
        const cacheTimeKey = "hh3d_vandap_cache_time";
        const ONE_HOUR = 60 * 60 * 1000; // 1 hour in ms

        try {
            const cachedData = localStorage.getItem(cacheKey);
            const cachedTime = localStorage.getItem(cacheTimeKey);
            const now = Date.now();

            if (cachedData && cachedTime && (now - parseInt(cachedTime, 10) < ONE_HOUR)) {
                try {
                    this.questionDataCache = JSON.parse(cachedData);
                    console.log("[Vấn Đáp] ✌ Đã tải dữ liệu đáp án từ LocalStorage Cache.");
                    return;
                } catch (err) {
                    console.warn("[Vấn Đáp] Lỗi parse JSON từ cache LocalStorage, tiến hành tải mới:", err);
                }
            }

            // ép tải mới bằng cách thêm timestamp
            const urlWithBypass = this.QUESTION_DATA_URL + "?t=" + now;
            const response = await fetch(urlWithBypass, { cache: "no-store" });
            if (!response.ok) {
                throw new Error(`HTTP error! Status: ${response.status}`);
            }
            const data = await response.json();
            this.questionDataCache = data;

            // Lưu lại cache
            try {
                localStorage.setItem(cacheKey, JSON.stringify(data));
                localStorage.setItem(cacheTimeKey, now.toString());
            } catch (storageErr) {
                console.warn("[Vấn Đáp] Không thể lưu cache vào LocalStorage:", storageErr);
            }

            console.log("[Vấn Đáp] ❌ Đã tải dữ liệu mới từ hub và lưu vào cache:", urlWithBypass);
        } catch (e) {
            showNotification('Lỗi khi tải đáp án. Vui lòng thử lại.', 'error');
            throw e;
        }
    }

    /**
     * Kiểm tra câu hỏi trong cache và gửi đáp án lên server.
     */
    /**
     * Tìm câu trả lời đúng cho một câu hỏi và gửi nó đi.
     */
    @param {object} question Đối tượng câu hỏi từ máy chủ.
    @param {object} headers Headers cần yêu cầu gửi đi.
    @returns {Promise<boolean>} True nếu câu trả lời được gửi thành công, ngược lại là false.
    */
    async checkAnswerAndSubmit(question, headers, securityToken) {

        const normalizedIncomingQuestion = this.normalizeText(question.question); // system question

        //check log câu hỏi từ hệ thống và câu hỏi từ hub
        console.log("Incoming Question (gửi):", question.question);
        console.log("Incoming Question (normalized):", normalizedIncomingQuestion);
        // console.log("Stored keys(normalized):", Object.keys(this.questionDataCache.questions).map(k => this.
        normalizeText(k)));

        let foundAnswer = null;
        let matchedQuestionKey = null;

        for (const storedQuestionKey in this.questionDataCache.questions) {
            const normalizedStoredQuestionKey = this.normalizeText(storedQuestionKey);
            if (normalizedStoredQuestionKey === normalizedIncomingQuestion) {
                matchedQuestionKey = storedQuestionKey;
            }
        }
    }

```

```

        foundAnswer = this.questionDataCache.questions[storedQuestionKey];
        break;
    }
}

if (!foundAnswer) {
    console.warn("[Vấn Đáp] ❌ Không tìm thấy câu hỏi trong cache:", question.question, "Normalized Question:", normalizedIncomingQuestion);
    showNotification(`<b>Vấn Đáp:</b> Không tìm thấy đáp án cho câu hỏi: <i>${question.question}</i>`,
    'error');
    return false;
}

// Tìm chỉ mục (Index) trong options
// Ưu tiên 1: Tìm chính xác (Exact Match)
let answerIndex = question.options.findIndex(option =>
    this.normalizeText(option) === this.normalizeText(foundAnswer));
// console.log("Answer index:", answerIndex, "Option chosen:", question.options[answerIndex]);

// Ưu tiên 2: Nếu không thấy, tìm theo điểm trùng từ (Similarity Score)
if (answerIndex === -1) {
    let maxScore = -1;
    let bestIdx = -1;
    const targetTokens = this.tokenize(foundAnswer);
    question.options.forEach((option, idx) => {
        const optTokens = this.tokenize(option);
        const intersection = optTokens.filter(token => targetTokens.includes(token));
        const score = intersection.length;

        if (score > maxScore) {
            maxScore = score;
            bestIdx = idx;
        }
    });

    if (bestIdx > -1 && maxScore > 0) {
        answerIndex = bestIdx;
        console.log(`[Vấn Đáp] ✅ Chọn option theo điểm cao nhất (${maxScore}): ${question.options[
bestIdx]}`);
    }
}

// Nếu vẫn không tìm thấy
if (answerIndex === -1) {
    console.warn("[Vấn Đáp] ❌ Không khớp option nào.");
    console.warn("Options (gốc):", question.options);
    console.warn("Options (normalized):", question.options.map(opt => this.normalizeText(opt)));
    console.warn("Đáp án từ GitHub:", foundAnswer);
    console.log("Đáp án từ GitHub (normalized):", this.normalizeText(foundAnswer));
    //console.log("Unicode từng ký tự đáp án:", [...foundAnswer].map(c => c.charCodeAt(0).toString(16)));
    showNotification(`[Vấn Đáp] Câu hỏi: <i>${question.question}</i> không có đáp án đúng trong server.`
    'error');
    return false;
}

const payloadSubmitAnswer = new URLSearchParams();
const actionSave = (typeof hData !== 'undefined' && hData?.act?.vdSave) ? hData.act.vdSave :
'save_quiz_result';
payloadSubmitAnswer.append('action', actionSave);
payloadSubmitAnswer.append('question_id', question.id);
payloadSubmitAnswer.append('answer', answerIndex);
payloadSubmitAnswer.append('security_token', securityToken);

//console.log("Submit payload:", { question_id: question.id, answer: answerIndex, security_token:
securityToken });

try {
    const responseSubmit = await fetch(this.ajaxUrl, {
        method: 'POST',
        headers: headers,
        body: payloadSubmitAnswer,
        credentials: 'include'
    });

    const dataSubmit = await responseSubmit.json();

    //console.log("Server response:", dataSubmit);

    if (dataSubmit.success) {
        return { success: true };
    } else {
        const msg = typeof dataSubmit.data === 'string' ? dataSubmit.data : (dataSubmit?.data?.message ||
dataSubmit?.message || "Server không trả về thông điệp gì");
        return { success: false, reason: "server_error", message: msg };
    }
} catch (error) {
    const msg = error?.message || JSON.stringify(error) || "Lỗi không xác định";
    return { success: false, reason: "exception", message: msg };
}
}

/**
 * Thực hiện toàn bộ quy trình vấn đáp.
 */
async doVanDap(nonce) {
    const securityToken = await getSecurityToken(weburl + 'van-dap-tong-mon?t');
    try {
        await this.loadAnswersFromHub();

        console.log('[HH3D Vấn Đáp] ▶️ Bắt đầu Vấn Đáp');
        const headers = {
            'Content-Type': 'application/x-www-form-urlencoded; charset=UTF-8',
            'X-Requested-With': 'XMLHttpRequest',
            'X-Wp-Nonce': nonce,
        };

        let correctCount = 0;
        let answeredThisSession = 0;
        const maxAttempts = 10;
        let currentAttempt = 0;
        let totalQuestions = 0;

        while (correctCount < 5 && currentAttempt < maxAttempts) {
            currentAttempt++;

```

```

const payloadLoadQuiz = new URLSearchParams();
const actionLoad = (typeof hData !== 'undefined' && hData?.act?.vdLoad) ? hData.act.vdLoad :
'load_quiz_data';

payloadLoadQuiz.append('action', actionLoad);
payloadLoadQuiz.append('security_token', securityToken);

const responseQuiz = await fetch(this.ajaxUrl, {
  method: 'POST',
  headers: headers,
  body: payloadLoadQuiz,
  credentials: 'include'
});

const dataQuiz = await responseQuiz.json();

if (!dataQuiz.success || !dataQuiz.data) {
  showNotification(`Vấn Đáp: ${dataQuiz.data || 'Lỗi khi lấy câu hỏi'}`, 'warn');
  return;
}

if (dataQuiz.data.completed) {
  showNotification('Đã hoàn thành vấn đáp hôm nay.', 'success');
  taskTracker.markTaskDone(accountId, 'diemdanh');
  return;
}

if (!dataQuiz.data.questions) {
  showNotification(`Vấn Đáp: Không có câu hỏi nào được tải`, 'warn');
  return;
}

const questions = dataQuiz.data.questions;
totalQuestions = questions.length;
correctCount = dataQuiz.data.correct_answers || 0;

// Bộ lọc câu hỏi bao hàm 2 trường hợp
const questionsToAnswer = questions.filter(q => {
  if ('is_correct' in q) {
    return String(q.is_correct) === "0"; // chỉ lấy câu chưa đúng
  }
  return true; // nếu chưa có is_correct thì lấy tất cả
});

if (questionsToAnswer.length === 0) {
  break;
}

let newAnswersFound = false;

for (const question of questionsToAnswer) {
  //console.log(`Đang xử lý câu hỏi #${question.id}: ${question.question}`);
  const result = await this.checkAnswerAndSubmit(question, headers, securityToken);

  if (result.success) {
    answeredThisSession++;
    // correctCount++;
    newAnswersFound = true;
    //showNotification(`Trúng lời đúng câu hỏi #${question.id}`, 'success'); // thêm noti
  } else {
    if (result.reason === "not_in_cache") {
      console.warn(`Không tìm thấy đáp án trong cache cho câu hỏi #${question.id}`);
      showNotification(`Không tìm thấy đáp án trong cache cho câu hỏi #${question.id}`,
'error'); // thêm noti
    } else if (result.reason === "not_in_options") {
      console.warn(`Đáp án có trong cache nhưng không khớp option server cho câu hỏi #${
question.id}`);
      showNotification(`Đáp án không khớp option server cho câu hỏi #${question.id}`,
'error'); // thêm noti
    } else if (result.reason === "server_error") {
      console.error(`Server từ chối câu hỏi #${question.id}: ${result.message}`);
      showNotification(`Server từ chối câu hỏi #${question.id}: ${result.message}`,
'warn'); // thêm noti
    } else if (result.reason === "exception") {
      console.error(`Lỗi khi gửi câu hỏi #${question.id}: ${result.message}`);
      showNotification(`Lỗi khi gửi câu hỏi #${question.id}: ${result.message}`, 'warn'); /
/ thêm noti
    }
  }
}

if (!newAnswersFound) {
  showNotification(`Vấn Đáp: Không tìm thấy câu trả lời mới, dừng lại.`, 'warn');
  break;
}

if (correctCount < 5) {
  await new Promise(resolve => setTimeout(resolve, 1000));
}

// Tạm ngưng trạng thái cuối cùng để báo cáo chính xác
const finalPayload = new URLSearchParams();
finalPayload.append('action', 'load_quiz_data');
finalPayload.append('security_token', securityToken);

const finalResponse = await fetch(this.ajaxUrl, {
  method: 'POST',
  headers: headers,
  body: finalPayload,
  credentials: 'include'
});
const finalData = await finalResponse.json();
if (finalData.success && finalData.data) {
  correctCount = finalData.data.correct_answers || correctCount;
  totalQuestions = finalData.data.questions.length || totalQuestions;
}

showNotification(
  `Hoàn thành Vấn Đáp. Đã trả lời thêm ${answeredThisSession} câu. Tổng số câu đúng: ${correctCount}/${
totalQuestions}`,
  'success'
);

} catch (e) {
  console.error(`[HH3D Vấn Đáp] Lỗi xảy ra:`, e);
  showNotification(`Lỗi khi thực hiện Vấn Đáp: ${e.message}`, 'error');
}

```

```

    }
}

// =====
// ĐIỂM DANH
// =====
async function doDailyCheckin(nonce) {
    try {
        console.log('[HH3D Daily Check-in] ▶ Bật đầu Daily Check-in');
        const url = weburl + 'wp-json/hh3d/v1/action';
        const headers = {
            'Content-Type': 'application/json',
            'X-Wp-Nonce': nonce,
            'X-Requested-With': 'XMLHttpRequest'
        };

        const bodyPayload = {
            action: 'daily_check_in'
        };

        const response = await fetch(url, {
            method: 'POST',
            headers: headers,
            body: JSON.stringify(bodyPayload),
            credentials: 'include',
            referer: weburl + 'diem-danh',
            mode: 'cors'
        });

        const data = await response.json();

        if (response.ok && data.success) {
            showNotification(`Điểm danh: ${data.message} (${data.streak} ngày)`, 'success');
        } else {
            showNotification(`Điểm danh: ${data.message} || 'Lỗi không xác định'`, 'warn');
        }
    } catch (e) {
        console.error('[HH3D Daily Check-in] ❌ Lỗi xảy ra:', e);
        showNotification(`Lỗi khi thực hiện Daily Check-in: ${e.message}`, 'error');
    }
}

// =====
// TÌM TỔNG MÓN
// =====
async function doClanDailyCheckin(nonce) {
    const securityToken = await getSecurityToken(weburl + 'danh-sach-thanh-vien-tong-mon?t');
    try {
        console.log('[HH3D Clan Check-in] ▶ Bật đầu Clan Check-in');

        // Gợi định 'weburl' được định nghĩa ở scope bên ngoài
        const url = weburl + "wp-json/tong-mon/v1/te-le-tong-mon";

        // --- 1. CẬP NHẬT HEADERS ---
        const headers = {
            "Content-Type": "application/json",
            "X-WP-Nonce": nonce,
            "security_token": securityToken
        };

        // --- 2. CẬP NHẬT BODY ---
        const bodyPayload = {
            action: "te_le_tong_mon",
            security_token: securityToken
        };

        const response = await fetch(url, {
            "credentials": "include",
            "headers": headers, // (Đã cập nhật)
            "referrer": weburl + "danh-sach-thanh-vien-tong-mon",
            "body": JSON.stringify(bodyPayload), // <-- THAY ĐỔI TỪ "{}"
            "method": "POST",
            "mode": "cors"
        });

        // Logic xử lý response giữ nguyên
        const data = await response.json();
        if (response.ok && data.success) {
            showNotification(`Tìm món: ${data.message} (${data.cong_hien_points})`, 'success');
        } else {
            showNotification(`Tìm món: ${data.message} || 'Lỗi không xác định'`, 'warn');
        }
    } catch (e) {
        console.error('[HH3D Clan Check-in] ❌ Lỗi xảy ra:', e);
        showNotification(`Lỗi khi thực hiện Clan Check-in: ${e.message}`, 'error');
    }
}

// =====
// HÀM ĐỔ THẠCH
// =====

/**
 * Lớp quản lý tính năng Đổ Thạch (Dice Roll).
 */
* Hướng dẫn sử dụng:
* 1. Tạo một thực thể của lớp, cung cấp các phụ thuộc cần thiết.
*   const doThachManager = new DoThach();
*
* 2. Gọi phương thức run với chỉ số lược mong muốn ('tài' hoặc 'xửu').
*   await doThachManager.run('tài');
*/
class DoThach {
    constructor() {
        this.ajaxUrl = ajaxUrl;
        this.webUrl = weburl;
        this.getSecurityNonce = getSecurityNonce;
        this.doThachUrl = this.webUrl + 'do-thach-hh3d?t';
    }

    // --- Các phương thức private để gọi API và lấy nonce ---

    // async #getLoadDataNonce() {
    //     return this.getSecurityNonce(this.doThachUrl, /action: 'load_do_thach_data', [\s\S]*?security: '([a-f0-9]+)'/);

```

```

    // }

    // async #getPlaceBetNonce() {
    //     return this.getSecurityNonce(this.doThachUrl, /action: 'place_do_thach_bet',[\s\S]*?security: '([a-f0-
9]+)'/);
    // }

    // async #getClaimRewardNonce() {
    //     return this.getSecurityNonce(this.doThachUrl, /action: 'claim_do_thach_reward',[\s\S]*?security: '([a-f0-
9]+)'/);
    // }

    /**
     * Lầy thông tin phiên đố thách hiện tại.
     * @param {string} securityNonce - Nonce cho yêu cầu.
     * @returns {Promise<object|null>} Dữ liệu phiên hoặc null nếu có lỗi.
     */
    async #getDiceRollInfo() {
        console.log('[HH3D Đố Thách] ▶ Đang lấy thông tin phiên...');
        const securityToken = await getSecurityToken(this.doThachUrl);
        //const payload = new URLSearchParams({ action: 'load_do_thach_data', security_token: securityToken,
security: securityNonce });
        const payload = new URLSearchParams({ action: 'load_do_thach_data', security_token: securityToken });
        const headers = {
            'Content-Type': 'application/x-www-form-urlencoded; charset=UTF-8',
            'X-Requested-With': 'XMLHttpRequest',
        };

        try {
            const response = await fetch(this.ajaxUrl, { method: 'POST', headers, body: payload });
            const data = await response.json();
            if (data.success) {
                console.log('[HH3D Đố Thách] □ Tỉ lệ thông tin phiên thành công.');
```

return data.data;

```
            }
            console.error('[HH3D Đố Thách] □ Lỗi từ API:', data.data || 'Lỗi không xác định');
            return null;
        } catch (e) {
            console.error('[HH3D Đố Thách] □ Lỗi mạng:', e);
            return null;
        }
    }

    /**
     * Đặt cược vào một viên đá cu th.
     * @param {object} stone - Đối tượng đá để đặt cược.
     * @param {number} betAmount - Số tiền cược.
     * @param {string} placeBetSecurity - Nonce để đặt cược.
     * @returns {Promise<boolean>} True nếu đặt cược thành công.
     */
    async #placeBet(stone, betAmount) {
        console.log('[HH3D Đặt Cược] □ Đang cược ${betAmount} Tiền Ngọc vào ${stone.name}...');
        const securityToken = await getSecurityToken(this.doThachUrl);
        const payload = new URLSearchParams({
            action: 'place_do_thach_bet',
            security_token: securityToken,
            //security: placeBetSecurity, b security nonce
            stone_id: stone.stone_id,
            bet_amount: betAmount
        });
        const headers = {
            'Accept': '*/',
            'Content-Type': 'application/x-www-form-urlencoded; charset=UTF-8',
            'X-Requested-With': 'XMLHttpRequest',
        };

        try {
            const response = await fetch(this.ajaxUrl, { method: 'POST', headers, body: payload });
            const data = await response.json();

            if (data.success) {
                showNotification('□ Cược thành công vào ${stone.name}!<br>Tỉ lệ <b>x${stone.reward_multiplier}</b>',
'success');
                this._alreadyClaimedReward = false; // reset flag
                return true;
            }
            else if (data.data === 'Vui lòng nhận thưởng kỳ trước rồi mới tiếp tục đặt cược.') {
                if (!this._alreadyClaimedReward) {
                    if (await this.#claimReward()) {
                        this._alreadyClaimedReward = true;
                        //return await this.#placeBet(stone, betAmount, placeBetSecurity);
                        return await this.#placeBet(stone, betAmount);
                    } else {
                        showNotification('□ Không thể nhận thưởng kỳ trước, vui lòng thử lại.', 'error');
                    }
                }
            }
            else {
                showNotification('□ Đã thử nhận thưởng nhưng vẫn không cược được.', 'error');
            }
            this._alreadyClaimedReward = false; // reset flag
            return false;
        }

        const errorMessage = data.data || data.message || 'Lỗi không xác định.';
        showNotification('□ Lỗi cược: ${errorMessage}', 'error');
        this._alreadyClaimedReward = false;
        return false;
    } catch (e) {
        showNotification('□ Lỗi mạng khi cược: ${e.message}', 'error');
        this._alreadyClaimedReward = false;
        return false;
    }
    }

    /**
     * Nhận thưởng cho một lần cược thắng.
     * @returns {Promise<boolean>} True nếu nhận thưởng thành công.
     */
    async #claimReward() {
        console.log('[HH3D Nhận Thưởng] □ Đang nhận thưởng...');
        const securityToken = await getSecurityToken(this.doThachUrl);
        const payload = new URLSearchParams({ action: 'claim_do_thach_reward', security_token: securityToken });
        const headers = {
            'Accept': 'application/json, text/javascript, */*; q=0.01',
            'Content-Type': 'application/x-www-form-urlencoded; charset=UTF-8',
        };

```

```

        'X-Requested-With': 'XMLHttpRequest',
    };

    try {
        const response = await fetch(this.ajaxUrl, { method: 'POST', headers, body: payload });
        const data = await response.json();
        if (data.success) {
            const rewardMessage = data.data?.message || `Nhận thưởng thành công!`;
            showNotification(rewardMessage, 'success');
            return true;
        }
        const errorMessage = data.data?.message || `Lỗi không xác định khi nhận thưởng.`;
        showNotification(errorMessage, 'error');
        return false;
    } catch (e) {
        console.error(e);
        showNotification(`Lỗi mạng khi nhận thưởng: ${e.message}`, 'error');
        return false;
    }
}

// --- Phương thức public để chạy toàn bộ quy trình ---

/**
 * Chạy toàn bộ quy trình để thách dựa trên chiến lược đã chọn.
 * @param {string} stoneType - Chiến lược đặt cược ('tài' hoặc 'xỉu').
 */
async run(stoneType) {
    console.log(`[HH3D Đố Thách] Bắt đầu quy trình với chiến lược: ${stoneType}...`);

    const sessionData = await this.#getDiceRollInfo();

    if (!sessionData) {
        console.error(`[HH3D Đố Thách] Không thể lấy dữ liệu phiên, dừng lại.`);
        return;
    }

    const userBetStones = sessionData.stones.filter(stone => stone.bet_placed);

    // Bước 2: Kiểm tra trạng thái phiên và hành động (nhận thưởng hoặc đặt cược)
    if (sessionData.winning_stone_id) {
        console.log(`[HH3D Đố Thách] Đã có kết quả. Kiểm tra nhận thưởng...`);
        const claimableWin = userBetStones.find(s => s.stone_id === sessionData.winning_stone_id && !s.reward_claimed);
        const alreadyClaimed = userBetStones.find(s => s.stone_id === sessionData.winning_stone_id && s.reward_claimed);

        if (claimableWin) {
            console.log(`[HH3D Đố Thách] Trúng rồi! Đã cược: ${claimableWin.name}. Đang nhận thưởng...`);
            await this.#claimReward();
        } else if (alreadyClaimed) {
            console.log(`[HH3D Đố Thách] Đã nhận thưởng cho phiên này.`);
        } else if (userBetStones.length > 0) {
            showNotification(`[Đố Thách] Rút tỉc, bạn không trúng phiên này.`, 'info');
        } else {
            showNotification(`[Đố Thách] Bạn không tham gia phiên này.`, 'info');
        }
        taskTracker.updateTask(accountId, 'dothach', 'reward_claimed', 'true')
        return;
    }

    // Bước 3: Nếu đang trong giờ cược, tiến hành đặt cược
    console.log(`[HH3D Đố Thách] Đang trong thời gian đặt cược.`);
    const userBetCount = userBetStones.length;

    if (userBetCount >= 2) {
        showNotification(`[Đố Thách] Đã cược đủ 2 lần. Chờ phiên sau.`, 'warn');
        taskTracker.updateTask(accountId, 'dothach', 'betplaced', true);

        return;
    }

    const sortedStones = [...sessionData.stones].sort((a, b) => b.reward_multiplier - a.reward_multiplier);
    const availableStones = sortedStones.filter(stone => !stone.bet_placed);

    if (availableStones.length === 0) {
        showNotification(`[Đố Thách] Không còn đá nào để cược!`, 'warn');
        return;
    }

    const betAmount = 20;
    const stonesToBet = [];
    const normalizedStoneType = stoneType.toLowerCase();
    const betsRemaining = 2 - userBetCount;

    if (normalizedStoneType === 'tài' || normalizedStoneType === 'tai') {
        stonesToBet.push(...availableStones.slice(0, betsRemaining));
    } else if (normalizedStoneType === 'xỉu' || normalizedStoneType === 'xiu') {
        const xiuStones = availableStones.slice(2, 4);
        stonesToBet.push(...xiuStones.slice(0, betsRemaining));
    } else {
        console.log(`[HH3D Đố Thách] Chiến lược không hợp lệ. Vui lòng chọn "tài" hoặc "xỉu".`);
        return;
    }

    if (stonesToBet.length === 0) {
        console.log(`[HH3D Đố Thách] Không có đá nào phù hợp chiến lược hoặc đã cược đủ.`);
        return;
    }

    let successfulBets = 0;
    for (const stone of stonesToBet) {
        //const success = await this.#placeBet(stone, betAmount, placeBetSecurity);
        const success = await this.#placeBet(stone, betAmount);
        if (success) {
            successfulBets++;
        }
    }

    // Kiểm tra và cập nhật trạng thái ngay sau khi cược
    if (userBetCount + successfulBets >= 2) {
        taskTracker.updateTask(accountId, 'dothach', 'betplaced', true);
        //console.log(taskTracker.getTaskStatus(accountId, 'dothach'));
    }
}

```

```

}

//=====
// TIỀN DUYÊN
//=====
class TienDuyen {
  nonce;

  constructor() {
    this.apiUrl = weburl + "wp-json/hh3d/v1/action";
  }

  async init() {
    console.log("Chạy init tiên duyên");
    this.nonce = await getNonce();
    //console.log("getNonce type:", typeof getNonce);
    //console.log("getNonce source:", getNonce.toString());
    console.log("Tiên Duyên Nonce (init):", this.nonce);

    this.securityToken = await getSecurityToken(weburl + "tien-duyen?t");
    console.log("Tiên Duyên SecurityToken (init):", this.securityToken);
  }

  async #post(action, body = {}) {
    const res = await fetch(this.apiUrl, {
      credentials: "include",
      method: "POST",
      headers: {
        "Accept": "*/*",
        "Content-Type": "application/json",
        "X-WP-Nonce": this.nonce
      },
      body: JSON.stringify({ action, ...body })
    });
    return res.json();
  }

  // [] Lấy danh sách phòng cưới
  async getWeddingRooms() {
    console.log("Tiên Duyên SecurityToken (GetWedding Rooms):", this.securityToken);
    return await this.#post("show_all_wedding", {
      security_token: this.securityToken
    });
  }

  // [] Chúc phúc
  async addBlessing(weddingRoomId, message = "Chúc phúc trăm năm hạnh phúc []") {
    return await this.#post("hh3d_add_blessing", {
      wedding_room_id: weddingRoomId,
      message
    });
  }

  // [] Chúc phúc H[]ng Nhan (endpoint riêng)
  async addHongNhanBlessing(weddingRoomId, message = "[] Một đoạn h[]ng duyên, vạn ph[]n cơ ngộ! Chúc mừng cơ duyên
đẹp giữa ch[]n h[]ng tr[]n. []") {
    const res = await fetch(weburl + "wp-json/hh3d/v1/hong-nhan/bless", {
      credentials: "include",
      method: "POST",
      headers: {
        "Accept": "*/*",
        "Content-Type": "application/json",
        "X-WP-Nonce": this.nonce
      },
      body: JSON.stringify({ wedding_room_id: weddingRoomId, message })
    });
    return res.json();
  }

  // [] Nhận lì xì
  async receiveLiXi(weddingRoomId) {
    return await this.#post("hh3d_receive_li_xi", {
      wedding_room_id: weddingRoomId
    });
  }

  // [] Duyên: chúc phúc + nhận lì xì
  async doTienDuyen(isManual = false) {
    if (!this.nonce || !this.securityToken) {
      console.log("Chưa init, đang chạy init trong doTienDuyen...");
      await this.init();
    }

    const lastCheck = taskTracker.getLastCheckTienDuyen(accountId);
    const now = new Date();
    if (!isManual && lastCheck && (now - lastCheck < 1800000)) return;

    const list = await this.getWeddingRooms();
    if (!list?.data) {
      showNotification("Không có danh sách phòng cưới", "warn");
      return;
    }

    let processedCount = 0;
    for (const room of list.data) {
      taskTracker.setLastCheckTienDuyen(accountId, now);
      const isHongNhan = room.room_type === 'hong_nhan';

      // Server tr[] has_blessed sai cho phòng H[]ng Nhan → ch[] skip n[]u là Đạo Lữ đã chúc & không có lì xì
      if (!isHongNhan && room.has_blessed === true && room.has_li_xi === false) {
        continue;
      }

      processedCount++;
      console.log(`[] K[]m tra phòng ${room.wedding_room_id} [${room.room_type}]`);

      // Đạo Lữ: ch[] chúc khi chưa chúc; H[]ng Nhan: luôn th[] (server tự báo l[]i n[]u đã chúc)
      if (isHongNhan || room.has_blessed === false) {
        const name1 = isHongNhan ? (room.nguyen_chu_name || room.user1_name) : room.user1_name;
        const name2 = isHongNhan ? (room.hong_nhan_name || room.user2_name) : room.user2_name;
        const bless = isHongNhan
          ? await this.addHongNhanBlessing(room.wedding_room_id)
          : await this.addBlessing(room.wedding_room_id);
        if (bless && bless.success === true) {
          const label = isHongNhan ? '[] H[]ng Nhan' : '[] Đạo Lữ';
          showNotification(

```

```

        `Bạn đã gửi lời chúc phúc [${label}] cho cặp đôi <br><b>${name1} & ${name2}</b>`,
        "success"
    );
    }
}

if (room.has_li_xi === true) {
    const liXi = await this.receiveLiXi(room.wedding_room_id);
    if (liXi && liXi.success === true) {
        showNotification(
            `Nhận lì xì phòng cưới ${room.wedding_room_id} được <b>${liXi.data.amount}</b> ${liXi.data.name}</b>!`,
            "success"
        );
    }
}

// Chờ 1 giây tránh spam
await new Promise(r => setTimeout(r, 1000));
}

if (isManual && processedCount === 0) {
    showNotification("Không có phòng cưới mới hoặc lì xì chưa nhận!", "info");
}
}

}

// =====
// TIỀN DUYÊN - TẶNG HOA
// =====

class TangHoa {
    nonce;
    initialized = false;

    constructor() {
        this.apiUrl = weblurl + "wp-json/hh3d/v1/action";
        this.accountId = accountId;
    }

    async init() {
        // console.log("chạy tặng hoa");
        this.nonce = await getNonce();
        // console.log("getNonce type:", typeof getNonce());
        // console.log("getNonce source:", getNonce.toString());
        this.securityToken = await getSecurityToken(weblurl + 'tien-duyen?t');
        this.initialized = true;
    }

    async #post(action, body = {}) {
        const res = await fetch(this.apiUrl,
            {
                credentials: "include",
                method: "POST",
                headers: {
                    "Accept": "*/*",
                    "Content-Type": "application/json",
                    "X-WP-Nonce": this.nonce
                },
                body: JSON.stringify({
                    action, ...body
                })
            });
        return res.json();
    }

    // Lấy danh sách bạn bè
    async getFriends() {
        return await this.#post("get_friends_td");
    }

    // Kiểm tra giới hạn quà tặng
    async checkGiftLimit(friendId, costType = "tien_ngoc") {
        return await this.#post("check_daily_gift_limit",
            {
                user_id: this.accountId,
                friend_id: friendId, // user_id c là bạn bè
                cost_type: costType
            });
    }

    // Tặng quà
    async giftToFriend(friendId, giftType = "hoa_hong", costType = "tien_ngoc") {
        return await this.#post("gift_to_friend",
            {
                user_id: this.accountId,
                friend_id: friendId, // user_id c là bạn bè
                gift_type: giftType,
                cost_type: costType
            });
    }

    // Hàm chính: tặng hoa cho đúng số người đã chọn
    async run(selectedCount) {
        try {
            if (!this.initialized) {
                await this.init();
            }

            const count = parseInt(selectedCount, 10);
            if (Number.isNaN(count) || count <= 0) {
                showNotification(`⚠ Số lượng chọn không hợp lệ: ${selectedCount}`, 'warn');
                return;
            }

            const friendsRes = await this.getFriends();
            const list = Array.isArray(friendsRes?.data)
                ? friendsRes.data
                : (Array.isArray(friendsRes) ? friendsRes : []);

            if (!Array.isArray(list) || list.length === 0) {
                showNotification("⚠ Không có danh sách bạn bè", 'error');
                return;
            }

            // Lấy đúng số người đã chọn trên menu
            const targetFriends = list.slice(0, count);
            // showNotification(`🎁 Sẽ tặng cho ${targetFriends.length} người.`, 'info');
            let processed = 0;

```

```

    for (const friend of targetFriends) {
        const friendId = friend?.user_id;
        if (!friendId) {
            showNotification("⚠️ Thi👤 user_id trong item", 'warn');
            continue;
        }

        processed++;
        showNotification(`📦 B👤t đ👤u tặng cho ID ${friendId}
        `) (${processed}
        )/${targetFriends.length}
        }`, 'info');

        let giftsForThisFriend = 0;
        const maxGiftsPerFriend = 3;

        while (giftsForThisFriend < maxGiftsPerFriend) {
            const check = await this.checkGiftLimit(friendId);
            const remaining = Number(check?.remaining_free_gifts ?? 0);

            if (remaining <= 0) {
                showNotification(`📦 H👤t lượt tặng cho ID ${friendId}
                `), dừng tặng người này.', 'warn');
                break;
            }

            const gift = await this.giftToFriend(friendId);
            if (gift?.success === true) {
                giftsForThisFriend++;
                //const remainingAfter = gift?.remaining_free_gifts ?? 'N/A';
                showNotification(`📦 Tặng l👤n ${giftsForThisFriend}
                `/${maxGiftsPerFriend}
                `), 'success');
            } else {
                showNotification(`⚠️ Tặng quà th👤t bại cho ID ${friendId}
                `), 'error');
                break;
            }

            await new Promise(r => setTimeout(r, 2000));
        }

        await new Promise(r => setTimeout(r, 1000));
    }

    showNotification(`📦 K👤t thúc: đã x👤 lý ${processed}
    `/${targetFriends.length}
    `) người', 'success');
} catch (err) {
    showNotification(`📦 L👤i trong run(): ${err?.message ?? err}
    `), 'error');
}
}
}

// =====
// TIẾN DUYÊN - C👤U NGUYỄN
// =====

async function docaunguyen(accountId) {
    const logPrefix = "[C👤u Nguyễn tiên duyên]";
    try {

        console.log(logPrefix, "▶️👤 Đang thực hiện...");

        const nonce = await getNonce();
        if (!nonce) {
            showNotification("Không l👤y được nonce", "error");
            return false;
        }

        const url = webserv + "wp-json/hh3d/v1/action";
        const bodyPayload = { action: "make_wish_tree" };

        const response = await fetch(url, {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
                "X-Wp-Nonce": nonce,
                "X-Requested-With": "XMLHttpRequest"
            },
            body: JSON.stringify(bodyPayload),
            credentials: "include"
        });

        const data = await response.json();

        if (
            data.success ||
            (data.message && data.message.includes("Chưa có đạo lữ đ👤 ước nguyện Tiên Duyên Thự"))
        ) {
            showNotification(`${logPrefix} 📦 ${data.message}`, "info");
            taskTracker.markTaskDone(accountId, "tienduyen");
            return true;
        } else {
            showNotification(`${logPrefix}⚠️ ${data.message || "Th👤t bại"}`, "warn");
            return false;
        }
    } catch (e) {
        console.error(e);
        showNotification(`${logPrefix}: ${e.message}`, "error");
        return false;
    }
}

// =====
// THỊ LUYỆN TÔNG MÔN
// =====

async function doThiLuyenTongMon() {
    console.log(`[HH3D Thị Luyện Tông Môn] ▶️👤 B👤t đ👤u Thị Luyện Tông Môn`);

    // Bước 1: L👤y security token

```

```

const securityToken = await getSecurityToken(weburl + 'thi-luyen-tong-mon-hh3d?t');
if (!securityToken) {
    showNotification('Lỗi khi lấy security token cho Thí Luyện Tổng Môn.', 'error');
    throw new Error('Lỗi khi lấy security token cho Thí Luyện Tổng Môn.');
```

```

}

const url = ajaxUrl;
const payload = new URLSearchParams();
payload.append('action', 'open_chest_tltm');
payload.append('security_token', securityToken);
```

```

console.log([...payload.entries()]);
```

```

const headers = {
    'Accept': 'application/json, text/javascript, */*; q=0.01',
    'Content-Type': 'application/x-www-form-urlencoded; charset=UTF-8',
    'X-Requested-With': 'XMLHttpRequest',
};
```

```

try {
    const response = await fetch(url, {
        method: 'POST',
        headers: headers,
        body: payload,
        credentials: 'include' // Quan trọng để giữ cookies
    });

    // Đọc body một lần
    const text = await response.text();
    console.log(text);
    // Parse JSON từ text
    const data = JSON.parse(text);

    if (data.success) {
        // Trường hợp thành công
        const message = data.data && data.data.message ? data.data.message : 'Mở rương thành công!';
        showNotification(message, 'success');
    } else {
        // Trường hợp thất bại
        const errorMessage = data.data && data.data.message ? data.data.message : 'Lỗi không xác định khi mở rương.';

        if (errorMessage.includes("Đã hoàn thành Thí Luyện Tổng Môn hôm nay")) {
            showNotification(errorMessage, 'info');
            taskTracker.markTaskDone(accountId, 'thiluyen');
        } else {
            showNotification(errorMessage, 'error');
        }
    }
}

} catch (e) {
    showNotification('Lỗi mạng khi thực hiện Thí Luyện: ${e.message}', 'error');
}
}
```

```

// =====
// PHÚC LỢI
// =====
```

```

async function doPhucLoiDuong() {
    const logPrefix = '[HH3D Phúc Lợi Đường]';
    console.log(`${logPrefix} ▶ Bắt đầu nhiệm vụ Phúc Lợi Đường.`);
    // Bước 1: Lấy security nonce từ trang hoặc fallback sang AJAX
    let securityNonce = null;
    try {
        const resp = await fetch(weburl + 'phuc-loi-duong?t');
        const jsonConfigMatch = await resp.text();
        const match = jsonConfigMatch.match(/"securityToken"\s*:\s*"([^\"]+)"/);
        if (match && match[1]) {
            securityNonce = match[1];
            console.log(`[HH3D Phúc Lợi Đường] Nonce từ HTML:`, securityNonce);
        }
    } catch (err) {
        console.warn(`[HH3D Phúc Lợi Đường] ⚠ Lỗi khi parse HTML:`, err);
    }

    if (!securityNonce) {
        try {
            const resAjax = await fetch(weburl + 'wp-content/themes/halimovies-child/hh3d-ajax.php', {
                method: 'POST',
                headers: { 'Content-Type': 'application/x-www-form-urlencoded; charset=UTF-8' },
                body: 'action=get_next_time_pl',
                credentials: 'include'
            });
            const dataAjax = await resAjax.json();
            if (dataAjax && dataAjax.security_token) {
                securityNonce = dataAjax.security_token;
                console.log(`[HH3D Phúc Lợi Đường] Nonce từ AJAX:`, securityNonce);
            }
        } catch (err) {
            console.error(`[HH3D Phúc Lợi Đường] ⚠ Lỗi khi gọi AJAX:`, err);
        }
    }

    if (!securityNonce) {
        showNotification('Lỗi khi lấy security nonce cho Phúc Lợi Đường.', 'error');
        return;
    }

    const url = ajaxUrl;
    const headers = {
        'Accept': 'application/json, text/javascript, */*; q=0.01',
        'Content-Type': 'application/x-www-form-urlencoded; charset=UTF-8',
        'X-Requested-With': 'XMLHttpRequest',
    };

    // Bước 2: Lấy thông tin thời gian còn lại và cập nhật độ rương
    console.log(`[HH3D Phúc Lợi Đường] Đang kiểm tra thời gian mở rương...`);
    const securityToken = await getSecurityToken(weburl + 'phuc-loi-duong?t');
    const payloadTime = new URLSearchParams();
    payloadTime.append('action', 'get_next_time_pl');
    payloadTime.append('security_token', securityToken);
    payloadTime.append('security', securityNonce);

    try {
```

```

const responseTime = await fetch(url, {
  method: 'POST',
  headers: headers,
  body: payloadTime,
  credentials: 'include'
});
const dataTime = await responseTime.json();

if (dataTime.success) {
  const { time, chest_level: chest_level_string } = dataTime.data;
  const chest_level = parseInt(chest_level_string, 10);

  if (chest_level >= 4) {
    showNotification('Phúc Lợi Đường đã hoàn tất hôm nay!', 'success');
    taskTracker.markTaskDone(accountId, 'phucloi');
    return;
  }

  const isVip = localStorage.getItem('generalVipMode') === 'true';
  if (time === '00:00' || isVip) {
    console.log('[HH3D Phúc Lợi Đường] Đang mở rương cấp ${chest_level + 1}...');
    const payloadOpen = new URLSearchParams();
    payloadOpen.append('action', 'open_chest_pl');
    payloadOpen.append('security_token', securityToken);
    payloadOpen.append('security', securityNonce);
    payloadOpen.append('chest_id', chest_level + 1);

    const responseOpen = await fetch(url, {
      method: 'POST',
      headers: headers,
      body: payloadOpen,
      credentials: 'include'
    });
    const dataOpen = await responseOpen.json();

    if (dataOpen.success) {
      const message = dataOpen.data && dataOpen.data.message ? dataOpen.data.message : 'Mở rương
thành công!';

      showNotification(message, 'success');
      if (message.includes('đã hoàn thành Phúc Lợi ngày hôm nay')) {
        taskTracker.markTaskDone(accountId, 'phucloi');
      } else {
        const nextDelay = isVip ? '07:33' : '30:00';
        taskTracker.adjustTaskTime(accountId, 'phucloi', timePlus(nextDelay));
      }
    } else {
      const errorMessage = dataOpen.data && dataOpen.data.message ? dataOpen.data.message : 'Lỗi
không xác định khi mở rương.';
      if (isVip) {
        console.log('[HH3D Phúc Lợi Đường] Bạn Thử mở rương VIP thất bại (chưa đến thời gian hoặc
lỗi khác): ${errorMessage}');
        if (!errorMessage.toLowerCase().includes('thời gian') && !errorMessage.toLowerCase().
includes('đợi')) {
          showNotification(errorMessage, 'error');
        } else {
          showNotification('VIP: Vui lòng đợi thêm để mở rương tiếp theo.', 'info');
        }
        const parts = time.split(':').map(Number);
        const serverWaitMs = parts.length === 2 ? (parts[0] * 60 + parts[1]) * 1000 : 0;
        const vipWaitMs = 7 * 60 * 1000 + 33 * 1000; // 7.55 minutes
        const waitMs = (serverWaitMs > 0 && serverWaitMs < vipWaitMs) ? serverWaitMs : vipWaitMs;
        taskTracker.adjustTaskTime(accountId, 'phucloi', Date.now() + waitMs);
      } else {
        showNotification(errorMessage, 'error');
      }
    }
  } else {
    showNotification('Vui lòng đợi ${time} để mở rương tiếp theo.', 'warn');
    taskTracker.adjustTaskTime(accountId, 'phucloi', timePlus(time));
  }
} else {
  const errorMessage = dataTime.data && dataTime.data.message ? dataTime.data.message : 'Lỗi không xác
định khi lấy thời gian.';
  showNotification(errorMessage, 'error');
}
} catch (e) {
  showNotification('Lỗi mạng khi thực hiện Phúc Lợi Đường: ${e.message}', 'error');
}
}

async function phucloiclaimbonus() {
  const logPrefix = "[HH3D Phúc Lợi Claim Bonus]";
  const ajaxUrl = weblink + "wp-content/themes/halimovies-child/hh3d-ajax.php";

  // Bước 1: Lấy security token từ trang hoặc fallback sang AJAX
  let securityToken = null;
  try {
    const resp = await fetch(weblink + "phuc-loi-duong?t", { credentials: "include" });
    const html = await resp.text();
    const match = html.match(/"securityToken"\s*:\s*"([^\"]+)"/);
    if (match && match[1]) {
      securityToken = match[1];
      console.log(`${logPrefix} Security token từ HTML:`, securityToken);
    }
  } catch (err) {
    console.warn(`${logPrefix} Lỗi khi parse HTML:`, err);
  }

  if (!securityToken) {
    try {
      const resAjax = await fetch(ajaxUrl, {
        method: "POST",
        headers: { "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8" },
        body: "action=get_next_time_pl",
        credentials: "include"
      });
      const dataAjax = await resAjax.json();
      if (dataAjax && dataAjax.security_token) {
        securityToken = dataAjax.security_token;
        console.log(`${logPrefix} Security token từ AJAX:`, securityToken);
      }
    } catch (err) {
      console.error(`${logPrefix} Lỗi khi gọi AJAX:`, err);
    }
  }

  if (!securityToken) {

```

```

        console.error(`${logPrefix} ❌ Không lấy được security token.`);
        return;
    }

    // Bước 2: Thử chest_id từ 1 đến 4
    for (let chestId = 1; chestId <= 4; chestId++) {
        const payload = new URLSearchParams();
        payload.append("action", "claim_bonus_reward");
        payload.append("security_token", securityToken);
        payload.append("chest_id", chestId);

        try {
            const response = await fetch.ajaxUrl, {
                method: "POST",
                headers: {
                    "Accept": "application/json, text/javascript, */*; q=0.01",
                    "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
                    "X-Requested-With": "XMLHttpRequest"
                },
                body: payload,
                credentials: "include"
            });

            const data = await response.json();
            // console.log(`${logPrefix} Chest ${chestId}:`, data);

            if (data.success) {
                showNotification(`${logPrefix} 🏆 Chest ${chestId}: ${data.data?.message}`, "success");
            } else {
                showNotification(`${logPrefix} ⚠️ Chest ${chestId}: ${data.data?.message}`, "warn");
            }
        } catch (err) {
            showNotification(`${logPrefix} ❌ Lỗi khi gọi chest ${chestId}: ${err}`, "error");
        }
    }
}

// =====
// Bí CànH
// =====
class BiCanh {
    constructor() {
        this.weburl = weburl;
        this.logPrefix = '[HH3D Bí CànH]';
    }

    async doBiCanh() {
        console.log(`${this.logPrefix} ▶️ Bật đầu nhiệm vụ Bí CànH Tổng Môn.`);

        // 1. Lấy nonce
        const nonce = await this.getNonce();
        if (!nonce) {
            showNotification('Lỗi: Không thể lấy nonce cho Bí CànH Tổng Môn.', 'error');
            throw new Error('Lỗi nonce bí cànH');
        }

        // 2. Lấy boss status
        let statusResp = await this.getBossStatus(nonce);
        console.log("[DEBUG lấy boss status]", statusResp);

        // Cập nhật % HP boss lên UI quest item Bí CànH
        if (statusResp?.boss?.hp_percentage != null) {
            window.bicanhBossHp = Number(statusResp.boss.hp_percentage);
            const bicanhProgressSpan = document.querySelector('.nv-quest-item[data-task-id="bicanh"] .quest-progress');
            if (bicanhProgressSpan) {
                const baseText = bicanhProgressSpan.textContent.replace(/s*\[^\]]*%s*HP\)/g, '').trimEnd();
                bicanhProgressSpan.textContent = baseText + ` (${window.bicanhBossHp.toFixed(2)}% HP)`;
            }
        }

        // 3. Nếu có thưởng pending thì nhận
        if (statusResp?.has_pending_reward) {
            console.log(`${this.logPrefix} 🎁 Có thưởng chưa nhận, tiến hành nhận...`);
            const rewardResponse = await this.sendApiRequest('wp-json/tong-mon/v1/claim-boss-reward', 'POST', nonce, {});
            console.log("[DEBUG ClaimReward response]", rewardResponse);
            if (rewardResponse?.success) {
                showNotification(rewardResponse.message, 'success');
            }
            // Sau khi nhận thưởng, lấy lại boss status sạch
            statusResp = await this.getBossStatus(nonce);
            console.log("[DEBUG BossStatus sau khi nhận thưởng]", statusResp);
        }

        // 4. Kiểm tra cooldown
        await this.sleep(500);
        console.log("[DEBUG Trước khi check cooldown, statusResp]", statusResp);
        const canAttack = await this.checkAttackCooldown(nonce, statusResp);
        console.log("[DEBUG Kiểm tra checkAttackCooldown: canattack", canAttack);
        if (!canAttack) return;

        // 5. Tấn công boss
        await this.sleep(500);
        await this.attackBoss(nonce);
    }

    async getNonce() {
        const nonce = await getSecurityNonce(weburl + 'bi-canh-tong-mon?t', "nonce");
        return nonce || null;
    }

    async getBossStatus(nonce) {
        console.log(`${this.logPrefix} 📡 Gọi API getBossStatus...`);
        const resp = await this.sendApiRequest('wp-json/tong-mon/v1/get-boss-status', 'POST', nonce, {});
        console.log("[DEBUG getBossStatus response]", resp);
        return resp;
    }

    sleep(ms) {
        return new Promise(resolve => setTimeout(resolve, ms));
    }

    async isReserveHold() {
        const nonce = await this.getNonce();
    }
}

```

```

    if (!nonce) return false;

    // Lấy thông tin boss status
    const statusResp = await this.sendApiRequest('wp-json/tong-mon/v1/get-boss-status', 'POST', nonce, {});
    //console.log(`${this.logPrefix} Debug boss status:`, statusResp);

    const remaining = Number(statusResp?.attack_info?.remaining ?? 0);
    const reserve = Number(localStorage.getItem('reserveBiCanhAttacks') || '0');
    console.log("[DEBUG isReserveHold] reserveBiCanhAttacks:", reserve);
    console.log("[DEBUG isReserveHold] remaining:", remaining);

    // Nếu có cấu hình giữ lượt và số lượt còn lại <= số lượt giữ → coi như đang giữ
    return reserve > 0 && remaining <= reserve;
}

async checkAttackCooldown(nonce, statusResp) {
    console.log(`${this.logPrefix} Đang kiểm tra thời gian hồi chiêu...`);
    const endpoint = 'wp-json/tong-mon/v1/check-attack-cooldown';

    try {
        const response = await this.sendApiRequest(endpoint, 'POST', nonce, {});
        console.log("[DEBUG checkAttackCooldown response]", response);
        // const statusResp = await this.getBossStatus(nonce);
        // console.log("[DEBUG BossStatus]", statusResp);

        // Nếu có thể tấn công
        if (response?.success && response.can_attack) {
            // Kiểm tra giữ lượt
            if (await this.isReserveHold()) {
                const msg = `Đang giữ lượt, không tấn công.`;
                console.log(`${this.logPrefix} ${msg}`);
                showNotification(msg, 'info');
                // Không hiển thị countdown trong trường hợp giữ lượt
                // taskTracker.adjustTaskTime(accountId, 'bicanh', null);
                return false;
            }
            const statusResp = await this.sendApiRequest('wp-json/tong-mon/v1/get-boss-status', 'POST', nonce, {});

            const remaining = Number(statusResp?.attack_info?.remaining ?? 0);
            console.log(`${this.logPrefix} Có thể tấn công. Còn ${remaining} lượt.`);
            return true;
        }

        // Boss chết
        if (response?.success && response.message === 'Không có boss để tấn công') {
            await this.sleep(1000);
            const rewardResponse = await this.sendApiRequest('wp-json/tong-mon/v1/claim-boss-reward', 'POST', nonce, {});

            if (rewardResponse?.success) showNotification(rewardResponse.message, 'success');

            const contributionResponse = await this.sendApiRequest('wp-json/tong-mon/v1/contribute-boss', 'POST', nonce, {});
            if (contributionResponse) {
                showNotification(contributionResponse.message, contributionResponse.success ? 'success' : 'warn');
            }

            taskTracker.adjustTaskTime(accountId, 'bicanh', null);
            return false;
        }

        // Countdown hiển thị khi có cooldown_remaining (trừ trường hợp giữ lượt)
        if (Number.isFinite(response?.cooldown_remaining)) {
            console.log("[DEBUG Cooldown_remaining]", response.cooldown_remaining);
            taskTracker.adjustTaskTime(accountId, 'bicanh', Date.now() + response.cooldown_remaining * 1000);
        } else {
            console.log("[DEBUG Không có cooldown_remaining, reset taskTime]");
            taskTracker.adjustTaskTime(accountId, 'bicanh', null);
        }

        const message = response?.message || 'Không thể tấn công vào lúc này.';
        showNotification(`${message}`, 'info');
        return false;
    } catch (e) {
        showNotification(`${this.logPrefix} Kiểm tra cooldown: ${e.message}`, 'error');
        return false;
    }
}

async attackBoss(nonce) {
    console.log(`${this.logPrefix} Đang khiêu chiến boss...`);
    const endpoint = 'wp-json/tong-mon/v1/attack-boss';

    try {
        const response = await this.sendApiRequest(endpoint, 'POST', nonce, {});
        if (response && response.success) {
            console.log("[DEBUG attackBoss response]", response);
            const message = response.message || `Gây ${response.damage} sát thương.`;
            showNotification(message, 'success');
            taskTracker.adjustTaskTime(accountId, 'bicanh', timePlus('07:00'));
        } else {
            const errorMessage = response?.message || 'Lỗi không xác định khi tấn công.';
            console.log("[DEBUG attackBoss errorMessage]", errorMessage);
            showNotification(errorMessage, 'error');
        }
    } catch (e) {
        showNotification(`Lỗi mạng khi tấn công boss Bí Cảnh: ${e.message}`, 'error');
    }
}

async isDailyLimit() {
    const endpoint = 'wp-json/tong-mon/v1/check-attack-cooldown';
    const nonce = await this.getNonce();
    if (!nonce) return false;

    try {
        const response = await this.sendApiRequest(endpoint, 'POST', nonce, {});
        return response && response.success && response.cooldown_type === 'daily_limit';
    } catch (e) {
        console.error(`${this.logPrefix} Kiểm tra cooldown:`, e);
        return false;
    }
}

async sendApiRequest(endpoint, method, nonce, body = {}) {

```

```

    try {
      const url = `${this.weburl}${endpoint}`;
      const headers = {
        "Content-Type": "application/json",
        "X-WP-Nonce": nonce,
        "Accept": "*/*",
        "Accept-Language": "vi,en-US;q=0.5",
        "X-Requested-With": "XMLHttpRequest",
      };
      const response = await fetch(url, {
        method,
        headers,
        body: JSON.stringify(body),
        credentials: 'include'
      });
      return await response.json();
    } catch (error) {
      console.error(`${this.logPrefix} ❌ Lỗi khi gửi yêu cầu tới ${endpoint}:`, error);
      throw error;
    }
  }
}

// =====
// BÍ CÃNH HIỆN TỖ
// =====
class BiCanhHienTe {
  constructor() {
    this.weburl = weburl;
    this.logPrefix = "[HH3D Bí Cảnh Socket]";
    this.biCanhSocketActive = false;
    this.biCanhSocketWaiter = null;
    this.handledBossIds = new Set();
    this.currentDay = this.getTodayKey();
    this.currentBossId = null;
  }

  getTodayKey() {
    const d = new Date();
    return `${d.getFullYear()}-${d.getMonth() + 1}-${d.getDate()}`;
  }

  resetIfNewDay() {
    const today = this.getTodayKey();
    if (today !== this.currentDay) {
      this.currentDay = today;
      this.handledBossIds.clear();
      console.log(`${this.logPrefix} 🔄 Reset handledBossIds vì sang ngày mới`);
      showNotification(`${this.logPrefix} 🔄 Reset handledBossIds vì sang ngày mới`, 'info');
    }
  }

  async sendApiRequest(endpoint, method, nonce, body = {}) {
    try {
      const url = `${this.weburl}${endpoint}`;
      const headers = {
        "Content-Type": "application/json",
        "X-WP-Nonce": nonce,
        "Accept": "*/*",
        "Accept-Language": "vi,en-US;q=0.5",
        "X-Requested-With": "XMLHttpRequest",
      };
      const response = await fetch(url, {
        method,
        headers,
        body: JSON.stringify(body),
        credentials: 'include'
      });
      return await response.json();
    } catch (error) {
      console.error(`${this.logPrefix} ❌ Lỗi khi gửi yêu cầu tới ${endpoint}:`, error);
      showNotification(`${this.logPrefix} ❌ Lỗi khi gửi yêu cầu tới ${endpoint}:`, error);
      throw error;
    }
  }

  async updateCurrentBossId() {
    const nonce = await getNonce();
    if (!nonce) {
      console.error(`${this.logPrefix} ❌ Không lấy được nonce`);
      showNotification(`${this.logPrefix} ❌ Không lấy được nonce`, 'info');
      return null;
    }
    const res = await this.sendApiRequest("wp-json/tong-mon/v1/get-boss-status", "POST", nonce, {});
    // console.log(`${this.logPrefix} 📄 Response get-boss-status:`, res);
    // showNotification(`${this.logPrefix} 📄 Response get-boss-status:`, res, 'info');

    if (res?.boss?.id) {
      this.currentBossId = res.boss.id;
      // console.log(`${this.logPrefix} 📄 Current boss id set to ${this.currentBossId}`);
      // showNotification(`${this.logPrefix} 📄 Current boss id set to ${this.currentBossId}`);
    } else {
      console.warn(`${this.logPrefix} ⚠️ Không tìm thấy boss id trong response`);
      showNotification(`${this.logPrefix} ⚠️ Không tìm thấy boss id trong response`, 'info');
    }
    return this.currentBossId;
  }

  async handleBossDefeated(bossId) {
    this.resetIfNewDay();
    if (this.handledBossIds.has(bossId)) return;
    this.handledBossIds.add(bossId);

    showNotification(`👤 Boss ${bossId} chết! Chuồn bị hiên t...`, "warn");

    // Delay ngẫu nhiên trước khi bắt đầu
    const initialDelay = Math.floor(Math.random() * 1000) + 2000;
    await new Promise(resolve => setTimeout(resolve, initialDelay));

    const nonce = await getNonce();
    if (!nonce) {
      console.error(`${this.logPrefix} ❌ Không lấy được nonce`);
      showNotification(`${this.logPrefix} ❌ Không lấy được nonce`, 'info');
      return;
    }

```

```

}

// Hàm gọi API với retry/backoff - retry tối đa 3 lần
const safeApiCall = async (endpoint, body = {}, retries = 3) => {
  for (let i = 0; i < retries; i++) {
    try {
      return await this.sendApiRequest(endpoint, "POST", nonce, body);
    } catch (err) {
      if (i < retries - 1) {
        const backoff = (i + 1) * 1000; // tăng dần: 1s, 2s, 3s
        console.warn(`${this.logPrefix} ⚠️ Lỗi khi gọi ${endpoint}, thử lại sau ${backoff}ms`);
        showNotification(`${this.logPrefix} ⚠️ Lỗi khi gọi ${endpoint}, thử lại sau ${backoff}ms`,
'info');

        await new Promise(r => setTimeout(r, backoff));
      } else {
        console.error(`${this.logPrefix} ❌ Gọi ${endpoint} thất bại sau ${retries} lần`);
        throw err;
      }
    }
  }
};

// Nhận thưởng
const reward = await safeApiCall("wp-json/tong-mon/v1/claim-boss-reward");
if (reward?.success) showNotification(reward.message, "success");

// Delay trước khi hiển thị
const contribDelay = Math.floor(Math.random() * 1000) + 2000;
await new Promise(resolve => setTimeout(resolve, contribDelay));

// Hiển thị
const contrib = await safeApiCall("wp-json/tong-mon/v1/contribute-boss");
if (contrib?.success) showNotification(contrib.message, "success");
}

// Check event
async processBossEvent(event, data) {
  if (!this.biCanhSocketActive || !data) return;
  const bossId = data.boss_id;

  if (bossId !== this.currentBossId) return;
  console.log(`[BiCanh] ${event} cho boss ${bossId}:`, data);
  if (event === "boss_tm_hp_update") {

    // Hiển thị rõ thông tin HP boss
    const { current_hp, max_hp, hp_percentage } = data;
    showNotification(`[BiCanh] Boss ${bossId}: HP ${current_hp}/${max_hp} (${Number(hp_percentage).toFixed(
2)})%`, 'info');

    // Cập nhật trực tiếp vào UI quest item Bí Canh
    window.bicanhBossHp = Number(hp_percentage);
    const bicanhProgressSpan = document.querySelector('.nv-quest-item[data-task-id="bicanh"] .quest-
progress');
    if (bicanhProgressSpan) {
      // Giữ nguyên text tỉ lệ g/c (ví dụ: " 5/5"), chỉ thay phần HP
      const baseText = bicanhProgressSpan.textContent.replace(/\\s*\\([^)]*%s*HP\\)/g, '').trimEnd();
      bicanhProgressSpan.textContent = baseText + ` (${Number(hp_percentage).toFixed(2)}% HP)`;
    }

    if (Number(current_hp) === 0) {
      await this.handleBossDefeated(bossId);
    }
  } else if (event === "boss_tm_defeated") {
    showNotification(`[BiCanh] Boss ${bossId} đã bị đánh bại!`, 'warn');
    await this.handleBossDefeated(bossId);
  } else {
    // Các event khác giữ nguyên
    showNotification(`[BiCanh] ${event} cho boss ${bossId}:`, data);
  }
}

async startBossSocketListener() {
  if (this.biCanhSocketActive) return;
  this.biCanhSocketActive = true;
  // console.log(`${this.logPrefix} ▶️ Bật đầu startBossSocketListener`);
  // showNotification(`${this.logPrefix} ▶️ Bật đầu startBossSocketListener`);
  if (this.biCanhSocketWaiter) clearInterval(this.biCanhSocketWaiter);
  this.biCanhSocketWaiter = setInterval(async () => {
    if (typeof socket !== "undefined" && typeof socket.on === "function") {
      clearInterval(this.biCanhSocketWaiter);
      this.biCanhSocketWaiter = null;

      // console.log(`${this.logPrefix} ❌ Socket ngừng, gọi updateCurrentBossId...`);
      // showNotification(`${this.logPrefix} ❌ Socket ngừng, gọi updateCurrentBossId...`, "info");
      await this.updateCurrentBossId();
      //showNotification(`${this.logPrefix} theo dõi bossId=${this.currentBossId}`, "info");
      if (typeof socket.off === "function") {
        socket.off("boss_tm_hp_update");
        socket.off("boss_tm_defeated");
      }
    }

    socket.on("boss_tm_hp_update", (data) => this.processBossEvent("boss_tm_hp_update", data));
    socket.on("boss_tm_defeated", (data) => this.processBossEvent("boss_tm_defeated", data));

    showNotification(`Đã kết nối socket Bí Canh (theo dõi bossId=${this.currentBossId}`, "info");
  }, 200);
}

stopBossSocketListener() {
  //showNotification(`${this.logPrefix} (bossId=${this.currentBossId}) trước khi clear`, 'info');
  this.biCanhSocketActive = false;
  this.handledBossIds.clear();
  // console.log(`${this.logPrefix} 🔄 Reset handledBossIds khi stop listener (bossId=${this.
currentBossId})`);
  // showNotification(`${this.logPrefix} 🔄 Reset handledBossIds khi stop listener (bossId=${this.
currentBossId})`, 'info');
  if (this.biCanhSocketWaiter) {
    clearInterval(this.biCanhSocketWaiter);
    this.biCanhSocketWaiter = null;
  }
  if (typeof socket !== "undefined" && typeof socket.off === "function") {

```

```

socket.off("boss_tm_hp_update");
socket.off("boss_tm_defeated");
}

showNotification(`${Socket listener Bí Cónh đã tụt (bossId=${this.currentBossId})`, "warn");
}
// =====
// HOANG VUC
// =====
class HoangVuc {
    constructor() {
        this.ajaxUrl = `${weburl}wp-content/themes/halimovies-child/hh3d-ajax.php`;
        this.adminAjaxUrl = `${weburl}wp-admin/admin-ajax.php`;
        this.logPrefix = "[HH3D Hoang Vuc]";
        this.headers = {
            "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
            "X-Requested-With": "XMLHttpRequest",
            "referer": weburl + 'hoang-vuc'
        };
        this._hvContext = null;
        this._hvLoadPromise = null;
        this._hvAttackPromise = null;
        this._hvPendingRequest = null;
        this._hvRunPromise = null;
    }

    hvString(value) {
        return typeof value === 'string' && value.trim() && !/^(\undefined|null)$/.test(value.trim())
            ? value.trim() : null;
    }

    hvLiteral(source, names) {
        source = String(source || '');
        for (const name of names) {
            const key = name.replace(/[\.*?^${}()|\\|\]/g, '\\$&');
            const re = new RegExp('(?:^[^\w$])(?:[\\\\"']?' + key + '[\\\\"']?)?(?:\s*\n)?\s*[:=]\s*(["\\\'])((?:\\.|(?!\n)[^\\n])*)\\1', 'g');
            let match;
            while ((match = re.exec(source))) {
                const value = match[2].replace(/\u{([da-f]{4})}|x{([da-f]{2})}|(\\\/'|"))/gi,
                    (_, u, x, escaped) => escaped || String.fromCharCode(parseInt(u || x, 16));
                if (this.hvString(value)) return value;
            }
        }
        return null;
    }

    hvObjectText(source, name) {
        source = String(source || '');
        const key = name.replace(/[\.*?^${}()|\\|\]/g, '\\$&');
        const match = new RegExp('(?:^[^\w$])[\\\\"']?' + key + '[\\\\"']?' + '(?:\s*\n)?\s*[:=]\s*[\\\\"']?.exec(
source);
        if (!match) return '';
        const start = match.index + match[0].lastIndexOf('{');
        let depth = 0, quote = '', escape = false;
        for (let i = start; i < source.length; i++) {
            const ch = source[i];
            if (quote) {
                if (escape) escape = false;
                else if (ch === '\\') escape = true;
                else if (ch === quote) quote = '';
            } else if (ch === '"' || ch === "'") quote = ch;
            else if (ch === '{') depth++;
            else if (ch === '}') && --depth === 0) return source.slice(start, i + 1);
        }
        return '';
    }

    parseHoangVucPage(html) {
        const doc = new DOMParser().parseFromString(String(html || ''), 'text/html');
        const scripts = Array.from(doc.querySelectorAll('script'), node => node.textContent || '').join('\n');
        const hh3dText = this.hvObjectText(scripts, 'hh3dData');
        const currentBossText = this.hvObjectText(scripts, 'HH3DBossConfig');
        const bossText = currentBossText || this.hvObjectText(scripts, 'hoangVucData');
        const actText = this.hvObjectText(hh3dText, 'act') || this.hvObjectText(bossText, 'act') || this.hvObjectText(scripts, 'hh3dData.act');
        const act = Object.create(null);
        for (const key of ['bossGet', 'bossAttack', 'bossTimer', 'bossHistory', 'bossBuy', 'bossBalance', 'bossChangeElem', 'bossCheckElem', 'bossOpenChest']) {
            const value = this.hvLiteral(actText, [key]);
            if (value) act[key] = value;
        }

        // v7.4.34: ưu tiên nonce/attackToken từ HH3DBossConfig do chính trang Hoang Vuc cõp.
        let nonce = this.hvLiteral(currentBossText, ['nonce']);
        nonce ||= this.hvLiteral(scripts, ['ajax_boss_nonce', 'boss_nonce', 'bossNonce']);
        nonce ||= this.hvLiteral(this.hvObjectText(hh3dText, 'nonces'), ['boss', 'bossNonce']);
        nonce ||= this.hvLiteral(bossText, ['nonce']);
        if (!nonce && (act.bossGet || act.bossAttack)) nonce = this.hvLiteral(hh3dText, ['nonce']);
        nonce ||= this.hvString(doc.querySelector('input[name="ajax_boss_nonce"], input[name="boss_nonce"]')?.
value);

        const securityToken = this.hvLiteral(hh3dText, ['securityToken', 'security_token'])
            || this.hvLiteral(bossText, ['securityToken', 'security_token'])
            || this.hvLiteral(scripts, ['securityToken', 'security_token']);
        const attackToken = this.hvLiteral(currentBossText, ['attackToken'])
            || this.hvLiteral(scripts, ['boss_attack_token', 'attack_token', 'attackToken']);

        const countText = doc.querySelector('.remaining-attacks')?.textContent || '';
        const count = countText.match(/\d+/);
        const elementNode = doc.elementSelector('#user-nguanh-image');
        const elementSrc = elementNode?.getAttribute('src') || elementNode?.getAttribute('data-src') || '';
        const element = elementSrc.match(/ngu-hanh/(moc|thu|haoj|tho|kim)\.(gif|png|webp)/i);

        // Main.js vñn đã có bộ giõi mã hh3dData; tận dụng nó đợ bõ sung action map nũu trang mã hóa act.
        try {
            const decoded = parseHh3dData(String(html || '')) || {};
            if (decoded.securityToken && !securityToken) this._hvDecodedSecurityToken = decoded.securityToken;
            if (decoded.act && typeof decoded.act === 'object') {
                for (const [key, value] of Object.entries(decoded.act)) {
                    if (this.hvString(value) && !act[key]) act[key] = value;
                }
            }
        } catch {}
    }
}

```

```

        return {
            nonce,
            securityToken: securityToken || this._hvDecodedSecurityToken || null,
            attackToken,
            act,
            ajaxUrl: this.hvLiteral(currentBossText, ['ajaxUrl']),
            adminAjaxUrl: this.hvLiteral(currentBossText, ['adminAjaxUrl']),
            remainingAttacks: count ? Number(count[0]) : null,
            myElement: element ? element[1].toLowerCase() : null,
            loadedAt: Date.now()
        };
    }

    readHoangVucLiveData() {
        if (!/^\/hoang-vuc\/?$/i.test(location.pathname)) return null;
        try {
            const page = typeof unsafeWindow !== 'undefined' ? unsafeWindow : window;
            const config = page.hh3dData || {};
            const currentBoss = page.HH3DBossConfig || {};
            const boss = page.hh3dBossData || page.hoangVucData || {};
            const data = this.parseHoangVucPage(document.documentElement.outerHTML);
            data.nonce = this.hvString(currentBoss.nonce) || this.hvString(page.ajax_boss_nonce) || this.hvString(
config.bossNonce)
                || this.hvString(config.nonces?.boss) || this.hvString(boss.nonce) || data.nonce;
            data.securityToken = this.hvString(config.securityToken) || this.hvString(boss.securityToken) || data.
securityToken;
            data.attackToken = this.hvString(currentBoss.attackToken) || this.hvString(page.boss_attack_token)
                || this.hvString(boss.attack_token) || data.attackToken;
            data.ajaxUrl = this.hvString(currentBoss.ajaxUrl) || data.ajaxUrl;
            data.adminAjaxUrl = this.hvString(currentBoss.adminAjaxUrl) || data.adminAjaxUrl;
            for (const key of ['bossGet', 'bossAttack', 'bossTimer', 'bossHistory', 'bossBuy', 'bossBalance',
'bossChangeElem', 'bossCheckElem', 'bossOpenChest']) {
                const value = this.hvString(config.act?.[key]) || this.hvString(boss.act?.[key]);
                if (value) data.act[key] = value;
            }
            return data.nonce ? data : null;
        } catch (_) {
            return null;
        }
    }

    async getHoangVucData(pageUrl = weburl + 'hoang-vuc', force = false) {
        if (this._hvLoadPromise) return this._hvLoadPromise;
        if (!force && this._hvContext && (this._hvAttackPromise || Date.now() - this._hvContext.loadedAt < 15000))
return this._hvContext;

        const load = async () => {
            const base = new URL(weburl);
            const url = new URL(pageUrl, weburl);
            if (url.origin !== base.origin) throw new Error('Hoang Vuc: địa chỉ khác miền đang mở.');
            url.searchParams.set('_hh3d_hv', String(Date.now()));
            const response = await fetch(url.href, { credentials: 'include', cache: 'no-store' });
            if (!response.ok) throw new Error('Hoang Vuc: tài trạng thất bại (HTTP ${response.status}).');
            if (response.url && !/^\/hoang-vuc\/?$/i.test(new URL(response.url, weburl).pathname)) {
                throw new Error('Hoang Vuc: trang bị chuyển hướng. Hãy kiểm tra đăng nhập/quyển vào Hoang Vuc.');
            }
            let data = this.parseHoangVucPage(await response.text());
            if (!data.nonce || !data.securityToken || !data.attackToken || !Object.keys(data.act).length) &&
!force) {
                data = this.readHoangVucLiveData() || data;
            }
            if (!data.nonce) throw new Error('Hoang Vuc: không tìm thấy nonce boss trong trang trỏ v');

            for (const key of ['ajaxUrl', 'adminAjaxUrl']) {
                if (!this.hvString(data[key])) continue;
                const endpoint = new URL(data[key], weburl);
                if (endpoint.origin !== base.origin) throw new Error('Hoang Vuc: API khác miền đang mở.');
                this[key] = endpoint.href;
            }
            this._hvContext = data;
            hData = hData || {};
            hData.act = { ...(hData.act || {}), ...data.act };
            hData.attackToken = data.attackToken;
            return data;
        };

        this._hvLoadPromise = load();
        try { return await this._hvLoadPromise; }
        catch (error) { this._hvContext = null; throw error; }
        finally { this._hvLoadPromise = null; }
    }

    getHoangVucAction(key, legacy = null) {
        const context = this._hvContext;
        if (this.hvString(context?.act?.[key])) return context.act[key];
        if (key === 'bossChangeElem' && this.hvString(context?.act?.bossCheckElem)) return context.act.
bossCheckElem;
        if (key === 'bossCheckElem' && this.hvString(context?.act?.bossChangeElem)) return context.act.
bossChangeElem;
        if (context && Object.keys(context.act || {}).length && ![ 'bossChangeElem', 'bossCheckElem',
'bossOpenChest' ].includes(key)) {
            throw new Error('Hoang Vuc: trang thiêu mã action ${key}. Hãy tải lại trang.');
```

```

const context = this._hvContext;
if (context && this.hvString(data?.attack_token)) context.attackToken = data.attack_token;
else if (context) context.attackToken = null;
let remaining = data?.remaining_attacks === null || data?.remaining_attacks === undefined || data?.
remaining_attacks === ''
  ? NaN : Number(data.remaining_attacks);
if (!Number.isInteger(remaining) && Number.isInteger(context?.remainingAttacks) && context.remainingAttacks
> 0) {
  remaining = context.remainingAttacks - 1;
}
if (Number.isInteger(remaining) && remaining >= 0 && context) context.remainingAttacks = remaining;
if (hData) hData.attackToken = context?.attackToken || null;
return remaining;
}

hvWatchAttack(requestId) {
  let socket;
  try { socket = (typeof unsafeWindow !== 'undefined' ? unsafeWindow : window).socket; } catch (_) {}
  if (!socket || typeof socket.on !== 'function' || typeof socket.off !== 'function') {
    return {
      wait: async () => ({ unconfirmed: true, error: 'Hoang Vuc: máy chủ đang xử lý lượt đánh nhưng thiêu
kết nối nhận kết quả. Kiểm tra lượt đánh rồi tiếp lại trang.' }),
      cancel() {}
    };
  }
  let finish, timer = null, settled = false;
  const event = 'attack_result_' + requestId;
  const result = data => {
    if (!data || (data.request_id && data.request_id !== requestId)) return;
    finish({ data });
  };
  const fallback = data => { if (data?.request_id === requestId) result(data); };
  const failure = data => {
    if (data?.request_id === requestId) finish({ error: this.hvString(data.error) || 'Hoang Vuc: xử lý lượt
đánh thất bại.' });
  };
  const promise = new Promise(resolve => {
    finish = value => {
      if (settled) return;
      settled = true;
      if (timer !== null) clearTimeout(timer);
      socket.off(event, result);
      socket.off('attack_result_fallback', fallback);
      socket.off('attack_boss_error', failure);
      resolve(value);
    };
  });
  socket.on(event, result);
  socket.on('attack_result_fallback', fallback);
  socket.on('attack_boss_error', failure);
  return {
    wait: () => {
      if (!settled && timer === null) timer = setTimeout(() => finish({
        unconfirmed: true,
        error: 'Hoang Vuc: chưa nhận kết quả sau 20 giây. Kiểm tra lượt đánh rồi tiếp lại trang; Auto
chưa gửi lại lệnh.'
      }), 20000);
      return promise;
    },
    cancel: () => finish({ cancelled: true })
  };
}

async getMyElement() {
  const context = await this.getHoangVucData(weburl + 'hoang-vuc');
  console.log(`${this.logPrefix} [] attackToken: ${context.attackToken ? 'OK' : 'THIẾU'}`);
  return { element: context.myElement, attackToken: context.attackToken };
}

getTargetElement(bossElement, maximizeDamage) {
  const rules = {
    'kim': { khac: 'moc', bi_khac: 'hoa' },
    'moc': { khac: 'tho', bi_khac: 'kim' },
    'thuy': { khac: 'hoa', bi_khac: 'tho' },
    'hoa': { khac: 'kim', bi_khac: 'thuy' },
    'tho': { khac: 'thuy', bi_khac: 'moc' },
  };
  const suitableElements = [];
  if (maximizeDamage) {
    for (const myElement in rules) {
      if (rules[myElement].khac === bossElement) { suitableElements.push(myElement); break; }
    }
  } else {
    for (const myElement in rules) {
      if (rules[myElement].bi_khac !== bossElement) suitableElements.push(myElement);
    }
  }
  return suitableElements;
}

async claimHoangVucRewards(nonce) {
  const context = await this.getHoangVucData(weburl + 'hoang-vuc');
  const action = this.getHoangVucAction('bossOpenChest', 'claim_chest');
  const payload = new URLSearchParams({ action, nonce: context.nonce || nonce || '' });
  console.log(`${this.logPrefix} [] Đang nhận thưởng...`);
  const response = await fetch(this.adminAjaxUrl, {
    method: 'POST', headers: this.headers, body: payload, credentials: 'include', cache: 'no-store'
  });
  const data = await response.json();
  if (data.success) {
    const rewards = data.total_rewards || data.data?.total_rewards || {};
    const message = `[] Nhận thưởng thành công: +${rewards.tinh_thach ?? 0} Tinh Thạch, +${rewards.tu_vi ??
0} Tu Vi.`;
    console.log(message);
    showNotification(message, 'success');
    this._hvContext = null;
    return true;
  }
  const message = this.hvApiError(data);
  console.error(`${this.logPrefix} [] Lỗi khi nhận thưởng:`, message);
  showNotification(message, 'error');
  return false;
}

async attackHoangVucBoss(bossId, _legacyNonce) {
  if (this._hvAttackPromise) return this._hvAttackPromise;

```

```

const run = async () => {
  try {
    if (this._hvPendingRequest) {
      showNotification('Hoang Vuc: lượt đánh trước chưa có kết quả xác nhận. Kiểm tra lượt đánh rồi lại trang.', 'info');
      return false;
    }
    for (let attempt = 0; attempt < 2; attempt++) {
      let context = await this.getHoangVucData(weburl + 'hoang-vuc', attempt > 0);
      if (!context.securityToken || !context.attackToken) {
        if (attempt === 0) context = await this.getHoangVucData(weburl + 'hoang-vuc', true);
        if (!context.securityToken || !context.attackToken) {
          throw new Error('Hoang Vuc: thiếu security_token hoặc attack_token; chưa gửi lệnh đánh.');
        }
      }
      if (!/^\\d+$/.test(String(bossId))) throw new Error('Hoang Vuc: boss_id không hợp lệ.');
```

```

const payload = new URLSearchParams({
  action: this.getHoangVucAction('bossAttack', 'attack_boss'),
  boss_id: String(bossId),
  security_token: context.securityToken,
  nonce: context.nonce,
  attack_token: context.attackToken,
  request_id: 'req_' + Math.random().toString(36).slice(2, 10) + '_' + Date.now()
});
const requestId = payload.get('request_id');
const watcher = this.hvWatchAttack(requestId);
console.log(`${this.logPrefix} [T]n công boss ${bossId} (nonce/token lấy từ HH3DBossConfig).`);

try {
  const response = await fetch(this.ajaxUrl, {
    method: 'POST', headers: this.headers, body: payload,
    credentials: 'include', cache: 'no-store'
  });
  let result;
  try { result = await response.json(); }
  catch (_) { throw new Error('Hoang Vuc: phản hồi không phải JSON. Chưa xác định kết quả lượt đánh.');
```

```

  if (this.hvString(result?.data?.attack_token)) context.attackToken = result.data.

  if (response.ok && result?.success === true) {
    let data = result.data || {};
    if (data.status === 'processing') {
      this._hvPendingRequest = requestId;
      const outcome = await watcher.wait();
      if (outcome.unconfirmed) throw new Error(outcome.error);
      this._hvPendingRequest = null;
      if (outcome.error) throw new Error(outcome.error);
      data = { ...outcome.data, attack_token: this.hvString(outcome.data?.attack_token) };
    }
    if (data.damage == null || !Number.isFinite(Number(data.damage))) {
      throw new Error('Hoang Vuc: phản hồi thiếu kết quả sát thương; chưa xác nhận đánh thành công.');
```

```

    }
    const remaining = this.hvUpdateAttackResult(data);
    const done = Number.isInteger(remaining) ? (5 - remaining) : '?';
    showNotification(`Gây ${data.damage} sát thương. Số lượt: ${done}/5`, 'success');
    if (remaining === 0) taskTracker.markTaskDone(accountId, 'hoangvuc');
    return true;
  }
  if (attempt === 0 && this.hvAuthRejected(result)) {
    this._hvContext = null;
    continue;
  }
  const message = this.hvApiError(result);
  if (/hết lượt tấn công trong ngày/i.test(message)) {
    this.hvUpdateAttackResult({ remaining_attacks: 0 });
    taskTracker.markTaskDone(accountId, 'hoangvuc');
    showNotification(message, 'info');
    return true;
  }
  throw new Error(message);
} finally {
  watcher.cancel();
}
}
return false;
} catch (error) {
  this._hvContext = null;
  this._hvPendingRequest = null;
  console.error(`${this.logPrefix} [E]`, error);
  showNotification(error.message || 'Hoang Vuc: lỗi kết nối; hãy kiểm tra kết quả trước khi đánh lại.', 'error');
```

```

  return false;
}
}
this._hvAttackPromise = run();
try { return await this._hvAttackPromise; }
finally { this._hvAttackPromise = null; }
}

async changeElementUntilSuitable(currentElement, bossElement, maximizeDamage, nonce) {
  let myElement = currentElement;
  if (!myElement || !['kim', 'moc', 'thuy', 'hoa', 'tho'].includes(myElement)) return myElement;
  let changeAttempts = 0;
  const MAX_ATTEMPTS = 5;
  const rules = {
    'kim': { khác: 'moc', bi_khac: 'hoa' },
    'moc': { khác: 'tho', bi_khac: 'kim' },
    'thuy': { khác: 'hoa', bi_khac: 'tho' },
    'hoa': { khác: 'kim', bi_khac: 'thuy' },
    'tho': { khác: 'thuy', bi_khac: 'moc' },
  };
  const isOptimal = el => rules[el]?.khac === bossElement;
  const isNeutral = el => rules[el]?.bi_khac !== bossElement;

  while (changeAttempts < MAX_ATTEMPTS) {
    changeAttempts++;
    if (isNeutral(myElement)) {
      if (maximizeDamage && isOptimal(myElement)) return myElement;
      if (!maximizeDamage) return myElement;
    }
    const context = await this.getHoangVucData(weburl + 'hoang-vuc');
    const action = this.getHoangVucAction('bossChangeElem', this.getHoangVucAction('bossCheckElem',

```

```

'change_user_element'));
    const payloadChange = new URLSearchParams({ action: context.nonce || nonce || '' });
    const changeData = await (await fetch(this.ajaxUrl, {
        method: 'POST', headers: this.headers, body: payloadChange, credentials: 'include', cache: 'no-
store'
    })).json();
    if (changeData.success) {
        myElement = changeData.data?.new_element || myElement;
        if (this._hvContext) this._hvContext.myElement = myElement;
        console.log(`${this.logPrefix} [L] khi [L] ${changeAttempts} -> ${myElement}`);
        await new Promise(resolve => setTimeout(resolve, 500));
    } else {
        console.error(`${this.logPrefix} [L] khi [L] i:`, this.hvApiError(changeData));
        return myElement;
    }
}
return myElement;
}

async getNonceAndRemainingAttacks(url) {
    try {
        const context = await this.getHoangVucData(url || weburL + 'hoang-vuc');
        return { attackToken: context.attackToken, remainingAttacks: context.remainingAttacks, nonce: context.
nonce };
    } catch (e) {
        console.error(`${this.logPrefix} [L] khi [L] y dữ liệu Hoang Vuc:`, e);
        return { attackToken: null, remainingAttacks: null, nonce: null };
    }
}

async doHoangVucOnce(_rewardReload = false) {
    const maximizeDamage = localStorage.getItem('hoangvucMaximizeDamage') === 'true';
    console.log(`${this.logPrefix} [L] B[ ]t [L]u nhiệm vụ với chỉ[ ]n lược: ${maximizeDamage ? 'T[ ]i đa hóa Sát
thương' : 'Không gi[ ]m Sát thương'}`);
    const context = await this.getHoangVucData(weburL + 'hoang-vuc');
    const remainingAttacks = context.remainingAttacks;
    const nonce = context.nonce;

    if (remainingAttacks === 0) {
        taskTracker.markTaskDone(accountId, 'hoangvuc');
        return true;
    }

    const payloadBossInfo = new URLSearchParams({
        action: this.getHoangVucAction('bossGet', 'get_boss'),
        nonce
    });
    const bossInfoResponse = await fetch(this.ajaxUrl, {
        method: 'POST', headers: this.headers, body: payloadBossInfo,
        credentials: 'include', cache: 'no-store'
    });
    const bossInfoData = await bossInfoResponse.json();
    if (!bossInfoData.success) throw new Error(this.hvApiError(bossInfoData));

    const boss = bossInfoData.data;
    if (_rewardReload && boss.has_pending_rewards) {
        throw new Error('Hoang Vuc: nhận thư[ ]ng chưa hoàn t[ ]t, đã dừng [L] tránh lặp yêu c[ ]u.');
```

```

        try { return await this._hvRunPromise; }
        catch (error) {
            console.error(`${this.logPrefix} `, error);
            showNotification(error.message || 'Hoàng Vực: không t[i được dữ liệu.', 'error');
            return false;
        } finally {
            this._hvRunPromise = null;
        }
    }
}

// =====
// HOANG VUC SHOP
// =====

class HoangVucShop {
    constructor() {
        this.ajaxUrl = `${weburl}wp-content/themes/halimmovies-child/hh3d-ajax.php`;
        this.logPrefix = "[HH3D Hoàng Vuc Shop]";
        this.dailyLimit = 5;
    }

    async getHoangVucNonce() {
        try {
            // Dùng cùng ngu[ן nonce v7.4.34 với module Hoàng Vực chính.
            const context = await hoangvuc.getHoangVucData(`${weburl}hoang-vuc`);
            return context?.nonce || null;
        } catch (err) {
            console.error(`${this.logPrefix} [ L[i khi l[yy nonce:`, err);
            return null;
        }
    }

    getTodayKey() {
        return new Date().toLocaleDateString('en-CA', { timeZone: 'Asia/Ho_Chi_Minh' });
    }

    resetPurchase() {
        const todayKey = this.getTodayKey();
        localStorage.setItem('hoangvuc-purchase', JSON.stringify({
            date: todayKey,
            count: 0,
            lastPurchaseTime: null
        }));
    }

    getPurchasedToday() {
        const saved = JSON.parse(localStorage.getItem('hoangvuc-purchase') || '{}');
        const todayKey = this.getTodayKey();

        if (!saved.date || saved.date !== todayKey) {
            this.resetPurchase();
            return 0;
        }
        return saved.count || 0;
    }

    setPurchasedToday(count) {
        const todayKey = this.getTodayKey();
        const now = new Date().toLocaleString('en-US', { timeZone: 'Asia/Ho_Chi_Minh' });
        localStorage.setItem('hoangvuc-purchase', JSON.stringify({
            date: todayKey,
            count,
            lastPurchaseTime: now
        }));
    }

    isDailyLimit() {
        return this.getPurchasedToday() >= this.dailyLimit;
    }

    async muaRuongLinhBao(quantity) {
        const nonce = await this.getHoangVucNonce();
        if (!nonce) {
            showNotification(" [ Không l[yy được nonce, vui lòng đăng nhập lại", "error");
            return null;
        }

        let purchased = this.getPurchasedToday();
        if (purchased >= this.dailyLimit) {
            showNotification(" [ Bạn đã mua đ[ 5 rương hôm nay", "warn");
            return null;
        }

        let allowedQuantity = quantity;
        if (purchased + quantity > this.dailyLimit) {
            allowedQuantity = this.dailyLimit - purchased;
            showNotification(` [ Ch[ có th[ mua thêm ${allowedQuantity} rương đ[ đạt t[i đa 5/ngà[`, "warn");
        }

        try {
            const body = new URLSearchParams({
                action: `${(typeof hData !== 'undefined' && hData.act) ? hData.act : 'bossBuy'}` +
                    "purchase_item_shop_boss",
                item_id: "ruong_linh_bao",
                item_type: "tinh_thach",
                quantity: allowedQuantity,
                nonce: nonce
            });

            const response = await fetch(this.ajaxUrl, {
                method: "POST",
                body,
                credentials: "include"
            });

            const res = await response.json();
            console.log(`${this.logPrefix} [ Response:`, res);

            if (res.success) {
                purchased += allowedQuantity;
                this.setPurchasedToday(purchased);
                showNotification(` [ ${res.data.message}`, "success");
            } else {

```

```

        showNotification(`⚠️ ${res.data}`, "warn");
    }
    return res;
} catch (err) {
    console.error(`${this.logPrefix} ❌ L[i] khi mua rương:`, err);
    showNotification(`❌ L[i] khi g[i] yêu c[u] mua rương`, "error");
    return null;
}
}
}

// =====
// LUYỆN ĐẠN ĐƯỜNG
// =====
class LuyenDan {
    constructor() {
        this.weburl = weburl;
        this.logPrefix = '[HH3D Luyện Đan]';
        this.luyenDanToken = null;
        this.luyenDanTokenExpires = 0;
        this.inviteSentTime = null;
    }

    updateProgress(text) {
        try {
            const accountId = localStorage.getItem('hh3d_account_id') || '';
            const localStorageKey = `luyenDanLastProgress_${accountId}`;
            if (text) {
                localStorage.setItem(localStorageKey, text);
            } else {
                localStorage.removeItem(localStorageKey);
            }
            const span = document.querySelector('.nv-quest-item[data-task-id="luyenDan"] .quest-progress');
            if (span) {
                span.textContent = text ? ` (${text})` : '';
            }
        } catch (e) {
            console.error(`${this.logPrefix} ❌ khi cập nhật UI progress:`, e);
        }
    }

    masterLevelCurve(m) {
        m = m || {};
        const maxLevel = Math.max(1, m.max_level | 0 || 10);
        const perLevel = {};
        const steps = m.level_xp && typeof m.level_xp === 'object';
        let l;
        if (steps) {
            for (l = 1; l < maxLevel; l++) {
                const sk = String(l);
                const need = m.level_xp[sk] != null ? m.level_xp[sk] : m.level_xp[l];
                perLevel[l] = Math.max(1, need | 0);
            }
        } else {
            const flat = Math.max(1, m.xp_per_level | 0 || 1000);
            for (l = 1; l < maxLevel; l++) perLevel[l] = flat;
        }
        const floors = { 1: 0 };
        let acc = 0;
        for (l = 1; l < maxLevel; l++) {
            acc += perLevel[l] || 1000;
            floors[l + 1] = acc;
        }
        return { max_level: maxLevel, per_level: perLevel, floors: floors };
    }

    computeDanRank(pts, m) {
        const curve = this.masterLevelCurve(m);
        const maxLevel = curve.max_level;
        const floors = curve.floors;
        const perLevel = curve.per_level;
        const xp = Math.max(0, pts | 0);
        let level = 1;
        for (let lv = maxLevel; lv >= 1; lv--) {
            if (xp >= (floors[lv] | 0)) {
                level = lv;
                break;
            }
        }
        const isMax = level >= maxLevel;
        const floor = floors[level] | 0;
        let xpIn, per, pct, xpToNext;
        if (isMax) {
            per = Math.max(1, perLevel[maxLevel - 1] | 0 || 1);
            xpIn = Math.max(0, xp - floor);
            pct = 100;
            xpToNext = 0;
        } else {
            const next = floors[level + 1] | 0;
            per = Math.max(1, next - floor);
            xpIn = xp - floor;
            pct = Math.min(100, Math.max(0, (xpIn / per) * 100));
            xpToNext = Math.max(0, next - xp);
        }

        let levelName = 'Luyện Đan Sư · Bậc ' + level;
        if (m && m.level_names && m.level_names[String(level)]) {
            levelName = String(m.level_names[String(level)]).trim();
        } else if (m && m.level_meta && m.level_meta[String(level)] && m.level_meta[String(level)].display) {
            levelName = String(m.level_meta[String(level)].display).trim();
        }

        return {
            level: level,
            xp_total: xp,
            xp_in_level: xpIn,
            xp_per_level: per,
            xp_to_next: xpToNext,
            pct: pct,
            level_name: levelName,
            is_max: isMax
        };
    }
}

updateAlchemistUI(rankXp, danMaster) {

```

```

    try {
      const questItem = document.querySelector('.nv-quest-item[data-task-id="luyenDan"]');
      if (questItem) {
        const infoDiv = questItem.querySelector('.quest-alchemist-info');
        if (infoDiv) infoDiv.remove();
      }
    } catch (e) { }
  }
}

async getNonce() {
  const nonce = await getNonce();
  return nonce || null;
}

async ensureLuyenDanToken() {
  const now = Math.floor(Date.now() / 1000);
  if (this.luyenDanToken && this.luyenDanTokenExpires > now + 30) {
    return this.luyenDanToken;
  }

  const wpNonce = await this.getNonce();
  if (!wpNonce) {
    console.error(`${this.logPrefix} ❌ Không có WP rest nonce`);
    return null;
  }

  try {
    const url = `${this.weburl}wp-json/hh3d/v1/luyen-dan/session-token`;
    const response = await fetch(url, {
      method: "GET",
      headers: {
        "X-WP-Nonce": wpNonce
      },
      credentials: "include"
    });
    const res = await response.json();
    if (res?.data?.security_token) {
      this.luyenDanToken = res.data.security_token;
      this.luyenDanTokenExpires = res.data.expires_at || (now + 1800);
      return this.luyenDanToken;
    } else {
      console.error(`${this.logPrefix} ❌ Không thể lấy session token`);
      return null;
    }
  } catch (e) {
    console.error(`${this.logPrefix} ❌ Lỗi lấy session token:`, e);
    return null;
  }
}

async sendLdRequest(path, method = "GET", body = null) {
  const token = await this.ensureLuyenDanToken();
  if (!token) {
    throw new Error("Không có Luyện Đan session token");
  }
  const wpNonce = await this.getNonce();
  const url = `${this.weburl}wp-json/hh3d/v1/luyen-dan${path}`;
  const headers = {
    "X-WP-Nonce": wpNonce,
    "X-LD-Token": token,
    "Accept": "application/json"
  };
  if (body) {
    headers["Content-Type"] = "application/json";
  }
  const response = await fetch(url, {
    method,
    headers,
    body: body ? JSON.stringify(body) : undefined,
    credentials: "include"
  });

  let res;
  try {
    res = await response.json();
  } catch (err) {
    res = { code: "parse_error", message: "Phản hồi không phải JSON" };
  }
  if (!response.ok || res.code) {
    return { error: true, code: res.code || "http_error", message: res.message || `Lỗi HTTP ${response.status}` };
  }
  return res;
}

async doLuyenDan(isManual = false) {
  // Mutex guard: tránh 2 luồng chạy đồng thời (click thẻ công + scheduleTask cùng lúc)
  if (this.isProcessing) {
    console.warn(`${this.logPrefix} Đang xử lý, bỏ qua lần gọi trùng lặp.`);
    return 10000;
  }
  this.isProcessing = true;
  try {
    // Dùng accountId toàn cục
    const autoLuyenDan = localStorage.getItem('autoLuyenDan') !== '0';
    const stateRes = await this.sendLdRequest("/state?fresh=1", "GET");
    if (!stateRes || !stateRes.data) {
      countdownTimer.remove('luyenDan');
      showNotification("❌ Không lấy được trạng thái Lò Đan", "warning");
      this.updateProgress("Lỗi kết nối");
      return 15000;
    }
  }

  const data = stateRes.data;

  // Cập nhật thông tin cấp bậc Đan Sư lên UI
  this.updateAlchemistUI(data.rank_xp, data.dan_master || data.danMaster);

  // 1. Tự động nhận lời mời làm Đan Đồng (Auto-Accept)
  const autoAccept = localStorage.getItem('luyenDanAutoAcceptInvite') === 'true';
  if (autoAccept && data.dong_invites_in && data.dong_invites_in.length > 0) {
    const acceptAll = localStorage.getItem('luyenDanAcceptAllInvites') === 'true';
    const selectedIds = (localStorage.getItem('luyenDanSelectedFriendIds') || '').split(',').filter(
      Boolean;

      for (const inv of data.dong_invites_in) {

```

```

const oid = String(inv.owner_id);
if (acceptAll || selectedIds.includes(oid)) {
  console.log(`${this.logPrefix} Tự động chấp nhận lời mời Đan Đẳng từ Đan Ch ${inv.
owner_name || oid}...`);
  this.updateProgress(`Nhận Đan Đẳng ${inv.owner_name || oid}`);
  try {
    const res = await this.sendLdRequest("/dong/respond", "POST", { owner_id: inv.owner_id,
accept: true });
    if (res && !res.error) {
      showNotification(`Đã tự động nhận làm Đan Đẳng cho Đan Ch: ${inv.owner_name ||
oid}`, "success");
      return 3000;
    }
  } catch (err) {
    console.error(`${this.logPrefix} Lỗi khi chấp nhận lời mời Đan Đẳng:`, err);
  }
}
}

// 2. Tự động rời Đan Đẳng sau 5 phút (Auto-Leave)
const autoLeave = localStorage.getItem('luyenDanAutoLeave') === 'true';
const furnaceState = data.furnace || 'idle';
if ((autoLeave || isManual) && data.dong_serving) {
  const serving = data.dong_serving;
  const oid = serving.owner_id || 0;
  const unstableLeftSec = data.craft ? (data.craft.unstable_left_sec || 0) : 0;

  let canLeave = false;
  if (furnaceState === 'exploded') {
    canLeave = true;
  } else if (unstableLeftSec <= 0) {
    const dongOwnersForMe = data.dong_owners_for_me || [];
    const row = dongOwnersForMe.find(o => (o.owner_id || 0) === oid);
    if (row) {
      canLeave = !!row.can_leave;
    } else {
      canLeave = furnaceState === 'crafting' || furnaceState === 'ready' || furnaceState ===
'idle';
    }
  }

  if (canLeave) {
    console.log(`${this.logPrefix} Tự động rời vai Đan Đẳng của Đan Ch ${serving.owner_name ||
oid}...`);
    this.updateProgress(`Rời Đan Đẳng ${serving.owner_name || oid}`);
    try {
      const res = await this.sendLdRequest("/dong/leave", "POST", { owner_id: oid });
      if (res && !res.error) {
        showNotification(`Đã tự động rời vai Đan Đẳng của Đan Ch ${serving.owner_name ||
oid}`, "success");
        return 3000;
      }
    } catch (err) {
      console.error(`${this.logPrefix} Lỗi khi rời vai Đan Đẳng:`, err);
    }
  } else if (isManual) {
    if (unstableLeftSec > 0) {
      showNotification(`Chưa th rời vai Đan Đẳng. Giai đoạn nhảy côm còn lại: ${
unstableLeftSec} giây.`, "warning");
    } else {
      showNotification(`Chưa th rời vai Đan Đẳng (chờ máy cập nhật trạng thái).`,
"warning");
    }
  }
}

// Tự động quét và phân giói đan được phm chít kém trong túi
const autoDecompose = localStorage.getItem('luyenDanAutoDecompose') === 'true';
const minStars = parseInt(localStorage.getItem('luyenDanMinStars') || '4', 10);
if (autoLuyenDan && autoDecompose) {
  let stacks = [];
  if (data.pill_stacks && data.pill_stacks.length > 0) {
    stacks = data.pill_stacks.map(s => {
      const parsedStars = parseInt(s.stars || s.star || 0, 10);
      return {
        tier: s.tier,
        stars: parsedStars,
        count: parseInt(s.count || 0, 10),
        stack_id: s.stack_id || `${s.tier}:${parsedStars}`
      };
    });
  } else if (data.pills && data.pills.length > 0) {
    const map = {};
    data.pills.forEach(p => {
      const stars = parseInt(p.stars || p.star || 0, 10);
      const sid = p.id || `${p.tier}:${stars}`;
      const key = `${p.tier}-${stars}`;
      if (!map[key]) {
        map[key] = { tier: p.tier, stars, count: 0, stack_id: sid };
      }
      map[key].count++;
    });
    stacks = Object.keys(map).map(k => map[k]);
  }

  // Quét DOM fallback nếu dữ liệu API trống
  if (stacks.length === 0 && typeof document !== 'undefined') {
    const pillCells = document.querySelectorAll('.ld-cell--pill');
    if (pillCells.length > 0) {
      console.log(`${this.logPrefix} Quét đan được từ giao diện hiển thị (DOM fallback)...`);
      pillCells.forEach(cell => {
        const tier = cell.dataset.tier;
        const stars = parseInt(cell.dataset.stars || 0, 10);
        const qtyEl = cell.querySelector('.ld-qty');
        const count = qtyEl ? parseInt(qtyEl.textContent || 1, 10) : 1;
        if (tier && stars < minStars) {
          stacks.push({
            tier,
            stars,
            count,
            stack_id: `${tier}:${stars}`,
            isDomFallback: true,
            cell: cell
          });
        }
      });
    }
  }
}

```

```

    });
  }
}

const lowStarStacks = stacks.filter(s => s.stars < minStars && s.count > 0);
if (lowStarStacks.length > 0) {
  console.log(`${this.logPrefix} Phát hiện đơn được phò m chít kém trong túi:`, lowStarStacks);
  for (const stack of lowStarStacks) {
    for (let i = 0; i < stack.count; i++) {
      const pid = stack.stack_id;
      showNotification(`❌ Tự động phân gi❌i đơn trong túi: ${stack.tier.toUpperCase()} ${
stack.stars}* (C❌n >= ${minStars})*`, "info");
      this.updateProgress(`Phân gi❌i ${stack.stars}*...`);
      try {
        const decompRes = await this.sendLdRequest("/decompose", "POST", { pill_id: String(
pid) });
        if (decompRes && !decompRes.error) {
          showNotification(`✅ Đã phân gi❌i thành công đơn ${stack.stars}*`, "success");

          // Cập nhật DOM ngay lập tức đ❌i đ❌ng bộ và tránh lặp lại quét
          if (stack.isDomFallback && stack.cell) {
            const qtyEl = stack.cell.querySelector('.ld-qty');
            const currentQty = qtyEl ? parseInt(qtyEl.textContent || 1, 10) : 1;
            if (currentQty <= 1) {
              stack.cell.remove();
            } else {
              qtyEl.textContent = String(currentQty - 1);
            }
          }
        } else {
          showNotification(`❌ L❌i phân gi❌i: ${decompRes?.message || 'không thành
công'}`, "warning");
        }
      } catch (err) {
        console.error(`${this.logPrefix} L❌i khi phân gi❌i đơn ${pid}:`, err);
      }
      await new Promise(resolve => setTimeout(resolve, 1000));
    }
  }
  return 10000; // Quay lại chu kỳ sau 10s đ❌i nạp trạng thái mới sạch sẽ và tránh nh❌p nháy UI
}

const furnace = data.furnace || "idle";
const craft = data.craft || null;
const materials = data.materials || {};
const recipes = data.recipes || {};

if (data.dong_serving) {
  countdownTimer.remove('luyenDan');
  const tuneCount = craft ? (craft.tune_count | 0) : 0;
  this.updateProgress(`Đ❌n đ❌ng (${tuneCount}/3)`);

  const stability = craft ? (craft.stability_pct != null ? parseFloat(craft.stability_pct) : 100) :
100;

  const autoTune = localStorage.getItem('luyenDanAutoTune') === 'true';

  if (autoTune && stability <= 60) {
    console.log(`${this.logPrefix} Đ❌n Đ❌ng tự động đi❌u h❌a hộ Đ❌n Ch❌ (Độ ❌n định: ${stability.
toFixed(1)}%)...`);
    try {
      const tuneRes = await this.sendLdRequest("/tune", "POST", {});
      if (tuneRes && !tuneRes.error) {
        showNotification(`✅ Đ❌n Đ❌ng đã tự động đi❌u h❌a giúp Đ❌n Ch❌!`, "success");
        return 10000;
      }
    } catch (err) {
      console.error(`${this.logPrefix} L❌i khi Đ❌n Đ❌ng đi❌u h❌a:`, err);
    }
  }
  return 15000;
}

console.log(`${this.logPrefix} Trạng thái Lò Đ❌n: ${furnace.toUpperCase()}`);

if (furnace === "exploded") {
  countdownTimer.remove('luyenDan');
  this.updateProgress("❌ Bị n❌ lò");
  if (!autoLuyenDan) {
    return 15000;
  }
  showNotification("❌ Đ❌n Lô bị n❌! Đang dọn dẹp lò...", "warning");
  const jobId = craft?.id || data.craftJobId;
  if (jobId) {
    const ackRes = await this.sendLdRequest("/ack-explosion", "POST", { job_id: jobId });
    if (ackRes && !ackRes.error) {
      showNotification("✅ Đã dọn dẹp lò đ❌n bị n❌", "success");
    } else {
      showNotification("❌ L❌i khi dọn dẹp lò đ❌n bị n❌", "error");
    }
  }
  console.warn(`${this.logPrefix} Không tìm th❌y job_id c❌a lò bị n❌`);
  return 10000;
}

if (furnace === "ready") {
  countdownTimer.remove('luyenDan');
  this.updateProgress("Chờ thu hoạch");
  if (!autoLuyenDan) {
    return 15000;
  }
  showNotification("✅ Luyện đ❌n hoàn t❌t! Thu hoạch...", "info");
  const jobId = craft?.id || data.craftJobId;
  if (jobId) {
    const collectRes = await this.sendLdRequest("/collect", "POST", { job_id: jobId });
    // API tr❌i v❌i { ok: true, data: {...} } khi thành công
    if (collectRes && !collectRes.error && (collectRes.ok || collectRes.data)) {
      const collectData = collectRes.data || collectRes;
      const pillName = collectData.pill_name || collectData.tier_label || "Đ❌n được";
      const stars = parseInt(collectData.stars || collectData.star || 1, 10);
      showNotification(`✅ Thu hoạch thành công: ${pillName} ${"*".repeat(stars)}`, "success");

      const minStars = parseInt(localStorage.getItem('luyenDanMinStars') || '4', 10);
      const autoUse = localStorage.getItem('luyenDanAutoUse') === 'true';
      const autoDecompose = localStorage.getItem('luyenDanAutoDecompose') === 'true';

```

```

// [v2.17.1-local] Lọc lại state mới để tìm pill_id thực tế trong pill_stacks
let pillId = null;
try {
  const freshState = await this.sendLdRequest("/state?fresh=1", "GET");
  const freshData = freshState?.data || freshState;

  const getStacksFromData = (d) => {
    if (!d) return [];
    if (d.pill_stacks && d.pill_stacks.length > 0) {
      return d.pill_stacks.map(s => ({
        tier: s.tier,
        stars: parseInt(s.stars || s.star || 0, 10),
        count: parseInt(s.count || 0, 10),
        stack_id: s.stack_id || `${s.tier}:${parseInt(s.stars || s.star || 0, 10)}`
      }));
    }
    if (d.pills && d.pills.length > 0) {
      const map = {};
      d.pills.forEach(p => {
        const stars = parseInt(p.stars || p.star || 0, 10);
        const sid = p.id || `${p.tier}:${stars}`;
        const key = `${p.tier}-${stars}`;
        if (!map[key]) {
          map[key] = { tier: p.tier, stars, count: 0, stack_id: sid };
        }
        map[key].count++;
      });
      return Object.keys(map).map(k => map[k]);
    }
    return [];
  };

  const stacks = getStacksFromData(freshData);
  // Tìm stack khớp tier và stars vừa thu hoạch
  const craftedTier = collectData.tier || craft?.ui_tier || data.tier;
  const cleanTierName = (t) => {
    if (!t) return '';
    const s = String(t).toLowerCase();
    if (s.includes('cuc')) return 'cuc';
    if (s.includes('thuong')) return 'thuong';
    if (s.includes('trung')) return 'trung';
    if (s.includes('ha')) return 'ha';
    return s;
  };
  const targetTierClean = cleanTierName(craftedTier);
  const matchStack = stacks.find(s =>
    cleanTierName(s.tier) === targetTierClean && parseInt(s.stars || s.star || 0, 10)
  ) || stacks.find(s => parseInt(s.stars || s.star || 0, 10) === stars);

  if (matchStack) {
    const matchStars = parseInt(matchStack.stars || matchStack.star || 0, 10);
    pillId = String(matchStack.stack_id || `${matchStack.tier}:${matchStars}`);
    console.log(`${this.logPrefix} Tìm thấy pill trong túi: ${pillId} (${matchStack.tier} ${matchStars} x ${matchStack.count})`);
  } else {
    console.warn(`${this.logPrefix} Không tìm thấy đơn trong túi sau thu hoạch.
  }
} catch (err) {
  console.error(`${this.logPrefix} Lỗi khi lọc state mới:`, err);
}

if (pillId) {
  if (stars >= minStars) {
    if (autoUse) {
      console.log(`${this.logPrefix} Tự động sử dụng đơn chất cao (${stars}*)...`);
      this.updateProgress(`Sử dụng đơn ${stars}*)`);
      const useRes = await this.sendLdRequest("/use-pill", "POST", { pill_id: String(
pillId) });

      if (useRes && !useRes.error && (useRes.ok || useRes.data)) {
        const useData = useRes.data || useRes;
        showNotification(`Đã sử dụng đơn. Tu Vi nhận: ${useData?.tu_vi_granted
|| useData?.tu_vi || "thành công"}`, "success");
      } else {
        showNotification(`Lỗi sử dụng đơn: ${useRes?.message || 'không thành
công'}`, "warning");
      }
    } else {
      showNotification(`Nhận đơn (${stars}*)$. Giữ lại trong túi để`, "info");
      this.updateProgress(`Giữ đơn ${stars}*)`);
    }
  } else if (autoDecompose) {
    console.log(`${this.logPrefix} Tự động phân giải đơn chất kém (${stars}*) < ${
minStars}*)...`);
    this.updateProgress(`Phân giải đơn ${stars}*)`);
    const decompRes = await this.sendLdRequest("/decompose", "POST", { pill_id: String(
pillId) });

    if (decompRes && !decompRes.error && (decompRes.ok || decompRes.data)) {
      showNotification(`Đã phân giải đơn phẩm chất kém (${stars}*)`, "success");
    } else {
      showNotification(`Lỗi phân giải đơn: ${decompRes?.message || 'không thành
công'}`, "warning");
    }
  } else {
    showNotification(`Nhận đơn (${stars}*)$. Giữ lại trong túi để`, "info");
    this.updateProgress(`Giữ đơn ${stars}*)`);
  }
} else {
  showNotification(`Thu đơn thành công (${stars}*) nhưng không xác định được ID đơn
trong túi.`, "info");
}
} else {
  showNotification(`Thu đơn thất bại`, "error");
}
} else {
  console.warn(`${this.logPrefix} Không tìm thấy job_id để thu đơn`);
}
return 10000;
}

if (furnace === "crafting") {
  const unstableLeftSec = craft ? (craft.unstable_left_sec || 0) : 0;
  const timerLeftSec = craft ? (craft.timer_left_sec || 0) : 0;
  const stability = craft ? (craft.stability_pct != null ? parseFloat(craft.stability_pct) : 100) :

```

```

100;

const tuneCount = craft ? (craft.tune_count | 0) : 0;
const tuneSurvivalMin = (data.danMaster && data.danMaster.rng && data.danMaster.rng.
tuneSurvivalMin) ? (data.danMaster.rng.tuneSurvivalMin | 0) : 3;
const isSurvival = tuneCount >= tuneSurvivalMin;
const isSafe = unstableLeftSec <= 0 || isSurvival;

console.log(`${this.logPrefix} Đang luyện. Còn: ${timerLeftSec}s. Giai đoạn nhảy c[m: ${
unstableLeftSec}s. Độ [n định: ${stability.toFixed(1)}%. Giữ l[a: ${tuneCount}/${tuneSurvivalMin}`);
localStorage.setItem('luyenDanStability_${accountId}', String(stability.toFixed(0)));
localStorage.setItem('luyenDanTuneCount_${accountId}', String(tuneCount));
localStorage.setItem('luyenDanTuneSurvivalMin_${accountId}', String(tuneSurvivalMin));
localStorage.setItem('luyenDanIsSafe_${accountId}', isSafe ? 'true' : 'false');
countdownTimer.set('luyenDan', timerLeftSec * 1000);

// Cập nhật ti[n trình hi[n thị trên UI kèm s[ l[n Đ[i[u H[a
this.updateProgress('Đang luyện (${stability.toFixed(0)}% - ${tuneCount}/${tuneSurvivalMin})`);

// N[u giai đoạn nhảy c[m đã qua hoặc đã đ[ s[ l[n giữ l[a (đã an toàn)
if (isSafe) {
  console.log(`${this.logPrefix} Lò đan đã an toàn tuyệt đ[. Chờ hoàn thành sau ${timerLeftSec}s.
..`);

  const m = Math.floor(timerLeftSec / 60);
  const s = timerLeftSec % 60;
  const timeStr = `${m}:${String(s).padStart(2, '0')}`;
  this.updateProgress('đã đ[i[u ho[ ${tuneCount} l[n thời gian ${timeStr}`);
  countdownTimer.set('luyenDan', timerLeftSec * 1000);
  return timerLeftSec * 1000 + 3000;
}

// N[u v[n [ giai đoạn nhảy c[m và chưa đ[ s[ l[n giữ l[a
if (!autoLuyenDan) {
  return 10000;
}
const autoTune = localStorage.getItem('luyenDanAutoTune') === 'true';
if (autoTune && stability <= 68) {
  console.log(`${this.logPrefix} Đan Ch[ tự động đ[i[u h[a (Độ [n định: ${stability.toFixed(1)}%)..
.);

  let successCount = 0;
  for (let i = 0; i < 3; i++) {
    showNotification('Đ[i[u H[a l[n ${i + 1}]/3...`', "warning");
    this.updateProgress('Đ[i[u H[a ${i + 1}]/3`);
    try {
      const tuneRes = await this.sendLdRequest("/tune", "POST", {});
      if (tuneRes && !tuneRes.error) {
        successCount++;
        console.log(`${this.logPrefix} Đ[i[u H[a l[n ${i + 1} thành công.`);
        const updatedTuneCount = tuneRes.data?.craft?.tune_count || tuneRes.data?.

        if (updatedTuneCount != null && updatedTuneCount >= tuneSurvivalMin) {
          console.log(`${this.logPrefix} Đã đạt đ[ s[ l[n giữ l[a sau l[n tune ${i + 1}.
.);

          break;
        }
      } else {
        console.log(`${this.logPrefix} Đ[i[u H[a l[n ${i + 1} th[ thất bại: ${tuneRes?.message
|| 'l[i Rest'`);
      }
    } catch (err) {
      console.error(`${this.logPrefix} L[i khi gọi Đ[i[u H[a l[n ${i + 1}:`, err);
    }
    if (i < 2) {
      const tuneDelay = 10000 + Math.floor(Math.random() * 2000);
      await new Promise(resolve => setTimeout(resolve, tuneDelay));
    }
  }
  showNotification('Hoàn t[ Đ[i[u H[a! Thành công: ${successCount}`, "success");
  return 10000;
}

return 10000; // Định kỳ 10s/l[n
}

if (furnace === "idle") {
  countdownTimer.remove('luyenDan');
  this.updateProgress("Lò tr[ng");
  if (!autoLuyenDan) {
    return 15000;
  }
  const autoStart = localStorage.getItem('luyenDanAutoStart') === 'true';
  if (!autoStart) {
    console.log(`${this.logPrefix} Tự động khai lò đã t[ thất, giữ lò tr[ng.`);
    return 15000;
  }
  const targetTier = localStorage.getItem('luyenDanTargetTier') || 'auto';
  let TiersOrder = ["cuc", "thuong", "trung", "ha"];
  if (targetTier !== 'auto') {
    TiersOrder = [targetTier];
  }
  let selectedTier = null;

  for (const tier of TiersOrder) {
    const rec = recipes[tier];
    const isUnlocked = rec ? !!rec.craft_unlocked : false;
    if (isUnlocked) {
      const vec = rec.vector || {};
      let hasEnoughMats = true;
      // [v2.17.1-local] Ki[m tra T[ C[ nguyên liệu (k[ c[ linh_phong_thao, huyen_van_thao...)
      for (const el of Object.keys(vec)) {
        const need = vec[el] || 0;
        if ((materials[el] || 0) < need) {
          console.log(`${this.logPrefix} Thi[u [ ${el}] cho ph[m [ ${tier}]: c[n [ ${need}, có [
materials[el] || 0`);

          hasEnoughMats = false;
          break;
        }
      }
    }
    if (hasEnoughMats) {
      selectedTier = tier;
      break;
    }
  }

  if (selectedTier) {
    // Tự động mời Đan Đ[ng và chờ họ tham gia

```

```

const autoInvite = localStorage.getItem('luyenDanAutoInvite') === 'true';
if (autoInvite) {
  const slots = data.dong_slots || [];
  const filledSlots = slots.filter(s => s != null);
  const isAllFilled = filledSlots.length >= 2;
  const waitSeconds = parseInt(localStorage.getItem('luyenDanWaitInviteSeconds') || '60', 10);

  if (!isAllFilled) {
    const selectedIds = (localStorage.getItem('luyenDanSelectedFriendIds') || '').split(',').filter(Boolean);

    if (!this.inviteSentTime) {
      console.log(`${this.logPrefix} Bật đồng mời Đan Đẳng...`);
      this.updateProgress("Gửi lời mời Đan Đẳng");
      for (const buddyId of selectedIds) {
        showNotification(`Đang gửi lời mời tới Đan Đẳng ID: ${buddyId}`, "info");
        try {
          await this.sendLdRequest("/dong/invite", "POST", { buddy_id: parseInt(buddyId, 10) });
        } catch (err) {
          console.error(`${this.logPrefix} Lỗi khi mời Đan Đẳng ${buddyId}:`, err);
        }
        this.inviteSentTime = Date.now();
        return 10000; // Quay lại check sau 10s
      }
    } else {
      const elapsed = Math.floor((Date.now() - this.inviteSentTime) / 1000);
      if (elapsed < waitSeconds) {
        const remaining = waitSeconds - elapsed;
        console.log(`${this.logPrefix} Đang chờ Đan Đẳng tham gia. Đã chờ: ${elapsed}s, còn lại: ${remaining}s.`);
        this.updateProgress(`Chờ Đan Đẳng ${filledSlots.length}/2 (${remaining}s`);
        return 10000; // Quay lại check sau 10s
      } else {
        console.log(`${this.logPrefix} Quá thời gian chờ Đan Đẳng (${waitSeconds}s, tiến hành Khai lò...`);
      }
    }
  } else {
    console.log(`${this.logPrefix} Đan Đẳng đã tham gia đầy đủ, tiến hành Khai lò...`);
  }

  // Reset mức thời gian chờ
  this.inviteSentTime = null;

  showNotification(`Khai lò luyện đan phẩm: ${selectedTier.toUpperCase()}...`, "info");
  this.updateProgress(`Khai lò phẩm ${selectedTier.toUpperCase()}`);
  const startRes = await this.sendLdRequest("/start", "POST", { tier: selectedTier });
  if (startRes && !startRes.error) {
    showNotification(`Khai lò Luyện Đan phẩm ${selectedTier.toUpperCase()} thành công!`, "success");
    return 10000;
  } else {
    showNotification(`Khai lò thất bại: ${startRes?.message || 'lỗi'}`, "error");
    return 10000;
  }
} else {
  console.log(`${this.logPrefix} Không đủ nguyên liệu ngũ hành. Tìm gói linh dược...`);
  const matBundles = data.mat_bundles || [];
  if (matBundles.length > 0) {
    const bundle = matBundles[0];
    const bundleKey = bundle.bundle_key;
    showNotification(`Phát hiện gói linh dược ${bundle.name || bundleKey}. Đang tự động mở...`, "info");
    this.updateProgress("Mở gói linh dược");
    const openRes = await this.sendLdRequest("/open-mat-bundle", "POST", { job_id: String(bundleKey), bundle_key: String(bundleKey) });
    if (openRes && !openRes.error) {
      showNotification(`Mở gói linh dược thành công!`, "success");
      return 3000;
    } else {
      showNotification(`Mở gói linh dược thất bại: ${openRes?.message || 'lỗi'}`, "error");
      return 10000;
    }
  }
} else {
  countdownTimer.remove('luyenDan');
  showNotification(`Hết nguyên liệu và gói linh dược. Dừng Luyện Đan.`, "warning");
  this.updateProgress("Hết nguyên liệu");
  const accountId = await getAccountId();
  if (accountId) {
    taskTracker.markTaskDone(accountId, 'luyenDan');
    updateAllQuestButtons().catch(() => {});
  }
  return null;
}
}
} catch (e) {
  countdownTimer.remove('luyenDan');
  console.error(`${this.logPrefix} Lỗi trong doLuyenDan:`, e);
  this.updateProgress("Lỗi");
  showNotification(`Lỗi Luyện Đan: ${e.message}`, "error");
  return 15000;
} finally {
  this.isProcessing = false;
}
}

//===== Pháp Tướng/ KHÁC TRẦN VĂN=====//

// =====
// LUẬN VÔ
// =====

class LuanVo {
  constructor() {
    this.weburl = weburl;
    this.accountId = accountId;
    this.logPrefix = '[Luận Vô]';
    this.config = null;
  }
}

```

```

        this.currentTarget = null;
        this.targetLeft = 0;
        this.sent = 0;
        this.maxBattles = 5;
    }

    async setupConfig() {
        const nonce = await getNonce();
        const securityToken = await getSecurityToken(this.weburl + 'luan-vo-duong?t');
        this.config = { nonce, token: securityToken };
    }

    async sendApiRequest(endpoint, method, body = {}) {
        try {
            const url = `${this.weburl}${endpoint}`;
            const headers = {
                "Content-Type": "application/json",
                "X-Requested-With": "XMLHttpRequest",
                "X-WP-Nonce": this.config.nonce,
                "x-lv-token": this.config.token
            };

            const response = await fetch(url, {
                method,
                headers,
                body: JSON.stringify(body),
                credentials: 'include'
            });

            const contentType = response.headers.get("content-type");
            let data = null;
            if (contentType && contentType.includes("application/json")) {
                data = await response.json();
            } else {
                data = await response.text();
            }
            if (!response.ok) {
                console.warn(`${this.logPrefix} API Lỗi ${response.status}:`, data);
                return data;
            }
            return data;
        } catch (error) {
            console.error(`${this.logPrefix} Lỗi khi gửi yêu cầu tới ${endpoint}:`, error);
            return null;
        }
    }
}

/**
 * Hàm hỗ trợ: Đợi một khoảng thời gian.
 */
async delay(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
}

/**
 * Lấy thông tin lượt gửi/nhận
 */
async fetchLvPageInfo() {
    try {
        const url = "/luan-vo-duong?t=" + Date.now();
        const resp = await fetch(url, {
            credentials: "include",
            cache: "no-store"
        });
        if (!resp.ok) {
            throw new Error("Không load được trang Luận Võ");
        }

        const html = await resp.text();
        const doc = new DOMParser().parseFromString(html, "text/html");

        // ===== PARSE ĐÃ GỬI / ĐÃ NHẬN =====
        let sent = 0;
        let received = 0;

        doc.querySelectorAll("p").forEach(p => {
            const t = p.innerText || "";
            if (t.includes("Đã gửi")) {
                const m = t.match(/(\d+)\s*\s*\s*5/);
                if (m) {
                    sent = Number(m[1]);
                }
            }
            if (t.includes("Đã nhận")) {
                const m = t.match(/(\d+)\s*\s*\s*5/);
                if (m) {
                    received = Number(m[1]);
                }
            }
        });

        this.sent = sent;
        this.received = received;
        console.log(`${this.logPrefix} Đã gửi: ${this.sent}, Đã nhận: ${this.received}`);

        return { sent, received };
    } catch (error) {
        console.error("Lỗi khi lấy thông tin Luận Võ:", error);
        return { sent: 0, received: 0 };
    }
}

/**
 * Đảm bảo tính năng tự động chấp nhận khiêu khích được bật.
 */
async ensureAutoAccept() {
    const toggleEndpoint = 'wp-json/luan-vo/v1/toggle-auto-accept';
    const result = await this.sendApiRequest(toggleEndpoint, 'POST', {});
    //console.log('Check Status Auto Accept: ${result.message}');
    if (result && result.success && result.message.includes('Đã bật')) {
        taskTracker.updateTask(this.accountId, 'luanvo', 'auto_accept', true);
        return true;
    }
}

```

```

    }
    await this.sendApiRequest(toggleEndpoint, 'POST', {}); //bật lại nếu có manual tắt autoaccept
    //console.log("Đã bật lại tự động chấp nhận");
    taskTracker.updateTask(this.accountId, 'luanvo', 'auto_accept', true);
    return false;
}

/**
 * Lọc danh sách tất cả user đang theo dõi
 * Gồm các phần: id, name, avatar, points, auto_accept, can_receive_count, profile_link, role, role_color,
description, challenges_remaining, challenge_exists, challenge_id, is_following, is_joined_today, can_send_count,
max_batch_count
 */

async getFollowingUsers() {
    console.log(`${this.logPrefix} Đang lọc danh sách người theo dõi...`);
    const endpoint = 'wp-json/luan-vo/v1/get-following-users';
    const body = { page: 1 };
    const data = await this.sendApiRequest(endpoint, 'POST', body);

    if (data && data.success) {
        const users = data.data.users;
        console.log(`${this.logPrefix} Tìm thấy ${users.length} người dùng.`);
        return users;
    } else {
        console.error(`${this.logPrefix} Lỗi khi lọc Online Users: ${data?.message || data?.data}`);
        return [];
    }
}

async getOnlineUsers() {
    console.log(" Đang lọc danh sách người dùng online...");
    const endpoint = 'wp-json/luan-vo/v1/online-users';
    const body = { page: 1 };
    const data = await this.sendApiRequest(endpoint, 'POST', body);

    if (data && data.success) {
        const users = data.data.users;
        console.log(`${this.logPrefix} Tìm thấy ${users.length} người online.`);
        return users;
    } else {
        console.error(`${this.logPrefix} Lỗi khi lọc Online Users: ${data?.message || data?.data}`);
        return [];
    }
}

async sendChallenge(userId, nonce, token) {
    console.log(`${this.logPrefix} Đang gửi khiêu chiến người chơi ID: ${userId}...`);
    const challengeMode = localStorage.getItem('luanVoChallengeMode') || 'auto';

    const sendEndpoint = 'wp-json/luan-vo/v1/send-challenge';
    const sendBody = { target_user_id: userId };
    const sendResult = await this.sendApiRequest(sendEndpoint, 'POST', nonce, token, sendBody);

    if (sendResult && sendResult.success) {
        console.log(`${this.logPrefix} Gửi khiêu chiến thành công! Challenge ID: ${sendResult.data.challenge_id}`);

        // Bước mới: Kiểm tra nếu đã tắt auto_accept
        if (sendResult.data.auto_accept || challengeMode === 'manual') {
            console.log(`${this.logPrefix} Đã tắt tự động chấp nhận, đang hoàn tất trận đấu...`);

            const approveEndpoint = 'wp-json/luan-vo/v1/auto-approve-challenge';
            const approveBody = {
                challenge_id: sendResult.data.challenge_id,
                target_user_id: userId
            };

            const approveResult = await this.sendApiRequest(approveEndpoint, 'POST', nonce, token, approveBody);

            if (approveResult && approveResult.success) {
                showNotification('Luận võ: ' + sendResult.data.message + '!', 'success');
                return true;
            } else {
                const message = approveResult?.data?.message || 'Lỗi không xác định khi hoàn tất trận đấu.';
                showNotification(`${this.logPrefix} Lỗi hoàn tất trận đấu: ${message}`, 'error');
                return false;
            }
        } else {
            showNotification('Đã gửi khiêu chiến đến ' + userId + '! Đang chờ đối thủ chấp nhận.', 'success');
            return true;
        }
    } else {
        const message = sendResult?.data?.message || JSON.stringify(sendResult) || 'Lỗi không xác định.';
        showNotification(`${this.logPrefix} Gửi khiêu chiến thất bại: ${message}`, 'error');
        return false;
    }
}

async attackCurrentTarget() {
    if (!this.currentTarget) return;

    // Nếu mình đã đấu 5 lượt → STOP
    if (this.sent >= this.maxBattles) {
        console.log("Đã đấu 5 lượt đấu - dừng auto", "success");
        return;
    }

    // target hết lượt
    if (this.targetLeft <= 0) {
        showNotification("Target hết lượt - chuyển đối tượng", "warn");
        this.currentTarget = null;
        return this.huntFromOnline();
    }

    const nonce = this.config.nonce;
    const token = this.config.token;
    const success = await this.sendChallenge(this.currentTarget.id, nonce, token);
    if (success) {
        this.sent++; // tăng số lượt mình đã gửi
        this.targetLeft--; // giảm số lượt còn lại của target
        await this.delay(4400);
    }
}

```

```

    return this.attackCurrentTarget();
}

async huntLuanVoTargets() {
    // Kiểm tra chế độ
    const challengeMode = localStorage.getItem('luanVoChallengeMode') || 'auto';

    if (challengeMode === 'manual') {
        // Chế độ Theo ID
        const targetUserId = localStorage.getItem(`luanVoTargetUserId_${this.accountId}`) || '';
        if (!targetUserId) {
            showNotification("Chưa có hình ID người chơi!", "error");
            return;
        }

        return this.huntSpecificUser(targetUserId);
    }

    // Chế độ Tự động (giữ logic cũ)
    const users = await this.getFollowingUsers();
    if (!Array.isArray(users) || users.length === 0) {
        showNotification("Không lấy được danh sách follow", "error");
        return this.huntFromOnline();
    }

    const target = users.find(u => Number(u.can_receive_count) > 0);
    if (!target) {
        showNotification("⚠️ Follow full – chuyển ONLINE", "warn");
        return this.huntFromOnline();
    }

    this.currentTarget = target;
    const myRemaining = this.maxBattles - this.sent;
    this.targetLeft = Math.min(target.can_receive_count, myRemaining);

    console.log(`Chọn ${target.name} còn ${this.targetLeft} lượt`);
    return this.attackCurrentTarget();
}

async huntSpecificUser(userId) {
    console.log(`Đang đánh theo ID: ${userId}`);

    // Đặt target với ID đã nhập
    this.currentTarget = {
        id: userId,
        user_id: userId,
        name: `User ${userId}`
    };

    // Gửi tối đa 5 lượt cho user này
    while (this.sent < this.maxBattles) {
        console.log(`${this.logPrefix} × Gửi lượt ${this.sent + 1}/${this.maxBattles} cho ID: ${userId}`);

        const nonce = this.config.nonce;
        const token = this.config.token;
        const success = await this.sendChallenge(userId, nonce, token);
        if (success) {
            this.sent++;
            await this.delay(1000);
        } else {
            // Nếu thất bại, thử lại sau 2 giây
            console.warn(`${this.logPrefix} ⚠️ Gửi thất bại, thử lại sau 2s...`);
            await this.delay(2000);
        }
    }

    console.log(`${this.logPrefix} Hoàn thành ${this.sent} lượt cho ID: ${userId}`);
}

async huntFromOnline() {
    const users = await this.getOnlineUsers();
    if (!Array.isArray(users) || users.length === 0) {
        showNotification("Online full – nghỉ 60s", "info");
        return setTimeout(() => this.huntLuanVoTargets(), 60000);
    }

    const target = users.find(u => Number(u.challenges_remaining) > 0);
    if (!target) {
        showNotification("⚠️ Không có online khả dụng", "warn");
        return;
    }

    this.currentTarget = target;
    const myRemaining = this.maxBattles - this.sent;
    this.targetLeft = Math.min(target.challenges_remaining, myRemaining);
    // showNotification(`× Online: ${target.name} còn ${this.targetLeft} lượt`, "info");
    return this.attackCurrentTarget();
}

async receiveReward() {
    console.log(`${this.logPrefix} Đang gửi yêu cầu nhận thưởng...`);

    const endpoint = 'wp-json/luan-vo/v1/receive-reward';
    const body = {};

    try {
        const response = await this.sendApiRequest(endpoint, 'POST', {});
        if (!response) {
            return;
        }
        if (response.success === true) {
            showNotification(`Luận võ: ${response.message}`, 'success');
            taskTracker.markTaskDone(accountId, 'luanvo');
            return;
        } else if (response.message === "Đạo hữu đã nhận thưởng trong ngày hôm nay.") {
            showNotification(`Bạn đã nhận thưởng Luận Võ hôm nay rồi!`, 'info');
            taskTracker.markTaskDone(accountId, 'luanvo');
            return;
        } else {
            // const errorMessage = response.message || 'Lỗi không xác định khi nhận thưởng.';
        }
    }

```

```

        //console.error(`${this.logPrefix} ❌ Lỗi khi nhận thư: ${response?.message || response?.data}`);
        showNotification('❌ Lỗi khi nhận thư: ${response?.message || response?.data}', 'error');
    }
} catch (error) {
    showNotification('❌ Lỗi mạng khi gửi yêu cầu nhận thư: ${error}', 'error');
}
}

async startLuanVo() {
    if (!this.config) {
        await this.setupConfig();
    }

    // Lấy số lượt đã gửi/nhận từ server
    await this.fetchLvPageInfo();

    // Bước 2: Tham gia trận đấu
    if (!taskTracker.getTaskStatus(accountId, 'luanvo').battle_joined) {
        const joinResult = await this.sendApiRequest(
            'wp-json/luan-vo/v1/join-battle', 'POST', { action: 'join_battle', security_token: this.config.token }
        );
        console.log('Check tham gia trận đấu: ${joinResult}');

        if (joinResult && joinResult.success === true) {
            console.log('✅ ${joinResult.message || 'Tham gia luận võ thành công.'}');
            taskTracker.updateTask(accountId, 'luanvo', 'battle_joined', true);
        } else if (joinResult.message === 'Bạn đã tham gia Luận Võ Đường hôm nay rồi!') {
            console.log('✅ ${joinResult.message}');
            taskTracker.updateTask(accountId, 'luanvo', 'battle_joined', true);
        } else {
            console.error(`${this.logPrefix} ❌ ${joinResult?.message}`);
            showNotification(joinResult?.message, 'error');
        }
    } else {
        console.log(`${this.logPrefix} ✅ Đã tham gia luận võ trước đó`);
    }

    // Bước 3: Mởm b mở tự động chấp nhận khiêu chiến
    // if (!taskTracker.getTaskStatus(accountId, 'luanvo').auto_accept) {
    const autoAcceptSuccess = await this.ensureAutoAccept();
    console.log(`${autoAcceptSuccess}`);
    if (autoAcceptSuccess) {
        console.log(`${this.logPrefix} ✅ Tự động chấp nhận đã được bật.`);
    } else {
        console.log('⚠️ Đã bật tự động chấp nhận trước đó hoặc Đã bật lại tự động lúc nữa');
    }
}

// }

}

async doLuanVo(autoChallenge = true) {
    await this.startLuanVo();
    // if (!autoChallenge) {
    //     return this.goToLuanVoPage();
    // }
    console.log("👉 Bật đấu auto Luận Võ...");
    await this.huntLuanVoTargets();
    await this.receiveReward();
}

/**Thuê Tiêu Viêm để hoàn thành khiêu chiến */
async thueTieuViem(securityToken) {
    const nonce = await getNonce();
    if (!nonce) {
        showNotification('❌ Lỗi: Không thể lấy nonce cho Luận Võ.', 'error');
        return;
    }

    try {
        while (true) {
            const res = await fetch(weburl + "wp-json/luan-vo/v1/send-bot-challenge", {
                method: "POST",
                credentials: "include",
                headers: {
                    "Content-Type": "application/json",
                    "X-WP-Nonce": nonce,
                    "X-LV-Token": securityToken
                },
                body: JSON.stringify({ bot_id: -1 })
            });

            if (!res.ok) {
                console.error("❌ Request thất bại:", res.status);
                break;
            }

            const data = await res.json();
            if (data.success) {
                showNotification(data.message, 'success');
            } else if (data.message === "Đạo hữu đã đạt tối đa nhận khiêu chiến trong ngày.") {
                showNotification('[Luận võ] Hoàn thành khiêu chiến Viêm Trưư', 'info');
                break;
            }
            // chờ 1-2 giây để tránh spam quá nhanh
            await new Promise(r => setTimeout(r, 1500));
        }
    } catch (error) {
        console.error("❌ Lỗi:", error);
    }
}

goToLuanVoPage() {
    const luanVoUrl = `${this.weburl}/luan-vo-duong?t`;
    if (confirm("Bạn có muốn chuyển đến trang Luận Võ Đường không?")) {
        window.location.href = luanVoUrl;
    }
}
}

```

```

/// =====
function extractActionTokens(html) {
    const map = {};

    // BƯỚC 1: Tìm tất cả các object có chứa 'action:' (bất kỳ giá trị)
    const actionPosRegex = /['"]?action["']?\s*[:=]/gi;
    let m;

    while ((m = actionPosRegex.exec(html)) !== null) {
        const actionIdx = m.index;

        // Tìm vị trí '{' gần nhất trước action
        let openPos = -1;
        for (let i = actionIdx; i >= 0 && i > actionIdx - 1000; i--) {
            if (html[i] === '{') { openPos = i; break; }
            if (html[i] === ';') break;
        }

        if (openPos === -1) continue;

        // Parse toàn bộ object từ '{' đến '}'
        let i = openPos;
        let depth = 0;
        let inStr = null;
        let escaped = false;
        let inLineComment = false;
        let inBlockComment = false;

        for (; i < html.length; i++) {
            const ch = html[i];
            if (inLineComment) {
                if (ch === '\n') inLineComment = false;
                continue;
            }
            if (inBlockComment) {
                if (ch === '*' && html[i + 1] === '/') { inBlockComment = false; i++; continue; }
                continue;
            }
            if (inStr) {
                if (escaped) { escaped = false; continue; }
                if (ch === '\\') { escaped = true; continue; }
                if (ch === inStr) { inStr = null; continue; }
                continue;
            }
            if (ch === '"' || ch === "'" || ch === '`') { inStr = ch; continue; }
            if (ch === '/' && html[i + 1] === '/') { inLineComment = true; i++; continue; }
            if (ch === '/' && html[i + 1] === '*') { inBlockComment = true; i++; continue; }
            if (ch === '{') depth++;
            else if (ch === '}') {
                depth--;
                if (depth === 0) { i++; break; }
            }
        }

        const objSlice = html.slice(openPos, i);

        // BƯỚC 2: Extract security từ object (bất kỳ vị trí)
        const securityMatch = /['"]?security["']?\s*[:=]\s*["']?([^\"]+)[']/i.exec(objSlice);
        if (!securityMatch) continue; // Không có security trong object này

        const securityToken = securityMatch[1];

        // BƯỚC 3: Extract action names (cả static, dynamic và ternary)
        const actionNames = new Set(); // Dùng Set để tránh trùng lặp

        // 3.1: Tìm action line trong object
        const actionLineMatch = /['"]?action["']?\s*[:=]\s*([^\"]+)[']/i.exec(objSlice);
        if (actionLineMatch) {
            const actionValue = actionLineMatch[1].trim();

            // 3.1a: Static string: action: "enter_mine" hoặc action: 'enter_mine'
            const staticMatch = /^["']?([^\"]+)[']/i.exec(actionValue);
            if (staticMatch) {
                actionNames.add(staticMatch[1]);
            }

            // 3.1b: Dynamic hh3dData.act.xxx
            const dynamicMatches = actionValue.matchAll(/hh3dData\.act\.[a-zA-Z0-9_]+/gi);
            for (const dm of dynamicMatches) {
                actionNames.add(dm[1]); // kmBuy, kmEnter, kmReward, etc.
            }

            // 3.1c: Ternary fallback: ... ? xxx : 'fallback_string'
            const ternaryMatch = /\s*["']?([^\"]+)[']/i.exec(actionValue);
            if (ternaryMatch) {
                actionNames.add(ternaryMatch[1]); // buy_item_khoang, enter_mine, etc.
            }
        }

        // BƯỚC 4: Map tất cả action names với security token
        for (const actionName of actionNames) {
            if (actionName && actionName.trim()) {
                const trimmedName = actionName.trim();
                // Chỉ map nếu chưa có hoặc update với token mới hơn
                if (!map[trimmedName] || map[trimmedName] !== securityToken) {
                    map[trimmedName] = securityToken;
                }
            }
        }

        // BƯỚC 5: Tìm các action đơn lẻ không nằm trong object (legacy support)
        const legacyActionRegex = /['"]?action["']?\s*[:=]\s*["']?([^\"]+)[']/gi;
        let lm;
        while ((lm = legacyActionRegex.exec(html)) !== null) {
            const actionName = lm[1];
            if (map[actionName]) continue; // Đã có rồi, skip

            // Tìm security gần đó
            const slice = html.slice(lm.index, lm.index + 1000);
            const nm = /(?:["']?(?:security|nonce)["']?\s*[:=]\s*["']?([^\"]+)[']/i.exec(slice);
            if (nm) map[actionName] = nm[1];
        }

        // BƯỚC 6: Thêm các nguôn khác (input hidden, meta, bí ẩn global)
        let mm;
    }
}

```

```

const inputRegex = /<input[>]+name=["'](?:security|nonce|wpnonce)["'][>]*value=["'](?:["']+)|["']/i;
while ((mm = inputRegex.exec(html)) !== null) map._global = map._global || mm[1];

const meta = /<meta[>]+name=["'](?:wpnonce|security|nonce)["'][>]*content=["'](?:["']+)|["']/i;
if (meta) map._meta = map._meta || meta[1];

const varRegex = /(?:var|let|const)\s*([a-zA-Z0-9_$]*nonce[a-zA-Z0-9_$]*)\s*=\s*["'](?:["']+)|["']/i;
while ((mm = varRegex.exec(html)) !== null) map[mm[1]] = mm[2];

// BƯỚC 7: Xử lý đặc biệt cho "nonce" và "restNonce"
// Nếu action name là "nonce" hoặc "restNonce", lấy giá trị từ JSON pattern
if (!map['nonce']) {
    const nonceMatch = html.match(/"nonce"\s*:\s*"([a-f0-9]+)"/i);
    if (nonceMatch) map['nonce'] = nonceMatch[1];
}

if (!map['restNonce']) {
    const restNonceMatch = html.match(/"restNonce"\s*:\s*"([a-f0-9]+)"/i);
    if (restNonceMatch) map['restNonce'] = restNonceMatch[1];
}

return map;
}

function extractWpRestNonce(html) {
    const m = html.match(/"restNonce"\s*:\s*"([a-f0-9]+)"/i);
    return m ? m[1] : null;
}

function extractWpNonce(html) {
    const m = html.match(/"nonce"\s*:\s*"([a-f0-9]+)"/i);
    return m ? m[1] : null;
}

function extractRedeemNonce(html) {
    // Match: const redeemNonce = '0e14dbd03c'; hoặc "redeemNonce": "abc"
    const m = html.match(/(?:var|let|const)\s+redeemNonce\s*=\s*"([a-f0-9]+)"/i);
    || html.match(/"redeemNonce"\s*[:=]\s*"([a-f0-9]+)"/i);
    return m ? m[1] : null;
}

//===== KHOẢNG MẠCH =====

class KhoangMach {
    constructor() {
        this.ajaxUrl = ajaxUrl;
        this.khoangMachUrl = weburl + 'khoang-mach?t';
        this.logPrefix = '[Khoảng Mạch]';
        this.headers = {
            "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
            "X-Requested-With": "XMLHttpRequest",
        };
        this.getUsersInMineNonce = null;
        this.securityToken = null;
        this.buffBought = false;
        this.lastOuterUserIds = '';
        this.lastOuterNotificationTime = 0;
    }

    delay(ms) {
        return new Promise(resolve => setTimeout(resolve, ms));
    }

    async #getNonce(actionName) {
        return getSecurityNonce(this.khoangMachUrl, actionName);
    }

    async loadMines(mineType) {
        const nonce = await getSecurityNonce(this.khoangMachUrl, 'load_mines_by_type');
        if (!nonce) { showNotification('Lỗi nonce (load_mines).', 'error'); return null; }
        const payload = new URLSearchParams({ action: hData && hData.act ? hData.act.kmList : 'load_mines_by_type',
mine_type: mineType, security: nonce });
        try {
            const r = await fetch(this.ajaxUrl, { method: 'POST', headers: this.headers, body: payload, credentials:
'include' });
            const d = await r.json();
            return d.success ? d.data : (showNotification(d.message || 'Lỗi tải mine', 'error'), null);
        } catch (e) { console.error(`${this.logPrefix} Lỗi mạng (tải mine):`, e); return null; }
    };

    async getAllMines(forceRefresh = false) {
        const mineTypes = ['gold', 'silver', 'copper'];
        const cacheKey = "HH3D_allMines";
        const cacheRaw = localStorage.getItem(cacheKey);

        // Kiểm tra cache (không có thời hạn - chỉ xóa khi bấm load lại)
        if (!forceRefresh && cacheRaw && cacheRaw.length > 0) {
            try {
                const cache = JSON.parse(cacheRaw);
                const cacheTypes = new Set((cache?.data || []).map(m => String(m?.type || '')));
                const cacheHasAllTypes = mineTypes.every(t => cacheTypes.has(t));

                if (cache.data && cache.data.length > 0 && cacheHasAllTypes) {
                    console.log("[HH3D] Dừng dữ liệu mine từ cache");
                    showNotification("Dữ liệu mine đã được tải từ cache.", "info");
                    return {
                        optionsHtml: cache.optionsHtml,
                        minesData: cache.data
                    };
                } else {
                    localStorage.removeItem(cacheKey);
                }
            } catch (e) {
                console.warn("[HH3D] Lỗi đọc cache:", e);
            }
        }

        // --- Nếu chưa có cache hoặc đã hết hạn, tải mới ---
        const nonce = await getSecurityNonce(this.khoangMachUrl, 'load_mines_by_type');
        if (!nonce) {
            showNotification('Lỗi nonce (getAllMines).', 'error');
            return { optionsHtml: '', minesData: [] };
        }

        // --- Load từng loại + kiểm tra đủ 3 loại ---
        const minesByType = new Map();
        const missingTypes = new Set(mineTypes);

```

```

const fetchMinesByType = async (type) => {
  const payload = new URLSearchParams({
    action: hData && hData.act ? hData.act.kmList : 'load_mines_by_type',
    mine_type: type,
    security: nonce
  });

  try {
    const r = await fetch(this.ajaxUrl, {
      method: 'POST',
      headers: this.headers,
      body: payload,
      credentials: 'include'
    });
    const d = await r.json();

    if (d && d.success && Array.isArray(d.data)) {
      const typed = d.data.map(mine => ({ ...mine, type }));
      minesByType.set(type, typed);
      missingTypes.delete(type);
      return true;
    }

    showNotification((d && (d.message || d?.data?.message)) || `Lỗi tải mine loại ${type}.`, 'error');
    return false;
  } catch (e) {
    console.error(`${this.logPrefix} Lỗi mạng (tải mine ${type}):`, e);
    return false;
  }
};

const loadTypes = async (typesToLoad) => {
  await Promise.all((typesToLoad || []).map(t => fetchMinesByType(t)));
};

// 1) Load lần đầu
await loadTypes(mineTypes);

// 2) Retry loại bị thiêu (tải đã 2 lần)
for (let attempt = 1; attempt <= 2 && missingTypes.size > 0; attempt++) {
  const retryTypes = Array.from(missingTypes);
  console.warn(`${this.logPrefix} ⚠️ getAllMines thiêu loại: ${retryTypes.join(', ')}. Retry lần ${
attempt}/2...`);
  await this.delay(500 * attempt);
  await loadTypes(retryTypes);
}

const allMines = [];
mineTypes.forEach(t => {
  const arr = minesByType.get(t);
  if (arr && arr.length) allMines.push(...arr);
});

if (missingTypes.size > 0) {
  const missing = Array.from(missingTypes);
  showNotification(`Chưa tải đủ 3 loại mine. Thiêu: ${missing.join(', ')}. (Không cache dữ liệu thiêu)`,
'error');
}

// --- Sắp xếp ---
allMines.sort((a, b) => {
  const typeOrder = { 'gold': 1, 'silver': 2, 'copper': 3 };
  const typeComparison = typeOrder[a.type] - typeOrder[b.type];
  if (typeComparison === 0) {
    return a.name.localeCompare(b.name, 'vi', { sensitivity: 'base' });
  }
  return typeComparison;
});

// --- Sinh HTML ---
const mineOptionsHtml = allMines.map(mine => {
  let typePrefix = '';
  if (mine.type === 'gold') typePrefix = '[Thượng] ';
  else if (mine.type === 'silver') typePrefix = '[Trung] ';
  else if (mine.type === 'copper') typePrefix = '[Hạ] ';
  return `<option value="${mine.id}">${typePrefix}${mine.name} (${mine.id})</option>`;
}).join('');

// --- Lưu cache (không có thời hạn) ---
if (missingTypes.size === 0) {
  localStorage.setItem(cacheKey, JSON.stringify({
    data: allMines,
    optionsHtml: mineOptionsHtml
  }));
}

return {
  optionsHtml: mineOptionsHtml,
  minesData: allMines
};
}

async enterMine(mineId) {
  // Lấy nonce
  const nonce = await this.#getNonce('enter_mine');
  if (!nonce) {
    showNotification('Lỗi nonce (enter_mine).', 'error');
    return false;
  }

  if (!nonce) {
    showNotification('Lỗi nonce (enter_mine).', 'error');
    return false;
  }

  // Hàm gửi request
  const post = async (payload) => {
    const r = await fetch(this.ajaxUrl, {
      method: 'POST',
      headers: this.headers,
      body: new URLSearchParams(payload),
      credentials: 'include'
    });
    return r.json();
  };
  this.securityToken = await getSecurityToken(this.khoangMachUrl);
}

```

```

    try {
        const d = await post({ action: hData && hData.act ? hData.act.kmEnter : 'enter_mine', mine_id: mineId,
security_token: this.securityToken, security: nonce });

        if (d.success) {
            showNotification(d.data.message, 'success');
            return true;
        }
        console.log(`${this.logPrefix} ❌ Vào mỏ thất bại:`, JSON.stringify(d));
        const msg = d.data.message || 'Lỗi vào mỏ.';

        if (msg.includes('đạt đủ thưởng ngày')) {
            taskTracker.markTaskDone(accountId, 'khoangmach');
            showNotification(msg, 'error');
        }
        else if (msg.includes('Có phn thưởng chưa nhận')) {
            // Nếu bị sát hại tại khoáng mạch → nhận thưởng trước
            const nonce = await this.#getNonce('claim_reward_km');
            if (!nonce) {
                showNotification('Lỗi nonce (claim_reward_km).', 'error');
                return false;
            }
            this.securityToken = hData.securityToken || await getSecurityToken(this.khoangMachUrl);
            const reward = await post({ action: hData && hData.act ? hData.act.kmReward : 'claim_reward_km',
security_token: this.securityToken, security: nonce });
            if (reward.success) {
                showNotification(`Nhận thưởng <b>${reward.data.total_tuvi}</b> tu vi và ${reward.data.
total_tinh_thach} tinh thạch</b> tại khoáng mạch ${reward.data.mine_name}`, 'info');
                return this.enterMine(mineId); // gọi lại đ vào mỏ
            } else {
                showNotification('Lỗi nhận thưởng khi bị đánh ra khỏi mỏ khoáng', 'warn');
            }
        }
        else {
            showNotification(msg, 'error');
            console.log(`${this.logPrefix} ❌ ${msg}`);
        }
        // Kiểm tra thông tin mỏ và kiểm tra ngoại tông
        let mineInfo = await this.getUsersInMine(mineId);
        if (!mineInfo) throw new Error('Lỗi lấy thông tin chi tiết trong mỏ');
        const users = mineInfo.users || [];
        if (users.length === 0) {
            console.log(`[Khoáng mạch] Mỏ ${mineId} trống.`);
            showNotification('Mỏ trống trơn??', 'warn');
            throw new Error('Mỏ trống trơn??');
        }

        // Kiểm tra ngoại tông
        const outerUsers = users.filter(u => !u.lien_minh && !u.dong_mon);
        this.checkAndNotifyOuterEnemies(outerUsers, mineId);
        return false;
    } catch (e) {
        console.error(`${this.logPrefix} ❌ Lỗi mạng (vào mỏ):`, e);
        return false;
    }
}

async decodeAvatar(encoded, viewerId) {
    try {
        // ❌ Validate input
        if (!encoded || typeof encoded !== 'string') {
            return null;
        }

        const key = (viewerId % 251) + 1;
        // ❌ Browser không có Buffer, dùng atob() để decode Base64
        const raw = atob(encoded);
        let result = '';
        for (let i = 0; i < raw.length; i++) {
            result += String.fromCharCode(raw.charCodeAt(i) ^ (key ^ (i % 7)));
        }
        return result;
    } catch (e) {
        console.error('decodeAvatar error:', e, 'Input:', encoded);
        return null;
    }
}

getIdFromAvatar(avatarUrl) {
    // ❌ Validate input
    if (!avatarUrl || typeof avatarUrl !== 'string') {
        return null;
    }

    // Tách user ID từ avatar URL: /ultimatemember/{ID}/
    const idFromAvatar = (avatarUrl.match(/\/ultimatemember\/(\\d+\\/)/i) || [])[1];
    if (idFromAvatar) return parseInt(idFromAvatar);
    return null;
}

async getUsersInMine(mineId) {
    // --- 1. Lấy 'security' nonce (giữ logic cache cũa bạn) ---
    let nonce = '';
    if (this.getUsersInMineNonce) {
        nonce = this.getUsersInMineNonce;
        console.log(`${this.logPrefix} ❌ Dùng 'security' nonce từ cache.`);
    } else {
        console.log(`${this.logPrefix} ➡️ Cache nonce không có, tải mới...`);
        // Gợi ý: this.#getNonce là hàm private của class bạn
        nonce = await this.#getNonce('get_users_in_mine');

        if (nonce) {
            this.getUsersInMineNonce = nonce; // lưu lại để dùng lần sau
        }
    }

    // Nếu page hiện tại là khoáng mạch thì lấy thưởng token từ đó
    // this.securityToken = await getSecurityToken(this.khoangMachUrl);
    this.securityToken = hData && hData.securityToken ? hData.securityToken : await getSecurityToken(this.
khoangMachUrl);

    // --- 3. Kiểm tra cũ hai token ---
    if (!nonce || !this.securityToken) {
        let errorMsg = 'Lỗi (get_users):';
        if (!nonce) errorMsg += " Không tìm thấy 'security' nonce.";
        if (!this.securityToken) errorMsg += " Không tìm thấy 'security_token' (hh3dData).";
        showNotification(errorMsg, 'error');
    }
}

```

```

        this.getUsersInMineNonce = null; // Xóa cache nonce hìng nìu có
        return null;
    }

    // --- 4. Tạo payload (Đã thêm security_token) ---
    const payload = new URLSearchParams({
        action: hData && hData.act ? hData.act.kmUsers : 'get_users_in_mine',
        mine_id: mineId,
        security_token: this.securityToken, // <-- THÊM DÒNG NÀY
        security: nonce
    });

    // --- 5. Gửi fetch ---
    try {
        const r = await fetch(this.ajaxUrl, { method: 'POST', headers: this.headers, body: payload, credentials:
'include' });
        const d = await r.json();
        console.log(`${this.logPrefix} [ ] Người trong m[ ]:`, d);
        // Logic tr[ ] v[ ] c[ ]a bạn (hoạt động t[ ]t)
        return d.success ? d.data : (showNotification(d.data.message || 'L[ ]i l[ ]y thông tin người chơi.',
'error'), null);

        } catch (e) {
            console.error(`${this.logPrefix} [ ] L[ ]i mạng (l[ ]y user):`, e);
            return null;
        }
    }

    async takeOverMine(mineId) {
        const nonce = await this.#getNonce('change_mine_owner');
        if (!nonce) { showNotification('L[ ]i nonce (take_over).', 'error'); return false; }
        this.securityToken = await getSecurityToken(this.khoangMachUrl);
        const payload = new URLSearchParams({ action: hData && hData.act ? hData.act.kmOwner : 'change_mine_owner',
mine_id: mineId, security_token: this.securityToken, security: nonce });
        try {
            const r = await fetch(this.ajaxUrl, { method: 'POST', headers: this.headers, body: payload, credentials:
'include' });
            const d = await r.json();
            if (d.success) {
                showNotification(d.data.message, 'success');
                return true;
            } else {
                showNotification(d.data.message || 'L[ ]i đoạt m[ ]:', 'error');
                return false;
            }
        } catch (e) { console.error(`${this.logPrefix} [ ] L[ ]i mạng (đoạt m[ ]):`, e); return false; }
    }

    async buyBuffItem() {
        const nonce = await this.#getNonce('buy_item_khoang');
        if (!nonce) { showNotification('L[ ]i nonce (buy_item).', 'error'); return false; }
        const payload = new URLSearchParams({ action: hData && hData.act ? hData.act.kmBuy : 'buy_item_khoang',
security: nonce, item_id: 4 });
        try {
            const r = await fetch(this.ajaxUrl, { method: 'POST', headers: this.headers, body: payload, credentials:
'include' });
            const d = await r.json();
            if (d.success) {
                showNotification(d.data.message || 'Đã mua Linh Quang Phù', 'success');
                this.buffBought = true;
                return true;
            } else {
                showNotification(d.data.message || 'L[ ]i mua Linh Quang Phù', 'error');
                return false;
            }
        } catch (e) { console.error(`${this.logPrefix} [ ] L[ ]i mạng (mua buff):`, e); return false; }
    }

    async claimReward(mineId) {
        const leaveMineToClaimReward = localStorage.getItem('khoangmach_leave_mine_to_claim_reward_${accountId}')
=== 'true';
        if (leaveMineToClaimReward) {
            const left = await this.leaveMine(mineId);
            if (!left) {
                showNotification('Không th[ ] rời m[ ] để nhận th[ ]ng.', 'error');
                return false;
            } else {
                await this.delay(500); // đợi 1s cho ch[ ]c
                const entered = await this.enterMine(mineId);
                if (!entered) {
                    showNotification('Không th[ ] vào lại m[ ] sau khi nhận th[ ]ng.', 'error');
                    return false;
                } else {
                    // Không đặt thời gian chờ lớn để duy trì việc ki[ ]m tra m[ ]i X phút
                    // taskTracker.adjustTaskTime(accountId, 'khoangmach', timePlus('30:00'));
                    return true;
                }
            }
        } else {
            const nonce = await this.#getNonce('claim_mycred_reward');
            if (!nonce) { showNotification('L[ ]i nonce (claim_reward).', 'error'); return false; }
            this.securityToken = hData.securityToken || await getSecurityToken(this.khoangMachUrl);
            const payload = new URLSearchParams({ action: hData && hData.act ? hData.act.kmClaim :
'claim_mycred_reward', mine_id: mineId, security_token: this.securityToken, security: nonce });
            try {
                const r = await fetch(this.ajaxUrl, { method: 'POST', headers: this.headers, body: payload,
credentials: 'include' });
                const d = await r.json();
                if (d.success) {
                    showNotification(d.data.message, 'success');
                    // Không đặt thời gian chờ lớn để duy trì việc ki[ ]m tra m[ ]i X phút
                    // taskTracker.adjustTaskTime(accountId, 'khoangmach', timePlus('30:00'));
                    return true;
                } else {
                    showNotification(d.data.message || 'L[ ]i nhận th[ ]ng.', 'error');
                    return false;
                }
            } catch (e) { console.error(`${this.logPrefix} [ ] L[ ]i mạng (nhận th[ ]ng):`, e); return false; }
        }
    }

    async attackUser(attackToken, mineId) {
        // [ ] Ki[ ]m tra cooldown: không cho t[ ]n công cách nhau dưới 5500ms
        const now = Date.now();
        if (this._lastAttackTime && (now - this._lastAttackTime) < 5500) {
            const remaining = Math.ceil((5500 - (now - this._lastAttackTime)) / 1000);

```

```

        showNotification(`Vui lòng chờ ${remaining}s trước khi tìn công tiếp.`, 'warn');
        return false;
    }

    const security = await this.#getNonce('attack_user_in_mine');
    const securityToken = await getSecurityToken(this.khoangMachUrl);
    if (!security) {
        showNotification(`Lỗi nonce (attack_user_in_mine).`, 'error');
        return false;
    }
    console.log(`${this.logPrefix} Đang tìn công người chơi trong m[] ${mineId} với attackToken: ${
attackToken}`);
    const payload = new URLSearchParams({ action: hData?.act?.kmAttack || 'attack_user_in_mine', attack_token:
attackToken, mine_id: mineId, security_token: securityToken, security: security });
    try {
        const r = await fetch(this.ajaxUrl, { method: 'POST', headers: this.headers, body: payload, credentials:
'include' });
        const d = await r.json();
        if (d.success) {
            this._lastAttackTime = Date.now(); // Ghi lại thời điểm tìn công thành công
            showNotification(d.data.message || JSON.stringify(d) || `Đã tìn công người chơi.`, 'success');
            return true;
        } else {
            showNotification(d.data.message || JSON.stringify(d) || `Lỗi tìn công người chơi.`, 'error');
            return false;
        }
    } catch (e) { console.error(`${this.logPrefix} Lỗi mạng (tìn công user):`, e); return false; }
}

async leaveMine(mineId) {
    const nonce = await this.#getNonce('leave_mine');
    if (!nonce) { showNotification(`Lỗi nonce (leave_mine).`, 'error'); return false; }
    this.securityToken = await getSecurityToken(this.khoangMachUrl);
    const payload = new URLSearchParams({ action: hData && hData.act ? hData.act.kmLeave : 'leave_mine',
mine_id: mineId, security_token: this.securityToken, security: nonce });
    try {
        const r = await fetch(this.ajaxUrl, { method: 'POST', headers: this.headers, body: payload, credentials:
'include' });
        const d = await r.json();
        if (d.success) {
            showNotification(d.data.message, 'success');
            return true;
        } else {
            showNotification(d.data.message || `Lỗi rời m[]`, 'error');
            return false;
        }
    } catch (e) { console.error(`${this.logPrefix} Lỗi mạng (rời m[]):`, e); return false; }
}

async doKhoangMach() {
    const selectedMineSetting = localStorage.getItem(`khoangmach_selected_mine_${accountId}`);
    if (!selectedMineSetting) {
        showNotification(`Vui lòng chọn một m[] trong cài đặt.`, 'error');
        throw new Error('Bạn chưa chọn m[]');
    }

    const selectedMineInfo = JSON.parse(selectedMineSetting);
    if (!selectedMineInfo || !selectedMineInfo.id || !selectedMineInfo.type) {
        showNotification(`Cài đặt m[] không hợp lệ.`, 'error');
        throw new Error('Cài đặt m[] không hợp lệ.');
```

```

        myIndex = i;
        break;
    }
}
if (myIndex === -1) {
    console.log(`[Khoáng mạch] Không tìm thấy vị trí. Bạn chưa vào mỏ ${targetMine.name}.`);
    showNotification(`Bạn chưa vào mỏ ${targetMine.name}.`, 'warn');
    return true;
}

// Kiểm tra ngoại tuyến
const outerUsers = users.filter(u => !u.lien_minh && !u.dong_mon);
this.checkAndNotifyOuterEnemies(outerUsers, targetMine);

let myInfo = users[myIndex];
console.log(`[Khoáng mạch] Vị trí: ${myIndex}, Tên: ${myInfo.name}, Time: ${myInfo.time_spent}`);

// Kiểm tra thời gian
if (myInfo.time_spent !== "Đạt tối đa") {
    const timeMatch = myInfo.time_spent.match(/(\d+)\s*phút/);
    const minutesSpent = timeMatch ? parseInt(timeMatch[1]) : 0;

    let shouldWait = false;
    let nextTime = null;

    if (rewardTimeSelected === 'max') {
        // Chờ đến khi đạt tối đa (30 phút)
        shouldWait = true;
        nextTime = Date.now() + Math.max(30 * 60 * 1000 - (minutesSpent * 60 * 1000), 0);
        showNotification(`Khoáng mạch chưa đủ thời gian.<br>Hiện đạt: <b>${myInfo.time_spent}</b>`);
    } else {
        // Kiểm tra với thời gian cụ thể
        const requiredMinutes = parseInt(rewardTimeSelected);
        if (minutesSpent < requiredMinutes) {
            shouldWait = true;
            nextTime = Date.now() + Math.max((requiredMinutes - minutesSpent) * 60 * 1000, 0);
            showNotification(`Khoáng mạch chưa đủ thời gian.<br>Hiện đạt: <b>${myInfo.time_spent}</b>`);
        }
    }

    if (shouldWait) {
        // Không đặt thời gian chờ lớn để duy trì việc kiểm tra mỏ X phút
        // taskTracker.adjustTaskTime(accountId, 'khoangmach', nextTime);
        break;
    }
}

// Kiểm tra trạng thái bonus
let bonus = mineInfo.bonus_percentage || 0;
let canClaim = false;
if (rewardMode === "any") {
    canClaim = true;
} else if (rewardMode === "20" && bonus >= 20) {
    canClaim = true;
} else if (rewardMode === "100" && bonus >= 100) {
    canClaim = true;
} else if (rewardMode === "110" && bonus === 110) {
    canClaim = true;
}

if (canClaim) {
    console.log(`[Khoáng mạch] Nhận thưởng tại mỏ ${targetMine.id}, bonus=${bonus}%`);
    await this.claimReward(targetMine.id); // Nhận thưởng
    break; // Thoát vòng lặp sau khi nhận thưởng
} else {
    console.log(`[Khoáng mạch] Bonus từ ${bonus}% chưa đạt ngưỡng ${rewardMode}%`);

    // Nếu có thể, thì takeover trước (option đoạt mỏ khi chưa buff)
    if (autoTakeover && mineInfo.can_takeover) {
        await this.delay(500);
        console.log(`[Khoáng mạch] Thử đoạt mỏ ${targetMine.id}...`);
        await this.takeOverMine(targetMine.id);
        continue;
    }

    // Nếu có thể, thì takeover trước (option đoạt mỏ bất kỳ buff)
    if (autoTakeoverRotation && mineInfo.can_takeover) {
        await this.delay(500);
        console.log(`[Khoáng mạch] Thử đoạt mỏ ${targetMine.id}...`);
        await this.takeOverMine(targetMine.id);
        continue;
    }

    // Nếu có chọn mua buff
    if (useBuff && bonus > 20 && !this.buffBought) {
        await this.delay(500);
        console.log(`[Khoáng mạch] Mua linh quang phù...`);
        await this.buyBuffItem(targetMine.id);
        // Đợi một chút để server xử lý
        await new Promise(resolve => setTimeout(resolve, 300));
        continue;
    }

    // Nếu không thì làm gì, thoát khỏi vòng lặp
    showNotification(`[Khoáng mạch] Bonus ${bonus}% chưa đạt ${rewardMode}%<br>Hiện không thể đoạt mỏ.`);
    break;
}

}
}

/**
 * Lấy danh sách tổng môn
 * @returns Promise<Array<{id: string, name: string, level: number}>> Mảng đối tượng tổng môn
 * ví dụ: [{id: "123", name: "Tổng Môn A", level: 6}, ...]
 */
async getListTongMon() {
    try {
        // 1. SỬA LỖI LOGIC URL:
        // Dùng đường dẫn tương đối "/" để tự động lấy domain hiện tại.
        // Không cần biến "weberl" (tránh lỗi weberl is not defined).
        const response = await fetch("/danh-sach-cac-tong-mon-tai-hoathinh3d");
        // Kiểm tra trạng thái HTTP
    }
}

```

```

    if (!response.ok) {
        throw new Error(`Lỗi khi gọi API: ${response.status} ${response.statusText}`);
    }

    // 2. Chuyển đổi dữ liệu
    const htmlText = await response.text();
    const parser = new DOMParser();
    const doc = parser.parseFromString(htmlText, "text/html");

    // Chọn danh sách hàng
    const rows = doc.querySelectorAll('table.guild-table tbody tr');
    const results = [];
    console.log(`Tìm thấy ${rows.length} hàng thông tin.`);

    rows.forEach(row => {
        // 1. CẤU TRÚC MẪU: Tìm guild-info-wrapper
        const guildWrapper = row.querySelector('.guild-info-wrapper');
        if (!guildWrapper) return;

        // 2. LẤY ID TỪ LINK /tong-mon/[ID]
        const link = guildWrapper.querySelector('a[href*="/tong-mon/"]');
        let id = null;
        if (link) {
            const href = link.getAttribute('href') || '';
            const match = href.match(/\/tong-mon\/(\d+)\//);
            if (match) id = match[1];
        }

        // Fallback: Lấy từ nút button nếu không tìm thấy từ link
        if (!id) {
            const btn = row.querySelector('button.join-group');
            id = btn ? btn.getAttribute('data-group-id') : null;
        }

        // 3. LẤY TÊN TỪ .guild-name
        const nameSpan = guildWrapper.querySelector('.guild-name');
        let nameText = '';
        if (nameSpan) {
            nameText = (nameSpan.textContent || '').trim()
                .replace(/^[\"\s]+|[\s\"]+$/g, ''); // Loại bỏ dấu ngoặc kép và khoảng trắng đầu/cuối
        }

        // 4. LẤY LEVEL TỪ .guild-level
        let levelNum = 0;
        const levelSpan = guildWrapper.querySelector('.guild-level');
        if (levelSpan) {
            const levelText = levelSpan.textContent || '';
            const match = levelText.match(/\d+/);
            if (match) levelNum = parseInt(match[0], 10);
        }

        // 5. CHỈ LƯU KHI CÓ ID VÀ TÊN HỢP LỆ
        if (id && nameText && nameText.length > 0) {
            results.push({
                id: id,
                name: nameText,
                level: levelNum
            });
            // console.log(`Tìm thấy thông tin: ID=${id}, Name=${nameText}, Level=${levelNum}`);
        }
    });

    return results;
} catch (error) {
    // Ghi log lỗi để debug
    console.error("Lỗi tại getListTongMon:", error);
    // Ném lỗi ra ngoài để hàm gọi bên ngoài biết là có lỗi
    throw error;
}

}

parseGroupRoleHtml(groupRoleHtml) {
    if (!groupRoleHtml || typeof groupRoleHtml !== 'string') {
        return { tongMonName: null, role: null };
    }
    try {
        const doc = new DOMParser().parseFromString(`<div>${groupRoleHtml}</div>`, 'text/html');
        const root = doc.body;
        const tongLink = root.querySelector('a.tong-link');
        const tongMonName = tongLink ? tongLink.textContent.trim() || null : null;
        const roleSpans = Array.from(root.querySelectorAll('span[data-tooltip]'))
            .filter(el => !el.classList.contains('tong-cap-wrapper'));
        const role = roleSpans.length ? roleSpans[roleSpans.length - 1].getAttribute('data-tooltip').trim() || null : null;
        return { tongMonName, role };
    } catch {
        return { tongMonName: null, role: null };
    }
}

/**
 * Tìm kiếm kẻ địch theo ID và/hoặc theo Tông Môn (ID tông).
 * @param {string[]} enemyList - danh sách userId (string)
 * @param {string[]} tongMonList - danh sách groupId tông môn (string)
 * @param {function} onProgressCallback - callback để cập nhật tiến độ UI
 * @returns {Promise<Array>}
 */

/**
 * Kiểm tra và hiển thị cảnh báo người chơi ngoại tông kèm cơ chế chống spam (cooldown 3 phút hoặc khi đổi danh sách)
 * @param {Array} outerUsers Danh sách user ngoại tông
 * @param {Object|string|number} mine Thông tin mine hoặc ID mine
 */
checkAndNotifyOuterEnemies(outerUsers, mine) {
    const outerNotification = localStorage.getItem('khoangmach_outer_notification') === 'true';
    if (!outerNotification || outerUsers.length === 0) return;

    const mineObj = typeof mine === 'object' && mine !== null ? mine : { id: mine, name: 'Mine ' + mine };
    const currentIds = outerUsers.map(u => u.profile_id || u.id).sort().join(',');
    const now = Date.now();
    const cooldown = 3 * 60 * 1000; // 3 phút

```

```

    if (currentIds !== this.lastOuterUserIds || (now - this.lastOuterNotificationTime > cooldown)) {
        this.lastOuterUserIds = currentIds;
        this.lastOuterNotificationTime = now;
        showNotification(`Cố <b>${outerUsers.length}</b> người chơi ngoại tông trong m[]`, 'warn');
        this.showOuterEnemyModal(outerUsers, mineObj);
    }
}

/**
 * Hi[]n thì modal c[]nh báo ngoại tông trong m[] hiện tại
 * @param {Array} outerUsers Danh sách user ngoại tông
 * @param {Object|String|Number} mine Thông tin m[] hiện tại
 */
async showOuterEnemyModal(outerUsers, mine) {
    const PANEL_ID = 'outerEnemyModal';
    const oldPanel = document.getElementById(PANEL_ID);
    if (oldPanel) oldPanel.remove();

    const mineObj = typeof mine === 'object' && mine !== null ? mine : { id: mine, name: 'M[] ' + mine };
    const accountId = await getAccountId();
    const esc = (v) => String(v ?? '').replace(/&g, ' & ');
    const esc = (v) => String(v ?? '').replace(/&g, ' & ');
    const esc = (v) => String(v ?? '').replace(/&g, ' & ');

    const usersHtml = (await Promise.all(outerUsers.map(async (u) => {
        const attackToken = u.attack_token || u.att || u.uid;
        const avatarUrl = u.avatar || await this.decodeAvatar(u.avatar, accountId);
        const id = u.profile_id || this.getIdFromAvatar(avatarUrl) || u.uid;
        const { tongMonName: group = null, role: _role = null } = u.group_role_html ? this.parseGroupRoleHtml(u.group_role_html) : {};
        const groupDisplay = group || 'Vô phái';
        const roleDisplay = _role || 'Thành viên';

        return `
            <div style="padding: 8px 0; border-bottom: 1px dashed #333; display: flex; justify-content: space-between; align-items: center; gap: 10px;">
                <div style="display: flex; align-items: center; gap: 10px; flex: 1;">
                    <a href="/profile/${id}" target="_blank" style="flex-shrink: 0; display: block;">
                        
                    </a>
                    <div style="display: flex; flex-direction: column; min-width: 0; flex: 1;">
                        <div style="color: #ff6b6b; font-weight: 500; overflow: hidden; text-overflow: ellipsis; white-space: nowrap;">${esc(u.name)} <span style="font-size: 11px; color: #777; overflow: hidden; text-overflow: ellipsis; white-space: nowrap;">${esc(groupDisplay)} - ${esc(roleDisplay)}</div>
                    </div>
                </div>
                <div style="display: flex; flex-direction: column; align-items: flex-end; gap: 4px; flex-shrink: 0;">
                    <div style="display: flex; gap: 5px;">
                        <button class="oe-btn-tuvi" data-uid="${id}" data-attack-token="${attackToken}" style="border: none; background: #039be5; color: white; border-radius: 3px; padding: 3px 8px; font-size: 11px; cursor: pointer; font-weight: bold;"></button>
                        <button class="oe-btn-attack" data-uid="${id}" data-attack-token="${attackToken}" data-mid="${mineObj.id}" style="border: none; background: #d32f2f; color: white; border-radius: 3px; padding: 3px 8px; font-size: 11px; cursor: pointer; font-weight: bold;"></button>
                    </div>
                    <div id="oe-info-${id}" style="font-size: 10px; color: #b0bec5; min-height: 14px;"></div>
                </div>
            </div>
        `;
    }))).join('');

    const panel = document.createElement('div');
    panel.id = PANEL_ID;
    panel.style.cssText = `
        position: fixed; right: 20px; bottom: 20px;
        width: 400px; max-width: 95vw;
        background: #1a1a1a; color: #e0e0e0;
        border: 1px solid #c62828; border-radius: 8px;
        box-shadow: 0 10px 25px rgba(0,0,0,0.7);
        z-index: 999999; font-family: sans-serif;
        display: flex; flex-direction: column;
        overflow: hidden; font-size: 13px;
    `;

    panel.innerHTML = `
        <div style="padding: 10px 12px; background: #c62828; border-bottom: 1px solid #b71c1c;">
            <div style="display: flex; justify-content: space-between; align-items: center;">
                <div style="font-weight: bold; font-size: 14px; color: #fff;">
                    [] Ngoại Tông Trong M[] <span style="background: #b71c1c; border-radius: 10px; padding: 1px 7px;">
                        ">${outerUsers.length}</span>
                </div>
                <div style="display: flex; gap: 5px;">
                    <button id="oe-goto" style="background: #fff3e0; color: #e65100; border: none; border-radius: 4px; padding: 4px 8px; font-size: 11px; cursor: pointer; font-weight: bold;"></button>
                    <button id="oe-close" style="background: #333; border: none; color: #fff; width: 28px; height: 28px; border-radius: 4px; cursor: pointer;"></button>
                </div>
            </div>
            <div style="font-size: 11px; color: #ffcd22; margin-top: 3px;">[] ${esc(mineObj.name)}</div>
        </div>
        <div style="padding: 8px 12px; max-height: 55vh; overflow-y: auto; background: #1a1a1a;">
            ${usersHtml}
        </div>
    `;

    document.body.appendChild(panel);

    panel.querySelector('#oe-goto').onclick = () => { window.location.href = this.khoangMachUrl; };
    panel.querySelector('#oe-close').onclick = () => panel.remove();

    panel.querySelectorAll('.oe-btn-tuvi').forEach(btn => {
        btn.onclick = async (e) => {
            e.stopPropagation();
            const uid = btn.getAttribute('data-uid');
            const resDiv = document.getElementById(`oe-info-${uid}`);
            btn.disabled = true; btn.textContent = '...';
            if (resDiv) resDiv.textContent = 'Đang xem...';
            try {
                const tierText = await hienTuviKM.getProfileTier(uid);
                if (resDiv) resDiv.textContent = tierText || 'K.Rô';
            } catch (err) {
                if (resDiv) resDiv.textContent = 'L[]i';
            } finally {
                btn.textContent = '[]'; btn.disabled = false;
            }
        }
    });

```

```

    }
  });
});

panel.querySelector('.oe-btn-attack').forEach(btn => {
  btn.onclick = (e) => {
    e.stopPropagation();
    const attackToken = btn.getAttribute('data-attack-token');
    const mid = btn.getAttribute('data-mid');
    btn.textContent = 'x';
    if (typeof khoangmach !== 'undefined' && khoangmach.attackUser) {
      khoangmach.attackUser(attackToken, mid);
      setTimeout(() => { btn.textContent = '✓'; }, 500);
    } else {
      showNotification('Lỗi: Không tìm thấy hàm tấn công!', 'error');
    }
  }
});
});
}

/**
 * Hiển thị kết quả tìm kiếm với thông tin nguồn và thời gian
 * @param {Array} foundUsers Danh sách kết quả tìm thấy
 * @param {Number} timestamp Thời gian dữ liệu được tạo (Date.now())
 * @param {String} source Nguồn dữ liệu ('Server' hoặc 'Quét trực tiếp')
 */
async showEnemySearchResults(foundUsers, timestamp, source = 'N/A') {
  // 1. Kiểm tra dữ liệu đầu vào
  if (!Array.isArray(foundUsers) || foundUsers.length === 0) {
    showNotification('Không tìm thấy kết quả nào phù hợp trong các mục.', 'info');
    return;
  }

  // Get account ID
  const accountId = await getAccountId();

  const PANEL_ID = 'enemyDashboard';
  const RESTORE_ID = 'enemyDashboardRestore';

  // 2. Xóa panel cũ
  const oldPanel = document.getElementById(PANEL_ID);
  if (oldPanel) oldPanel.remove();
  const oldRestore = document.getElementById(RESTORE_ID);
  if (oldRestore) oldRestore.remove();

  // 3. Tiện ích
  const esc = (v) => String(v ?? '').replace(/&g/, '&amp;').replace(/<g/, '&lt;').replace(/>g/, '&gt;').replace(/"/g, '&quot;');
  const timeSinceStr = (ts) => {
    if (!ts) return 'Vừa xong';
    const seconds = Math.floor((Date.now() - ts) / 1000);
    if (seconds < 60) return `${seconds} giây trước`;
    const minutes = Math.floor(seconds / 60);
    if (minutes < 60) return `${minutes} phút trước`;
    return 'Khả lâu trước';
  };

  // 4. Gom nhóm
  const minesMap = foundUsers.reduce((acc, u) => {
    const mid = String(u.mineId || 'unknown');
    if (!acc[mid]) {
      acc[mid] = {
        id: mid,
        name: u.mineName || 'Mìn Lạ',
        users: [],
        tongMons: new Set()
      };
    }
    acc[mid].users.push(u);
    if (u.tongMonName) acc[mid].tongMons.add(u.tongMonName);
    return acc;
  }, {});

  const sortedMines = Object.values(minesMap).sort((a, b) => b.users.length - a.users.length);

  // 5. Tạo Panel
  const panel = document.createElement('div');
  panel.id = PANEL_ID;
  panel.className = 'enemy-dashboard';
  panel.style.cssText = `
    position: fixed; right: 20px; bottom: 20px;
    width: 460px; max-width: 95vw;
    background: #1a1a1a; color: #e0e0e0;
    border: 1px solid #444; border-radius: 8px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.7);
    z-index: 999999; font-family: sans-serif;
    display: flex; flex-direction: column;
    overflow: hidden; font-size: 13px;
  `;

  const sourceColor = source.includes('Server') ? '#4caf50' : '#ff9800';

  // HTML Structure - Build mines HTML first
  const minesHtml = (await Promise.all(sortedMines.map(async (mine) => {
    const tongList = mine.tongMons.size > 0 ? Array.from(mine.tongMons).join(', ') : 'Vô phải';
    const usersHtml = (await Promise.all(mine.users.map(async (u) => {
      const isAlly = u.dong_mon || u.lien_minh;
      const attackToken = u.attack_token || u.att || u.id;
      const avatarUrl = u.avatar || await this.decodeAvatar(u.avatar, accountId);
      // console.log('Avatar URL for user ${u.name} (ID: ${u.id}): ${avatarUrl} (avatar: ${u.avatar})');
      const id = this.getIdFromAvatar(avatarUrl) || u.id;
      // console.log('User: ${u.name}, ID: ${id}, Đồng Môn: ${u.dong_mon}, Liên Minh: ${u.lien_minh}');
      const allyLabel = u.dong_mon ? '☑ Đồng Môn' : (u.lien_minh ? '☑ Liên Minh' : '');
      const nameColor = isAlly ? '#4caf50' : '#ff6b6b';
      return `
        <div style="padding: 6px 0; border-bottom: 1px dashed #333; display: flex; justify-content:
space-between; align-items: center; gap: 10px;">
          <div style="display: flex; align-items: center; gap: 10px; flex: 1;">
            <a href="/profile/${id}" target="_blank" style="flex-shrink:0;display:block">
              
            </a>
            <div style="display: flex; flex-direction: column; min-width: 0; flex: 1;">
              <div style="color: ${nameColor}; font-weight: 500; overflow: hidden; text-overflow:
ellipsis; white-space: nowrap;">${esc(u.name)} <span style="font-size: 10px;">${allyLabel}</span>` : ''
            </div>
          </div>
        `
      `;
    }
  }

```

```

        <div style="font-size: 11px; color: #777; overflow: hidden; text-overflow: ellipsis;
white-space: nowrap;">${esc(u.tongMonName || 'Vô phải')} - ${esc(u.role || 'Thành viên')}

```

```

        if (e.target.tagName === 'BUTTON') return;
        const targetId = header.getAttribute('data-target');
        const content = document.getElementById(targetId);
        const arrow = header.querySelector('.arrow');
        if (content) {
            const isOpen = content.style.display === 'block';
            content.style.display = isOpen ? 'none' : 'block';
            if (arrow) arrow.textContent = isOpen ? '▼' : '▲';
        }
    }
});

// Mũi đầu tiên
const firstHeader = panel.querySelector('.mine-header');
if (firstHeader) firstHeader.click();

// Biện quản lý trạng thái nút Global
const btnWeakGlobal = panel.querySelector('#btn-attack-weak-global');
let weakCountGlobal = 0;

// =====
// ❌ LOGIC: HELPER HÀM ĐƠN TỰ ĐỘNG (Dùng chung)
// =====
const runBatchAttack = async (targets, statusBtn) => {
    if (targets.length === 0) {
        showNotification('Không có mục tiêu nào!', 'warning');
        return;
    }

    if (!confirm(`Tìm thấy ${targets.length} mục tiêu "Không tên lượt".\nBắt đầu đấm? (Delay 6s/người)`) ) {
        return;
    }

    const originalText = statusBtn.textContent;
    statusBtn.disabled = true;

    for (let i = 0; i < targets.length; i++) {
        const btn = targets[i];

        // Cập nhật trạng thái nút
        statusBtn.textContent = ` ${i + 1}/${targets.length} (Chờ 6s)`;

        // Thực hiện đấm
        btn.click();

        // Xóa class
        btn.classList.remove('is-weak-target');
        btn.style.border = 'none';

        // Delay 6s (Trừ người cuối cùng)
        if (i < targets.length - 1) {
            await new Promise(r => setTimeout(r, 6000));
        }
    }

    statusBtn.textContent = 'Xong';
    setTimeout(() => {
        statusBtn.style.display = 'none'; // Ẩn nút sau khi xong
        statusBtn.disabled = false;
        statusBtn.textContent = originalText;
    }, 3000);
    showNotification('Đã xử lý xong danh sách!', 'success');
};

// =====
// ❌ LOGIC: CHECK TỬ VI
// =====
panel.querySelectorAll('.btn-check-tuvi').forEach(btn => {
    btn.onclick = async (e) => {
        e.stopPropagation();
        const uid = btn.getAttribute('data-uid');
        const resDiv = document.getElementById(`info-res-${uid}`);
        const attackBtn = btn.parentElement.querySelector('.btn-attack');

        btn.disabled = true;
        btn.textContent = '...';
        resDiv.textContent = 'Đang xem...';

        try {
            // const data = await hienTuviKM.getProfileTier(uid);
            // const tierText = await hienTuviKM.getProfileTier(uid);

            // if (data) {
            //     const tuViStr = new Intl.NumberFormat('vi-VN').format(data.tuvi || 0);
            //     let rightSideHtml = '';

            //     // ❌ KÈO THƠM: KHÔNG TÊN LƯỢT
            //     if (data.notCountAttack) {
            //         rightSideHtml = `<span style="color: #ea80fc; font-weight: bold; text-shadow: 0 0 5px rgba(234,128,252,0.5);">> Không tên lượt</span>`;
            //     }

            //     // Đánh dấu nút tên công
            //     if (attackBtn) {
            //         attackBtn.classList.add('is-weak-target');
            //         attackBtn.style.border = '1px solid #ea80fc';
            //         attackBtn.style.boxShadow = '0 0 5px #ea80fc';

            //         // 1. Cập nhật nút Global
            //         weakCountGlobal++;
            //         btnWeakGlobal.style.display = 'block';
            //         btnWeakGlobal.textContent = ` Đấm Kĩ Y ( ${weakCountGlobal} )`;

            //         // 2. Cập nhật nút Local (Cấp mũi)
            //         const mid = attackBtn.getAttribute('data-mid');
            //         const btnWeakMine = document.getElementById(`btn-weak-mine-${mid}`);
            //         if (btnWeakMine) {
            //             btnWeakMine.style.display = 'block';
            //             // Tăng đấm cho mũi (lưu vào attribute data-count)
            //             let currentCount = parseInt(btnWeakMine.getAttribute('data-count') || 0) + 1;
            //             btnWeakMine.setAttribute('data-count', currentCount);
            //             btnWeakMine.textContent = ` Đấm Kĩ Y ( ${currentCount} )`;
            //         }
            //     }
            // } else {
            //     // Kèo thường

```

```

winRateRaw}%`;
//      const winRateRaw = data.winRate || '?';
//      const winRateDisplay = String(winRateRaw).includes('%') ? winRateRaw : `${
//
//      let rateNumber = parseInt(String(winRateRaw).replace('%', ''));
//      if (isNaN(rateNumber)) rateNumber = -1;
//
//      let rateColor = '#ffffff';
//      if (rateNumber === -1) rateColor = '#808080';
//      else if (rateNumber < 25) rateColor = '#ff5f5f';
//      else if (rateNumber > 75) rateColor = '#00ff00';
//
//      rightSideHtml = `<span style="color: ${rateColor}; font-weight: bold;">${
winRateDisplay}</span>`;
//
//      // Xóa dấu hiệu nếu user soi lại và thây không còn ngon
//      if (attackBtn && attackBtn.classList.contains('is-weak-target')) {
//          attackBtn.classList.remove('is-weak-target');
//          attackBtn.style.border = 'none';
//          attackBtn.style.boxShadow = 'none';
//      }
//
//      resDiv.innerHTML = `<span style="color: #4fc3f7;">${tuViStr}</span> | ${rightSideHtml}`;
//  } else
//  {
//      resDiv.textContent = 'K.Rõ';
//      resDiv.textContent = `${tierText ? tierText : 'K.Rõ'}`;
//      resDiv.style.color = '#ff5252';
//  }
//  } catch (err) {
//      console.error(err);
//      resDiv.textContent = 'Lỗi!';
//  } finally {
//      btn.textContent = '□';
//      btn.disabled = false;
//      btn.classList.add('checked-done');
//  }
//  });
//  });
//  =====
//  × LOGIC: ATTACK (Đón l)
//  =====
panel.querySelectorAll('.btn-attack').forEach(btn => {
    btn.onclick = (e) => {
        e.stopPropagation();
        const uid = btn.getAttribute('data-uid');
        const attackToken = btn.getAttribute('data-attack-token');
        const mid = btn.getAttribute('data-mid');
        btn.textContent = '×';

        if (typeof khoangmach !== 'undefined' && khoangmach.attackUser) {
            khoangmach.attackUser(attackToken, mid);
            setTimeout(() => {
                btn.textContent = '✓';
                // btn.style.opacity = '0.5';
            }, 500);
        } else {
            showNotification("Lỗi: Không tìm thấy hàm tấn công!", "error");
        }
    };
});
//  =====
//  □ LOGIC: SOI HÀNG LOẠT (Global & Local)
//  =====
const runBatchScan = async (buttons) => {
    if (!buttons || buttons.length === 0) return;

    // Khi soi mới, cần reset các biến đếm nếu muốn chính xác tuyệt đối,
    // nhưng đây ta cứ cộng dồn cho đơn giản hoặc user tự tắt bật lại panel.
    showNotification(`Đang soi ${buttons.length} mục tiêu...`, 'info');

    for (const btn of buttons) {
        if (!btn.disabled && !btn.classList.contains('checked-done')) {
            btn.click();
            await new Promise(r => setTimeout(r, 500)); // Delay soi 500ms
        }
    }
    showNotification('Đã soi xong.', 'success');
};

panel.querySelector('#btn-scan-all').onclick = () => {
    // Reset đếm toàn cục khi soi lại từ đầu (tuỳ chọn)
    weakCountGlobal = 0;
    btnWeakGlobal.style.display = 'none';

    const allBtns = panel.querySelectorAll('.btn-check-tuvi');
    runBatchScan(allBtns);
};

panel.querySelectorAll('.btn-scan-mine').forEach(btn => {
    btn.onclick = (e) => {
        e.stopPropagation();
        const targetId = btn.getAttribute('data-target');
        const mineContainer = document.getElementById(targetId);

        // Reset đếm cục bộ của m này
        const mid = targetId.replace('m-', '');
        const btnWeakMine = document.getElementById(`btn-weak-mine-${mid}`);
        if (btnWeakMine) {
            btnWeakMine.style.display = 'none';
            btnWeakMine.setAttribute('data-count', 0);
        }

        if (mineContainer && mineContainer.style.display === 'none') mineContainer.style.display = 'block';
        if (mineContainer) {
            runBatchScan(mineContainer.querySelectorAll('.btn-check-tuvi'));
        }
    };
});
//  =====
//  □ LOGIC: Đếm K Y (X lý sự kiện click)
//  =====
//  1. Sự kiện nút Tng (Global)

```

```

btnWeakGlobal.onclick = () => {
    const targets = panel.querySelectorAll('.btn-attack.is-weak-target');
    runBatchAttack(targets, btnWeakGlobal);
};

// 2. Sự kiện nút Tùm Mìn (Local)
panel.querySelectorAll('.btn-attack-weak-mine').forEach(btn => {
    btn.onclick = (e) => {
        e.stopPropagation(); // Không đóng mìn accordion
        const targetId = btn.getAttribute('data-target'); // m-xxxx
        const mineContainer = document.getElementById(targetId);
        if (mineContainer) {
            // Chỗ tìm kìn yểu trong mìn này
            const targets = mineContainer.querySelectorAll('.btn-attack.is-weak-target');
            runBatchAttack(targets, btn);
        }
    };
});
}

}

//=====
// RƯƠNG HOẠT ĐỘNG NGAY
//=====
class HoatDongNgay {
    constructor() {
        this.ajaxUrl = we Burl + "/wp-content/themes/halimmovies-child/hh3d-ajax.php";
    }

    // 1. Lầy rương (Daily Chest)
    async getDailyChest(stage) {
        if (stage !== "stage1" && stage !== "stage2") {
            console.error("Lỗi: Stage phải là 'stage1' hoặc 'stage2'.");
            return false;
        }

        const bodyData = new URLSearchParams({
            action: hData?.act?.hdnReward || "daily_activity_reward",
            stage: stage,
            security_token: hData?.securityToken || securityToken // thêm token vào đây
        });

        console.log("Đang nhận rương ${stage} với dữ liệu:", bodyData.toString());
        try {
            const response = await fetch(this.ajaxUrl, {
                credentials: "include",
                headers: {
                    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:142.0) Gecko/20100101 Firefox/142.0",
                    "Accept": "*/*",
                    "Accept-Language": "vi,en-US;q=0.5",
                    "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
                    "X-Requested-With": "XMLHttpRequest"
                },
                body: bodyData,
                method: "POST",
                mode: "cors"
            });

            const data = await response.json();
            if (data.success || data.data.message === "Đạo hữu đã nhận phần thưởng này rồi.") {
                return true;
            } else {
                console.error("Lỗi khi nhận rương ${stage}: ", JSON.stringify(data));
                showNotification("Lỗi nhận rương hàng ngày: ${data.data.message || data.message || JSON.stringify(data)}", "error");
                return false;
            }
        } catch (error) {
            console.error("Lỗi khi lầy rương ${stage}: ", error);
            showNotification("Lỗi khi lầy rương ${stage}: ${error.message || JSON.stringify(error)}", "error");
            return false;
        }
    }

    // 2. Spin vòng quay phúc vận
    async spinLottery() {
        const nonce = await getSecurityNonce(we Burl + '?t=' + Date.now(), "restNonce");
        //console.log("Nonce spinLottery:", nonce);

        if (!nonce) {
            showNotification("Lỗi: Không thể lấy nonce cho vòng quay phúc vận", "error");
            return false;
        }

        const spinURL = we Burl + "wp-json/lottery/v1/" + (hData && hData.act ? hData.act.lotterySpin : '815b016f');
        console.log("URL vòng quay phúc vận:", spinURL);
        let remainingSpins = 4;

        do {
            try {
                const response = await fetch(spinURL, {
                    credentials: "include",
                    headers: {
                        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:142.0) Gecko/20100101 Firefox/142.0",
                        "Accept": "*/*",
                        "Accept-Language": "vi,en-US;q=0.5",
                        "X-Security-Token": hData?.securityToken || securityToken,
                        "X-WP-Nonce": nonce,
                        "Content-Type": "application/json"
                    },
                    method: "POST",
                    mode: "cors"
                });

                const data = await response.json();
                if (data.success) {
                    showNotification("Vòng quay phúc vận: ${data.message}", "success");
                    remainingSpins = data.user_info.remaining_spins;
                    if (remainingSpins === 0) {
                        return true;
                    }
                } else if (data.message === "Đạo hữu đã hết lượt quay hôm nay.") {

```

```

        return true;
    } else {
        showNotification(`❌ Lỗi khi quay vòng quay phúc vận: ${data.message}`, "error");
        return false;
    }
} catch (error) {
    console.error("Lỗi khi spin:", error);
    return false;
}

// ⏱ Chờ 1 giây tránh spam
await new Promise(r => setTimeout(r, 1000));
} while (remainingSpins > 0);
}

extractPtTokenFromHtml(html) {
    if (!html) return null;
    const match = html.match(/pt_token\s*[:=]\s*"([a-f0-9]{32,})"['"]/i) ||
        html.match(/ptToken\s*[:=]\s*"([a-f0-9]{32,})"['"]/i) ||
        html.match(/"pt-token"\s*[:=]\s*"([a-f0-9]{32,})"['"]/i) ||
        html.match(/pt_token['"]?\s*[:=]\s*"([a-f0-9]{32,})"['"]/i);
    return match ? match[1] : null;
}

async getPtToken() {
    // 1. Check globals
    if (typeof unsafeWindow !== 'undefined') {
        if (unsafeWindow.ptToken) return unsafeWindow.ptToken;
        if (unsafeWindow.pt_token) return unsafeWindow.pt_token;
    }
    if (typeof window !== 'undefined') {
        if (window.ptToken) return window.ptToken;
        if (window.pt_token) return window.pt_token;
    }

    // 2. Check current document body/scripts
    const currentHTML = document.documentElement.outerHTML;
    let token = this.extractPtTokenFromHtml(currentHTML);
    if (token) return token;

    // 3. Fallback: fetch nhiệm-vu-hang-ngay page
    try {
        console.log("[HH3D Auto] Fetching /nhiem-vu-hang-ngay to retrieve pt-token...");
        const response = await fetch(weburl + 'nhiem-vu-hang-ngay?t=' + Date.now());
        if (response.ok) {
            const html = await response.text();
            token = this.extractPtTokenFromHtml(html);
            if (token) return token;
        }
    } catch (e) {
        console.error("Lỗi khi fetch pt-token:", e);
    }
    return null;
}

// ⚙ Lấy nonce và token từ trang Pháp Tướng (logic từ ThamKhao.js)
async getPtConfig() {
    const logPrefix = "[HH3D Khóc Trần Văn]";
    try {
        const response = await fetch(weburl + "/trieu-hoi-phap-tuong?t", {
            headers: {
                "accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8",
                "sec-fetch-dest": "iframe",
                "sec-fetch-mode": "navigate",
                "sec-fetch-site": "same-origin",
                "referrer": weburl + "/tu-bao-cac?t" + Math.random().toString(36).substring(2, 8)
            },
            credentials: "include"
        });
        const html = await response.text();

        const regex = /PHAP_TUONG_CONFIG\s*=\s*{(?:\s*\S*\s*?)};/;
        const match = html.match(regex);

        if (match && match[1]) {
            const config = eval("(" + match[1] + ")");
            console.log(logPrefix, "⚙ Nonce:", config.nonce);
            console.log(logPrefix, "⚙ Token:", config.token);
            return config;
        } else {
            showNotification("❌ Không tìm thấy PHAP_TUONG_CONFIG trong HTML", "error");
            return null;
        }
    } catch (e) {
        console.error("[HH3D Khóc Trần Văn]", e);
        showNotification("❌ Lỗi khi tải trang Pháp Tướng để lấy nonce/token", "error");
        return null;
    }
}

// ⚙ Nhận lượt Khóc Trần Văn (Daily Turns) - logic từ ThamKhao.js
async claimDailyTurns() {
    const logPrefix = "[HH3D Khóc Trần Văn]";
    const config = await this.getPtConfig();
    if (!config?.nonce || !config?.token) return false;

    try {
        const response = await fetch(weburl + "wp-json/phap-tuong/v1/claim-daily-turns", {
            method: "POST",
            credentials: "include",
            headers: {
                "Content-Type": "application/json",
                "X-WP-Nonce": config.nonce,
                "X-Pt-Token": config.token
            },
            referrer: weburl + "/tu-bao-cac?t",
        });

        const data = await response.json();

        // ✅ Thành công
        if (data.success) {
            showNotification(`${logPrefix} ✅ ${data.message}`, "success");
            console.log(logPrefix, "✅ Nhận thêm ${data.turns_claimed} lượt");
            return true;
        }
    }

```

```

    }

    // Đã nhận r[i] (coi như OK)
    if (data.message && data.message.includes("Đạo hữu đã nhận lượt VIP hôm nay r[i]")) {
        showNotification(`${logPrefix} [i] ${data.message}`, "info");
        return true;
    }

    // Tính năng dành riêng cho VIP (coi như OK)
    if (data.message && data.message.includes("Tính năng dành riêng cho VIP")) {
        showNotification(`${logPrefix} [i] ${data.message}`, "error");
        return true;
    }

    // Các lỗi khác
    showNotification(`${logPrefix} [i] ${data.message}`, "error");
    return false;
} catch (err) {
    console.error(logPrefix, "[i] khi claim lượt VIP:", err);
    return null;
}
}

// Thực hiện toàn bộ hoạt động ngày
async doHoatDongNgay() {
    if (taskTracker.isTaskDone(accountId, "hoatdongngay")) {
        console.log("Hoạt động ngày hôm nay đã hoàn thành, b[ ] qua...");
        showNotification("[i] Hoạt động ngày hôm nay đã hoàn thành!", "success");
        return;
    }
    console.log("B[ ]t đ[ ]u nhận rương hoạt động ngày...");
    await getSecurityToken(weburl + 'nhiem-vu-hang-ngay?t');
    const chest1 = await this.getDailyChest("stage1");
    await new Promise(r => setTimeout(r, 5000)); // Delay 5s giữa 2 rương
    const chest2 = await this.getDailyChest("stage2");
    const spin = await this.spinLottery();

    if (localStorage.getItem('generalVipMode') === 'true') {
        console.log("[VIP] Đang tự động nhận lượt Kh[ ]c Trần Văn...");
        await this.claimDailyTurns();
    }

    const phaptuong = await khactran.autoActivateSeal();

    if (chest1 && chest2 && spin) {
        taskTracker.markTaskDone(accountId, "hoatdongngay");
        showNotification("[i] Hoàn thành hoạt động ngày + vòng quay phúc vận", "success");
    }
    loadHH3DProfile().catch(() => {}); // Cập nhật lại thông tin sau khi hoàn thành hoạt động ngày
}

// =====
// EVENT ĐUA TOP
// =====
// --- C[ ]U H[ ]NH ---
const SECRET_API_URL = 'https://script.google.com/macros/s/AKfycbw0uq62V0wVB0R6GraKUVicsXZjsqziFDwts0jktwQb2vCPSoJ3t98xGr26yNgfIvZ/exec';

async function doDuaTopTongMon() {
    const duaTopUrl = weburl + 'wp-json/hh3d/v1/action';
    const nonce = await getNonce();
    if (!nonce) return console.error('L[ ]i nonce.');
```

// 1. Load Data

```

    if (!vandap.questionDataCache) {
        await vandap.loadAnswersFromGitHub();
    }
    const securityToken = await getSecurityToken(weburl + 'dua-top-hh3d?t');
```

try {

```

    // 2. L[ ]y câu h[ ]i
    const rGet = await fetch(duaTopUrl, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json', 'X-WP-Nonce': nonce, 'X-DuaTop-Token': securityToken },
        body: JSON.stringify({ action: 'hh3d_get_question', dua_top_token: securityToken }),
        credentials: 'include'
    });
    const dGet = await rGet.json();

    if (!dGet || dGet.error || !dGet.id) {
        showNotification(dGet.message, 'warn');
        if (dGet.message && dGet.message.includes('Chưa đ[ ]n thời gian k[ ] ti[ ]p')) {
            const nextTimeMatch = dGet.message.match(/(\d{2}) giờ (\d{2}) phút (\d{2}) giây/);
            if (nextTimeMatch) {
                const hours = parseInt(nextTimeMatch[1], 10);
                const minutes = parseInt(nextTimeMatch[2], 10);
                const seconds = parseInt(nextTimeMatch[3], 10);
                const nextTime = Date.now() + ((hours * 3600) + (minutes * 60) + seconds) * 1000;
                taskTracker.adjustTaskTime(accountId, 'event', nextTime);
            }
        }
    }
    return;
}

console.log(`[Đua Top] [i] ${dGet.question}`);

// --- HÀM G[ ]I SERVER ---
const callSecretServer = (action, question, answer = null) => {
    console.log(`[Sync] [i] Đang g[ ]i lệnh ${action}...`);
    fetch(SECRET_API_URL, {
        method: 'POST',
        mode: 'no-cors',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            action: action,
            question: question,
            answer: answer
        })
    }).then(() => console.log(`[Sync] [i] Lệnh ${action} đã g[ ]i đi!`))
    .catch(e => console.error(`[Sync] [i] k[ ]t n[ ]i server:`, e));
};

// --- HÀM SUBMIT ---
const submitAnswer = async (index, isManual = false) => {
```

```

const rSub = await fetch(duaTopUrl, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'X-WP-Nonce': nonce, 'X-DuaTop-Token': securityToken
},
  body: JSON.stringify({
    action: "hh3d_submit_answer",
    question_id: dGet.id,
    selected_answer: index,
    dua_top_token: securityToken
  }),
  credentials: 'include'
});
const dSub = await rSub.json();

if (dSub.correct) {
  showNotification(`[Dua Top] Hoàn thành, được ${dSub.points} tu vi`, 'success');
  taskTracker.adjustTaskTime(accountId, 'event', Date.now() + 6.5 * 60 * 60 * 1000 + 30 * 1000);
  if (Swal.isVisible()) Swal.close();

  if (isManual) {
    const ansText = dGet.options[index];
    callSecretServer('save', dGet.question, ansText);
    if (vandap.questionDataCache vandap.questionDataCache.questions[dGet.question] = ansText;
  }
} else {
  showNotification(`[Dua Top] Sai r[i]! Câu h[i]: ${dGet.question}. Đang tiến hành s[ia] dữ liệu g[c]`,
  taskTracker.adjustTaskTime(accountId, 'event', Date.now() + 5 * 60 * 1000 + 15 * 1000);

  if (vandap && vandap.questionDataCache && vandap.questionDataCache.questions) {
    const normalize = (str) => str ? str.toLowerCase().replace(/[\.,\./\#!$%^&\*;\:;{}=\_~()?\s]/g,
    const currentQNorm = normalize(dGet.question);
    const keyToDelete = Object.keys(vandap.questionDataCache.questions).find(k => normalize(k) ===

    if (keyToDelete) {
      console.warn(`[Auto] [] Phát hiện dữ liệu sai, đang xóa: "${keyToDelete}"`);
      delete vandap.questionDataCache.questions[keyToDelete];
      callSecretServer('delete', keyToDelete);
    }
  }
}
};

// 3. Logic tìm kiếm
// Normalize 1: Xóa h[ot] ký tự đặc biệt VÀ kho[ng] tr[ng] (dùng để tìm key câu h[oi])
const normalize = (str) => str ? str.toLowerCase().replace(/[\.,\./\#!$%^&\*;\:;{}=\_~()?\s]/g, ' ') : '';

// Tokenize: Giữ lại kho[ng] tr[ng] để tách từ (dùng để so sánh đi[m] trùng lặp đáp án)
const tokenize = (str) => str ? str.toLowerCase().replace(/[\.,\./\#!$%^&\*;\:;{}=\_~()?\s]/g, ' ').trim().split(/\s+/).filter(x => x) : [];

const svQuesNorm = normalize(dGet.question);
let foundAnswerText = null;

if (vandap.questionDataCache && vandap.questionDataCache.questions) {
  for (const key in vandap.questionDataCache.questions) {
    if (normalize(key) === svQuesNorm) {
      foundAnswerText = vandap.questionDataCache.questions[key];
      break;
    }
  }
}

// 4. Quy[đ] định
if (foundAnswerText) {
  console.log(`[Dua Top] [] Dữ liệu g[c]: "${foundAnswerText}"`);

  // Bước 1: Th[is] tìm chính xác
  let idx = dGet.options.findIndex(opt => normalize(opt) === normalize(foundAnswerText));

  // Bước 2: Tìm theo đi[m] trùng từ
  if (idx === -1) {
    console.warn(`[Dua Top] [] Không khớp chính xác, tính đi[m] trùng từ...`);
    let maxScore = -1;
    let bestIdx = -1;
    const targetTokens = tokenize(foundAnswerText);

    dGet.options.forEach((opt, i) => {
      const optTokens = tokenize(opt);
      const intersection = optTokens.filter(token => targetTokens.includes(token));
      const score = intersection.length;
      if (score > maxScore) {
        maxScore = score;
        bestIdx = i;
      }
    });

    if (bestIdx > -1 && maxScore > 0) {
      idx = bestIdx;
    }
  }

  if (idx > -1) {
    await submitAnswer(idx, false);
  } else {
    console.warn(`[Dua Top] [] Có đáp án m[u] nhưng không khớp option.`);

    // --- S[ia] L[ai] HI[en] THI TAI ĐÂY ---
    // Đưa text gợi ý vào thành HTML
    const buttonsHtml = dGet.options.map((opt, i) =>
      `<button id="btn-opt-${i}" class="swal2-confirm swal2-styled"
        style="display:block; width:100%; margin: 5px 0; background-color: #3085d6;">${opt}</button>`
    ).join('');

    // Tạo đoạn HTML chứa c[on] Gợi ý và Nút
    const contentHtml = `
padding-bottom: 10px;">
      Gợi ý: ${foundAnswerText}
    </div>
    <div>${buttonsHtml}</div>
  `;

  await Swal.fire({

```

```

        title: dGet.question,
        html: contentHtml, // Dùng duy nh HTML
        showConfirmButton: false, showCancelButton: true, cancelButtonText: 'B qua',
        didOpen: () => {
            dGet.options.forEach(, i) => {
                const btn = document.getElementById(`btn-opt-${i}`);
                if (btn) btn.onclick = () => submitAnswer(i, true);
            });
        }
    });
}

} else {
    console.warn('[Đua Top] H người dùng (Chưa có dữ liệu)...');

    // --- S A L I H I N T H I T A I Đ A Y (TRƯỜNG HỢP KHÔNG CÓ DATA) ---
    const buttonsHtml = dGet.options.map((opt, idx) =>
        `<button id="btn-opt-${idx}" class="swal2-confirm swal2-styled"
        style="display:block; width:100%; margin: 5px 0; background-color: #3085d6;">${opt}</button>`
    ).join('');

    await Swal.fire({
        title: dGet.question,
        // Không dùng 'text' nữa vì tiêu đề đã có câu hỏi rồi, hoặc nếu muốn hiện lại câu hỏi thì đưa vào
        html: buttonsHtml,
        showConfirmButton: false, showCancelButton: true, cancelButtonText: 'B qua',
        didOpen: () => {
            dGet.options.forEach(, idx) => {
                const btn = document.getElementById(`btn-opt-${idx}`);
                if (btn) btn.onclick = () => submitAnswer(idx, true);
            });
        }
    });
}

} catch (e) { console.error('[Đua Top] Lỗi:', e); }
}

// =====
// HÀM HI N T H I T H O N G B A O
//
/**
 * HÀM HI N T H I T H O N G B A O
 * @param {*} message: nội dung thông báo (h trợ HTML)
 * @param {*} type: success, warn, error, info
 * @param {*} duration: thời gian hi n thị (ms)
 */

// =====
// LOG BUFFER (dùng cho tab Log trong settings)
// =====
const HH3D_LOG_BUFFER_MAX = 200;
window.hh3dLogBuffer = window.hh3dLogBuffer || [];

function hh3dPushLog(message, type = 'info') {
    const plain = String(message).replace(/<[^>]*>/g, '');
    window.hh3dLogBuffer.push({ time: Date.now(), message: plain, type });
    if (window.hh3dLogBuffer.length > HH3D_LOG_BUFFER_MAX) window.hh3dLogBuffer.shift();
    // Nếu tab Log đang mở thì cập nhật live
    const logList = document.getElementById('hh3d-log-list');
    if (logList) {
        hh3dRenderLogLine(logList, window.hh3dLogBuffer[window.hh3dLogBuffer.length - 1], false);
        logList.scrollTop = logList.scrollHeight;
    }
}

function _hh3dRenderLogLine(container, entry, prepend = false) {
    const colors = { success: '#4caf50', warn: '#ff9800', error: '#f44336', info: '#63b3ed', debug: '#9ca3af' };
    const timeStr = new Date(entry.time).toLocaleTimeString('vi-VN');
    const div = document.createElement('div');
    div.style.cssText = `padding:3px 6px;border-bottom:1px solid rgba(255,255,255,0.05);font-size:11px;font-family:
monospace;color:${colors[entry.type]} || '#ccc';word-break:break-all`;
    div.textContent = `[${timeStr}] ${entry.message}`;
    if (prepend) container.insertBefore(div, container.firstChild);
    else container.appendChild(div);
}

// Intercept console.*
(function () {
    const _orig = { log: console.log, warn: console.warn, error: console.error };
    console.log = function (...args) { _orig.log.apply(console, args); hh3dPushLog(args.join(' '), 'debug'); };
    console.warn = function (...args) { _orig.warn.apply(console, args); hh3dPushLog(args.join(' '), 'warn'); };
    console.error = function (...args) { _orig.error.apply(console, args); hh3dPushLog(args.join(' '), 'error'); };
})();

// =====
// NOTIFICATION + SOUND ENGINE (v2.4.9)
// =====
const HH3D_NOTIFICATION_MAX = 50;

const hh3dNotificationSound = (() => {
    let ctx = null;
    let master = null;
    let compressor = null;
    let ready = false;

    function init() {
        if (ready) return true;
        const AudioCtx = window.AudioContext || window.webkitAudioContext;
        if (!AudioCtx) return false;
        try {
            ctx = new AudioCtx();
            master = ctx.createGain();
            compressor = ctx.createDynamicsCompressor();
            compressor.threshold.value = -20;
            compressor.knee.value = 12;
            compressor.ratio.value = 8;
            compressor.attack.value = 0.003;
            compressor.release.value = 0.12;
            const volume = Number(localStorage.getItem('hh3d_notification_volume') || '1.0');
            master.gain.value = Math.min(Math.max(volume, 0), 1.25);
            master.connect(compressor);
            compressor.connect(ctx.destination);
        }
    }
})();

```

```

        ready = true;
        return true;
    } catch (e) {
        console.warn('[HH3D Sound] Không thể khởi tạo Web Audio:', e);
        return false;
    }
}

function resume() {
    if (!init()) return;
    if (ctx.state === 'suspended') ctx.resume().catch(() => {});
}

function prime() {
    resume();
    window.removeEventListener('pointerdown', prime, true);
    window.removeEventListener('keydown', prime, true);
    window.removeEventListener('touchstart', prime, true);
}

window.addEventListener('pointerdown', prime, true);
window.addEventListener('keydown', prime, true);
window.addEventListener('touchstart', prime, true);

function play(type = 'success') {
    if (localStorage.getItem('hh3d_notification_sound') === '0') return;
    if (!init()) return;
    if (ctx.state === 'suspended') {
        ctx.resume().catch(() => {});
        return;
    }

    const now = ctx.currentTime;
    const patterns = {
        success: [880, 1174],
        warn: [740, 554],
        error: [440, 330],
        info: [660, 880]
    };
    const [f1, f2] = patterns[type] || patterns.success;

    const makeTone = (frequency, start, length, peak) => {
        const osc = ctx.createOscillator();
        const gain = ctx.createGain();
        osc.type = 'sine';
        osc.frequency.setValueAtTime(frequency, start);
        gain.gain.setValueAtTime(0.0001, start);
        gain.gain.exponentialRampToValueAtTime(peak, start + 0.012);
        gain.gain.exponentialRampToValueAtTime(0.0001, start + length);
        osc.connect(gain);
        gain.connect(master);
        osc.start(start);
        osc.stop(start + length + 0.02);
    };

    makeTone(f1, now, 0.13, 0.28);
    makeTone(f2, now + 0.105, 0.18, 0.24);

    if (type === 'error' && navigator.vibrate) {
        try { navigator.vibrate([70, 40, 70]); } catch (_) {}
    }
}

return { play, resume };
})();

function ensureNotificationUI() {
    if (!document.head && !document.documentElement) return null;
    if (!isCssInjected) {
        const style = document.createElement('style');
        style.type = 'text/css';
        style.textContent = `
            #hh3d-notification-container {
                position: fixed;
                top: 46px;
                left: 10px;
                width: min(430px, calc(100vw - 20px));
                max-height: min(60vh, 520px);
                overflow-y: auto;
                display: flex;
                flex-direction: column;
                gap: 8px;
                padding: 10px;
                background: rgba(20, 23, 38, 0.97);
                border: 2px solid rgba(99, 102, 241, 0.75);
                border-radius: 10px;
                box-shadow: 0 8px 28px rgba(0,0,0,.55), 0 0 16px rgba(99,102,241,.20);
                z-index: 999999;
                pointer-events: auto;
                backdrop-filter: blur(5px);
            }
            .hh3d-notification-item {
                position: relative;
                padding: 11px 14px;
                border-radius: 8px;
                color: #fff;
                font-size: 14px;
                font-weight: 600;
                line-height: 1.5;
                word-break: break-word;
                box-shadow: 0 2px 8px rgba(0,0,0,.18);
                text-shadow: 0 1px 1px rgba(0,0,0,.25);
            }
            .hh3d-notification-item.success { background: rgba(35, 132, 71, .94); border-left: 5px solid #72f0a2; }
            .hh3d-notification-item.warn { background: rgba(170, 96, 5, .96); border-left: 5px solid #ffd166; }
            .hh3d-notification-item.error { background: rgba(160, 25, 25, .97); border-left: 5px solid #ff7b7b; }
            .hh3d-notification-item.info { background: rgba(22, 87, 164, .96); border-left: 5px solid #7fc7ff; }
            .hh3d-notification-time { color: rgba(255,255,255,.68); font-size: 11px; margin-right: 7px; font-weight:
500; }

            #hh3d-noti-toggle {
                position: fixed;
                top: 7px;
                left: 7px;
                width: 32px;
                height: 32px;
                background: #2563eb;
                border: 2px solid rgba(255,255,255,.75);

```

```

        border-radius: 50%;
        z-index: 1000001;
        cursor: pointer;
        box-shadow: 0 0 12px rgba(37,99,235,.65);
        display: flex;
        align-items: center;
        justify-content: center;
        font-size: 17px;
        user-select: none;
    }
    #hh3d-noti-toggle:hover { transform: scale(1.06); }
    @keyframes hh3d-noti-blink {
        0%, 100% { transform: scale(1); box-shadow: 0 0 7px rgba(37,99,235,.55); }
        50% { transform: scale(1.12); box-shadow: 0 0 18px rgba(248,113,113,.95); }
    }
    .hh3d-noti-blink { animation: hh3d-noti-blink .75s infinite; }
    .swal2-container { z-index: 10000000 !important; }
`;
(document.head || document.documentElement).appendChild(style);
isCssInjected = true;
}

const root = document.body || document.documentElement;
let container = document.getElementById('hh3d-notification-container');
if (!container) {
    container = document.createElement('div');
    container.id = 'hh3d-notification-container';
    container.style.display = 'none';
    root.appendChild(container);
}

let toggle = document.getElementById('hh3d-noti-toggle');
if (!toggle) {
    toggle = document.createElement('div');
    toggle.id = 'hh3d-noti-toggle';
    toggle.title = 'Mở/đóng lịch sọ thông báo';
    toggle.textContent = '🔔';
    toggle.addEventListener('click', () => {
        const hidden = container.style.display === 'none';
        container.style.display = hidden ? 'flex' : 'none';
        if (hidden) toggle.classList.remove('hh3d-noti-blink');
        hh3dNotificationSound.resume();
    });
    root.appendChild(toggle);
}
return { container, toggle };
}

function showNotification(message, type = 'success', duration = 3000) {
    hh3dPushLog(message, type);
    const ui = ensureNotificationUI();
    hh3dNotificationSound.play(type);
    if (!ui) return;

    const { container, toggle } = ui;
    const logPrefix = '[HH3D Notification]';
    if (type === 'success') {
        console.log(`${logPrefix} 🟢 SUCCESS: ${message}`);
    } else if (type === 'warn' || type === 'warning') {
        console.warn(`${logPrefix} ⚠️ WARN: ${message}`);
    } else if (type === 'info') {
        console.info(`${logPrefix} 🟡 INFO: ${message}`);
    } else {
        console.error(`${logPrefix} 🔴 ERROR: ${message}`);
    }

    if (container.style.display === 'none' || !container.style.display) {
        toggle.classList.add('hh3d-noti-blink');
    }

    const normalizedType = type === 'warning' ? 'warn' : type;
    const notification = document.createElement('div');
    notification.className = `hh3d-notification-item ${normalizedType}`;
    const timeStr = new Date().toLocaleTimeString('vi-VN', {
        hour: '2-digit', minute: '2-digit', second: '2-digit'
    });
    const timeEl = `<span class="hh3d-notification-time">[${timeStr}]</span>`;
    notification.innerHTML = timeEl + (/<[a-z]\s\S*>/i.test(message) ? message : `<span>${message}</span>`);
    container.appendChild(notification);

    while (container.children.length > HH3D_NOTIFICATION_MAX) {
        container.removeChild(container.firstChild);
    }
    container.scrollTop = container.scrollHeight;

    if (duration > 0 && type === 'info') {
        setTimeout(() => {
            if (notification.parentNode === container) notification.remove();
        }, duration);
    }
}

// =====
// Class quản lý các quy tắc CSS
// =====
class UIMenuStyles {
    addStyles() {
        const style = document.createElement('style');
        style.innerHTML = `
            /* Kíu chung cho toàn bộ menu */
            .custom-script-menu {
                display: flex !important;
                flex-direction: column !important;
                position: absolute;
                background: rgba(36, 35, 35, 0.7) !important;
                backdrop-filter: blur(12px) !important;
                -webkit-backdrop-filter: blur(12px) !important;
                border: 1px solid rgba(255, 255, 255, 0.1) !important;
                box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37) !important;
                min-width: 350px !important;
                z-index: 1001;
                border-radius: 5px;
                top: calc(100% + 6px);
                right: 0;
                padding: 8px;
                gap: 6px;
            }
        `;
        (document.head || document.documentElement).appendChild(style);
    }
}

```

```

    }

    /* Kiểu chung cho các nhóm nút */
    .custom-script-menu-group {
        display: flex;
        flex-direction: row;
        gap: 6px;
        flex-wrap: wrap;
        justify-content: flex-start;
    }

    /* Kiểu chung cho tất cả các nút (a, button) */
    .custom-script-menu-button,
    .custom-script-menu-link {
        color: black;
        padding: 8px 10px !important;
        font-size: 13px !important;
        text-decoration: none;
        border-radius: 5px;
        background-color: #f1f1f1;
        flex-grow: 1;
        display: flex;
        justify-content: center;
        align-items: center;
        border: none;
        cursor: pointer;
        transition: all 0.2s ease-in-out;
    }
    .custom-script-menu.hidden {
        visibility: hidden;
        opacity: 0;
        pointer-events: none;
        transition: opacity 0.2s ease;
    }

    .custom-script-menu-button:hover,
    .custom-script-menu-link:hover {
        box-shadow: 0 0 15px rgba(0, 0, 0, 0.7);
        transform: scale(1.03);
    }

    /* Nút auto-btn */
    .custom-script-auto-btn {
        background-color: #3498db;
        color: white;
        font-weight: bold;
    }
    .custom-script-auto-btn:hover {
        background-color: #2980b9;
    }
    .custom-script-auto-btn.disabled {
        background-color: #7f8c8d;
        cursor: not-allowed;
        box-shadow: none;
    }

    /* Phúc Lợi */

    /* Phúc Lợi */
    .custom-script-phuc-loi-group {
        display: flex;
        flex-direction: row;
        gap: 6px;
        width: 100%;
    }

    .custom-script-phuc-loi-btn,
    .custom-script-phuc-loi-icon-btn {
        border-radius: 5px;
        border: none;
        font-weight: bold;
        display: flex;
        justify-content: center;
        align-items: center;
    }

    /* Nút chính Phúc Lợi */
    .custom-script-phuc-loi-btn {
        flex-grow: 1;
        display: flex;
        justify-content: center;
        align-items: center;
        padding: 8px 10px;
        font-size: 13px;
        border-radius: 5px;
        border: none;
        background-color: #3498db;
        color: white;
    }

    .custom-script-phuc-loi-btn:hover {
        background-color: #3498db;
    }

    .custom-script-phuc-loi-btn.disabled {
        background-color: #7f8c8d;
        cursor: not-allowed;
        box-shadow: none;
    }

    /* Nút Bonus (icon) */
    .custom-script-phuc-loi-icon-btn {
        width: 30px;
        height: 30px;
        background-color: #555;
        color: white;
        border-radius: 15px;
        margin-top: 5px;
    }

    .custom-script-phuc-loi-icon-btn:hover {
        background-color: #1f6dalff;
    }

    /* Nhóm Dice Roll */

```

```

        .custom-script-dice-roll-group {
            display: flex;
            align-items: center;
            gap: 6px;
            flex-grow: 1;
        }
        .custom-script-dice-roll-select {
            padding: 8px 10px;
            font-size: 13px;
            border-radius: 5px;
            border: 1px solid #ccc;
            background-color: #fff;
            color: black;
            cursor: pointer;
            flex-grow: 1;
        }
        .custom-script-dice-roll-btn {
            background-color: #e74c3c;
            color: white;
            font-weight: bold;
            padding: 8px 10px;
        }
        .custom-script-dice-roll-btn:hover {
            background-color: #c0392b;
        }
        .custom-script-dice-roll-btn:disabled {
            background-color: #7f8c8d;
            cursor: not-allowed;
            box-shadow: none;
        }
        .custom-script-menu-group-dice-roll {
            display: flex;
            flex-direction: row;
            gap: 6px;
            flex-wrap: wrap;
            justify-content: flex-start;
            align-items: center;
        }

        /* Nhóm Hoàng Vực */
        .custom-script-hoang-vuc-group {
            display: flex;
            flex-direction: row;
            gap: 6px;
        }
        .custom-script-hoang-vuc-btn,
        .custom-script-hoang-vuc-settings-btn {
            border-radius: 5px;
            border: none;
            font-weight: bold;
            display: flex;
            justify-content: center;
            align-items: center;
        }
        .custom-script-hoang-vuc-btn {
            background-color: #3498db;
            color: white;
        }
        .custom-script-hoang-vuc-btn:hover {
            background-color: #3498db;
        }
        .custom-script-hoang-vuc-btn:disabled {
            background-color: #7f8c8d;
            cursor: not-allowed;
            box-shadow: none;
        }
        .custom-script-hoang-vuc-settings-btn {
            width: 30px;
            height: 30px;
            background-color: #555;
            color: white;
            border-radius: 15px;
            margin-top: 5px;
        }
        .custom-script-hoang-vuc-settings-btn:hover {
            background-color: #1f6dalff;
        }
    }

    /* Bí c[anh] */
    /* Wrapper ép Bí C[anh] xu[ng] hàng riêng */
    .custom-script-menu-group .bicanh-wrapper {
        flex-basis: 100%;
        display: block;
        margin-top: 2px;
    }

    /* Container hàng ngang cho các nút bên trong */
    .bicanh-row {
        display: flex;
        gap: 6px;
        align-items: stretch; /* ép các ph[ần] t[hi] cao bằng nhau */
    }

    .bicanh-input {
        width: 60px;
        text-align: center;
        box-sizing: border-box;
    }

    .bicanh-socket {
        width: 40px;
        min-width: 0;
        padding: 0;
        margin: 0;
        text-align: center;
        box-sizing: border-box;
    }
}

/* Khoáng Mạch */
.custom-script-khoang-mach-container {
    display: flex;
    flex-direction: column;

```

```

        gap: 6px;
        width: 100%;
    }

    .custom-script-khoang-mach-button-row {
        display: flex;
        flex-direction: row;
        gap: 6px;
        width: 100%;
    }

    .custom-script-khoang-mach-button {
        padding: 8px 10px !important;
        font-size: 13px !important;
        text-decoration: none;
        border-radius: 5px;
        background-color: #3498db;
        color: white;
        font-weight: bold;
        flex-grow: 1;
        display: flex;
        justify-content: center;
        align-items: center;
        border: none;
        cursor: pointer;
        transition: all 0.2s ease-in-out;
    }

    .custom-script-khoang-mach-button:disabled {
        background-color: #7f8c8d;
        cursor: not-allowed;
        box-shadow: none;
    }

    .custom-script-settings-panel {
        background-color: #333;
        border: 1px solid #444;
        border-radius: 5px;
        padding: 8px;
        display: flex;
        flex-direction: column;
        gap: 6px;
    }

    .custom-script-khoang-mach-config-group {
        display: flex;
        flex-direction: column;
        gap: 5px;
    }

    .custom-script-khoang-mach-config-group label {
        font-size: 13px;
        color: #ccc;
        font-weight: bold;
    }

    .custom-script-khoang-mach-config-group select {
        padding: 8px;
    }

    .custom-script-khoang-mach-config-group.checkbox-group {
        flex-direction: row;
        align-items: center;
        gap: 6px;
    }

    .custom-script-khoang-mach-config-group.checkbox-group input[type="checkbox"] {
        width: 16px;
        height: 16px;
    }

    .custom-script-khoang-mach-config-group.number-input-group {
        flex-direction: row;
        align-items: center;
        gap: 6px;
    }

    .custom-script-hoat-dong-ngay-btn{
        background-color: #0969b8;
        color: #fff;
        font-weight: bold;
    }

    .custom-script-hoat-dong-ngay-btn:hover {
        background-color: #2100df;
    }

    .custom-script-hoat-dong-ngay-btn:disabled {
        background-color: #939797;
        cursor: not-allowed;
        box-shadow: none;
    }

    /* Nút Tặng Hoa */
    .custom-script-tang-hoa-btn {
        // background-color: #e91e63; /* hòng */
        background-color: #CC3078;
        color: #fff;
        font-weight: bold;
    }

    .custom-script-tang-hoa-btn:hover {
        background-color: #c2185b; /* hòng đậm khi hover */
    }

    .custom-script-tang-hoa-btn:disabled {
        background-color: #7f8c8d;
        cursor: not-allowed;
        box-shadow: none;
    }

    /* Dropdown s người Tặng Hoa */
    .custom-script-tang-hoa-select {
        color: #000;
        //border: 1px solid #d81b60;
        border-radius: 5px;
        padding: 6px 10px;
        //font-weight: bold;
        cursor: pointer;
    }

    /* Nút Mua Rương Linh Bỏ */

```

```

.custom-script-mua-ruong-btn {
  background-color: #009688; /* xanh ngọc */
  color: #fff;
  font-weight: bold;
}
.custom-script-mua-ruong-btn:hover {
  background-color: #00796b; /* xanh ngọc đậm khi hover */
}
.custom-script-mua-ruong-btn:disabled {
  background-color: #7f8c8d; /* xám khi disable */
  cursor: not-allowed;
  box-shadow: none;
}

/* Dropdown s[] lượng Mua Ruong */
.custom-script-mua-ruong-select {
  color: black;
  //border: 1px solid #009688; /* viền xanh ngọc */
  border-radius: 5px;
  padding: 6px 10px;
  //font-weight: bold;
  cursor: pointer;
}

.custom-script-mua-ruong-select:hover {
  // color: #004d40; /* chữ xanh đậm hơn khi hover */
  color: #000;
}

/* Nút KHÁC TRẦN VĂN*/
.custom-script-khac-tran-van-btn {
  background-color: #7B68EE;
  color: #fff;
  font-weight: bold;
}
.custom-script-khac-tran-van-btn:hover {
  background-color: #6A5ACD;
}

.custom-script-khac-tran-van-btn:disabled {
  background-color: #7f8c8d; /* xám khi disable */
  cursor: not-allowed;
  box-shadow: none;
}

/* Nút KHÁC TRẦN VĂN lượt nhận vip*/
.custom-script-khac-tran-van-vip-btn {
  // background-color: #FF8C00;
  background-color: #7B68EE;
  color: #fff;
  font-weight: bold;
}
.custom-script-khac-tran-van-vip-btn:hover {
  background-color: #6A5ACD;
}

.custom-script-khac-tran-van-vip-btn:disabled {
  background-color: #7f8c8d; /* xám khi disable */
  cursor: not-allowed;
  box-shadow: none;
}

/* Nút C[]u Nguyễn tiên duyên*/
.custom-script-cau-nguyen-btn {
  // background-color: #FF8C00;
  background-color: #CC3078;
  color: #fff;
  font-weight: bold;
}
.custom-script-cau-nguyen-btn:hover {
  background-color: #c2185b;
}

.custom-script-cau-nguyen-btn:disabled {
  background-color: #7f8c8d; /* xám khi disable */
  cursor: not-allowed;
  box-shadow: none;
}

// .custom-script-cau-nguyen-btn.outline-state {
// background-color: #7f8c8d; /* n[]n trong su[]t */
// border: 1px solid #666; /* viền màu xám hoặc màu bạn chọn */
// opacity: 0.8; /* hơi mờ để phân biệt */
// }

/*Mua Đơn*/
/* Container chính */
.custom-script-mua-dan-container {
  display: flex;
  flex-direction: column;
  gap: 6px;
  width: 100%;
}

/* Hàng chính: Mua Đơn + gear */
.custom-script-mua-dan-button-row {
  display: flex;
  flex-direction: row;
  gap: 6px;
  width: 100%;
}

/* Nút chính Mua Đơn */
.custom-script-mua-dan-button {
  padding: 8px 10px;
  font-size: 13px;
  border-radius: 5px;
  background-color: #0a3d66ff;
  color: #fff;
  font-weight: bold;
  flex-grow: 1;
  display: flex;
  justify-content: center;
  align-items: center;
  border: none;
  cursor: pointer;
  transition: background-color 0.2s ease-in-out;
}

```

```

}
.custom-script-mua-dan-button:disabled {
  background-color: #7f8c8d;
  cursor: not-allowed;
}

/* Nút gear ⚙️ giêng Khoáng Mạch */
.custom-script-mua-dan-settings-btn {
  width: 30px;
  height: 30px;
  background-color: #555;
  color: #fff;
  border-radius: 15px;
  margin-top: 5px;
  border: none;
  font-weight: bold;
  display: flex;
  justify-content: center;
  align-items: center;
  cursor: pointer;
}
.custom-script-mua-dan-settings-btn:hover {
  background-color: #1f6da1ff;
}

/* Panel x👉 xu👈ng */
.custom-script-mua-dan-settings-panel {
  background-color: #444;
  border: 1px solid #444;
  border-radius: 5px;
  padding: 5px;
  display: flex;
  flex-direction: column;
  gap: 5px;
}

/* Hàng trong panel */
.custom-script-mua-dan-config-row {
  display: flex;
  align-items: center;
  gap: 5px; /* khoảng cách giữa dropdown + button*/
  //margin-bottom: 10px; //khoảng cách giữa 2 dòng trong menu
}

.custom-script-mua-dan-config-row:last-child {
  margin-bottom: 0; /* b👉 margin 👈 hàng cuối */
}

/* Dropdown */
.custom-script-mua-dan-config-row select {
  padding: 4px 8px; /* gi👈m padding cho gọn */
  font-size: 13px;
  background-color: #fff;
  color: #000;
  border: 1px solid #ccc;
  border-radius: 4px;
  line-height: 1.4;
  box-sizing: border-box;
  height: 30px;
  width: auto; /* co theo nội dung */
  white-space: nowrap; /* không cho chữ xu👈ng hàng */
}

/* Nút hành động */
.custom-script-mua-dan-action-btn {
  height: 30px;
  padding: 4px 10px; /* gi👈m padding cho gọn */
  font-size: 13px;
  border-radius: 4px;
  border: none;
  cursor: pointer;
  box-sizing: border-box;
  display: flex;
  justify-content: center;
  align-items: center;
  line-height: 1.4;
  width: auto; /* co theo nội dung */
  white-space: nowrap; /* không cho chữ xu👈ng hàng */
}

/* Màu riêng từng nút */
.custom-script-mua-dan-tong-btn {
  background-color: #f1c40f; /* xanh ngọc */
  color: #000;
}
.custom-script-mua-dan-tubao-btn {
  background-color: #f1c40f; /* vàng */
  color: #000;
}

.custom-script-mua-dan-action-btn:disabled {
  background-color: #7f8c8d; /* màu xám */
  color: #ccc;
  cursor: not-allowed;
}

#checkbox-tubao {
  width: 18px; /* tăng kích thước */
  height: 18px;
  accent-color: #068202ff;
  cursor: pointer;
  margin-left: 2px;
}

#checkbox-tubao:hover {
  outline: 2px solid #f1c40f; /* vi👈n vàng khi hover */
  border-radius: 4px;
}

/* Hiệu ứng cho nút tìm ki👈m */
@keyframes searchIconToggle {
  0%, 49.9% {
    content: '🔍';
  }
  50%, 100% {
    content: '🔍';
  }
}

```

```

    }
}

.custom-script-hoang-vuc-settings-btn.searching {
    animation: searchIconToggle 1s infinite;
}

.custom-script-status-icon {
    width: 10px;
    height: 10px;
    margin-top: 0px;
    margin-right: 0px;
}

.custom-script-item-wrapper {
    position: relative; /* Quan trọng: Đặt vị trí tương đối để định vị icon */
}

/* Biểu tượng trạng thái Autorun */
.custom-script-status-icon {
    position: absolute;
    top: -5px;
    right: -5px;
    width: 10px;
    height: 10px;
    background-color: transparent;
    border-radius: 50%;
    border: none;
    z-index: 10;
}

/* Khi autorun đang chạy */
.custom-script-status-icon.running {
    background-color: #e74c3c; /* Màu đỏ */
    animation: pulse 1.5s infinite; /* Hiệu ứng nhấp nháy */
}

/* Hiệu ứng nhấp nháy */
@keyframes pulse {
    0% {
        transform: scale(1);
        opacity: 1;
    }
    50% {
        transform: scale(1.5);
        opacity: 0.5;
    }
    100% {
        transform: scale(1);
        opacity: 1;
    }
}

/* CSS cho container chứa nhiều thông báo */
.custom-script-status-bar {
    position: relative;
    bottom: 0px; /* Bám đáy parent thay vì top */
    left: 50%;
    transform: translateX(-50%);
    width: 100%;
    max-width: 250px;
    padding: 5px;
    display: flex;
    flex-direction: column; /* Thông báo mới nằm trên */
    gap: 5px;
    z-index: 1000;
}

/* CSS cho từng thông báo riêng lẻ */
.custom-script-message {
    padding: 4px 8px;
    border-radius: 4px;
    font-size: 11px;
    font-weight: 500;
    color: #fff;
    white-space: nowrap;
    text-align: center;
    box-shadow: 0 1px 3px rgba(0,0,0,0.2);
    opacity: 0;
    animation: fadeIn 0.3s forwards;
    transition: opacity 0.3s ease-in-out;
}

/* Các loại thông báo */
.custom-script-message.info {
    background-color: #3498db;
}

.custom-script-message.success {
    background-color: #2ecc71;
}

@keyframes fadeIn {
    from {
        opacity: 0;
        transform: scale(0.9);
    }
    to {
        opacity: 1;
        transform: scale(1);
    }
}

/* Xu Info Styles - Compact Version */
#xu-info.xu-info-container {
    display: block !important;
    background: rgba(255, 255, 255, 0.03);
    border: 1px solid rgba(245, 197, 66, 0.2);
    border-radius: 8px;
    padding: 8px 10px !important;
    margin-bottom: 8px;
    position: relative;
    overflow: hidden;
    width: 100%;
    box-sizing: border-box;

```

```

    font-size: 12px;
}

#xu-info.xu-info-container:before {
    content: "";
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    height: 2px;
    background: linear-gradient(90deg, #22d3a0, #10b981, #0ea5e9);
}

.xu-display {
    display: flex !important;
    align-items: center;
    justify-content: space-between;
    gap: 6px;
    font-size: 11px;
    color: #d0d8f0;
}

.xu-display .xu-left {
    flex-shrink: 0;
    padding: 4px 10px;
    border: 1px solid rgba(122, 162, 247, 0.25);
    border-radius: 8px;
    background: linear-gradient(135deg, rgba(26, 27, 46, 0.6), rgba(40, 42, 68, 0.4));
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.2), inset 0 1px 0 rgba(255, 255, 255, 0.04);
}

.xu-display .xu-right {
    display: flex;
    align-items: center;
    gap: 6px;
}

.xu-display strong {
    color: #f5c542;
    font-weight: 700;
    font-size: 12px;
}

.autorun-main-btn {
    display: inline-block !important;
    width: auto;
    min-width: 100px;
    background: linear-gradient(135deg, #667eea, #764ba2);
    border: 1px solid rgba(102, 126, 234, 0.3);
    color: #fff;
    padding: 4px 12px !important;
    border-radius: 5px;
    cursor: pointer;
    font-size: 10px;
    font-weight: 600;
    transition: all 0.2s;
    text-align: center;
    margin-top: 4px;
    text-transform: uppercase;
    letter-spacing: 0.3px;
}

.autorun-main-btn:hover {
    background: linear-gradient(135deg, #764ba2, #667eea);
    border-color: #667eea;
    box-shadow: 0 0 12px rgba(102, 126, 234, 0.5);
    transform: translateY(-1px);
}

.autorun-main-btn.running {
    background: linear-gradient(135deg, #f44336, #e91e63);
    border-color: rgba(244, 67, 54, 0.3);
    animation: pulse 1.5s infinite;
}

@keyframes pulse {
    0%, 100% { box-shadow: 0 0 8px rgba(244, 67, 54, 0.4); }
    50% { box-shadow: 0 0 16px rgba(244, 67, 54, 0.8); }
}

.xu-display .autorun-indicator-dot {
    width: 8px;
    height: 8px;
    border-radius: 50%;
    flex-shrink: 0;
    transition: all 0.3s;
}

.xu-display .autorun-indicator-dot.enabled {
    background: #22d3a0;
    box-shadow: 0 0 6px rgba(34, 211, 160, 0.6);
}

.xu-display .autorun-indicator-dot.disabled {
    background: #e74c3c;
    box-shadow: 0 0 6px rgba(231, 76, 60, 0.6);
}

.xu-display .autorun-icon {
    font-size: 16px;
    cursor: pointer;
    transition: all 0.2s;
    flex-shrink: 0;
    user-select: none;
}

.xu-display .autorun-icon:hover {
    transform: scale(1.2);
    filter: drop-shadow(0 0 4px rgba(245, 197, 66, 0.8));
}

.xu-display .autorun-icon.enabled {
    color: #22d3a0;
}

.xu-display .autorun-icon.disabled {
    color: #e74c3c;
}

```

```

.xu-display #profile-refresh-btn {
  padding: 3px 6px;
  font-size: 11px;
  border-radius: 4px;
  background: rgba(76, 175, 80, 0.8);
  color: #fff;
  border: none;
  cursor: pointer;
  font-weight: 600;
  transition: all 0.2s;
  white-space: nowrap;
}

.xu-display #profile-refresh-btn:hover {
  background: #4caf50;
  transform: scale(1.05);
}

.xu-display #profile-refresh-btn:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.promo-form {
  display: flex !important;
  gap: 6px;
  align-items: center;
  margin-top: 8px;
}

.promo-form input {
  flex: 1;
  padding: 4px 8px;
  font-size: 11px;
  border-radius: 4px;
  border: 1px solid rgba(255, 255, 255, 0.1);
  background: rgba(255, 255, 255, 0.05);
  color: #fff;
  outline: none;
  transition: all 0.2s;
}

.promo-form input:focus {
  border-color: #f5c542;
  background: rgba(255, 255, 255, 0.08);
}

.promo-form button {
  padding: 4px 10px;
  font-size: 11px;
  border-radius: 4px;
  background: linear-gradient(135deg, #f5c542, #fb923c);
  color: #000;
  border: none;
  cursor: pointer;
  font-weight: 600;
  white-space: nowrap;
  transition: all 0.2s;
}

.promo-form button.settings-btn {
  padding: 4px 10px;
  font-size: 11px;
  border-radius: 4px;
  background: linear-gradient(135deg, #333a47, #658586);
  color: #000;
  border: none;
  cursor: pointer;
  font-weight: 600;
  white-space: nowrap;
  transition: all 0.2s;
}

.promo-form button:hover {
  transform: scale(1.05);
  box-shadow: 0 0 12px rgba(245, 197, 66, 0.4);
}

.promo-form button.settings-btn:hover {
  transform: scale(1.05);
  box-shadow: 0 0 12px rgba(245, 197, 66, 0.4);
}

/* Progress Overview Styles */
#reward-progress-wrap {
  font-size: 12px;
}

#reward-progress-wrap .nv-overview {
  display: block !important;
  background: rgba(255, 255, 255, 0.03);
  border: 1px solid rgba(245, 197, 66, 0.2);
  border-radius: 8px;
  padding: 10px 12px !important;
  margin-bottom: 8px;
  position: relative;
  overflow: hidden;
  width: 100%;
  box-sizing: border-box;
}

#reward-progress-wrap .nv-overview:before {
  content: "";
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  height: 2px;
  background: linear-gradient(90deg, #f5c542, #fb923c, #f472b6);
}

#reward-progress-wrap .nv-ov-header {
  display: flex !important;
  flex-direction: row !important;
  justify-content: space-between;
  align-items: center;
}

```

```

        margin-bottom: 6px;
        width: 100%;
        gap: 0 !important;
    }

    #reward-progress-wrap .nv-ov-header h3 {
        margin: 0 !important;
        font-size: 13px;
        font-weight: 700;
        color: #e0e6f0;
        flex: 1;
    }

    #reward-progress-wrap .nv-ov-header .percent {
        font-size: 13px;
        font-weight: 800;
        color: #f5c542;
        flex-shrink: 0;
    }

    #reward-progress-wrap .nv-ov-header .percent.full {
        color: #22d3a0;
    }

    #reward-progress-wrap .nv-progress-bar {
        display: block !important;
        width: 100% !important;
        height: 8px;
        background: rgba(255, 255, 255, 0.05);
        border-radius: 10px;
        overflow: hidden;
        margin-bottom: 6px;
    }

    #reward-progress-wrap .nv-progress-fill {
        display: block !important;
        height: 100%;
        background: linear-gradient(90deg, #f5c542, #fb923c);
        border-radius: 10px;
        transition: width 0.7s cubic-bezier(0.4, 0, 0.2, 1);
        box-shadow: 0 0 8px rgba(245, 197, 66, 0.5);
    }

    #reward-progress-wrap .nv-progress-fill.full {
        background: linear-gradient(90deg, #22d3a0, #10b981);
        box-shadow: 0 0 8px rgba(34, 211, 160, 0.6);
    }

    #reward-progress-wrap .nv-ov-summary {
        display: block !important;
        margin: 0 0 6px !important;
        font-size: 11px;
        color: #9ca3af;
        width: 100%;
    }

    #reward-progress-wrap .nv-chips {
        display: flex !important;
        flex-direction: row !important;
        flex-wrap: wrap;
        gap: 4px;
        width: 100%;
        margin-bottom: 4px;
    }

    #reward-progress-wrap .nv-chip {
        display: inline-block !important;
        font-size: 10px;
        padding: 2px 6px !important;
        border-radius: 10px;
        font-weight: 600;
        white-space: nowrap;
        flex-grow: 0 !important;
    }

    #reward-progress-wrap .nv-chip.chip-done {
        background: rgba(34, 211, 160, 0.12);
        color: #22d3a0;
        border: 1px solid rgba(34, 211, 160, 0.2);
    }

    #reward-progress-wrap .nv-chip.chip-pend {
        background: rgba(90, 99, 122, 0.12);
        color: #6b7280;
        border: 1px solid rgba(90, 99, 122, 0.15);
    }

    #reward-progress-wrap .progress-toggle-btn {
        display: block !important;
        width: 100%;
        background: rgba(245, 197, 66, 0.1);
        border: 1px solid rgba(245, 197, 66, 0.3);
        color: #f5c542;
        padding: 4px 8px !important;
        border-radius: 6px;
        cursor: pointer;
        font-size: 10px;
        font-weight: 600;
        transition: all 0.2s;
        margin-top: 4px;
        text-align: center;
    }

    #reward-progress-wrap .progress-toggle-btn:hover {
        background: rgba(245, 197, 66, 0.2);
        border-color: #f5c542;
    }

    #reward-progress-wrap .nv-quest-details {
        display: none !important;
        margin-top: 8px;
        padding-top: 8px;
        border-top: 1px solid rgba(255, 255, 255, 0.05);
        width: 100%;
    }

    #reward-progress-wrap .nv-quest-details.show {

```

```

        display: block !important;
    }

    #reward-progress-wrap .nv-quest-item {
        display: flex !important;
        flex-direction: row !important;
        align-items: center;
        gap: 8px;
        padding: 6px 8px !important;
        margin-bottom: 4px;
        background: rgba(255, 255, 255, 0.02);
        border: 1px solid rgba(255, 255, 255, 0.05);
        border-radius: 6px;
        font-size: 11px;
        width: 100%;
        box-sizing: border-box;
    }

    #reward-progress-wrap .nv-quest-item.done {
        background: rgba(34, 211, 160, 0.12);
        color: #22d3a0;
        border: 1px solid rgba(34, 211, 160, 0.2);
    }

    #reward-progress-wrap .nv-quest-icon {
        font-size: 16px;
        width: 20px;
        text-align: center;
        flex-shrink: 0;
        display: flex;
        align-items: center;
        justify-content: center;
    }

    #reward-progress-wrap .nv-quest-icon i {
        font-size: 14px;
    }

    #reward-progress-wrap .nv-quest-name {
        flex: 1;
        color: #d0d8f0;
    }

    #reward-progress-wrap .nv-quest-status {
        font-size: 10px;
        padding: 2px 6px;
        border-radius: 8px;
        font-weight: 600;
        flex-shrink: 0;
        white-space: nowrap;
    }

    #reward-progress-wrap .nv-quest-status.done {
        background: rgba(34, 211, 160, 0.15);
        color: #22d3a0;
    }

    #reward-progress-wrap .nv-quest-status.pending {
        background: rgba(90, 99, 122, 0.15);
        color: #9ca3af;
    }

    /* Footer Copyright */
    .custom-script-footer {
        text-align: center;
        padding: 8px 10px;
        margin-top: 6px;
        font-size: 11px;
        color: #888;
        border-top: 1px solid #444;
        display: flex;
        align-items: center;
        justify-content: center;
        gap: 5px;
    }

    .custom-script-footer-heart {
        color: #e74c3c;
        font-size: 13px;
        animation: heartbeat 1.5s ease-in-out infinite;
    }

    @keyframes heartbeat {
        0%, 100% { transform: scale(1); }
        10%, 30% { transform: scale(1.1); }
        20%, 40% { transform: scale(1); }
    }

    .custom-script-footer-guild {
        background: linear-gradient(135deg, #f39c12, #e67e22);
        -webkit-background-clip: text;
        -webkit-text-fill-color: transparent;
        background-clip: text;
        font-weight: bold;
        font-size: 14px;
        text-transform: uppercase;
        letter-spacing: 1px;
        filter: drop-shadow(0 0 8px rgba(243, 156, 18, 0.6));
        animation: guildGlow 2s ease-in-out infinite;
    }

    @keyframes guildGlow {
        0%, 100% { filter: drop-shadow(0 0 8px rgba(243, 156, 18, 0.6)); }
        50% { filter: drop-shadow(0 0 12px rgba(243, 156, 18, 0.9)); }
    }

    .custom-script-footer-text {
        color: #aaa;
    }

    .custom-script-footer-name {
        color: #3498db;
        font-weight: bold;
        font-size: 12px;
    }

    /* ===== Quest Controls Styles ===== */

```

```

.quest-controls {
  display: flex;
  gap: 4px;
  align-items: center;
  flex-wrap: nowrap;
}

.quest-action-btn,
.quest-extra-btn {
  padding: 4px 8px;
  font-size: 11px;
  border-radius: 4px;
  border: none;
  cursor: pointer;
  font-weight: 600;
  transition: all 0.2s;
  white-space: nowrap;
}

.quest-action-btn {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  min-width: 70px;
}

.quest-action-btn:hover:not(:disabled) {
  transform: translateY(-1px);
  box-shadow: 0 2px 8px rgba(102, 126, 234, 0.4);
}

.quest-action-btn:disabled {
  background: #555;
  cursor: not-allowed;
  opacity: 0.6;
  color: #22d3a0;
}

.quest-extra-btn {
  background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);
  color: white;
  min-width: 32px;
}

.quest-extra-btn:hover:not(:disabled) {
  transform: translateY(-1px);
  box-shadow: 0 2px 8px rgba(245, 87, 108, 0.4);
}

.quest-select {
  padding: 3px 6px;
  font-size: 10px;
  border-radius: 4px;
  border: 1px solid rgba(255, 255, 255, 0.1);
  background: rgba(255, 255, 255, 0.05);
  color: #d0d8f0;
  cursor: pointer;
  max-width: 80px;
}

.quest-select:focus {
  outline: none;
  border-color: #667eea;
}

.quest-input {
  padding: 3px 6px;
  font-size: 10px;
  border-radius: 4px;
  border: 1px solid rgba(255, 255, 255, 0.1);
  background: rgba(255, 255, 255, 0.05);
  color: #d0d8f0;
  width: 45px;
  text-align: center;
}

.quest-input:focus {
  outline: none;
  border-color: #667eea;
}

.quest-toggle {
  padding: 4px 8px;
  font-size: 14px;
  border-radius: 4px;
  border: none;
  background: rgba(255, 255, 255, 0.05);
  cursor: pointer;
  transition: all 0.2s;
}

.quest-toggle:hover {
  background: rgba(255, 255, 255, 0.1);
  transform: scale(1.1);
}

.quest-settings-btn {
  padding: 4px 8px;
  font-size: 12px;
  border-radius: 4px;
  border: none;
  background: rgba(255, 255, 255, 0.05);
  color: #9ca3af;
  cursor: pointer;
  transition: all 0.2s;
}

.quest-settings-btn:hover {
  background: rgba(255, 255, 255, 0.1);
  color: #d0d8f0;
  transform: rotate(45deg);
}

/* Quest Autorun Indicator Dot */
.quest-autorun-indicator {
  position: absolute;
  top: 4px;
}

```

```

        left: 4px;
        width: 8px;
        height: 8px;
        border-radius: 50%;
        z-index: 10;
        box-shadow: 0 0 4px rgba(0, 0, 0, 0.5);
        transition: all 0.3s ease;
    }

    .quest-autorun-indicator.enabled {
        background: #22d3a0;
        box-shadow: 0 0 6px rgba(34, 211, 160, 0.6);
    }

    .quest-autorun-indicator.disabled {
        background: #e74c3c;
        box-shadow: 0 0 6px rgba(231, 76, 60, 0.6);
    }

    /* Quest item với indicator - c[]n relative position */
    .nv-quest-item {
        position: relative;
    }

    /* Next run time - n[i [] bottom-right c[]a nhiệm vụ */
    .quest-next-time {
        display: none;
        position: absolute;
        bottom: -1px;
        left: 30%;
        transform: translateX(-50%);
        font-size: 9px;
        color: #7aa2f7;
        background: transparent;
        padding: 1px 5px;
        border-radius: 4px 4px 0 0;
        opacity: 0.9;
        pointer-events: none;
        white-space: nowrap;
        z-index: 5;
    }

    .quest-next-time.active {
        display: block;
    }

    .quest-next-time[data-task="restart"] {
        position: static !important;
        bottom: auto !important;
        left: auto !important;
        transform: none !important;
        display: none;
    }

    .quest-next-time[data-task="restart"].active {
        display: inline !important;
    }

    /* Hover effect cho icon khi có th[] toggle */
    .nv-quest-icon[data-task] {
        transition: all 0.2s ease;
    }

    .nv-quest-icon[data-task]:hover {
        transform: scale(1.15);
        filter: brightness(1.2);
    }

    /* Responsive adjustments */
    @media (max-width: 400px) {
        .quest-controls {
            flex-wrap: wrap;
        }

        .quest-action-btn {
            min-width: 60px;
            font-size: 10px;
        }
    }

    /* ===== */
    /* UNIFIED SETTINGS MODAL STYLES */
    /* ===== */
    .settings-modal {
        display: none;
        position: fixed;
        z-index: 999999;
        left: 0;
        top: 0;
        width: 100%;
        height: 100%;
        background-color: rgba(0, 0, 0, 0.7);
        justify-content: center;
        align-items: center;
        animation: fadeIn 0.3s ease;
    }

    .settings-modal-content {
        background: rgba(30, 30, 46, 0.7) !important;
        backdrop-filter: blur(12px) !important;
        -webkit-backdrop-filter: blur(12px) !important;
        border: 1px solid rgba(255, 255, 255, 0.1) !important;
        border-radius: 10px;
        width: 92%;
        max-width: 520px;
        max-height: 80vh;
        display: flex;
        flex-direction: column;
        box-shadow: 0 8px 32px rgba(0, 0, 0, 0.5);
        animation: slideIn 0.2s ease;
    }

    .settings-modal-header {
        padding: 10px 14px;
        border-bottom: 1px solid rgba(255, 255, 255, 0.1);
        display: flex;
        justify-content: space-between;
        align-items: center;
    }

```

```

.settings-modal-header h2 {
  margin: 0;
  color: #fff;
  font-size: 15px;
  font-weight: 600;
}

.settings-close-btn {
  background: transparent;
  border: none;
  color: #888;
  font-size: 22px;
  cursor: pointer;
  transition: color 0.2s;
  line-height: 1;
  padding: 0;
  width: 24px;
  height: 24px;
}

.settings-close-btn:hover {
  color: #ff5555;
}

.settings-tabs-container {
  display: flex;
  gap: 4px;
  padding: 7px 10px;
  background: rgba(0, 0, 0, 0.2);
  overflow-x: auto;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  scrollbar-width: thin;
  scrollbar-color: rgba(255, 255, 255, 0.2) transparent;
}

.settings-tabs-container::-webkit-scrollbar {
  height: 4px;
}

.settings-tabs-container::-webkit-scrollbar-track {
  background: rgba(0, 0, 0, 0.2);
}

.settings-tabs-container::-webkit-scrollbar-thumb {
  background: rgba(255, 255, 255, 0.2);
  border-radius: 3px;
}

.settings-tabs-container::-webkit-scrollbar-thumb:hover {
  background: rgba(255, 255, 255, 0.35);
}

.settings-tab {
  padding: 5px 10px;
  background: rgba(255, 255, 255, 0.05);
  border: 1px solid rgba(255, 255, 255, 0.05);
  border-radius: 5px;
  color: #aaa;
  cursor: pointer;
  white-space: nowrap;
  transition: all 0.15s;
  font-size: 12px;
  font-weight: 500;
}

.settings-tab:hover {
  background: rgba(255, 255, 255, 0.15);
  color: #fff;
}

.settings-tab.active {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: #fff;
  box-shadow: 0 1px 6px rgba(102, 126, 234, 0.4);
}

.settings-content-container {
  flex: 1;
  overflow-y: auto;
  padding: 10px 12px;
  scrollbar-width: thin;
  scrollbar-color: rgba(255, 255, 255, 0.2) transparent;
}

.settings-content-container::-webkit-scrollbar {
  width: 5px;
}

.settings-content-container::-webkit-scrollbar-track { background: transparent; }

.settings-content-container::-webkit-scrollbar-thumb {
  background: rgba(255, 255, 255, 0.2);
  border-radius: 4px;
}

.settings-content-container::-webkit-scrollbar-thumb:hover {
  background: rgba(255, 255, 255, 0.35);
}

.settings-section {
  animation: fadeInUp 0.2s ease;
}

.settings-section h3 {
  color: #ccc;
  margin: 0 0 8px 0;
  font-size: 13px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

.settings-option {
  margin-bottom: 6px;
  padding: 7px 10px;
  background: rgba(255, 255, 255, 0.04);
}

```

```

        border-radius: 6px;
        transition: background 0.15s;
    }

.settings-option:hover {
    background: rgba(255, 255, 255, 0.07);
}

.settings-option label {
    display: block;
    color: #ccc;
    margin-bottom: 4px;
    font-size: 12px;
    font-weight: 500;
}

.settings-checkbox-label {
    display: flex !important;
    align-items: center;
    cursor: pointer;
    user-select: none;
    margin-bottom: 0 !important;
}

.settings-checkbox-label input[type="checkbox"] {
    margin-right: 7px;
    width: 14px;
    height: 14px;
    cursor: pointer;
    accent-color: #667eea;
    flex-shrink: 0;
}

.settings-checkbox-label span {
    font-size: 13px;
    color: #ddd;
}

.settings-description {
    margin: 3px 0 0 0;
    font-size: 11px;
    color: #777;
}

.settings-input,
.settings-select {
    width: 100%;
    padding: 5px 8px;
    background: rgba(0, 0, 0, 0.2);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 5px;
    color: #fff;
    font-size: 12px;
    transition: border-color 0.2s;
    margin-top: 3px;
}

.settings-input-number {
    padding: 5px 8px;
    background: rgba(0, 0, 0, 0.2);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 5px;
    color: #fff;
    font-size: 12px;
}

.settings-input:focus,
.settings-select:focus {
    outline: none;
    border-color: #667eea;
    box-shadow: 0 0 0 2px rgba(102, 126, 234, 0.2);
}

.settings-modal-footer {
    padding: 8px 12px;
    border-top: 1px solid rgba(255, 255, 255, 0.1);
    display: flex;
    justify-content: flex-end;
}

.settings-save-btn {
    padding: 7px 18px;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: #fff;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 13px;
    font-weight: 600;
    transition: transform 0.15s, box-shadow 0.15s;
}

.settings-save-btn:hover {
    transform: translateY(-1px);
    box-shadow: 0 3px 10px rgba(102, 126, 234, 0.4);
}

.settings-save-btn:active {
    transform: translateY(0);
}

.general-link-btn {
    padding: 4px 8px;
    font-size: 11px;
    border-radius: 4px;
    border: 1px solid rgba(255, 255, 255, 0.12);
    background: rgba(255, 255, 255, 0.06);
    color: #d0d8f0;
    cursor: pointer;
    transition: background 0.15s;
    white-space: nowrap;
}

.general-link-btn:hover {
    background: rgba(102, 126, 234, 0.25);
    border-color: #667eea;
    color: #fff;
}

```

```

    }

    /* Search task item */
    .nv-quest-item.km-search-item {
        flex-wrap: wrap;
        align-items: center;
    }
    .km-search-item .nv-quest-name,
    .km-search-item .nv-quest-icon,
    .km-search-item .quest-controls {
        flex-shrink: 0;
    }
    .km-search-item .nv-quest-name {
        flex: 1;
    }

    @keyframes fadeIn {
        from { opacity: 0; }
        to { opacity: 1; }
    }

    @keyframes slideIn {
        from {
            opacity: 0;
            transform: translateY(-20px);
        }
        to {
            opacity: 1;
            transform: translateY(0);
        }
    }

    @keyframes fadeInUp {
        from {
            opacity: 0;
            transform: translateY(10px);
        }
        to {
            opacity: 1;
            transform: translateY(0);
        }
    }

    /* Giảm 25% độ lớn chuông thông báo */
    .load-notification > a {
        transform: scale(0.75) !important;
        transform-origin: center !important;
        display: inline-flex !important;
    }
    `;

    document.head.appendChild(style);
}

// =====
// Class kh[i tạo và chèn menu vào DOM
// =====
class UIInitializer {
    constructor(selector, linkGroups, accountId) {
        this.selector = selector;
        this.linkGroups = linkGroups;
        this.accountId = accountId;

        this.retryInterval = 500;
        this.timeout = 15000;
        this.elapsedTime = 0;
        this.intervalId = null;
    }

    start() {
        console.log("[HH3D Script] Đang tìm kiếm vị trí để chèn menu...");
        this.intervalId = setInterval(() => this.checkAndInsert(), this.retryInterval);
    }

    checkAndInsert() {
        const notificationsDiv = document.querySelector(this.selector);

        if (notificationsDiv) {
            clearInterval(this.intervalId);
            console.log("[HH3D Script] Đã tìm thấy vị trí. Bắt đầu chèn menu.");
            this.createAndInjectMenu(notificationsDiv);
        } else {
            this.elapsedTime += this.retryInterval;

            if (this.elapsedTime >= this.timeout) {
                clearInterval(this.intervalId);
                console.error(
                    `[HH3D Script - Lỗi] Không tìm thấy phần tử "${this.selector}" sau ${this.timeout / 1000} giây.`
                );
            }
        }
    }

    createAndInjectMenu(notificationsDiv) {
        const parentNavItems = notificationsDiv.parentNode;

        if (parentNavItems && parentNavItems.classList.contains("nav-items")) {
            if (document.querySelector(".custom-script-item-wrapper")) {
                console.log("[HH3D Script] Menu đã tồn tại. Bỏ qua việc chèn lại.");
                return;
            }

            const customMenuWrapper = document.createElement("div");
            customMenuWrapper.classList.add("load-notification", "relative", "custom-script-item-wrapper");

            const newMenuButton = document.createElement("a");
            newMenuButton.href = "#";
            newMenuButton.setAttribute("data-view", "hide");

            // Tạo phần tử div cho biểu tượng trạng thái
            const statusIcon = document.createElement("div");
            statusIcon.classList.add("custom-script-status-icon");
            newMenuButton.appendChild(statusIcon);
            const iconDiv = document.createElement("div");

```

```

const iconSpan = document.createElement("span");
iconSpan.classList.add('material-icons-round1', "material-icons-menu");
iconSpan.textContent = "task";
iconDiv.appendChild(iconSpan);

const label = document.createElement('span');
label.classList.add('nav-label');
label.textContent = 'Auto';
newMenuButton.appendChild(iconDiv);
newMenuButton.appendChild(label);

const dropdownMenu = document.createElement("div");
dropdownMenu.className = "custom-script-menu hidden";

/* ===== Kh[?]i tạo Profile UI với Quest Skeleton ===== */
const infoBox = document.createElement("div");
infoBox.id = "autoProfileInfo";
infoBox.style.position = "relative";
infoBox.innerHTML = `
<div id="xu-info" class="xu-info-container"></div>
<div id="reward-progress-wrap" style="margin-top:6px;">${createQuestSkeletonUI()}</div>
`;
dropdownMenu.appendChild(infoBox);

// G[?]n sự kiện cho các nút trong quest skeleton
setTimeout(() => {
    attachQuestButtonHandlers();
    loadHH3DProfile();
}, 100);

// Không còn tạo menu groups vì t[?]t c[?] đã chuyển vào quest list
// LINK_GROUPS hiện tại r[?]ng
if (this.linkGroups.length > 0) {
    this.linkGroups.forEach(group => {
        const groupDiv = document.createElement("div");
        groupDiv.className = "custom-script-menu-group";
        dropdownMenu.appendChild(groupDiv);

        group.links.forEach(link => {
            if (!link.isAutorun) {
                const menuItem = document.createElement("a");
                menuItem.classList.add("custom-script-menu-link");
                menuItem.href = link.url;
                menuItem.textContent = link.text;
                menuItem.target = "blank";
                groupDiv.appendChild(menuItem);
            }
        });
    });
}

// --- Thanh trạng thái ---
const statusBar = document.createElement("div");
statusBar.className = "custom-script-status-bar";
dropdownMenu.appendChild(statusBar);
const tongMonName = localStorage.getItem('tm_name');
// --- Footer Copyright (B[?] theo yêu c[?]u) ---
// const footer = document.createElement("div");
// footer.className = "custom-script-footer";
// footer.innerHTML = `
// <span class="custom-script-footer-guild">${tongMonName}</span>
// `;
dropdownMenu.appendChild(footer);

customMenuWrapper.appendChild(newMenuButton);
customMenuWrapper.appendChild(dropdownMenu);
parentNavItems.insertBefore(customMenuWrapper, notificationsDiv.nextSibling);

console.log("[HH3D Script] [?] Chèn menu tùy ch[?]nh thành công!");

newMenuButton.addEventListener("click", e => {
    e.preventDefault();
    dropdownMenu.classList.toggle("hidden");
    iconSpan.textContent = dropdownMenu.classList.contains("hidden") ? "task" : "highlight_off";
});

document.addEventListener("click", e => {
    if (!customMenuWrapper.contains(e.target)) {
        dropdownMenu.classList.add("hidden");
        iconSpan.textContent = "task";
    }
});
} else {
    console.warn('[HH3D Script - C[?]nh báo] [?] Không tìm th[?]y ph[?]n t[?] cha ".nav-items". Không th[?] chèn menu.
');
}
}

// Hàm đ[?] cập nhật statusBar
updateStatusBar(message, type = "info", duration = null) {
    const statusBar = document.querySelector(".custom-script-status-bar");
    if (!statusBar) return;

    const messageElement = document.createElement("div");
    messageElement.className = "custom-script-message";
    messageElement.classList.add(type);
    messageElement.textContent = message;

    // Thêm thông báo vào cu[?]i danh sách
    statusBar.appendChild(messageElement);

    // Giới hạn 5 thông báo
    while (statusBar.children.length > 5) {
        statusBar.removeChild(statusBar.firstChild);
    }

    // Tự động xóa thông báo sau một kho[?]ng thời gian
    if (duration !== null) {
        setTimeout(() => {
            messageElement.style.animation = "fadeOut 0.3s forwards";
            messageElement.addEventListener("animationend", () => {
                if (messageElement.parentNode === statusBar) {

```

```

        statusBar.removeChild(messageElement);
    }
    });
    }, duration);
}

// Xóa tất cả thông báo
clearStatusBar() {
    const statusBar = document.querySelector(".custom-script-status-bar");
    if (statusBar) {
        while (statusBar.firstChild) {
            statusBar.removeChild(statusBar.firstChild);
        }
    }
}

// Hàm updateButtonState (DEPRECATED - no longer needed)
async updateButtonState(taskName) {
    // No longer needed since UIMenuCreator is removed
    // All task buttons are now in QUEST_CONFIG list
}

}

// =====
// Countdown Timer - 1 vòng lặp duy nhất cho tất cả task
// =====
class CountdownTimer {
    constructor() {
        this.tasks = {}; // { taskName: targetTimestamp }
        this.intervalId = null;
    }

    set(taskName, delayMs) {
        this.tasks[taskName] = Date.now() + delayMs;
        const targetName = taskName === 'luyenDanCheck' ? 'luyenDan' : taskName;
        const el = document.querySelector(`.quest-next-time[data-task="${targetName}"]`);
        if (el) el.classList.add('active');
        if (!this.intervalId) this._start();
    }

    remove(taskName) {
        delete this.tasks[taskName];
        if (taskName === 'luyenDan') {
            // KHÔNG xóa DOM span - span chỉ được di chuyển khi bấm updateProgress()
            // Việc remove() chỉ dùng để ngầm hiểu, không cần thiết vào text hiện tại
        } else if (taskName === 'luyenDanCheck') {
            const el = document.querySelector(`.quest-next-time[data-task="luyenDan"]`);
            if (el) { el.textContent = ''; el.classList.remove('active'); }
        } else {
            const el = document.querySelector(`.quest-next-time[data-task="${taskName}"]`);
            if (el) { el.textContent = ''; el.classList.remove('active'); }
        }
        if (Object.keys(this.tasks).length === 0) this._stop();
    }

    clear() {
        for (const name in this.tasks) this.remove(name);
        this._stop();
    }

    _start() {
        if (this.intervalId) return;
        this.intervalId = setInterval(() => this._tick(), 1000);
    }

    _stop() {
        if (this.intervalId) { clearInterval(this.intervalId); this.intervalId = null; }
    }

    _tick() {
        const now = Date.now();
        for (const taskName in this.tasks) {
            const remaining = this.tasks[taskName] - now;
            if (taskName === 'luyenDanCheck') {
                const el = document.querySelector(`.quest-next-time[data-task="luyenDan"]`);
                if (el) {
                    if (remaining <= 0) {
                        if (el.textContent !== '') {
                            el.textContent = '';
                        }
                        el.classList.remove('active');
                        delete this.tasks[taskName];
                    } else {
                        const s = Math.max(0, Math.floor(remaining / 1000));
                        const newText = ` ${s}s`;
                        if (el.textContent !== newText) {
                            el.textContent = newText;
                        }
                        el.classList.add('active');
                    }
                }
            }
            continue;
        }
    }

    if (taskName === 'luyenDan') {
        if (typeof luyendan !== 'undefined' && luyendan && luyendan.isProcessing) {
            continue;
        }
        const el = document.querySelector(`.nv-quest-item[data-task-id="luyenDan"] .quest-progress`);
        if (el) {
            if (remaining <= 0) {
                if (el.textContent !== '') {
                    el.textContent = '';
                }
                delete this.tasks[taskName];
            } else {
                const accountId = localStorage.getItem('hh3d_account_id') || '';
                const isSafe = localStorage.getItem(`luyenDanIsSafe_${accountId}`) === 'true';
                const tuneCount = localStorage.getItem(`luyenDanTuneCount_${accountId}`) || '0';
                const tuneSurvivalMin = localStorage.getItem(`luyenDanTuneSurvivalMin_${accountId}`) || '3';
                const stab = localStorage.getItem(`luyenDanStability_${accountId}`) || '100';
                const totalSec = Math.max(0, Math.floor(remaining / 1000));
                const minVal = Math.floor(totalSec / 60);
                const secVal = totalSec % 60;
                const timeStr = `${String(minVal).padStart(2, '0')}:${String(secVal).padStart(2, '0')}`;
                let text;
            }
        }
    }
}

```

```

        if (isSafe) {
            text = `đã đi u ho ${tuneCount} l\n thời gian ${timeStr}`;
        } else {
            text = `Đang luyện (${stab}% - ${tuneCount}/${tuneSurvivalMin})`;
        }
        const newText = ` (${text})`;
        if (el.textContent !== newText) {
            el.textContent = newText;
        }
        localStorage.setItem(`luyenDanLastProgress_${accountId}`, text);
    }
}
continue;
}
const el = document.querySelector(`.quest-next-time[data-task="${taskName}"]`);
if (!el) continue;
if (remaining <= 0) {
    if (el.textContent !== '') {
        el.textContent = '';
    }
    el.classList.remove('active');
    delete this.tasks[taskName];
} else {
    const h = Math.floor(remaining / 3600000);
    const m = Math.floor((remaining % 3600000) / 60000);
    const s = Math.floor((remaining % 60000) / 1000);
    const newText = h > 0 ? `${h}:${String(m).padStart(2, '0')}:${String(s).padStart(2, '0')}` : `${String(s).padStart(2, '0')}`;
    if (el.textContent !== newText) {
        el.textContent = newText;
    }
}
}
if (Object.keys(this.tasks).length === 0) this._stop();
}
}
const countdownTimer = new CountdownTimer();

// =====
// Automatic
// =====
class AutomationManager {
    constructor() {
        this.accountId = accountId;
        this.delay = HH3D_FAST_MODE ? 1500 : 5000;

        // Các khoảng thời gian kiểm tra (ms)
        this.CHECK_INTERVAL_TIEN_DUYEN = 30 * 60 * 1000;
        this.INTERVAL_HOANG_VUC = 3 * 60 * 1000 + this.delay;
        this.INTERVAL_PHUC_LOI = 5 * 60 * 1000 + this.delay;
        this.INTERVAL_THI_LUYEN = 5 * 60 * 1000 + this.delay;
        this.INTERVAL_BI_CANH = 7 * 60 * 1000 + this.delay;
        this.INTERVAL_KHOANG_MACH = localStorage.getItem('khoangmach_check_interval') ? parseInt(localStorage.getItem('khoangmach_check_interval')) * 60 * 1000 + this.delay : 5 * 60 * 1000 + this.delay;
        this.INTERVAL_HOAT_DONG_NGAY = 10 * 60 * 1000 + this.delay;
        this.INTERVAL_LUYEN_DAN = 5 * 60 * 1000 + this.delay;

        this.timeoutIds = {};
        this.isRunning = false;
    }

    async start() {
        if (isSecurityProtectionActive()) {
            console.warn('[Auto] Trang đang bật lớp bảo mật. Chuyển sang chế độ an toàn, không tự động khi chạy task.');
```

```

    if (autoKhoangMach) {
        runStaggered(() => {
            this.INTERVAL_KHOANG_MACH = localStorage.getItem('khoangmach_check_interval') ? parseInt(
                localStorage.getItem('khoangmach_check_interval')) * 60 * 1000 + this.delay : 5 * 60 * 1000 + this.delay;
            this.scheduleTask('khoangmach', () => khoangmach.doKhoangMach(), this.INTERVAL_KHOANG_MACH);
        });
    }

    if (autoBiCanh) {
        runStaggered(() => this.scheduleTask("bicanh", async () => {
            // Đọc giữ lượt đánh bí c[on]h
            const reserve = Number(localStorage.getItem("reserveBiCanhAttacks")) || "0";
            console.log("Giá trị reserveBiCanhAttacks trong autobicanh:", reserve);
            await bicanh.doBiCanh();
        }), this.INTERVAL_BI_CANH));
    }

    if (autoCauNguyen) { runStaggered(() => this.caunguyentienduyen()); }

    runStaggered(() => this.scheduleHoatDongNgay());
    const _sh = parseInt(localStorage.getItem('selfSchedule_h')) ?? '0', 10) || 0;
    const _sm = parseInt(localStorage.getItem('selfSchedule_m')) ?? '30', 10);
    this.selfSchedule(_sh, _sm, 0); // Lên lịch tự động chạy theo cài đặt (mặc định 00:30 hàng ngày)
    // this.applyPromoCode();
}

async eventSchedule() {
    const now = Date.now();
    const nextEventTime = taskTracker.getNextTime(accountId, 'event');

    // Logic tính thời gian chờ mặc định
    // Nếu chưa có lịch hoặc tính ra s[ố] âm (quá khứ) thì đợi 1s rồi check lại, ngược lại đợi đúng thời gian
    let waitTime = 1000;

    if (nextEventTime && now >= nextEventTime) {
        console.log("[Auto] Đã đến giờ sự kiện. Đang thực hiện...");
        try {
            // Thực hiện nhiệm vụ
            await doDuaTopTongMon();

            // QUAN TRỌNG: Hàm doDuaTopTongMon phải có lệnh cập nhật lại nextEventTime (taskTracker).
            // Nếu không cập nhật thời gian, nó sẽ lặp vô tận liên tục gây treo trình duyệt.
        } catch (error) {
            console.error("[Auto] Lỗi khi thực hiện sự kiện:", error);
        }

        // Sau khi chạy xong (dù lỗi hay không), đợi 5 giây rồi check lại lịch mới
        waitTime = 5000;
    } else {
        // Chưa đến giờ, tính thời gian chờ
        if (nextEventTime) {
            waitTime = nextEventTime - now;
            // Nếu âm b[ên]o không chờ s[ố] âm (nếu máy tính bị lag)
            if (waitTime < 0) waitTime = 1000;
        } else {
            // Nếu không tìm thấy lịch (null), mặc định check lại sau 5 phút
            waitTime = 5 * 60 * 1000;
        }
    }

    // Gọi đệ quy để duy trì vòng lặp vĩnh vi[ễn]
    // setTimeout(() => {
    //     this.eventSchedule();
    // }, waitTime + (this.delay || 0));
}

/**Lên lịch tự chạy lại vào lúc 1 giờ */
async selfSchedule(h = 1, m = 0, s = 0) {
    if (!this.isRunning) return;
    const now = Date.now();
    const timeToRerun = new Date();
    timeToRerun.setHours(h, m, s, 0);
    if (timeToRerun.getTime() <= now) {
        timeToRerun.setDate(timeToRerun.getDate() + 1);
    }
    const delay = timeToRerun.getTime() - now;
    console.log(`[Auto] Lên lịch tự chạy lại vào lúc ${h} giờ ${m} phút ${s} giây. Thời gian chờ: ${delay}ms.`);
    countdownTimer.set('restart', delay);
    setTimeout(() => { this.stop(); }, delay);
    setTimeout(() => { this.start(); }, delay + 1000);
}

async doInitialTasks() {
    if (!taskTracker.isTaskDone(this.accountId, 'diemdanh')) {
        try {
            const nonce = await getNonce()
            if (!nonce) return
            await doDailyCheckin(nonce);
            await doClanDailyCheckin(nonce);
            await vandap.doVanDap(nonce);
        } catch (e) {
            console.error("[Auto] Lỗi khi thực hiện đi[ể]m danh, t[ên] l[ưu], v[ấn] đáp:", e);
        }
    }
}

async caunguyentienduyen() {
    if (!taskTracker.isTaskDone(this.accountId, 'tienduyen')) {
        try {
            const nonce = await getNonce()
            if (!nonce) return
            const result = await docaunguyen(this.accountId);
            console.log("K[ết] quả c[on]g nguyên:", result);
        } catch (e) {
            console.error("[C[on]g nguyên Tiên duyên]", e);
        }
    }
}

async scheduleTienDuyenCheck() {
    const isEnabled = localStorage.getItem('autoTienDuyen') !== '0';
    if (!isEnabled) {
        if (this.tienduyenTimeout) {

```

```

        clearTimeout(this.tienduyenTimeout);
        this.tienduyenTimeout = null;
    }
    countdownTimer.remove('tienduyen');
    return;
}
const now = Date.now();
const lastCheckTienDuyen = taskTracker.getLastCheckTienDuyen(this.accountId);
let timeToNextCheck;

if (lastCheckTienDuyen === null || now - lastCheckTienDuyen >= this.CHECK_INTERVAL_TIEN_DUYEN) {
    console.log("[Auto] Đã đến giờ làm Tiên Duyên. Đang thực hiện...");
    try {
        await tienduyen.doTienDuyen();
    } catch (error) {
        console.error("[Auto] Lỗi khi thực hiện Tiên Duyên:", error);
    }
    timeToNextCheck = this.CHECK_INTERVAL_TIEN_DUYEN;
} else {
    timeToNextCheck = this.CHECK_INTERVAL_TIEN_DUYEN - (now - lastCheckTienDuyen);
    console.log("[Auto] Chưa đến giờ tiên duyên. Sẽ chờ ${timeToNextCheck}ms.");
}

// Hẹn giờ gọi lại chính nó sau khoảng thời gian đã tính
if (this.tienduyenTimeout) clearTimeout(this.tienduyenTimeout);
countdownTimer.set('tienduyen', timeToNextCheck);
this.tienduyenTimeout = setTimeout(() => this.scheduleTienDuyenCheck(), timeToNextCheck);
}

/**
 * Tạo lịch trình cho một nhiệm vụ cụ thể.
 * - Ví dụ: scheduleTask('thiluyen', () => thiluyen.doThiLuyen(), this.INTERVAL_THI_LUYEN, 'thiluyenTimeout')
 * @param {string} taskName Tên của nhiệm vụ, dùng để truy vấn trạng thái (ví dụ: 'thiluyen').
 * @param {Function} taskAction Hàm để thực thi nhiệm vụ (ví dụ: 'hoangvuc.doHoangVuc').
 * @param {number} interval Chu kỳ lặp lại của nhiệm vụ tính bằng mili giây.
 */
async scheduleTask(taskName, taskAction, interval) {
    if (this.timeoutIds[taskName]) clearTimeout(this.timeoutIds[taskName]);

    // Kiểm tra xem quest này có bị tắt chạy tự động không (bao gồm cả luyenDan)
    const quest = QUEST_CONFIG.find(q => q.taskId === taskName);
    if (quest && quest.autorunEnabled) {
        const isEnabled = localStorage.getItem(quest.autorunKey) !== '0';
        if (!isEnabled) {
            console.log("[Auto] Nhiệm vụ ${taskName} đã bị tắt tự động. Dừng lịch trình.");
            if (this.timeoutIds[taskName]) {
                clearTimeout(this.timeoutIds[taskName]);
                this.timeoutIds[taskName] = null;
            }
            countdownTimer.remove(taskName);
            if (taskName === 'luyenDan') countdownTimer.remove('luyenDanCheck');
            return;
        }
    }

    let isTaskDone;
    if (taskName === 'bicanh' && await bicanh.isDailyLimit()) {
        isTaskDone = true;
    } else if (taskName === 'luyenDan') {
        isTaskDone = false; // Luyện đan luôn chạy ngầm để hiển thị UI đúng ngược
    } else {
        isTaskDone = taskTracker.isTaskDone(this.accountId, taskName);
    }

    // Kiểm tra và dừng lịch trình nếu nhiệm vụ đã hoàn thành
    if (isTaskDone) {
        loadHH3DProfile().catch(() => {});
        return;
    }

    const now = Date.now();
    // luyenDan luôn chạy ngay khi scheduleTask được gọi (timeout đã xử lý delay rồi)
    // Không dùng nextTime của taskTracker cho luyenDan để tránh bị kẹt ở trạng thái cũ
    const nextTime = taskName === 'luyenDan' ? null : taskTracker.getNextTime(this.accountId, taskName);
    let timeToNextCheck;

    if (nextTime === null || now >= nextTime) {
        // Trễ phút ứng ngẫu nhiên mô phỏng hành vi người dùng thật (2s - 7s, riêng luyenDan chỉ 0.5s - 2s để
        // đảm bảo vòng lặp ổn định)
        const humanDelay = HH3D_FAST_MODE
            ? (HH3D_FAST_HUMAN_DELAY_MIN + Math.floor(Math.random() * (HH3D_FAST_HUMAN_DELAY_MAX - HH3D_FAST_HUMAN_DELAY_MIN + 1)))
            : (taskName === 'luyenDan' ? (500 + Math.floor(Math.random() * 1500)) : (2000 + Math.floor(Math.random() * 5000)));
        console.log("[Auto] Đã đến giờ làm nhiệm vụ: ${taskName}. Delay nội bộ: ${humanDelay}ms...");
        await new Promise(r => setTimeout(r, humanDelay));

        console.log("[Auto] Đang thực hiện nhiệm vụ: ${taskName}...");
        try {
            // Cho phép taskAction trễ vài delay thực tế (ms hoặc chuỗi thời gian)
            let result = await taskAction();
            // Cập nhật trạng thái UI sau khi task chạy xong
            // Không gọi loadHH3DProfile cho luyenDan vì nó sẽ xóa span progress đang hiển thị
            if (taskName !== 'luyenDan') {
                loadHH3DProfile().catch(() => {});
            } else {
                // Chỉ cập nhật nút trạng thái, không rebuild toàn bộ profile
                updateAllQuestButtons().catch(() => {});
            }
            // Nếu trễ vài giây, dùng làm delay
            if (typeof result === 'number' && !isNaN(result) && result > 0) {
                timeToNextCheck = result;
            } else if (typeof result === 'string') {
                // Nếu trễ vài chuỗi, parse ra ms
                const ms = parseDelayString(result);
                timeToNextCheck = ms > 0 ? ms : interval;
            } else {
                timeToNextCheck = interval;
            }
        }

        // Thêm jitter ngẫu nhiên vào chu kỳ kiểm tra (tránh các nhiệm vụ không phải luyenDan chạy đều đặn
        // tuyệt đối)
        if (taskName !== 'luyenDan') {
            // Jitter từ -15s đến +45s
            const checkJitter = Math.floor(Math.random() * 60000) - 15000;
            timeToNextCheck = Math.max(10000, timeToNextCheck + checkJitter);
            console.log("[Auto] Đã thêm jitter vào chu kỳ kiểm tra của ${taskName}. Thời gian check tiếp

```

```

theo: ${Math.round(timeToNextCheck / 1000)}s.`);
    }
    } catch (error) {
        console.error(`[Auto] Lỗi khi thực hiện nhiệm vụ ${taskName}:`, error);
        // Có thể đặt thời gian chờ ngắn hơn khi có lỗi để thử lại
        timeToNextCheck = 3 * 60 * 1000; // Thử lại sau 3 phút
    }
} else {
    timeToNextCheck = Math.max(nextTime - now, 0);
    console.log(`[Auto] Nhiệm vụ ${taskName} chưa đến giờ, sẽ chờ ${timeToNextCheck}ms.`);
}

// Hẹn giờ cho lần chạy tiếp theo
if (this.timeoutIds[taskName]) clearTimeout(this.timeoutIds[taskName]);
if (taskName === 'luyenDan' || !taskTracker.isTaskDone(accountId, taskName)) {
    const taskFullName = {
        hoangvuc: "Hoàng Vực",
        phucloi: "Phúc Lợi",
        thiluyen: "Thí Luyện",
        bicanh: "Bí Cảnh",
        khoangmach: "Khoảng Mách",
        luyenDan: "Luyện Đan"
    }[taskName];
    //showNotification
    if (taskName === 'bicanh') {
        const isReserveHold = await bicanh.isReserveHold();
        if (isReserveHold) {
            //createUI.updateStatusBar(`${taskFullName}: đang giữ lượt, không hẹn giờ`, 'info', 0);
            return; // dừng hñn, không hẹn giờ
        }
    }
    // Cập nhật countdown vào từng nhiệm vụ (dùng 1 vòng lặp chung)
    if (taskName === 'luyenDan') {
        countdownTimer.set('luyenDanCheck', timeToNextCheck);
    } else {
        countdownTimer.set(taskName, timeToNextCheck);
    }
    this.timeoutIds[taskName] = setTimeout(() => this.scheduleTask(taskName, taskAction, interval),
timeToNextCheck);
}

}

// Hàm parse chuỗi thời gian dạng "6 phút 0 giây" hoặc "0 phút 40 giây" thành ms
async parseDelayString(str) {
    if (!str) return 0;
    let m = 0, s = 0, h = 0;
    // Hở trợ cở dạng "6 phút 0 giây", "40 giây", "1 giờ 2 phút 3 giây"
    const hourMatch = str.match(/(\d+)\s*giờ/);
    if (hourMatch) h = parseInt(hourMatch[1]);
    const minMatch = str.match(/(\d+)\s*phút/);
    if (minMatch) m = parseInt(minMatch[1]);
    const secMatch = str.match(/(\d+)\s*giây/);
    if (secMatch) s = parseInt(secMatch[1]);
    return h * 3600000 + m * 60000 + s * 1000;
}

async scheduleLuanVo() {
    const isDone = taskTracker.isTaskDone(this.accountId, 'luanvo');
    if (isDone) {
        if (this.luanvoTimeout) clearTimeout(this.luanvoTimeout);
        return;
    }
    await luanvo.startLuanVo();
    let timeTo21h = new Date();
    timeTo21h.setHours(21, 1, 0, 0);
    const delay = timeTo21h.getTime() - Date.now();
    console.log(`[Auto] Lên lịch Luận Vô vào lúc 00:01. Thời gian chờ: ${delay}ms.`);
    if (this.luanvoTimeout) clearTimeout(this.luanvoTimeout);
    if (delay < 0) {
        await luanvo.thueTieuViem();
        await luanvo.doLuanVo(true);
    } else {
        countdownTimer.set('luanvo', delay);
        this.luanvoTimeout = setTimeout(() => this.scheduleLuanVo(), delay);
    }
}

async scheduleDoThach() {
    const isEnabled = localStorage.getItem('autoDoThach') !== '0';
    if (!isEnabled) {
        if (this.dothachTimeout) {
            clearTimeout(this.dothachTimeout);
            this.dothachTimeout = null;
        }
        countdownTimer.remove('dothach');
        return;
    }
    const status = taskTracker.getTaskStatus(accountId, 'dothach');
    const isBetPlaced = status.betplaced;
    const isRewardClaimed = status.reward_claimed;

    const currentHour = parseInt(
        new Date().toLocaleString('en-US', {
            timeZone: 'Asia/Ho_Chi_Minh',
            hour: 'numeric',
            hour12: false
        }),
        10
    );

    let nextActionTime; // Giờ hành động tiếp theo (ví dụ: 13, 16, 21, 6)
    let timeToNextCheck; // Thời gian chờ (mili giây)

    const calculateTimeToNextHour = (targetHour) => {
        const now = new Date();
        const nextTime = new Date(now.getFullYear(), now.getMonth(), now.getDate(), targetHour, 0, 0, 0);
        if (now.getHours() >= targetHour) {
            nextTime.setDate(nextTime.getDate() + 1); // Nếu giờ mục tiêu đã qua, chuyển sang ngày mai
        }
        return nextTime.getTime() - now.getTime();
    };

    if (isBetPlaced) {
        // Đã đặt cược, chờ đến giờ nhận thưởng
        if (currentHour >= 6 && currentHour < 13) {
            nextActionTime = 13; // Chờ đến 13h để nhận thưởng lần 1

```

```

    } else if (currentHour >= 16 && currentHour < 21) {
      nextActionTime = 21; // Chờ đến 21h để nhận thưởng lần 2
    } else {
      console.log('[Đ] Thạch] Đã đặt cược nhưng không trong khung giờ cược, chờ khung giờ nhận thưởng
tiếp theo.');
```

```

      if (currentHour < 13) {
        nextActionTime = 13;
      } else if (currentHour < 21) {
        nextActionTime = 21;
      } else {
        nextActionTime = 6; // Chờ đến 6h sáng mai
      }
    }
  } else if (isRewardClaimed) {
    // Đã nhận thưởng, chờ đến giờ đặt cược tiếp theo
    if (currentHour >= 13 && currentHour < 16) {
      nextActionTime = 16; // Chờ đến 16h để đặt cược lần 2
    } else {
      nextActionTime = 6; // Chờ đến 6h sáng hôm sau
    }
  } else {
    const stoneType = localStorage.getItem('dice-roll-choice') ?? 'tai';
    // Chưa đặt cược hoặc chưa nhận thưởng. Kiểm tra khung giờ hiện tại
    if (currentHour >= 6 && currentHour < 13) {
      console.log('[Đ] Thạch] Đang trong khung giờ 6h-13h. Đang đặt cược...');
      await dothach.run(stoneType); // Thực hiện đặt cược
      createUI.updateButtonState('dothach');
      nextActionTime = 13; // Sau khi cược, chờ đến 13h để kiểm tra thưởng
    } else if (currentHour >= 16 && currentHour < 21) {
      console.log('[Đ] Thạch] Đang trong khung giờ 16h-21h. Đang đặt cược...');
      await dothach.run(stoneType); // Thực hiện đặt cược
      createUI.updateButtonState('dothach');
      nextActionTime = 21; // Sau khi cược, chờ đến 21h để kiểm tra thưởng
      setTimeout(loadHH3DProfile, 100); // cập nhật profile sau khi đặt cược
    } else {
      // Không trong khung giờ nào, chờ đến khung giờ đặt cược tiếp theo
      console.log('[Đ] Thạch] Không trong khung giờ cược. Chờ...');
      if (currentHour < 6) {
        nextActionTime = 6;
      } else if (currentHour < 16) {
        nextActionTime = 16;
      } else {
        nextActionTime = 6; // Chờ đến 6h sáng mai
      }
    }
  }
}

timeToNextCheck = calculateTimeToNextHour(nextActionTime);

// Hủy timeout cũ nếu có và thiết lập timeout mới
if (this.dothachTimeout) clearTimeout(this.dothachTimeout);
countdownTimer.set('dothach', timeToNextCheck);
this.dothachTimeout = setTimeout(() => this.scheduleDoThach(), timeToNextCheck);

console.log(`[Đ] Thạch] Lần kiểm tra tiếp theo lúc: ${new Date(Date.now() + timeToNextCheck).
toLocaleTimeString('vi-VN', { timeZone: 'Asia/Ho_Chi_Minh' })}`);
}

async scheduleHoatDongNgay() {
  const isDone = taskTracker.isTaskDone(this.accountId, 'hoatdongngay');
  if (isDone) {
    console.log("[Auto] Hoạt Động Ngày đã hoàn thành, không cần lên lịch.");
    if (this.hoatdongngayTimeout) clearTimeout(this.hoatdongngayTimeout);
    loadHH3DProfile().catch(() => {});
    return;
  }
  const isHoangVucDone = taskTracker.isTaskDone(this.accountId, 'hoangvuc');
  const isPhucLoiDone = taskTracker.isTaskDone(this.accountId, 'phucloi');
  const isDiemDanhDone = taskTracker.isTaskDone(this.accountId, 'diemdanh');
  const isThiLuyenDone = taskTracker.isTaskDone(this.accountId, 'thiluyen');
  // const isLuanVoDone = taskTracker.isTaskDone(this.accountId, 'luanvo');
  if (isHoangVucDone && isPhucLoiDone && isDiemDanhDone && isThiLuyenDone) {
    console.log("[Auto] Điều kiện đã đủ, đang thực hiện Hoạt Động Ngày...");
    try {
      await hoatdongngay.doHoatDongNgay();
      if (this.hoatdongngayTimeout) clearTimeout(this.hoatdongngayTimeout);
      if (taskTracker.isTaskDone(this.accountId, 'hoatdongngay') && this.hoatdongngayTimeout) {
        return;
      }
    } else {
      countdownTimer.set('hoatdongngay', 5 * 60 * 1000);
      this.hoatdongngayTimeout = setTimeout(() => this.scheduleHoatDongNgay(), 5 * 60 * 1000);
    }
  }
  catch (e) {
    console.error("[Auto] Lỗi khi thực hiện Hoạt Động Ngày:", e);
  }
} else {
  if (this.hoatdongngayTimeout) clearTimeout(this.hoatdongngayTimeout);
  countdownTimer.set('hoatdongngay', this.INTERVAL_HOAT_DONG_NGAY);
  this.hoatdongngayTimeout = setTimeout(() => this.scheduleHoatDongNgay(), this.INTERVAL_HOAT_DONG_NGAY);
}
}

stop() {
  if (!this.isRunning || !window.isRunning) return;
  window.isRunning = false;
  for (const taskName in this.timeoutIds) {
    if (this.timeoutIds[taskName]) {
      clearTimeout(this.timeoutIds[taskName]);
      this.timeoutIds[taskName] = null; // Đặt lại giá trị để tránh rò rỉ bộ nhớ
      console.log(`[Auto] Đã hủy hẹn giờ cho nhiệm vụ: ${taskName}`);
    }
  }
  if (this.tienduyenTimeout) {
    clearTimeout(this.tienduyenTimeout);
    console.log('Đã dừng quá trình tự động tiên duyên');
  }
  if (this.dothachTimeout) {
    clearTimeout(this.dothachTimeout);
    console.log('Đã dừng quá trình tự động đặt cược');
  }
  if (this.hoatdongngayTimeout) {
    clearTimeout(this.hoatdongngayTimeout);
    console.log('Đã dừng quá trình tự động hoạt động ngày');
  }
  // Xóa tất cả countdown trừ 'restart' (vẫn đang chờ selfSchedule)

```

```

    Object.keys(countdownTimer.tasks).forEach(name => {
        if (name !== 'restart') countdownTimer.remove(name);
    });
    createUI.clearStatusBar();
}

checkAndStart() {
    if (localStorage.getItem('autorunEnabled') === null) {
        localStorage.setItem('autorunEnabled', '0');
    }
    if (localStorage.getItem('autoLuyenDan') === null) {
        localStorage.setItem('autoLuyenDan', '1'); // [v2.17.1-local] Mặc định BẬT tự động luyện đàn
    }

    if (isSecurityProtectionActive()) {
        console.warn('[Automation] Phát hiện lớp bảo mật. Không tự động khởi động để tránh xung đột.');
```

window.isRunning = false;

this.isRunning = false;

return;

let autorunEnabled = localStorage.getItem('autorunEnabled') === '1';

```

    if (autorunEnabled) {
        console.log('[Automation] Tự động khởi động Autorun...');

        // Tạo một hàm chờ để đảm bảo UI đã sẵn sàng
        const checkStatusIcon = () => {
            const statusIcon = document.querySelector('.custom-script-status-icon');
            if (statusIcon) {
                // Nếu icon đã tồn tại, cập nhật trạng thái và bắt đầu tác vụ
                this.start();
            } else {
                // Nếu icon chưa tồn tại, chờ 100ms và thử lại
                setTimeout(checkStatusIcon, 100);
            }
        };

        // Bắt đầu quá trình kiểm tra
        checkStatusIcon();
    } else {
        console.log('[Automation] Autorun không được bật, sẽ không khởi động tự động.');
```

```

}
// =====
// HIỆN TU VI KHOẢNG MẠCH
// =====
class hienTuViKhoangMach {
    constructor() {
        this.selfTuViCache = null;
        this.mineImageSelector = '.mine-image';
        this.attackButtonSelector = '.attack-btn';
        this.currentMineUsers = []; // Sẽ lưu dữ liệu người dùng tại đây
        this.tempObserver = null; // Bên để lưu MutationObserver tạm thời
        this.nonceGetUserInMine = null;
        this.nonce = null;
        this.headers = {
            'Content-Type': 'application/x-www-form-urlencoded; charset=UTF-8',
            'X-Requested-With': 'XMLHttpRequest'
        };
        this.currentMineId = null;
        this.tempObserverRearrange = null; // Bên để lưu MutationObserver tạm thời khi sắp xếp
    }

    async waitForElement(selector, timeout = 15000) {
        const found = document.querySelector(selector);
        if (found) return Promise.resolve(found);
        return new Promise((resolve) => {
            const obs = new MutationObserver(() => {
                const el = document.querySelector(selector);
                if (el) {
                    obs.disconnect();
                    clearTimeout(timer);
                    resolve(el);
                }
            });
            obs.observe(document.documentElement || document.body, { childList: true, subtree: true });
            const timer = setTimeout(() => {
                obs.disconnect();
                resolve(null);
            }, timeout);
        });
    }

    async getNonceGetUserInMine() {
        const htmlSource = document.documentElement.innerHTML;
        const tokens = extractActionTokens(htmlSource);
        const security_get_users = tokens["get_users_in_mine"];
        return security_get_users || null;
    }

    async getNonce() {
        if (typeof restNonce !== 'undefined' && restNonce) {
            return restNonce;
        }

        const scripts = document.querySelectorAll('script');
        for (const script of scripts) {
            const match = script.innerHTML.match(/"restNonce"\s*:\s*"([a-f0-9]+)"/i);
            if (match) {
                return match[1];
            }
        }

        try {
            const nonce = await getSecurityNonce(weburl + '?t', "restNonce");
            if (nonce) {
                return nonce;
            }
        } catch (error) {
            console.error("Failed to get security nonce", error);
        }

        return null;
    }
}

```

```

async getSelfTuVi(forceRefresh = false) {
  if (!forceRefresh && this.selfTuViCache !== null) {
    return this.selfTuViCache;
  }
  // ưu tiên header hiện tu vi
  const el = document.querySelector('#head_manage_acc');
  const text = (el?.textContent || '').trim();
  let num = text.match(/\d+/);
  if (num) {
    this.selfTuViCache = parseInt(num[0]);
    return this.selfTuViCache;
  }
  // fallback: quét body (trường hợp UI đ)
  const bodyText = (document.body?.innerText || '').slice(0, 5000);
  num = bodyText.match(/Tu\s*Vi\s*[:\s]*\s*(\d+)/i) || bodyText.match(/(\d{6,})/);
  if (num) {
    this.selfTuViCache = parseInt(num[1] || num[0]);
    return this.selfTuViCache;
  }
  return null;
}

async getProfileTier(userId) {
  if (!userId) return null;
  try {
    const res = await fetch(`${weburl}profile/${userId}/`);
    if (!res.ok) return null;

    const text = await res.text(); // ph) await
    const doc = new DOMParser().parseFromString(text, 'text/html');

    const h4 = doc.querySelector('.um-name h4');
    if (!h4) return null;

    // l) text từ <b> n) có, n) không fallback div (class thay đ) m) id) / h4
    const raw = h4.querySelector('b')?.textContent
      || h4.querySelector('div[class*=color_]')?.textContent
      || h4.querySelector('div')?.textContent
      || h4.textContent
      || "";
    console.log(`L) c)nh giới cho userId ${userId}:`, raw.trim());
    return raw.trim();
  } catch (e) {
    console.error(`${this.logPrefix} ) L) mạng (l) c)nh giới:`, e);
    return null;
  }
}

winRate(selfTuVi, opponentTuVi) {
  if (!selfTuVi || !opponentTuVi) return -1;
  if (typeof selfTuVi !== 'number' || typeof opponentTuVi !== 'number') return -1;
  if (selfTuVi <= 0 || opponentTuVi <= 0) return -1;
  if (selfTuVi >= 10 * opponentTuVi) return 100;
  if (opponentTuVi >= 10 * selfTuVi) return 0;
  let winChance = 50;
  const diff = selfTuVi - opponentTuVi;
  const ratio = diff > 0 ? selfTuVi / opponentTuVi : opponentTuVi / selfTuVi;
  const factor = ratio >= 8 ? 1 : ratio >= 7 ? 0.9 : ratio >= 6 ? 0.8 :
    ratio >= 5 ? 0.7 : ratio >= 4 ? 0.6 : ratio >= 3 ? 0.5 :
    ratio >= 2 ? 0.4 : 0.3;
  winChance += (diff / 1000) * factor;
  return Math.max(0, Math.min(100, winChance));
}

async upsertTuViInfo(btn, userId, opponentTuVi, myTuVi) {
  const cls = 'hh3d-tuvi-info';
  const next = btn.nextElementSibling;
  const opponentTuViText = typeof opponentTuVi === 'number' ? opponentTuVi : 'Unknown';

  // Tạo nội dung HTML một l)n duy nh)t
  const rate = this.winRate(myTuVi, opponentTuVi).toFixed(2);
  const rateNumber = parseFloat(rate);
  let rateColor;
  if (rateNumber === -1) {
    rateColor = '#808080'; // Grey
  }
  else if (rateNumber < 25) {
    rateColor = '#ff5f5f'; // Red
  }
  else if (rateNumber > 75) {
    rateColor = '#00ff00'; // Green
  }
  else {
    rateColor = '#ffff00ff'; // White
  }

  let displayRate = rate;
  if (rateNumber === 0.00) {
    displayRate = '0';
  }
  else if (rateNumber === 100.00) {
    displayRate = '100';
  }
  else if (rateNumber === -1) {
    displayRate = 'Không rõ';
  }
  let innerHTMLContent = '';
  if (myTuVi <= 10 * opponentTuVi) {
    innerHTMLContent = `
    <p><strong>Tu Vi</strong> <span style="font-weight: bold; color: #ffff00ff;">${opponentTuViText}</
    span></p>
    <p><strong>T) Lệ Th)ng</strong> <span style="font-weight: bold; color: ${rateColor}">${displayRate}%</
    span></p>
    `;
  }
  else {
    innerHTMLContent = `
    <p><strong>Tu Vi</strong> <span style="font-weight: bold; color: #ffff00ff;">${opponentTuViText}</
    span></p>
    <p><span style="font-weight: bold; color: #00ff00ff;">Không t)n lượt</span></p>
    `;
  }

  if (next && next.classList.contains(cls) && next.dataset.userId === String(userId)) {
    next.innerHTML = innerHTMLContent;
    return;
  }

  document.querySelectorAll(`.${cls}[data-user-id="${userId}"]`).forEach(el => {
    if (el !== next) el.remove();
  });
}

```

```

const info = document.createElement('div');
info.className = cls;
info.dataset.userId = String(userId);
info.style.fontSize = '12px';
info.style.color = '#fff';
info.style.marginTop = '3px';
info.style.backgroundColor = 'none';
info.style.padding = '0px 0px';
info.style.border = 'none';

// Sõ dụng biõn đã tạo õ trên
info.innerHTML = innerHTMLContent;

btn.insertAdjacentElement('afterend', info);
}

async upsertTierInfo(btn, userId) {
const cls = 'hh3d-tuvi-info';
const next = btn.nextElementSibling;
const tierText = await this.getProfileTier(userId);
console.log(`UserID: ${userId}, Tier: ${tierText}`);
if (!tierText) return;
if (next && next.classList.contains(cls) && next.dataset.userId === String(userId)) {
next.innerHTML = `<p><strong>Cõnh giõi:</strong> <span style="font-weight: bold; color: #ffff00ff;">${
tierText}</span></p>`;
return;
}

document.querySelectorAll(`.${cls}[data-user-id="${userId}"]`).forEach(el => {
if (el !== next) el.remove();
});
const info = document.createElement('div');
info.className = cls;
info.dataset.userId = String(userId);
info.style.fontSize = '12px';
info.style.color = '#fff';
info.style.marginTop = '3px';
info.style.backgroundColor = 'none';
info.style.padding = '0px 0px';
info.style.border = 'none';
info.innerHTML = `<p><strong>Cõnh giõi:</strong> <span style="font-weight: bold; color: #ffff00ff;">${
tierText}</span></p>`;
btn.insertAdjacentElement('afterend', info);
}

async getUsersInMine(mineId) {
let securityToken = null;

// Cách 1: Lõy từ unsafeWindow (Biõn thật cõa trang web)
if (typeof unsafeWindow !== 'undefined' && unsafeWindow.hh3dData && unsafeWindow.hh3dData.securityToken) {
securityToken = unsafeWindow.hh3dData.securityToken;
}
// Cách 2: Lõy từ hData
else if (typeof hData !== 'undefined' && hData.securityToken) {
securityToken = hData.securityToken;
}

// Cách 3: Nõu võn null -> Gõi hàm quét (Fallback cũõ cùng)
if (!securityToken) {
console.log(`${this.logPrefix} ð Token biõn global bị thiõu, đang fetch lại...`);
// Gõi hàm getSecurityToken chúng ta đã viõt õ trên
securityToken = await getSecurityToken(this.khoangMachUrl || window.location.href);
}
this.nonceGetUserInMine = await this.getNonceGetUserInMine();

if (!this.nonceGetUserInMine || !securityToken) {
let errorMsg = `Lõĩ (get_users):`;
if (!this.nonceGetUserInMine) errorMsg += " Nonce (security) chưa được cung cõp.";
if (!securityToken) errorMsg += " Không tìm thõy 'security_token' (hh3dData).";

showNotification(errorMsg, 'error');
return null;
}

const payload = new URLSearchParams({
action: hData && hData.act ? hData.act.kmUsers : 'get_users_in_mine',
mine_id: mineId,
security_token: securityToken,
security: this.nonceGetUserInMine
});

try {
const r = await fetch.ajaxUrl, {
method: 'POST',
headers: this.headers,
body: payload,
credentials: 'include'
});
const d = await r.json();
console.log(`${this.logPrefix} Dữ liệu người chơi trong mõ:`, d);
return d.success ? d.data : (showNotification(d.message || `Lõĩ lõy thông tin người chơi.`, 'error'),
null);
} catch (e) {
console.error(`${this.logPrefix} ð Lõĩ mạng (lõy user):`, e);
return null;
}
}

// async getTuVi(userId) {
// // 0. Chuõn bị Nonce & Headers
// if (!this.nonce) {
// this.nonce = await this.getNonce();
// }
// const nonce = this.nonce;
// if (!nonce) return null;

// const headers = {
// "Content-Type": "application/json",
// "X-WP-Nonce": nonce
// };
// const targetId = String(userId);

// // =====
// // CÁCH 1: LOGIC CÕ (GIỮ NGUYỄN BõN GõC)
// // =====

```

```

// try {
//   const res = await fetch(`${weburl}/wp-json/luan-vo/v1/search-users`, {
//     method: "POST",
//     headers: headers,
//     body: JSON.stringify({ query: targetId, page: 1 }),
//     credentials: "include",
//     mode: "cors"
//   });
//
//   // Logic gıc: Lầy user đầu tiên trong danh sách (users[0])
//   const points = res.ok ? (await res.json())?.data?.users?.[0]?.points ?? null : null;
//
//   // Nếu tìm thấy điểm -> Trỏ về luôn
//   if (points !== null && points !== undefined) {
//     return points;
//   }
// } catch (e) {
//   // Lỗi cách 1 -> Bỏ qua đi chạy xuống cách 2
// }
//
// // =====
// // CÁCH 2: FALLBACK (FOLLOW -> SCAN -> UNFOLLOW)
// // Chạy khi Cách 1 trả về null hoặc lỗi
// // =====
// console.log(`[GetTuVi] Cách 1 thất bại, đang dùng Fallback cho ID ${targetId}...`);
//
// let tuVi = null;
//
// try {
//   // B2.1: Follow
//   await fetch(`${weburl}/wp-json/luan-vo/v1/follow`, {
//     method: "POST",
//     headers: headers,
//     body: JSON.stringify({ followed_user_id: targetId }),
//     credentials: "include",
//     mode: "cors"
//   });
//
//   // B2.2: Lấy danh sách Following
//   const resList = await fetch(`${weburl}/wp-json/luan-vo/v1/get-following-users`, {
//     method: "POST",
//     headers: headers,
//     body: JSON.stringify({ page: 1 }),
//     credentials: "include",
//     mode: "cors"
//   });
//
//   if (resList.ok) {
//     const jsonList = await resList.json();
//     if (jsonList.success && jsonList.data && Array.isArray(jsonList.data.users)) {
//       // Danh sách follow thì phải tìm chính xác ID kẻo lấy nhầm người khác
//       const targetUser = jsonList.data.users.find(u => String(u.id) === targetId);
//       if (targetUser) {
//         tuVi = targetUser.points;
//       }
//     }
//   }
// } catch (e) {
//   console.error(`[GetTuVi] Fallback lỗi:`, e);
// } finally {
//   // B2.3: Unfollow (Luôn chạy để dọn rác)
//   try {
//     await fetch(`${weburl}/wp-json/luan-vo/v1/unfollow`, {
//       method: "POST",
//       headers: headers,
//       body: JSON.stringify({ unfollow_user_id: targetId }),
//       credentials: "include",
//       mode: "cors"
//     });
//   } catch (ignore) {}
// }
//
// return tuVi;
// }
}
}

async showTotalEnemies(mineId) {
  const data = await this.getUsersInMine(mineId);
  const currentMineUsers = data && data.users ? data.users : [];
  let totalEnemies = 0;
  let totalLienMinh = 0;
  let totalDongMon = 0;
  const myTuVi = await this.getSelfTuVi();
  let isInMine = currentMineUsers.some(user => user.id.toString() === accountId.toString());
  for (let user of currentMineUsers) {
    if (user.dong_mon) {
      totalDongMon++;
    } else if (user.lien_minh) {
      totalLienMinh++;
    } else {
      totalEnemies++;
    }
  }
}

const bonus_display = document.querySelector('#bonus-display');
const batquai_section = document.querySelector('#batquai-section');
const pagination = document.querySelector('.pagination');
const page_indicator = document.querySelector('#page-indicator');
if (bonus_display) {
  let existingInfo = document.querySelector('.hh3d-mine-info');
  if (!existingInfo) {
    existingInfo = document.createElement('div');
    existingInfo.className = 'hh3d-mine-info';
    //existingInfo.style.right = '5px';
    existingInfo.style.fontSize = '11px';
    existingInfo.style.color = 'fff';
    existingInfo.style.marginLeft = '-1px';
    existingInfo.style.backgroundColor = 'none';
    existingInfo.style.padding = '0px 0px';
    existingInfo.style.border = 'none';
    existingInfo.style.textAlign = 'left';
    existingInfo.style.fontFamily = 'Font Awesome 5 Free';
    bonus_display.prepend(existingInfo);
    bonus_display.style.display = 'block';
    batquai_section.style.display = 'block';
  }
}

```

```

const observer = new MutationObserver(() => {
  bonus_display.style.display = 'block';
  batquai_section.style.display = 'block';
  pagination.style.display = 'block';
  page_indicator.style.display = 'block';
});
observer.observe(bonus_display, { attributes: true, attributeFilter: ['style'] });
observer.observe(batquai_section, { attributes: true, attributeFilter: ['style'] });
observer.observe(pagination, { attributes: true, attributeFilter: ['style'] });
observer.observe(page_indicator, { attributes: true, attributeFilter: ['style'] });
}

existingInfo.innerHTML = `
<h style="color: #ff5f5f;">Kịch dich: <b>${totalEnemies}</b></h><br>
<h style="color: #ffff00;">Liên Minh: <b>${totalLienMinh}</b></h><br>
<h style="color: #9c59bdf;">Đông Môn: <b>${totalDongMon}</b></h>
`;
}

async addEventListenersToReloadBtn(mineId) {
  const reloadBtn = document.querySelector('#reload-btn');
  if (reloadBtn && !reloadBtn.dataset.listenerAdded) {
    reloadBtn.addEventListener('click', async () => {
      this.showTotalEnemies(mineId);
    });
    reloadBtn.dataset.listenerAdded = 'true';
  }
}

async addEventListenersToMines() {
  const mineImages = document.querySelectorAll(this.mineImageSelector);
  mineImages.forEach(image => {
    if (!image.dataset.listenerAdded) {
      image.addEventListener('click', async (event) => {
        const mineId = event.currentTarget.getAttribute('data-mine-id');
        if (mineId) {
          this.showTotalEnemies(mineId);
          this.addEventListenersToReloadBtn(mineId);
        }
      });
      image.dataset.listenerAdded = 'true';
    }
  });
}

async decodeAvatar(encoded, viewerId) {
  try {
    // Validate input
    if (!encoded || typeof encoded !== 'string') {
      return null;
    }

    const key = (viewerId % 251) + 1;
    // Browser không có Buffer, dùng atob() để decode Base64
    const raw = atob(encoded);
    let result = '';
    for (let i = 0; i < raw.length; i++) {
      result += String.fromCharCode(raw.charCodeAt(i) ^ (key ^ (i % 7)));
    }
    return result;
  } catch (e) {
    console.error('decodeAvatar error:', e, 'Input:', encoded);
    return null;
  }
}

async showTuVi(myTuVi) {
  if (!myTuVi) return;

  const rows = document.querySelectorAll('.user-row');
  for (const row of rows) {
    if (row.dataset.tuviAttached === '1') continue;
    row.dataset.tuviAttached = '1';

    // Lấy userId từ href profile hoặc src avatar
    const profileLink = row.querySelector('a[href*="/profile/"]');
    const avatarImg = row.querySelector('img[src*="/ultimatemember/"]');
    let userId = null;
    if (profileLink) {
      const m = profileLink.getAttribute('href').match(/\/profile\/(\d+)\//);
      if (m) userId = m[1];
    }
    if (!userId && avatarImg) {
      const m = avatarImg.getAttribute('src').match(/\/ultimatemember\/(\d+)\//);
      if (m) userId = m[1];
    }
    if (!userId) continue;
    // console.log('Đang xử lý userId ${userId}...');

    const btn = row.querySelector('.attack-btn');

    try {
      await new Promise(r => setTimeout(r, 500));
      if (btn) this.upsertTierInfo(btn, userId);
    } catch (e) {
      console.error('getTuVi error', e);
    }

    if (btn) {
      const mineId = btn.getAttribute('data-mine-id');
      if (mineId && mineId !== this.currentMineId) {
        this.currentMineId = mineId;
        this.showTotalEnemies(mineId);
        this.addEventListenersToReloadBtn(mineId);
      }
    }
    // nghỉ 1s tránh spam
    await new Promise(r => setTimeout(r, 1000));
  }
}

async startUp() {
  if (document.readyState === 'loading') {
    await new Promise(resolve => document.addEventListener('DOMContentLoaded', resolve, { once: true }));
  }
  this.nonceGetUserInMine = await this.getNonceGetUserInMine();
}

```

```

this.nonce = await this.getNonce();
await this.waitForElement('#head_manage_acc', 15000);

const getMyTuVi = async () => {
    const v = await this.getSelfTuVi(true);
    return v;
};

const firstTuVi = await getMyTuVi();
if (firstTuVi) {
    await this.showTuVi(firstTuVi);
}

// quan sát DOM để cập nhật khi các nút attack xuất hiện hoặc nội dung thay đổi
let __timeout = null;
const observer = new MutationObserver(() => {
    clearTimeout(__timeout);
    __timeout = setTimeout(async () => {
        const latest = await getMyTuVi();
        await this.showTuVi(latest);
    }, 200);
});
observer.observe(document.body, { childList: true, subtree: true });

this.addEventListenersToMines();
// MutationObserver chính để thêm listener cho các mìn mới
const mainObserver = new MutationObserver(() => {
    this.addEventListenersToMines();
});
mainObserver.observe(document.body, { childList: true, subtree: true });
}
}

// =====
// Bộ lọc tổng môn
// =====
async function getDivContent(url, selector) {
    const logPrefix = '[HH3D Auto]';

    console.log(`${logPrefix} ▶ Đang tải trang từ ${url} để lấy nội dung...`);
    try {
        const response = await fetch(url);
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }
        const html = await response.text();

        // Sử dụng DOMParser để phân tích mã HTML
        const parser = new DOMParser();
        const doc = parser.parseFromString(html, 'text/html');

        // Tìm phần tử bằng selector
        const element = doc.querySelector(selector);

        if (element) {
            const content = element.innerHTML;
            console.log(`${logPrefix} □ Đã trích xuất thành công nội dung: ${content}`);
            return content;
        } else {
            console.error(`${logPrefix} □ Không tìm thấy phần tử với bộ chọn: ${selector}`);
            return null;
        }
    } catch (e) {
        console.error(`${logPrefix} □ Lỗi khi tải trang hoặc trích xuất nội dung:`, e);
        return null;
    }
}

// □ HÀM LẤY ID TỔNG MÔN TỪ URL □NH
async function getTongMonId(url) {
    const logPrefix = '[HH3D Auto]';
    // console.log(`${logPrefix} ▶ Đang lấy ID tổng môn từ ${url}...`);
    try {
        const response = await fetch(url);
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }
        const html = await response.text();

        const parser = new DOMParser();
        const doc = parser.parseFromString(html, 'text/html');

        // Tìm link tổng môn trong link: 
        const link = doc.querySelector('.tm-guild-avatar-link');

        if (link) {
            const img = link.querySelector('img');
            if (img && img.src) {
                // Extract ID từ URL □nh: .../group-image/628390.jpg → 628390
                const match = img.src.match(/\/group-image\/(\d+)\.jpg/);
                if (match && match[1]) {
                    const tongMonId = match[1];
                    // console.log(`${logPrefix} □ Đã lấy ID tổng môn: ${tongMonId}`);

                    // Lấy tên tổng môn từ alt của img
                    localStorage.setItem('tm_name', img.alt ? img.alt.trim() : 'Không rõ');
                    console.log(`${logPrefix} □□ Tên tổng môn: ${img.alt} = ${localStorage.getItem('tm_name')}`);

                    return tongMonId;
                }
            }
        }
        console.error(`${logPrefix} □ Không tìm thấy ID tổng môn`);
        return null;
    } catch (e) {
        console.error(`${logPrefix} □ Lỗi khi lấy ID tổng môn:`, e);
        return null;
    }
}

async function kiemTraIdTong(tongMonId) {
    try {
        // Gọi GitHub để lấy danh sách ID tổng môn hợp lệ: ["628390", "123456", ...]
        const response = await fetch('https://raw.githubusercontent.com/krizk-tool/hh3d-tool/main/tm_hop_le.json');

        if (!response.ok) {

```

```

        throw new Error(`Không thể kiểm tra tổng môn: ${response.status} ${response.statusText}`);
    }

    const data = await response.json();

    // Response.json() đã trả về array trực tiếp: ["628390", "123456", ...]
    if (!data || !Array.isArray(data)) {
        throw new Error("Không tìm thấy danh sách tổng môn được phép.");
    }

    const danhSachTongId = data;
    // console.log(`[HH3D Auto] Danh sách ID tổng môn hợp lệ đã tải: ${danhSachTongId.length} tổng môn.`);
    // console.log(`[HH3D Auto] Danh sách ID tổng môn: ${danhSachTongId.join(', ')}');

    if (!tongMonId) {
        console.error('[HH3D Auto] ID tổng môn trống');
        return false;
    }

    // Chuyển hóa ID: trim và chuyển thành string
    const idChuanHoa = String(tongMonId).trim();
    // console.log(`[HH3D Auto] ID tổng môn sau khi chuyển hóa: "${idChuanHoa}"`);
    // Kiểm tra tên tại - so sánh trực tiếp với danh sách ID (as string)
    const isExist = danhSachTongId.some(id => String(id) === idChuanHoa);
    // console.log(`[HH3D Auto] Kết quả kiểm tra: ID "${idChuanHoa}" ${isExist ? 'TÌM THẤY' : 'KHÔNG TÌM THẤY'}`);

    trong danh sách`);

    return isExist;

} catch (error) {
    console.error('Lỗi khi kiểm tra tổng:', error);
    return false;
}

}

async function checkTongMon() {
    // Lấy ID tổng môn từ URL ảnh (628390.jpg) thay vì tên
    const tongMonId = await getTongMonId(weburl + 'danh-sach-thanh-vien-tong-mon?t=' + Date.now());
    // console.log(`[HH3D Auto] ID tổng môn hiện tại: ${tongMonId}`);

    if (!tongMonId) {
        console.error('[HH3D Auto] Không lấy được ID tổng môn');
        return false;
    }

    const isValid = await kiểmTraIdTong(tongMonId);
    console.log(`[HH3D Auto] Tổng môn ${isValid ? 'hợp lệ' : 'không hợp lệ'}`);

    if (isValid) {
        return true;
    } else {
        // Xóa menu nếu tổng môn không hợp lệ
        const menuWrapper = document.querySelector('.custom-script-item-wrapper');
        if (menuWrapper) {
            menuWrapper.remove();
            console.log('[HH3D Auto] Đã xóa menu do tổng môn không hợp lệ');
        }
        // showNotification("[HH3D Auto] ⚠️ Tổng môn không hợp lệ. Không thể tiếp tục sử dụng script.", "error",
10000);
        return false;
    }
}

}

// =====
// KHỞI ĐỘNG CHƯƠNG TRÌNH
// =====
// if(await checkTongMon()) {
//     console.log("[HH3D Auto] Tổng môn hợp lệ, tiếp tục khởi động script...");
//     showNotification("[HH3D Auto] Tổng môn hợp lệ, tiếp tục khởi động script...", "success");
// } else {
//     console.warn("[HH3D Auto] ⚠️ Tổng môn không hợp lệ, vui lòng tham gia tổng môn hợp lệ để sử dụng script.");
//     showNotification("[HH3D Auto] ⚠️ Tổng môn không hợp lệ, vui lòng tham gia tổng môn hợp lệ để sử dụng script.", "error", 10000);
//     return; // Dừng khởi động script nếu tổng môn không hợp lệ
// }

// .....
// Nếu xóa phỏn check tổng môn thì gọi riêng phỏn lấy ID tổng môn để lưu tên tổng môn vào localStorage và hiển thị
UI
await getTongMonId(weburl + 'danh-sach-thanh-vien-tong-mon?t=' + Date.now());

// =====
// KHỞI TẠO SCRIPT
// =====
const taskTracker = new TaskTracker();
accountId = await getAccountId();
if (accountId) {
    let accountData = taskTracker.getAccountData(accountId);
    // console.log(`[HH3D] Account ID: ${accountId}`);
    // console.log(`[HH3D] Đã lấy dữ liệu tài khoản: ${JSON.stringify(accountData)}`);
} else {
    console.warn("[HH3D] ⚠️ Không thể lấy ID tài khoản.");
}

const securityToken = await getSecurityToken();
if (!securityToken) {
    showNotification("[HH3D] ⚠️ Không thể lấy security token.", "error");
}

// Khởi tạo các class
const vandap = new VanDap();
const dothach = new DoThach();
const hoangvuc = new HoangVuc();
// const luanvo = new LuanVo();
const bicanh = new BiCanh();
const bicanhhiente = new BiCanhHienTe();
const khoangmach = new KhoangMach();
const hienTuviKM = new hienTuviKhoangMach();
const hoatdongngay = new HoatDongNgay();
const tanghoa = new TangHoa();
// await tanghoa.init();

const hvmuaruong = new HoangVucShop();
const luyendan = new LuyenDan();

```

```

if (securityToken) {
  console.log("[HH3D Auto] Đã lấy thành công token, tự động kiểm tra tiến độ luyện đơn...");
  luyendan.doLuyenDan().catch(err => {
    console.error("[HH3D Auto] Lỗi khi chạy luyện đơn ban đầu:", err);
  });
}

// Khởi tạo và chạy UI
const uiStyles = new UIMenuStyles();
uiStyles.addStyles();

const createUI = new UIInitializer(".load-notification.relative", LINK_GROUPS, accountId);
createUI.start();

const tienduyen = new TienDuyen();
//await tienduyen.init();

const automatic = new AutomationManager();
window.hh3dAutomatic = automatic; // Save to window for access from UI
window.isRunning = null; // null = chưa chạy, true = đang chạy, false = đã dừng

// Delay 1000ms để UI ổn định
await new Promise(resolve => setTimeout(resolve, 1000));

automatic.checkAndStart();
if (location.pathname.includes("khoang-mach") || location.href.includes("khoang-mach")) {
  hienTuviKM.startUp();
}
})();

```